4MW 双馈风电机组
主控制器综合技术手册

控制逻辑 · 数学模型 · 参数与工程验证

SY_SG_WG4000KDF_NPC
GSC / RSC · 背靠背 NPC 变流器

完整源码与 PSCAD 工程核对版

2026年10月12日

A4 打印版 · 公式原生数学排版

目 录

第1章 阅读指南与技术范围 3

第2章 双馈系统、物理量与基准 5

第3章 源码构建与初始化 13

第4章 调度、状态区与接口契约 18

第5章 参数、命令与数值换算 34

第6章 公共算法：数学模型与源码对照 59

第7章 网侧变流器 GSC 控制 85

第8章 机侧变流器 RSC 控制 109

第9章 频率支撑、穿越恢复与特殊分支 137

第10章 保护、联锁、锁存与复位 148

第11章 工况验证与结果判据 159

第12章 工程操作与故障排查 161

第13章 附录 参数全集与源码索引 162

第14章 PSCAD 工程与宿主接口核对 175

推荐阅读：系统与基准（第2章）→公共算法（第6章）→网侧和机侧（第7、8章）→保护和恢复（第9、10章）。工程使用按第3、4、5、12、13章查接口、构建、参数和操作。历史验证单独位于第11章。

表格跨页重复表头；公式按章节编号；PDF含书签与可点击目录。打印建议A4、纵向、100%或适合页面，双面打印可按阅读器设置。

第1章 阅读指南与技术范围

本手册说明 4 MW 双馈风电机组背靠背 NPC 变流器主控制器 SY_SG_WG4000KDF_NPC 的控制逻辑、数学模型、状态接口、参数换算与 PSCAD 工程实现。网侧 GSC 调节直流母线与网侧电流，机侧 RSC 根据功率命令生成转子电流参考，并与穿越、恢复和保护逻辑共同决定输出。

1.1 阅读路线与核对基准

源码基准为控制器核心代码下的源码目录，共 7 个 C++ 实现文件和 1 个接口头文件。函数、状态地址、分支与运算顺序以 2026 年 10 月 12 日核对的内容为准。工程侧以工程文件目录中的 PSCAD 工程及 Fortran 包装为对象，与原装基准目录比较。文件在目录中存在不代表宿主当前已加载该版本。

表 1　阅读路线

目的	章节
系统与物理量	第2章
构建、初始化、接口与调度	第3、4章
参数换算与命令	第5章和第13章
公共算法	第6章
两侧闭环与穿越	第7、8章
频率支撑、恢复与保护	第9、10章
验证与工程操作	第11、12章
完整 PSCAD 工程及包装接口	第14章

1.2 数值层次与证据范围

配置输入、初始化值、每拍写入值、缓存值、活动调节器值及最终输出描述不同层次。参数已写入不等于本拍已被活动对象读取；中间需求达到上限也不等于最终给定或物理输出达到该值。使能、模式、后写覆盖和状态保持按源码顺序解释。

本次核对覆盖 C++、接口头文件、PSCAD 工程和 Fortran 包装，完成静态内容核对与排版核验。第11章保留项目验证记录并说明复核方法，不把记录视为本次重新执行仿真的结果。电气动态性能仍须由同初态、同输入的运行测试确认。

1.3 数学符号与排版约定

表 2　数学符号约定

符号	含义	单位或约定
k 和 T_s	本拍索引与控制周期	x_{k-1} 为上一拍，T_s=250 μs
α β 与 d q	静止与旋转坐标分量	V、A 或明确标出的内部定标
上标 + 和 −	正序与负序	母线正负极另行说明
ω f θ	电角速度、频率与电角度	rad/s、Hz、rad
p[i]	局部控制段	p[i]=parameters[i+10]
0x 地址	状态区字节偏移	区分全局与机侧局部地址
clip(x,L,U)	先上界后下界限幅	上下限及非有限边界按 C++ 比较

分式、根号、上下标、分段函数、矩阵及多行表达以结构化数学格式排版，再固化为 PDF。配套源稿可在浏览器中直接打开，阅读和打印无需安装 LaTeX。公式按章编号；源码节选标出文件和一基行号。源码中的 1.414、1.732、0.577 等有限常数原样保留。

第2章　双馈系统、物理量与基准

2.1 功率拓扑与两侧职责

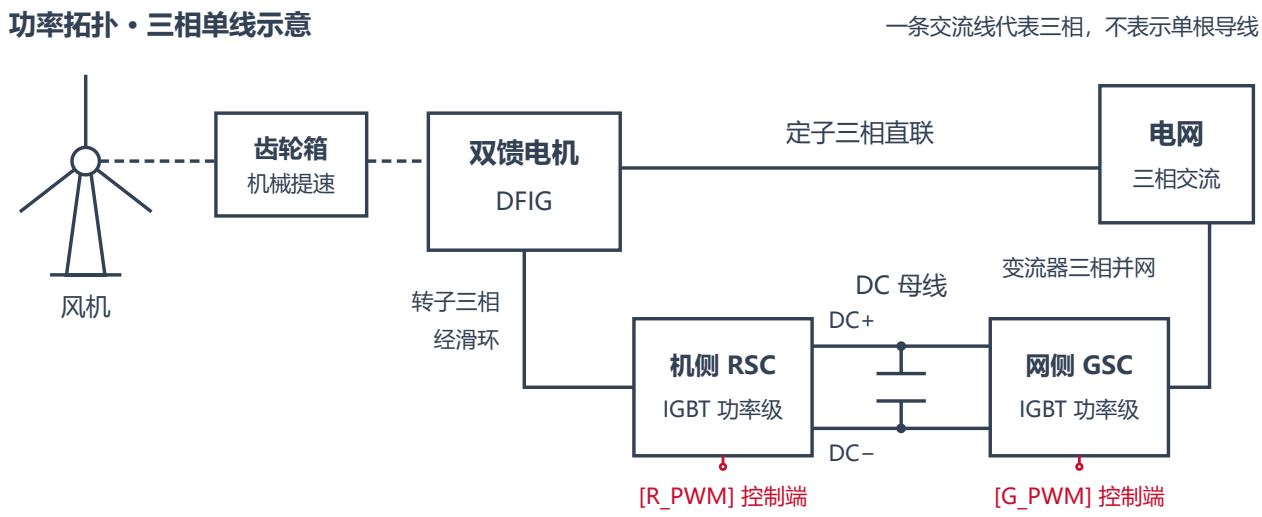


图 1　双馈系统功率拓扑与独立控制信号路径：三相单线示意

表 3　物理量、铭牌与控制基准

量与符号	当前值 / 单位	解释与来源
功率基准 S_base	4 000 000 W（4 MW）	Gen_1；工程采用4 MVA容量口径，功率给定按该基准标幺化
额定频率 f_base	50 Hz	Control_56；额定电角频率314.159265… rad/s
定子线电压有效值 V_s	1140 V	Gen_4=1.14 kV；保护门槛按线电压RMS换算
转子铭牌电压 V_r	3541 V	Gen_3=3.541 kV；电压比3541/1140=3.10614035
线电压幅值基准 V_LL,pk	1611.96 V	1140×1.414；不得误称相电压峰值
相基波幅值比较基准 V_ph,pk	930.69284065 V	1611.96/1.732；LVRT/HVRT参数门槛据此换算
基波提取器默认幅值基准	930.69 V	初始化时0x1780/0x1A20；与换算值差异源于有限常数

量与符号	当前值 / 单位	解释与来源
母线目标 U_dc*	1800 V（正、负母线各约900 V）	0x168；初始化上限0x170=2100 V
网侧电流换算基准 I_G	636.3 A	0x110；网侧电流换算基准，单位 A
机侧电流基准 I_R	989.8 A	机侧局部0x48；全局0x1B00
极对数 N_p / 额定转速 n	2 / 1680 r/min	Gen_10 / Gen_11；机械同步转速1500 r/min
Gen_12 原配置	3000 r/min	照录面板值；不作为50Hz、2对极的物理同步转速
内部定标 S_pu	8 388 608 = 2^23	内部调制与电压参考尺度，须区分工程pu和内部定标
控制周期 T_s / 仿真步长	250 µs / 10 µs	每25个仿真调用对应一次控制拍；历史录波间隔1ms

表 4　网侧与机侧分工

侧别	**GSC网侧**	**RSC机侧**
连接	电网—变流器—直流母线	直流母线—变流器—转子
主任务	直流稳压、无功支撑、滑差功率交换	转矩和励磁相关电流、定子有功和无功调节
外环	直流母线电压PI	命令换算、转矩/励磁和功率电流合成
内环	正负序各d/q四个电流PI	正负序各d/q四个电流PI
正常增益	正负序1200/20	正序3500/15、负序4500/25
故障策略	LVRT无功优先、HVRT吸收、母线与电流约束	LVRT强励、HVRT吸收、撬棒与过流闭锁

$$n_s = \frac{60 f_{\text{base}}}{N_p} = 1500 \text{ r / min} \tag{2-1}$$

$$f_{\text{slip}} = \frac{n N_p}{60} - f_{\text{base}} = 6 \text{ Hz} \tag{2-2}$$

$$s = \frac{n_s - n}{n_s} = -0.12, \qquad f_{\text{slip}} = -s f_{\text{base}} \tag{2-3}$$

超同步时转子滑差功率送至电网，次同步时电网向转子供功率。转差频率带单位Hz；教材无量纲转差率s符号相反，不应混用。

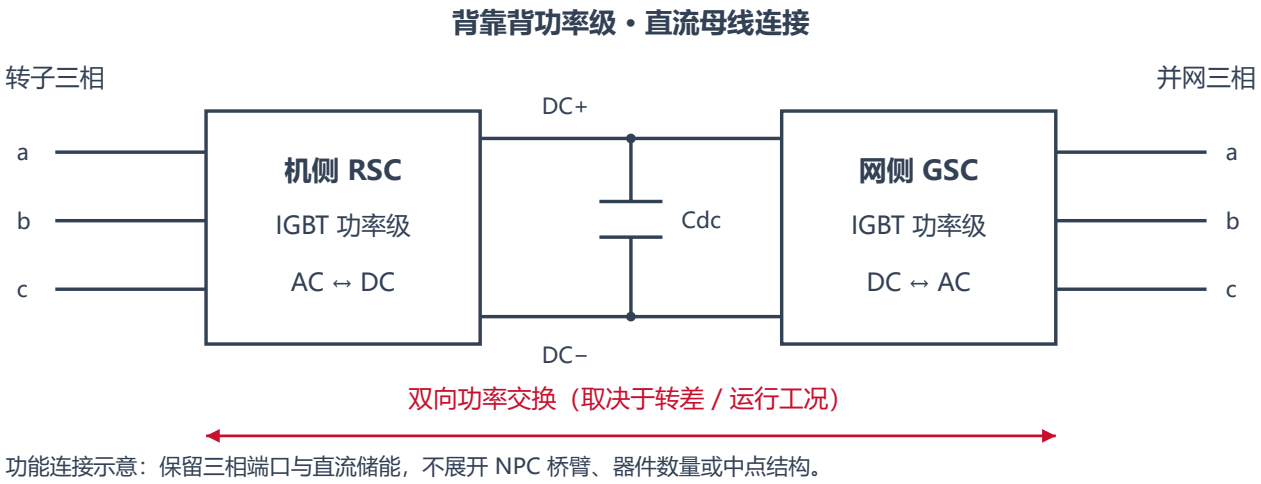


图 2　背靠背功率交换路径与直流环节

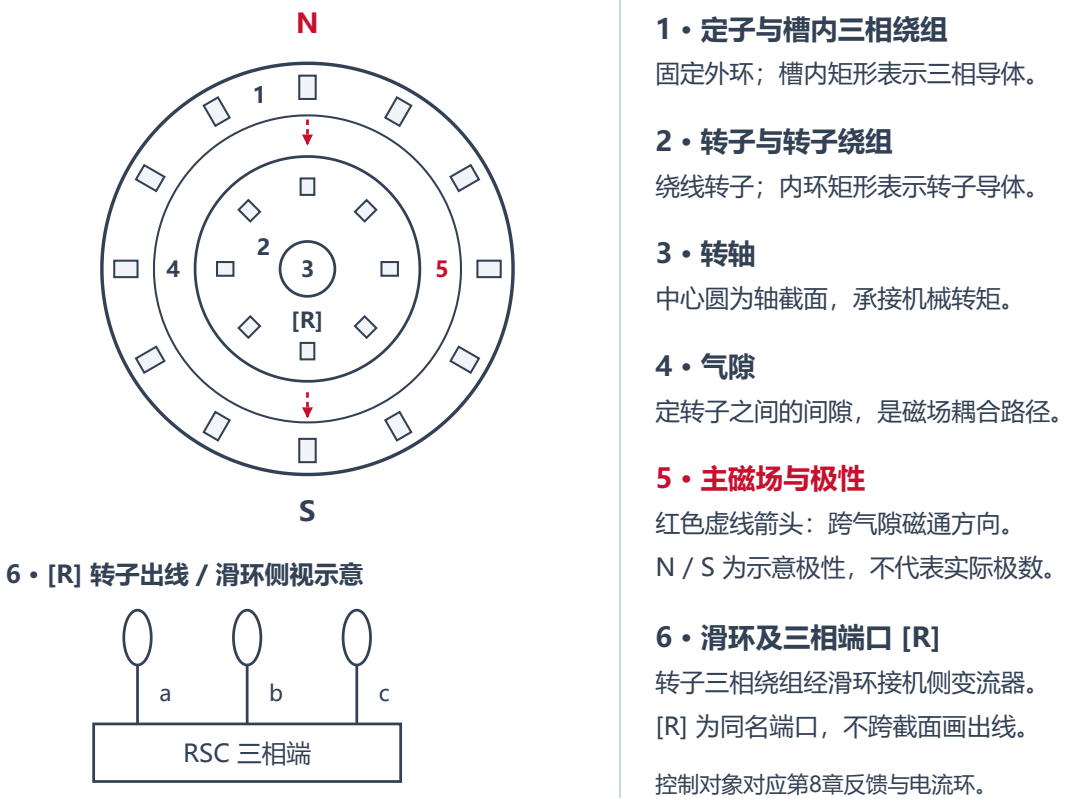


图 3　双馈电机定子、转子与机械轴示意

控制实现与逐拍关系

双馈机组与背靠背变流器

双馈机组定子直接并网，转子经背靠背变流器与电网交换滑差功率。电网50Hz、极对数2时机械同步转速1500r/min；1680r/min时为超同步，转子滑差功率经RSC—母线—GSC送回电网。Gen_12配置3000照录为工程面板值，不能与机械同步转速混用。

控制周期与状态区

PSCAD 仿真步长 10 μs；控制算法每 250 μs（4 kHz）执行一次，由总调度中的采样闸门控制。控制器全部中间量存放在一块 0x3748 字节的状态区中：前 0x1AB8 字节为网侧状态区，从 0x1AB8 起的 0x1C68 字节为机侧状态区。每个变量按固定字节偏移量（门牌号）读写。

每个控制周期内的调用序列

每个 250 μs 控制拍按以下顺序执行：

grid_input / rotor_input —— 把 PSCAD 输入搬进状态区（机侧同时算出转差频率）；

rotor_nameplate / parameter_input / protection_input / command_input —— 铭牌、参数、保护定值、命令换算；

grid_calculate —— 先执行网侧主算法。

仅全状态0x1E08==1.0时执行grid_frequency_support；随后0x1D58叠加0x768的有功给定修正，再执行motor_calculate。

grid_post / motor_post —— 两侧状态机与保护；

average_converter_output —— 直流母线电流折算（平均模型用）；

grid_output / rotor_output / view_output —— 打包输出给 PSCAD。

铭牌和两侧分工

表 5　铭牌和两侧分工：参数与状态关系

项目	数值	含义与来源
额定容量 Sn	4,000,000 VA（4 MVA）	铭牌参数 Gen_1；控制器功率基准 0x538
定子线电压	1140 V	铭牌 Gen_4 = 1.14 kV；线电压峰值基准 = 1140×1.414 ≈ 1611.96 V
转子线电压	3541 V	铭牌 Gen_3 = 3.541 kV；与定子电压比 = 3541/1140 = 3.10614
极对数 / 同步转速	2 对极 / 1500 rpm	铭牌 Gen_10；Gen12原面板值3000；物理同步转速1500，不混作同一量
直流母线电压	总额定 1800 V（±900 V）	控制器目标定值 0x168 = 1800 V、上限 0x170 = 2100 V；网侧电流基准 636.3 A（0x110）
网侧电流基准	636.3 A	网侧电流有名值换算基准；不是母线额定电压
机侧（转子）电流基准	989.8 A	机侧电流标幺→有名值的乘数

项目	数值	含义与来源
网侧电压基准（幅值）	930.69 V	用于网侧标幺电压计算（0x1780）
机侧电压基准（幅值）	1611.96 V	机侧控制局部幅值尺度，须按读者区分（0x410）
额定频率	50 Hz	Control_56 Rate_Freq；控制周期 250 μs = 每周期 80 个控制拍
定子电阻 Rs	0.0097 pu	Gen_5
定子漏抗 Xls / 转子漏抗 Xlr	0.0774 / 0.0999 pu	Gen_6 / Gen_8
转子电阻 Rr	0.0087 pu	Gen_7
励磁电抗 Xm	2.6962 pu	Gen_9；换算系数 Xm·Vs²/PMW = 0.87599538 存入状态区
额定转速（面板）	1680 rpm	Gen_11（额定风速下的转速定值）

表 6　铭牌和两侧分工：参数与状态关系

对比项	网侧变流器 GSC	机侧变流器 RSC
接在哪儿	直流母线 ↔ 电网	转子绕组 ↔ 直流母线
核心任务	把直流母线电压稳定在 1800 V 附近；把滑差功率送回电网	控制转子电流的幅值与频率（转差频率），从而控制电磁转矩与定子无功
控制外环	直流电压外环（PI：kp=5、ki=0.15）→ 有功电流给定	转矩给定（P*×1.045）与励磁给定（Q*+0.015）→ 电流给定
电流内环	正序、负序各一组 PI：kp=1200、ki=20	正序 PI：kp=3500、ki=15；负序 PI：kp=4500、ki=25
穿越策略	LVRT 增发无功（封顶 745.68 A）；HVRT 吸收无功（限 −53.57 ~ −424.24 A）	LVRT 强励（转子 d 轴电流封顶 976.24 A）；HVRT 吸收无功（限 −40.104 ~ −424.240 A）
锁相环	锁电网电压（kp=0.08、ki=0.0001）	同样锁电网电压（kp=0.08、ki=0.0001，参数入口每拍重写），保证转子电流相位与电网一致
调制方式	标准调制（零序注入 + 中点电位平衡）	标准调制版（0x6E8 = commands[9] = 0 每拍覆盖；备用调制仅在 0x6E8==1 时启用）

Gen1~12及门槛

表 7 Gen1~12及门槛：参数与状态关系

参数	面板值	含义	控制器里的用途
Gen_1 Generator_Power_W	4,000,000 W	额定功率（瓦）	控制容量基准 0x538 = 4 MVA；用于参数电流换算，与电机模型容量分开
Gen_2 Generator_Power_MW	4.0 MW	额定功率（兆瓦）	Gen2=4MW用于工程包装的 Xm×Vs²/Gen2；不是未取用的普通备注
Gen_3 Motor_Vrotor	3.541 kV	转子开路电压	与定子电压 1.14（铭牌槽[1]，参数契约 标注"额定电压换算"、无对应面板名，×1000 → 0x6D0）组成转子/定子电压比 3541/1140 = 3.10614。该比值双来源：rotor_nameplate 每拍算 0x5F0 = 0x6D8/0x6D0；motor_defaults 固定写 0x14A0（合并区 0x2F58）——对应换算关系 的 b 项因子取 0x2F58，两处数值相同。铭牌槽与 Gen 面板变量的确切对应需对照 PSCAD 工程判定
Gen_4 Motor_Vstator	1.14 kV	定子额定电压（线电压有效值）	即 1140 V —— 保护定值换算基准
Gen_5 Motor_Rs_pu	0.0097	定子电阻（标幺）	机侧电机模型参数
Gen_6 Motor_Xls_pu	0.0774	定子漏抗（标幺）	机侧电机模型参数
Gen_7 Motor_Rr_pu	0.0087	转子电阻（标幺）	机侧电机模型参数
Gen_8 Motor_Xlr_pu	0.0999	转子漏抗（标幺）	机侧电机模型参数
Gen_9 Motor_Xm_pu	2.6962	励磁电抗（标幺）	机侧电机模型参数
Gen_10 Motor_Pole_Pairs	2.0	极对数	转差频率公式（6-1）的极对数：额定 1680 rpm 时 转差 = 1680×2/60 − 50 = +6 Hz（超同步运行）
Gen_11 speed	1680 rpm	额定转速	转差计算用转速
Gen_12 syn_speed	3000 rpm	面板标注的同步转速	照录面板原值；控制器转差计算只用 Gen_10（极对数），不用 Gen_12

表 8 Gen1~12及门槛：参数与状态关系

门槛	标幺	运行时值	说明
LVRT 进入	0.9	837.624 V	正序电压幅值低于此值 → 进入低压穿越（网侧、机侧同一门槛）
LVRT 退出	0.95	884.158 V	回升超过此值才退出（滞回 0.05 防抖动）
HVRT 进入	1.13	1051.683 V	正序电压幅值高于此值 → 进入高压穿越
HVRT 退出	1.08 且负序 < 56.56 V；或 1.02	1005.148 / 949.31 V	双条件（任一满足、计数阈值 4；严格比较在第5拍动作）才退出；仅回落低于 1.08 pu 不够

术语表

表 9 术语表：参数与状态关系

术语	英文	一句话白话	章节
双馈电机	DFIG	定子直联电网、转子经变流器馈电的绕线式异步发电机；变流器只处理转差功率（约 1/3 总功率）	系统总览
锁相环	PLL	自动跟踪电网电压相位角的闭环回路；控制器靠它和电网"对齐"，后面的坐标变换全靠这个角度	第6章
dq / Park 变换	Park	把旋转的三相交流量变成两个直流量；直流才能被 PI 调节器无静差地控制	第6章
序分量	Sequence	把不对称的三相量拆成"正序（正常旋转）+ 负序（反向旋转）"；故障不平衡时靠负序通道来对付	第6章
PI 调节器	PI	比例（P）决定响应快慢，积分（I）消除稳态误差；本控制器所有环路共用同一公式	第6章 / D
抗饱和	Anti-windup	输出被限幅后，把"超出的量"回馈进积分器，防止积分越积越大、恢复时过冲	第6章

术语	英文	一句话白话	章节
SPWM 调制	SPWM	三角载波和正弦调制波比较，交点处开关动作，等效输出正弦电压	第6章 配图
平均模型	AVG	用"平均值"代替逐个开关动作的简化变流器模型，仿真快、波形干净	第8章
低电压穿越	LVRT	电网电压跌落期间保持并网不脱网，并按整定值注入无功电流支撑电网	第8章
惯量/频率支撑	Inertia / FFR	频率跌落时短时释放额外有功，模拟同步机的"惯性"，帮电网把频率顶 住	第6章
快照	Snapshot	14152 字节完整状态的存档/恢复机制；motor[9]==1000 时从快照恢 复状态	第4章

第3章 源码构建与初始化

3.1 源码文件职责

表 10 文件职责

文件	职责
recovered_primitives.h	结构体布局、函数声明与 ABI 约定
recovered_primitives.cpp	PI、变换、PLL、滤波、调制与输入输出
recovered_initialization.cpp	默认状态及控制对象初态
recovered_controller.cpp	实例、时间闸门、跨区复制与总调度
recovered_parameters.cpp	参数、保护定值、换算与活动增益装入
recovered_commands.cpp	功率命令、无功模式、使能与复位输入
recovered_grid.cpp	网侧控制、保护与频率支撑
recovered_motor.cpp	机侧控制、启动、穿越、恢复与闭锁

3.2 编译与接入步骤

本地 build_primitives.cmd 位于控制器核心代码目录。脚本调用 VCVARS64 指定的 vcvars64.bat 配置 MSVC 环境，编译 7 个 .cpp 并由它们包含 .h。选项为 /nologo /LD /O2 /fp:strict /EHsc /W4，产物为 build/recovered_primitives.dll；本地脚本没有 /utf-8 选项。构建前后分别运行 test_primitives.py 的 building 和 complete 记录阶段。

执行前核对 VCVARS64 安装路径、py 命令和记录脚本；在控制器核心代码目录运行 build_primitives.cmd，保存退出码、日志及构建指纹。随后核对目标位数、导出符号、调用约定、运行库和宿主数组布局。源码含中文，编译代码页与文件编码须对应，不能直接忽略全部 C4819 告警。

接入 PSCAD 前保存现用 DLL 的可恢复副本，停止加载对应 DLL 的仿真实例，在工程要求的位置替换主控制器 DLL，重新运行并核对实际加载路径和哈希。Fortran 包装中的数组次序、名称和类型属于接口契约。手册核对不改变源码、工程或 DLL，不构成控制器重新构建通过。

3.3 初始化、默认值与局部复位的边界

初始化不是把每个默认值写一遍的同义词。整体初始化先清除14152字节，再调用网侧和机侧默认值；两侧默认值仅覆盖指定偏移。以下按实际写入顺序解释上电初态，并把首次参数装载之后的活动值与初始化值分开。

3.3.1 初始化序列R、H、F的精确字段

initialize_r从p、q指定偏移复制两个系数到b和b+0x48，再把后者除以前者写入b+0x50；对称限值位于b+0x28和b+0x30，绝对值由0.577乘v偏移的基准生成。这是一段偏移写入序列，不能仅凭函数名把它等同于RegulatorState的任意字段布局；调用方常以调节器Kp位置作为b。

initialize_h分别写入1.414、314.159265358979、0.00025和正负62.8318530717958；initialize_f写入4、628.318530717958及0.00025。它们不清空整块滤波历史。period_count在第122～125行以CVTT向零截断(1/f)/0.00025，正常正频率下等于向下取整；频率0、NaN或超范围不是软件饱和处理。

$$N(f) = \operatorname{cvtt}\left(\frac{1/f}{0.00025}\right) \tag{3-1}$$

$$L_R = 0.577\,v \tag{3-2}$$

recovered_initialization.cpp L135-L158

```
inline void initialize_r(unsigned char* state, std::size_t b,
                         std::size_t p, std::size_t q, std::size_t v)
{
    store_double(state, b, load_double(state, p));
    store_double(state, b + 0x48, load_double(state, q));
    store_double(state, b + 0x50,
                 load_double(state, b + 0x48) / load_double(state, b));
    store_double(state, b + 0x28, C61D20 * load_double(state, v));
    store_double(state, b + 0x30, -load_double(state, b + 0x28));
}

inline void initialize_h(unsigned char* state, std::size_t b)
{
    store_double(state, b, C62060);
    store_double(state, b + 0x18, C62530);
    store_double(state, b + 0x08, C61970);
    store_double(state, b + 0x28, C623A0);
    store_double(state, b + 0x30, C629F0);
}

inline void initialize_f(unsigned char* state, std::size_t b)
{
    store_double(state, b + 0x08, C62200);
    store_double(state, b, C625A0);
```

recovered_initialization.cpp L159-L160

```
    store_double(state, b + 0x50, C61970);
}
```

初始化常数保留有限精度；不要把1.414和0.577替换为理想根式。

3.3.2 网侧默认值为局部覆盖而非全区清零

grid_defaults把启动计数与部分标志清零，并写入400、800等计数门槛；这些拍数的秒数解释以固定算法步长0.00025为前提。0x350是运行状态整数，0x50C是输出使能整数，0x4B8、0x4D0、0x4E8是三个double输出。不同类型必须按字节布局读取。

0x110默认636.3，供电流换算链使用；0x120为0.5倍该基准，0x138/0x140/0x148实际为1.25倍，即795.375，不能按附近注释写成1.2倍。0x118为5倍基准。函数没有memset，未列入store序列的状态保留原值；与整体初始化结合时这些未覆盖字节才因前置memset成为零。

$$1.25 \times 636.3 = 795.375 \tag{3-3}$$

recovered_initialization.cpp L175-L198

```
// 开关标志与计数器：0x000 启动计数、0x004 启动时长 400 步（0.1 秒）、
// 0x008 = 800 步（0.2 秒）、0x35C 停机时长 800 步；
// 0x350 运行状态、0x50C 输出使能、0x4B8/0x4D0/0x4E8 输出清零。
    store_int(s, 0x000, 0);
    store_int(s, 0x018, 0);
    store_int(s, 0x01C, 0);
    store_int(s, 0x004, 400);
    store_int(s, 0x008, 800);
    store_int(s, 0x35C, 800);
    store_int(s, 0x350, 0);
    store_double(s, 0x4B8, 0.0);
    store_double(s, 0x4D0, 0.0);
    store_double(s, 0x4E8, 0.0);
    store_int(s, 0x50C, 0);
    store_int(s, 0x500, 0);

// 直流电压基准与限幅：0x110 = 636.3V（直流母线额定电压），
// 0x120/0x138/0x140/0x148 = 636.3 的 0.5/1.2/1.2/1.2 倍（外环输出限幅），
// 0x118 = 5×636.3（积分限幅），0x080 = 8388608（大数，用作除法基准）。
    store_double(s, 0x110, C625B0);
    store_double(s, 0x120, C61CD0 * load_double(s, 0x110));
    store_double(s, 0x138, C62000 * load_double(s, 0x110));
    store_double(s, 0x140, C62000 * load_double(s, 0x110));
    store_double(s, 0x148, C62000 * load_double(s, 0x110));
```

recovered_initialization.cpp L199-L201

```
    store_double(s, 0x118, C62210 * load_double(s, 0x110));
    store_int(s, 0x158, 1);
    store_double(s, 0x080, C628B0);
```

每拍parameter_input会重写0x120和0x140等部分默认值。

0x080=8388608是内部定标；不是物理电压。

3.3.3 周期计数、谐振器重复写入与缓冲初态

0x3A0和0x464由period_count(f)写入每基波周期的采样点数；50Hz、0.00025s得到80。0x238按额定频率计算解耦系数，PLL缓存初值0x550=0.12、0x560=0.0012，不能把源码注释的0.15当执行值。

两个谐振对象不对称初始化：0x15B0和0x15B8被重复写入，而第二对象对应的0x1668、0x1670没有在该序列赋值。整体初始化后这些未写字节为零，局部grid_defaults调用则保持既有值。第354～360行另外分别清零0x670、0x698起的两个5点double缓冲，但不代表所有频率支撑状态均被清空。

$$N(50) = 80 \tag{3-4}$$

$$\omega_h = 1884.955592153874 \tag{3-5}$$

recovered_initialization.cpp L294-L317

```
    store_int(s, 0x3A0, period_count(f));
    store_int(s, 0x464, period_count(f));
    store_double(s, 0x238, (C625C0 * (C619D0 * f)) * C62060);
    store_double(s, 0x550, C61AF0);
    store_double(s, 0x560, C61990);
    store_double(s, 0x558, C61AF0);
    store_double(s, 0x568, C61930);
    store_double(s, 0xE20, C62210);
    store_double(s, 0xE28, C61B60);
    store_int(s, 0xEA8, 1);
    store_double(s, 0xE30, C622A0);

// 两组谐振器初值（0x15A8 起、0x1660 起）：中心频率 2π×300 = 1884.96（300Hz），
// 带宽 ±62.83（±10Hz），步长 0.00025，输出限幅 ±0.05×8388608。
// 原 DLL 会重复写 0x15B0/0x15B8 两处（重复但不影响），原样保留。
    store_double(s, 0x15B0, C62770);
    store_double(s, 0x15B8, C623A0);
    store_double(s, 0x1628, 0.0);
    store_double(s, 0x15A8, C61970);
    store_double(s, 0x1650, C61A80 * load_double(s, 0x080));
    store_double(s, 0x1658, C628C0 * load_double(s, 0x080));

    // The original repeats these offsets; it does not initialize 0x1668/0x1670.
    store_double(s, 0x15B0, C62770);
```

recovered_initialization.cpp L318-L321

```
    store_double(s, 0x15B8, C623A0);
    store_double(s, 0x16E0, 0.0);
    store_double(s, 0x1660, C61970);
    store_double(s, 0x1708, C61A80 * load_double(s, 0x080));
```

保留重复写入和遗漏写入的差异，不能凭对象相似性补齐。

3.3.4 整体初始化顺序及与命令复位的区别

controller_initialize 先 memset 整个 0x3748 字节，然后 grid_defaults(f,state)，再 motor_defaults(f,state+0x1AB8)，最后写入两侧启动命令、LVRT/HVRT使能及附加默认值。机侧基址0x1AB8为十进制6840；全局偏移与局部偏移必须相加，不能因两侧同名地址而混淆。

这些尾部数值如0x370=1596、0x380=1.1f、0x388=0.9f、0x1FD8=141400属于整体初始化初值。parameter_input随后每控制拍把对应部分写成1.4、1.1、0.9、5.3778等固定值，所以初态不能当运行时活动定值。command_input只写复位请求；两侧计算函数处理请求时调用各自defaults，而非这里的整区memset，局部复位与init入口、运行号变化引发的整体初始化应分开说明。

$$\mathrm{offset}_{global} = 0x1AB8 + \mathrm{offset}_{motor} \tag{3-6}$$

$$0x3748 = 14152 \tag{3-7}$$

recovered_initialization.cpp L373-L396

```
extern "C" __declspec(dllexport) void controller_initialize(double f, void* state)
{
    auto* s = static_cast<unsigned char*>(state);
    std::memset(state, 0, 0x3748);
    grid_defaults(f, state);
    motor_defaults(f, s + 0x1AB8);

    store_int(s, 0x348, 1);
    store_int(s, 0x1FAC, 1);
    store_int(s, 0x518, 1);
    store_int(s, 0x1FEC, 1);
    store_int(s, 0x51C, 1);
    store_int(s, 0x1FF0, 1);
    store_int(s, 0x2088, 1);
    store_double(s, 0x370, C62720);
    store_double(s, 0x378, C62480);
    store_double(s, 0x380, C61F40 * f);
    store_double(s, 0x388, C61E70 * f);
    store_double(s, 0x368, C62850);
    store_double(s, 0x390, C627D0);
    store_double(s, 0x398, C62640);
    store_double(s, 0x1FC8, C62830);
    store_double(s, 0x1FD0, C62720);
    store_double(s, 0x1FD8, C62880);
```

recovered_initialization.cpp L397-L397

```
}
```

end入口只清状态，没有调用本函数。

源码能证明调用顺序，不能证明任何宿主工程的实际首次输入。

第4章　调度、状态区与接口契约

4.1 三种时间尺度与状态记忆

仿真调用10μs、控制拍250μs、工程导出1ms分别负责积分步进、算法更新和录波存档。输出数组随有效控制拍更新；其余仿真步保持已有输出。1ms导出不能称控制器仅1ms更新，也不能称覆盖全部10μs仿真步。



图 4　时间尺度示意；图中录波与控制器输出更新须按正文区分

完整状态区14152字节（0x3748），保存滤波记忆、PI积分与抗饱和误差、正负序估计、模式、事件计数器、累计计时、输出缓存及门控。相同瞬时输入不意味着相同输出，比较必须采用相同初态或快照。机侧局部偏移基址0x1AB8；全局地址=0x1AB8+局部偏移。

控制实现与逐拍关系

采样周期闸门

PSCAD 每 10 μs 调用一次主入口，控制算法只在累计时间归零的那一步执行：

$$T_{acc} = T_{acc} + \Delta t \tag{4-1}$$

每次主入口先检查累计时间严格等于0，仅此时执行控制。随后累计时间加宿主delta，达到sample_period后置0供下一次主入口执行；非达到门槛同次立即控制。250μs/10μs为当前工程参数。运行号变化初始化，快照支持状态保存恢复。

输入输出映射

网侧8路依次三路线电压、正负母线电压、三相电流；机侧依次三相转子电流、1140V低压接入测量电流、编码器角和转速。各20路输出仅模式选择的1…6信号乘使能，指定零路直接置0，7…19不乘使能。模式0相电压/母线电流、1占空比、2调制波；非法模式保持1…6原缓冲。平均模型按m>0与

otherwise分流，网侧负号机侧正号。 两个输入数组的[8]为宿主元数据，主入口比较motor[8]与grid[8]决定是否保存快照；grid[9]为运行号，motor[9]==1000触发从snapshot.txt恢复14152字节状态。

$$I_{\text{nonpos}} = \eta \sum_{j:m_j \le 0} m_j i_j, \quad I_{\text{pos}} = \eta \sum_{j:m_j > 0} m_j i_j, \quad \eta_{\text{GSC}} = -1, \eta_{\text{RSC}} = 1 \tag{4-2}$$

保护复位计数

自动复归机制的潜在阈值为8000控制拍（2s），受0x3738以及两侧相关门控控制。当前源码内部没有把整链门控激活的写入路径，不能断言当前配置跳闸后自动重启。外部宿主或状态恢复可改变状态；重新验证时需记录实际初始状态。

18步执行序列

表 11　18步执行序列：参数与状态关系

步	模块	做什么（与关键状态区位置）
1	grid_input	网侧 8 路采样入状态区：[0..2] 线电压 vab/vbc/vca → 0xC0/0xC8/0xD0；[3..4] 直流母线正/负电 压 → 0x100/0x108；[5..7] 三相电流 ia/ib/ic → 0xD8/0xE0/0xE8
2	rotor_input	机侧8路：三相转子电流、1140V低压接入测量电流、编码器角、机械转速 → 0x228~0x6E0；并计算转差频 率 fslip = 转速×极对数/60 − 50 Hz（0x5F8）
3	rotor_nameplate	铭牌换算写入：额定功率 4,000,000（0x598）、定子电压 1.14×1000 = 1140（0x6D0）、转子电 压 3.541×1000 = 3541（0x6D8）、定子电压 1.14（0x6C8）、极对数 2（0x1E0）、电压比 0x6D8/0x6D0 = 3.10614（0x5F0）；另 0x14A0 的 3.10614 由 motor_defaults 固定写入，非本函数输出
4	parameter_input	读取外层parameters+10的80槽控制段中的指定项；按类型直接写入或换算门槛、PI系数及限幅
5	protection_input	28项保护定值写入：前16项偶数下标乘0x270/1.414，奇数时间项直接存；后12项频率/时间直接存
6	command_input	11 路命令 → 启动/使能/复位标志；有功给定 P*（0x1D58）、无功给定 Q*（0x1D60，含三种模 式换算）

步	模块	做什么（与关键状态区位置）
7	跨区搬移	电网电压复制到机侧可见区（0x1D10~0x1D20）；P* 副本 0x7C0（供频率支撑限幅用）；频率支撑使能 0xBD0
8	保护复位计时	门控0x3738和相关标志满足时才累计；8000拍为潜在复归阈值，当前整链未激活
9	grid_calculate	网侧主控制：PLL → 序分解 → 直流电压外环 → LVRT/HVRT 逻辑 → 正负序电流环 → 调制（第 5章）
10	grid_frequency_support	仅使能（0x1E08==1）时执行：惯量响应 / 一次调频（工程配置下未使能）
11	有功叠加	0x1D58 += 0x768：调频支撑功率叠加到有功给定（未使能时为 0）
12	频率传递	0x25E0 = 0x468：网侧实测频率传给机侧（机侧 0xB28）
13	motor_calculate	机侧主控制：PLL → 转子序分解 → 转矩/励磁给定 → LVRT/HVRT → 电流环 → 调制 → 撬棒/过流（机侧控制专题）
14	grid_post	网侧状态机（停机/运行/跳闸）+ 频率/RMS 测量 + 保护判定（保护专题）
15	motor_post	机侧状态机 + 网侧线电压重构的相电压RMS测量 + 故障锁存
16	跳闸互传	网侧状态 3（跳闸）→ 0x1FB0 通知机侧；机侧状态 3 → 0x34C 通知网侧
17	average_converter_output	按直流电压折算两侧直流母线电流（供平均模型使用）
18	输出打包	grid_output 20 路、rotor_output 20 路、view_output 监控输出（130 槽中 41 个有效值）

时间尺度与分支设置

表 12　时间尺度与分支设置：参数与状态关系

事项	时长或设置值	拍数或更新频率	说明
控制周期	250 µs	1	4 kHz；1 s = 4000 拍
网侧软启动	0.1 s	400	启动后前 400 拍走电压外环软起，之后转正常

事项	时长或设置值	拍数或更新频率	说明
机侧预同步确认	0.05 s	200	d 轴电压给定与反馈相等保持 200 拍后，再等 100 拍进入正常
穿越恢复标志计数	名义门槛0.2 s	800；第801次解除	退出时穿越模式已恢复0；恢复标志计数800次后，在第801次相应分支解除，不能理解为继续保持模式1或2。
直流电压平均窗	20 ms	80	每 80 拍算一次直流电压平均值
LVRT 进入确认	5 ms	20	计数阈值 20：连续满足、从首次满足起第 21 拍进入 LVRT（防抖动）
HVRT 进入确认	1 ms	4	计数阈值 4：连续满足、从首次满足起第 5 拍进入 HVRT
保护自动复位	2 s	8000	潜在复归机制；当前整链门控未激活
停机复位	0.2 s	800	停机计数从零递增，严格大于800时在第801个调用复位
中点平衡增益选择	5.0（增益）	每控制拍	滑差越限时按方向选极性及增益5.0；中间区增益0。此项不是延时。
惯量响应确认	2 s	—	频率支撑启动前的确认窗口（未使能）
惯量维持 / 一次调频最长	10 s / 5 s	—	Control_57 / Control_60（未使能）
二次故障锁存延时	4 s	—	二类故障延时 4 s 后跳闸

八数组及换算

表 13　八数组及换算：参数与状态关系

数组	长度	方向	内容
grid（网侧输入）	10	仿真 → 控制器	[0..2] 三相线电压 vab/vbc/vca；[3..4] 直流母线正/负电压（Vdc+ 0x100 / Vdc− 0x108）；[5..7] 三相电流 ia/ib/ic（0xD8/0xE0/0xE8）；[8] 握手号；[9] 运行号（每次重新 Run 自增，用于检测重新初始化）

数组	长度	方向	内容
motor（机侧输入）	10	仿真 → 控制器	[0..2]转子电流A；[3..5]低压接入测量电流A；[6]机器编码器角；[7]转速r/min； [8] 握手号；[9] 快照标志（=1000.0 表示从快照恢复状态）
parameters（参数）	90	仿真 → 控制器	[0..4] 铭牌换算值；[5..9] 保留；[10..89]为局部控制段p[0..79]；参数函数只读取指定槽。外层[65]同时是额定频率输入；[89]写入机侧动态量0x1C40，外部含义与更新频率需由宿主契约确认
protection（保护定值）	28个内部有效项	仿真 → 控制器	[0..15] 电压类 16 项；[16..27] 频率类 12 项（保护专题）
commands（命令）	11	仿真 → 控制器	启动、P*、Q*、使能、复位、模式（3.3 节）
grid_result（网侧输出）	20	控制器 → 仿真	使能、三相调制波/占空比、运行/故障标志、频率、电压基准等
motor_result（机侧输出）	20	控制器 → 仿真	同机侧
view_result（监控输出）	130	控制器 → 仿真	监控通道：130 槽中 41 个有效（PLL 角度、正负序分量、各环输出、功率 等）

4.2 状态结构、实例生命周期与主入口执行次序

完整状态是字节缓冲，结构体只为其中部分算法提供字段契约；两侧通过同一缓冲共享命令、模式和测量。主入口先处理运行号和快照，再检查时间闸门。函数注释中的10μs与250μs是典型解释，实际闸门输入为delta和sample_period。

4.2.1 调节器字段与每拍积分、跟踪系数

RegulatorState连续排列16个double，从reference到previous_error。kp位于结构偏移0x18，upper_limit/lower_limit位于0x40/0x48，output位于0x50，antiwindup_error位于0x58，ki_per_step位于0x60，tracking_gain位于0x68，derivative_gain位于0x70。

参数入口把输入Ki写入ki_per_step，把Ki/Kp写入tracking_gain；Ki/Kp不是再乘误差的每步积分系数。initialize_r使用以Kp字段为基址的相对写入序列，因此它的b+0x50对应完整调节器结构的0x68。参数表描述应依据字段实际位置，避免按附近注释误认。

$$K_t = K_i \,/\, K_p \tag{4-3}$$

recovered_primitives.h L43-L60

```
struct RegulatorState {
    double reference;
    double feedback;
    double error;
    double kp;
    double proportional;
    double integral;
    double derivative;
    double unsaturated;
    double upper_limit;
    double lower_limit;
    double output;
    double antiwindup_error;
    double ki_per_step;
    double tracking_gain;
    double derivative_gain;
    double previous_error;
};
```

这里只定义存储；精确离散更新见regulator实现。

4.2.2 坐标变换结构及保留槽

LineVoltageState由vab、vbc、reserved、alpha、beta组成；第三个double是占位，不能当vca参与变换。CurrentTransformState由ia、ib、ic、alpha、beta组成。两种结构尺寸均为五个double，却代表不同输入契约。

ParkState依次存alpha、beta、theta、d、q。正变换和逆变换共享同一类型，但输入/输出字段的作用由被调函数决定。结构声明没有单位变换或自动校验，宿主与算法必须使用一致的电压、电流和角度单位。

recovered_primitives.h L66-L89

```
struct LineVoltageState {
    double vab;
    double vbc;
    double reserved;
    double alpha;
    double beta;
};

// 三相电流 → αβ 变换（等幅值 Clarke 变换，系数 2/3）。
// ia/ib/ic 三相电流，alpha/beta 是变换结果。
struct CurrentTransformState {
    double ia;
    double ib;
    double ic;
    double alpha;
    double beta;
};

// Park 变换：把静止的 αβ 坐标系旋转到与电网电压同步的 dq 旋转坐标系。
// theta 是旋转角（通常来自锁相环 PLL），d、q 是变换结果。
// 在 dq 坐标系里，50Hz 的正弦量会变成直流量，PI 调节器才能无静差地控制它。
struct ParkState {
    double alpha;
    double beta;
```

recovered_primitives.h L90-L92

```
    double theta;
    double d;
    double q;
```

不能因结构大小相等而交换线电压变换和电流变换。

4.2.3 两态滤波器保存更新前后的内部状态

TwoStateFilter的step、scale之后是input/error以及x、next_x、z、next_z，尾部三个字段是x_coefficient、z_coefficient、gain。next状态和当前状态分别占位，不能把两者合并为一个数学符号后忽略更新次序。

结构声明只能证明这些字段的排列，不能单独证明该滤波器中心频率、传递函数或稳定范围；这些结论必须结合two_state_filter的实际算式及调用方系数。局部defaults未清写的历史在局部复位后可以保留。

recovered_primitives.h L98-L110

```
struct TwoStateFilter {
    double step;
    double scale;
    double input;
    double error;
    double x;
    double next_x;
    double z;
    double next_z;
    double x_coefficient;
    double z_coefficient;
    double gain;
};
```

滤波特性由初始化和每拍系数共同决定。

4.2.4 递归与残差滤波器的历史布局

RecursiveFilterState依次存三个系数、当前和上一输入、当前和上一输出，previous_output_coefficient的命名不自动确定递推中的加减号。公式必须以recursive_filter函数的执行表达式为准。

ResidualFilterState由两路input、两路lowpass和两路residual组成，各数组为两个double。d/q两个通道不等同于两侧控制器；调用方决定输入是什么。低通历史参与后续控制，整体清零与选择性复位得到的初态也可能不同。

recovered_primitives.h L114-L131

```
struct RecursiveFilterState {
    double input_coefficient;
    double previous_input_coefficient;
    double previous_output_coefficient;
    double input;
    double previous_input;
    double output;
    double previous_output;
};

// 残差滤波器：先对输入做低通滤波，再用“输入 - 低通结果”得到高频残差。
// 常用于提取谐波/纹波分量，然后反相注入以抑制谐波。
// 两通道（channel 0/1）分别对应 d 轴和 q 轴。
struct ResidualFilterState {
    double input[2];
    double lowpass[2];
    double residual[2];
};
```

头文件的注释为解释，不代替实现证据。

4.2.5 PLL对象与两侧缓存参数

PllState保存三路线电压、angular_speed、angle、正负序的decoupled/filtered/raw各二维量，以及voltage_magnitude、negative_magnitude和feedforward。尾部还包含step、LineVoltageState、ParkState、16个保留double与完整RegulatorState。

主控制器外部测得的正序幅值、PLL原始输入矢量模、低通dq幅值属于不同字段，不能仅因单位同为V互相替代。两侧PLL的输入缓存增益由参数入口刷新，活动调节器的跟踪系数是否同步改变必须查各calculate调用方，不能从该结构声明推导。

recovered_primitives.h L139-L157

```
struct PllState {
    double line_voltage[3];
    double angular_speed;
    double angle;
    double positive_decoupled[2];
    double negative_decoupled[2];
    double positive_filtered[2];
    double negative_filtered[2];
    double positive_raw[2];
    double negative_raw[2];
    double voltage_magnitude;
    double negative_magnitude;
    double feedforward[2];
    double step;
    LineVoltageState stationary;
    ParkState rotating;
    double reserved[16];
    RegulatorState regulator;
};
```

声明本身不保证任何一次运行的锁相误差或稳定性。

4.2.6 调制对象和整数尾字段

ModulationState依次保存alpha/beta、三相phase、最大最小和中间量、保留区、modulation、正负母线dc_plus/dc_minus、delta、中点correction、三相两路duty和gain/limit/deadband。保留数组参与内存跨度，占位字段不能删除以压缩状态。

尾部polarity为int32而reserved_tail为uint32，与前面的double不同。duty[3][2]只是调制对象的内部六路占空比；输出接口中的平均相电压、占空比、调制波三种模式由独立output函数选取，不能将duty字段称为所有模式统一输出。

recovered_primitives.h L165-L187

```
struct ModulationState {
    double alpha;
    double beta;
    double phase[3];
    double phase_max;
    double phase_min;
    double middle;
    double reserved_040[9];
    double modulation[3];
    double reserved_0A0[5];
    double dc_plus;
    double dc_minus;
    double delta;
    double reserved_0E0;
    double correction;
    double duty[3][2];
    double gain;
    double limit;
    double deadband;
    double reserved_138[14];
    std::int32_t polarity;
    std::uint32_t reserved_tail;
};
```

备用调制通过void*操作固定偏移，行为应查alternate_modulation。

4.2.7 算法导出声明与主调度导出的层次

头文件从第199行至234行共声明34个算法导出函数：10个结构算法regulator至modulation，随后为alternate_modulation、motor_calculate、grid_input、rotor_input、grid_output、rotor_output、motor_defaults、motor_post、grid_defaults、controller_initialize、average_converter_output、view_output、rotor_nameplate、grid_post、command_input、protection_input、parameter_input、grid_frequency_support、grid_calculate、adaptive_oscillator、current_sequence、rotor_current_sequence、resonant_filter和difference_filter。这些声明没有给出数组的运行时长度检查。

recovered_controller.cpp另外定义init、主入口和end三个导出，因此“头文件34项”与“八份实现中全部导出37项”是两个统计口径。parameter_input的头文件契约明确指针为public90+10、80个double可读，并要求状态跨度0x3748；频率支撑使用网侧区0x1AB8，环形缓冲长度1～5且索引应在有效范围。

$$p[n] = parameters[n + 10], \quad 0 \le n \le 79 \tag{4-4}$$

recovered_primitives.h L209-L232

```
extern "C" __declspec(dllexport) void alternate_modulation(void* state);
extern "C" __declspec(dllexport) void motor_calculate(double base_frequency, void* state);
extern "C" __declspec(dllexport) void grid_input(const double* input, void* state);
extern "C" __declspec(dllexport) void rotor_input(const double* input, double base_frequency, void*
state);
extern "C" __declspec(dllexport) void grid_output(double* output, const void* state);
extern "C" __declspec(dllexport) void rotor_output(double* output, const void* state);
extern "C" __declspec(dllexport) void motor_defaults(double base_frequency, void* state);
extern "C" __declspec(dllexport) void motor_post(double base_frequency, void* state);
extern "C" __declspec(dllexport) void grid_defaults(double base_frequency, void* state);
extern "C" __declspec(dllexport) void controller_initialize(double base_frequency, void* state);
extern "C" __declspec(dllexport) void average_converter_output(void* state);
extern "C" __declspec(dllexport) void view_output(double* output, const void* state);
extern "C" __declspec(dllexport) void rotor_nameplate(const double* parameters, void* state);
extern "C" __declspec(dllexport) void grid_post(double base_frequency, void* state);
extern "C" __declspec(dllexport) void command_input(const double* commands, void* state);
extern "C" __declspec(dllexport) void protection_input(const double* input, double base_frequency,
void* state);
// Input is public90 + 10 (80 readable doubles); state spans 0x3748 bytes.
extern "C" __declspec(dllexport) void parameter_input(const double* input, double base_frequency,
void* state);
// State spans 0x1AB8 bytes; ring lengths are 1..5, with indices in [0, length).
extern "C" __declspec(dllexport) void grid_frequency_support(double base_frequency, void* state);
extern "C" __declspec(dllexport) void grid_calculate(double base_frequency, void* state);
extern "C" __declspec(dllexport) void adaptive_oscillator(void* state, double base_frequency);
extern "C" __declspec(dllexport) void current_sequence(void* state);
extern "C" __declspec(dllexport) void rotor_current_sequence(void* state);
```

recovered_primitives.h L233-L234

```
extern "C" __declspec(dllexport) void resonant_filter(void* state);
extern "C" __declspec(dllexport) void difference_filter(void* state);
```

此契约不证明外部包装已经正确分配数组。

extern C避免C++改名；具体宿主ABI还需编译产物与包装验证。

4.2.8 实例、初始化与结束入口

instances按int32实例号保存独立ControllerState，map节点让已存实例的状态地址稳定。init以parameters[65]为额定频率调用整体初始化；end通过operator[]取得实例后只将字节清零，既不删除map元素，也不调用两侧默认值。

run_number以x86 CVTT把grid[9]的double转为int32，采用向零截断。与数学四舍五入不同，异常转换保留硬件INT_MIN语义。直接调用主入口而没有先init仍会由map新建全零状态，但这不是正确默认值；调用者应遵守生命周期。

recovered_controller.cpp L68-L91

```
// The original array capacity is unknown; storage remains stable per instance.
// instances: 按“实例号”管理多份状态。
// 一台仿真里可能放多个控制器（多个机组），每个实例号对应一块独立状态区。
// 原 DLL 内部同样是按实例号分开存储的，这里用 map 等价实现。
std::map<std::int32_t, ControllerState> instances;

std::int32_t run_number(double value) {
    return _mm_cvttsd_si32(_mm_set_sd(value));
}

} // namespace

// ---------------------------------------------------------------------------
// init: 初始化入口。PSCAD 传参数数组（第 66 个是额定频率）和实例号，
// 我们取出该实例的状态区，调用 controller_initialize 填默认值。
// ---------------------------------------------------------------------------
extern "C" __declspec(dllexport) void dfig_controller_sy_wg4000kdf_npc_init(
    const double* parameters, const std::int32_t* instance) {
    ControllerState& s = instances[*instance];
    controller_initialize(parameters[65], s.at());
}

extern "C" __declspec(dllexport) void dfig_controller_sy_wg4000kdf_npc_end(
    const std::int32_t* instance) {
```

recovered_controller.cpp L92-L93

```
    instances[*instance].clear();
}
```

实例状态分离，snapshot.txt文件名仍为共享相对路径。

map和相同实例的写入没有源码级并发保护。

4.2.9 运行号变化与快照恢复先于时间门控

主入口先将grid[9]转换成当前运行号写0x3730，与0x3734比较。改变时先清快照恢复标志0x3740；仅当motor[9]不等于1000时整体初始化，并重新写运行号。随后把当前号保存到0x3734。

若恢复标志为0且motor[9]精确等于1000，先读snapshot.txt恢复完整字节状态，再置标志1并用本次grid[9]覆写新旧运行号。这一流程在累计时间闸门之前，故非控制拍也可能恢复或重新初始化。读取文件失败会abort；第39～49行没有校验fread/fwrite返回量，源码不能保证截短或损坏快照安全恢复。

recovered_controller.cpp L109-L124

```
    const double frequency = parameters[65];
    s.i(0x3730, run_number(grid[9]));
    if (s.i(0x3734) != s.i(0x3730)) {
        s.i(0x3740, 0);
        if (motor[9] != 1000.0) {
            controller_initialize(frequency, s.at());
            s.i(0x3730, run_number(grid[9]));
        }
    }
    s.i(0x3734, s.i(0x3730));
    if (s.i(0x3740) == 0 && motor[9] == 1000.0) {
        s.snapshot(true);
        s.i(0x3740, 1);
        s.i(0x3730, run_number(grid[9]));
        s.i(0x3734, s.i(0x3730));
    }
```

恢复标志只阻止连续相同恢复请求重复读文件。

文件名为工作目录中的snapshot.txt；无实例后缀。

4.2.10 输入装载、跨区共享与容量基准时序

只有double累计时间0x3728严格等于0时进入控制序列。先网侧输入、机侧输入、铭牌、局部80槽参数、28项保护、11槽命令；parameter_input得到的是parameters+10。随后把网侧线电压及正负母线电压复制到机侧可见区。

0x538的容量基准在参数换算完成之后才由机侧全局0x2050刷新。因此宿主本拍修改铭牌容量，机侧nameplate先得到新值，而参数h/f/r换算该拍仍读取进入参数函数时的旧0x538；下拍才使用新容量。0xBD0复制机侧局部0x1C标志，0x7C0保存本拍原始有功给定，均是后续频率支撑使用的共享状态。

recovered_controller.cpp L130-L153

```
    if (s.d(0x3728) == 0.0) {
// 一个控制周期内的完整调用序列（顺序即原 DLL 的顺序）：
//   1) grid_input / rotor_input      —— 把 PSCAD 输入搬进状态区；
//   2) rotor_nameplate / parameter_input / protection_input / command_input
//                                    —— 铭牌、参数、保护定值、命令；
//   3) grid_calculate / motor_calculate —— 两侧主控制算法；
//   4) grid_post / motor_post        —— 两侧状态机与保护；
//   5) average_converter_output      —— 直流母线电流折算；
//   6) grid_output / rotor_output / view_output —— 打包输出给 PSCAD。
// 中间还有频率支撑（0x1E08 使能时）和 0x1D58 += 0x768（调频功率叠加到有功给定）。
        void* rotor = s.at(0x1AB8);
        grid_input(grid, s.at());
        rotor_input(motor, frequency, rotor);
        rotor_nameplate(parameters, rotor);
        parameter_input(parameters + 10, frequency, s.at());
        protection_input(protection, frequency, s.at());
        command_input(commands, s.at());
        s.d(0x1D10, s.d(0xC0));
        s.d(0x1D18, s.d(0xC8));
        s.d(0x1D20, s.d(0xD0));
        s.d(0x1D38, s.d(0x100));
        s.d(0x1D40, s.d(0x108));
        s.d(0x538, s.d(0x2050));
        s.i(0xBD0, s.i(0x1AD4));
```

recovered_controller.cpp L154-L154

```
        s.d(0x7C0, s.d(0x1D58));
```

rotor_input在nameplate之前执行，铭牌依赖输入还需区分更新前后。

0x1AC8=机侧0x10，0x1AD8=机侧0x20，属于共享字节而非互不相干的变量。

4.2.11 自动复归计数、两侧计算与输出次序

0x3738必须为1且机侧故障字0x1FB8或网侧故障字0x354非零，0x373C才每有效控制拍自增；不满足即清零。达到8000后写两侧复位请求0x520、0x1FF4。8000拍等于2s只在每拍间隔0.00025s时成立，计数属于控制拍而非10μs仿真步。

顺序为grid_calculate、条件调用grid_frequency_support、无条件把0x768加到0x1D58、将网侧频率0x468复制到机侧0x25E0、motor_calculate、grid_post、motor_post、跳闸互传、average_converter_output，最后三路输出打包。后处理发生在本拍calculate之后，跳闸互传也晚于两侧post；互传标志通常到下一次计算/状态判定才被消费。频率支撑未执行不等于0x768被此处主动清零。

$$P^{*}_{motor,in} = P^{*}_{command} + \Delta P_{support} \tag{4-5}$$

$$T_{reset} = 8000\,T_{control} \tag{4-6}$$

recovered_controller.cpp L157-L180

```
        if (s.i(0x3738) == 1 && (s.i(0x1FB8) != 0 || s.i(0x354) != 0)) {
            s.increment(0x373C);
            if (s.i(0x373C) >= 8000) {
                s.i(0x520, 1);
                s.i(0x1FF4, 1);
            }
        } else {
            s.i(0x373C, 0);
        }
        grid_calculate(frequency, s.at());
        if (s.d(0x1E08) == 1.0) {
            grid_frequency_support(frequency, s.at());
        }
        s.d(0x1D58, s.d(0x1D58) + s.d(0x768));
        s.d(0x25E0, s.d(0x468));
        motor_calculate(frequency, rotor);
        grid_post(frequency, s.at());
        motor_post(frequency, rotor);
        s.i(0x34C, s.i(0x1FB4) == 3 ? 1 : 0);
        s.i(0x1FB0, s.i(0x350) == 3 ? 1 : 0);
        average_converter_output(s.at());
        grid_output(grid_result, s.at());
        rotor_output(motor_result, rotor);
        view_output(view_result, s.at());
```

需分别核查0x3738和两侧post中的自动复位门控。

输入输出只在有效控制拍重新打包；其他步保持宿主数组已有值。

4.2.12 时间累计与快照保存边界

无论本次是否执行控制，末尾都把delta加到0x3728；达到sample_period后直接置0，下一次主入口才执行控制。超出的时间余量被丢弃，没有减sample_period的余量累积，也没有一次调用补算多拍的循环。典型10μs/250μs是外部输入选择，源码不把二者固定在此闸门。

快照保存的条件是motor[8]与grid[8]相等且motor[9]不等于1000；它比较元数据槽8，而非两个运行号9。条件成立就清恢复标志并写整个状态，即使此次没有进入控制闸门也会保存。故“非控制拍什么也不做”只适用于控制算法与输出重新打包，不能用于整个主入口。

$$t \leftarrow t + \Delta t;\quad t \geq T_{sample} \Rightarrow t \leftarrow 0 \tag{4-7}$$

recovered_controller.cpp L183-L194

```
// 采样计时器推进：累计本步时间 delta，到满一个采样周期就清零，
// 下一个 250μs 窗口再执行一次控制。
// 末尾：当两侧“运行号”相等且机侧不处于复位过程时，把状态区存成快照。
    s.d(0x3728, s.d(0x3728) + *delta);
    if (s.d(0x3728) >= *sample_period) {
        s.d(0x3728, 0.0);
    }
    if (motor[8] == grid[8] && motor[9] != 1000.0) {
        s.i(0x3740, 0);
        s.snapshot(false);
    }
}
```

若delta和sample_period组合改变，实际执行间隔需要按该离散闸门计算。

第5章　参数、命令与数值换算

5.1 面板、参数槽与活动状态的关系

parameters[0..9]为铭牌、辅助和保留区；p[0..79]指parameters+10。包装表复用Control15到p[15]、Control19到p[19]；未单列Control16/20不表示对应槽为零。p[45..54]、p[68..70]及部分尾槽保留补零；p[79]由外部P_pos动态写入。Control76=0存在面板，但不进入当前控制打包表。

表 14　初始化、参数拍与活动调节器增益

环节	初始化 **kp / ki**	当前参数 **kp / ki**	**tracking** 与限幅
两侧PLL	0.12 / 0.0012	0.08 / 0.0001	tracking=0.01保留初始化；机侧LVRT后的参数拍ki÷10。GSC正常/LVRT/HVRT限幅±15.708/±7.854/±62.832 rad/s；RSC LVRT±7.854、其他±62.832
网侧电压外环	4 / 0.15	5 / 0.15	tracking=0.03；电流上限795.375A，下限随负有功斜坡变化
网侧正、负序电流环	见初始化源码	1200 / 20	tracking≈0.0166667；输出±0.577×2^23内部单位
机侧正序电流环	见初始化源码	3500 / 15	tracking≈0.00428571；缓存到活动PI的复制受ready门控
机侧负序电流环	见初始化源码	4500 / 25	tracking≈0.00555556；缓存到活动PI的复制受ready门控

参数入口在算法之前执行；机侧LVRT模式影响下一参数拍的PLL积分增益。机侧电流环的每拍缓存值不等于活动PI本拍立即改变，ready阶段才复制。文中的“未读取”限定到当前恢复代码内部；宿主初始化、外部改写、快照恢复和模式改变须单独核验。

控制实现与逐拍关系

三个参数换算公式

控制参数p[]=外层90路parameters+10，与铭牌parameters[0..4]是不同索引段。换算保留1.414/1.732。1140为源码硬编码电压，与本工程定子额定线电压一致；下式a=d(0x110)、b=d(0x1B00)、c=d(0x2F58)。

$$h(n) = p_n\left((d_{0x538} / d_{0x270}) / 1.732\right)1.414 \tag{5-1}$$

$$f(n) = 1.414h(n) \tag{5-2}$$

$$r(n)=\frac{p_n((d_{0x538}\ /\ 1140)\ /\ 1.732)}{a\ /\ 1.414+(b\ /\ 1.414)c} \tag{5-3}$$

extra为真时，分母第二项(b/1.414)c再乘0.75；其余不变，各除数须非零。

PI 参数分布

工程kp/ki：PLL0.08/0.0001、网侧外环5/0.15、网侧正负序电流环1200/20、机侧正序3500/15与负序4500/25。ki_per_step（基址+0x60）保存ki，tracking_gain（+0x68）保存ki/kp。0x1AC8是机侧局部0x10的全局别名，motor_calculate实际写入穿越模式1/2/0；模式1期间参数入口把两侧PLL积分增益写成p[1]/10，进入或退出模式在下一参数拍反映。

LVRT/HVRT 门槛换算

p[12]/p[13] = 低压穿越门槛 0.9 / 滞回 0.05；p[16]/p[17] = 高压穿越门槛 1.13 / 滞回 0.05；p[14] = LVRT 无功系数（网侧 3.0）、p[18] = HVRT 无功系数（网侧 2.2841）。门槛换算：

$$V_{\mathrm{LV,in}}=d_{0x270}p_{12}\ /\ 1.732 \tag{5-4}$$

$$V_{\mathrm{LV,out}}=d_{0x270}(p_{12}+p_{13})\ /\ 1.732 \tag{5-5}$$

$$V_{\mathrm{HV,in}}=d_{0x270}p_{16}\ /\ 1.732 \tag{5-6}$$

$$V_{\mathrm{HV,out}}=d_{0x270}(p_{16}-p_{17})\ /\ 1.732 \tag{5-7}$$

d(0x270)=1611.96V为线电压幅值基准。当前参数LVRT进/退约837.62/884.16V，HVRT进/退約1051.68/1005.15V，分别0.9/0.95/1.13/1.08pu；固定1.02×930.69的另一条件单独列出。机侧参数写入定值与实际内部固定门槛须区分。

电流限幅换算

p20…25经f(n)=1.414h(n)换算有名值电流限幅。√2仅为理论峰值解释，执行式保留1.414。

保护定值换算

28 路保护定值：前 16 路电压类——偶数下标（阈值）乘电压基准 V_base/√2 有名值化，奇数下标（时间）直接存入；后 12 路频率类直接存入。外部28项电压、频率和时间定值全部由input[0..27]提供；函数没有缺项默认或异常回退机制。内部Setting表另行写入其他固定保护参数，不能与这28项外部输入混称。

无功给定三种模式（命令入口）

commands[8] 选择无功控制模式（用上一拍的模式值执行）：

模式0把01及异常浮点输入不受此钳位。有限非零输入按S=P/pf计算Q，负根号项归零，pf<0时取负：

$$S=\frac{P}{pf} \tag{5-8}$$

$$Q = \text{sign}(pf)\sqrt{S^2 - P^2} \tag{5-9}$$

模式 2（直接给定 + LVRT 无功增发）：运行标志0x1AD8为1且电压偏差（参考电压−机端电压标幺）达到或越过正负死区0x36A0时，按系数 0x3698 比例增发：

$$Q = Q^* + k_\Delta \Delta V \tag{5-10}$$

模式 1（直接给定）：限幅 ±0.33：

$$Q = \text{clip}\left(Q^*, -0.33, 0.33\right) \tag{5-11}$$

铭牌参数换算

铭牌parameters[0]原值存功率槽（本工程4000000W），parameters[1]/[2]为定子/转子电压kV，×1000存V；parameters[3]独立B，parameters[4]为极对数。电压比Vr/Vs，非电流/电压比。

$$k_{\psi r} = V_r \,/\, V_s \tag{5-12}$$

5.2 当前关键换算与穿越需求区间

表 15　当前值与物理含义速查

量	换算 / 当前结果	性质
h类电流换算	h(n)=p[n]×2025.849890… A	使用1611.96、1.732、1.414有限常数
f类峰值换算	f(n)=1.414 h(n)=p[n]×2864.551745… A	参数入口实际有名值
网侧LVRT / HVRT系数	2.315876 / 2.223863	r(14) / re(18)，不是面板3 / 2.2841原值
机侧HVRT系数	(re(19)×3)/4≈1.6678969	p19复用Control19；不会因缺Control20名称变0
LVRT进入 / 退出	837.624 / 884.158 V	0.9 / 0.95 ×1611.96/1.732
HVRT进入 / 参数退出	1051.683 / 1005.148 V	1.13 / 1.08 ×1611.96/1.732
HVRT兜底退出	949.31 V	1.02×930.6928，另一路比较
网侧LVRT需求 / q天花板	负值分支9.5445A、上限745.680A / q天花板≤636.216A	区分原始需求与最终给定
机侧LVRT需求	负值分支7.9184A，上限976.2397A	0.008 / 0.9863 ×989.8A
网侧HVRT参数需求区间	[-424.240,-53.567]A	f(37) / f(36)
机侧HVRT参数需求区间	[-424.240,-40.104]A	f(39) / f(38)
机侧LVRT If*钳位	约[-1145.821,-211.977]A	下界-f(22)，上界-h(44)×1.414
机侧LVRT余量上限	1088.816A	Control44=0.3801经f(43)，707A仅初始化

$$V_{LV,\text{in}} = \frac{1611.96}{1.732}p[12], \quad V_{LV,\text{out}} = \frac{1611.96}{1.732}(p[12] + p[13]) \tag{5-13}$$

$$V_{HV,\text{in}} = \frac{1611.96}{1.732}p[16], \quad V_{HV,\text{out}} = \frac{1611.96}{1.732}(p[16] - p[17]) \tag{5-14}$$

命令与Q三模式

表 16　命令与Q三模式：参数与状态关系

槽	面板名（工程值）	写入状态区	作用
[0]	Cmd_1 Start_Cmd（1）	0x348 / 0x1FAC	启动命令：置 1 使网侧/机侧状态机从停机转运行
[1]	P*（接口可变，工程保存常数 1.0）	0x1D58（=机侧 0x2A0）	有功给定（标幺，以 4 MVA 为基）；机侧每拍将其 ×1.045 后作为转矩参考（0x6E8 = commands[9] = 0；0x6E8 = 1 时改 ×1.04）
[2]	Q*（接口可变，工程保存常数 0.33）	0x1D60（=机侧 0x2A8）	无功相关给定（经模式换算，见下）；机侧每拍 +0.015 后作 为励磁参考
[3]	Cmd_2 LVRT_Enable（1）	0x518 / 0x1FEC	低压穿越功能总开关
[4]	Cmd_3 HVRT_Enable（1）	0x51C / 0x1FF0	高压穿越功能总开关
[5]	Cmd_4 Reset_Cmd（0）	0x520 / 0x1FF4（边沿触发）	复位：清空状态区并重写默认值（0→1 跳变时生效）
[6]	Cmd_5 Flag_LVRT_Restore（1）	0x2088	LVRT 恢复标志
[7]	保护字（工程保存开关值组合 0）	0x7D8 / 0x23D0	测试接口保护字示例 0x4000CC00；当前工程保护开关保存值对应0，见第14章
[8]	Cmd_6 Q_ctrol_mode（1）	0x21A4	无功控制模式 0/1/2（本拍用上一拍的模式值，原序保留）
[9]	Cmd_7 Flag_AVG_IGBT（0）	0x628 / 0x21A0	输出模式0平均模型、1占空比、2调制波；模式1亦选择机侧备用调制
[10]	Cmd_8 Flag_Interia_Freq（0）	0x740	频率支撑：0=关闭、1=惯量响应、2=一次调频（工程配置为关闭）

PI完整12项

表 17　PI完整12项：参数与状态关系

参数	p 下标	面板值	换算	运行时值	控制什么
Control_1 PLL_kp	p[0]	0.08	直接	0.08	网侧锁相环比例系数（0x550）
Control_2 PLL_ki	p[1]	0.0001	直接	0.0001	网侧锁相环积分系数（0x560）
Control_3 电压外环 kp	p[2]	5.0	直接	5.0	网侧直流电压外环 kp（0x1120）
Control_4 电压外环 ki	p[3]	0.15	直接	0.15	网侧直流电压外环 ki（0x1168）；ki/kp = 0.03（跟踪 增益）
Control_5 网侧正序电 流 kp	p[4]	1200.0	直接	1200.0	网侧正序 d 轴电流环 kp（0xF20）
Control_6 网侧正序电 流 ki	p[5]	20.0	直接	20.0	网侧正序 d 轴电流环 ki（0xF68）；ki/kp = 0.016667
Control_7 网侧负序电 流 kp	p[6]	1200.0	直接	1200.0	网侧负序电流环 kp（0x1020/0x10A0）
Control_8 网侧负序电 流 ki	p[7]	20.0	直接	20.0	网侧负序电流环 ki（0x1068/0x10E8）；ki/kp = 0.016667
Control_9 机侧正序电 流 kp	p[8]	3500.0	直接	3500.0	机侧正序电流环 kp（0x1CC0）
Control_10 机侧正序电 流 ki	p[9]	15.0	直接	15.0	机侧正序电流环 ki（0x1CC8）；ki/kp = 0.004286
Control_11 机侧负序电 流 kp	p[10]	4500.0	直接	4500.0	机侧负序电流环 kp（0x1CD0）

参数	p 下标	面板值	换算	运行时值	控制什么
Control_12 机侧负序电 流 ki	p[11]	25.0	直接	25.0	机侧负序电流环 ki（0x1CD8）；ki/kp = 0.005556

穿越和六项电流限幅

表 18　穿越和六项电流限幅：参数与状态关系

参数	p 下标	面板值	换算	运行时值	控制什么
Control_13 LVRT 门槛	p[12]	0.9	×930.6928	837.624 V	正序电压低于 837.624 V 进入低压穿越（两侧相同）
Control_14 LVRT 滞回	p[13]	0.05	×930.6928	46.535 V	退出门槛 = 0.9+0.05 = 0.95 pu = 884.158 V（防 抖动）
Control_15 LVRT 无功系数 G	p[14]	3.0	r(14)	2.315876	网侧 0x278 与机侧 0x418（同一数值，两侧共用）
Control_17 HVRT 门槛	p[16]	1.13	×930.6928	1051.683 V	正序电压高于 1051.683 V 进入高压穿越
Control_18 HVRT 滞回	p[17]	0.05	×930.6928	46.535 V	退出门槛 = 1.13−0.05 = 1.08 pu = 1005.148 V
Control_19 HVRT 无功系数 G	p[18]	2.2841	re(18)	p[19]=Control19= 2.2841；(r_e(19)×3)/4	网侧 r_e(18)=2.223863；机侧p[19]复用C19，(r_e(19)×3)/4≈1.6678969

表 19　穿越和六项电流限幅：参数与状态关系

参数	p 下标	面板值	运行时值	控制什么
Control_21 网侧有功电流 限幅	p[20]	0.2777	795.486 A	网侧有功电流上限（0x140）
Control_22 网侧无功电流 限幅	p[21]	0.1111	318.252 A	网侧无功电流上限（0x120）

参数	p 下标	面板值	运行时值	控制什么
Control_23 机侧有功电流 上限	p[22]	0.4	1145.821 A	机侧有功电流上限（0x1B08）
Control_24 机侧有功电流 下限	p[23]	0.0	0 A	机侧有功电流下限（0x1B10）
Control_25 机侧无功电流 上限	p[24]	0.2895	829.288 A	机侧无功电流上限（0x1B18）
Control_26 机侧无功电流 下限	p[25]	−0.2895	−829.288 A	机侧无功电流下限（0x1B20）

Control27~45

表 20　Control27~45：参数与状态关系

参数	p 下标	面板值	换算	运行时值	控制什么
Control_27 LVRT 无功计算电压	p[26]	1.0	直接	1.0	LVRT 无功公式的参考电压（网侧 0x638 / 机侧 0x6F0）：ΔV = 1.0 − V⁺/1611.96
Control_28 LVRT 无功初始系数	p[27]	1.0	直接	1.0	1.0参与机侧Ip*中的励磁叠加；不是没有读者
Control_29 LVRT 负序系数	p[28]	1.0	直接	1.0	网侧 0x5E8 / 机侧 0x630（负序回路未使 能，未取用）
Control_30 网侧 LVRT 无功最 大	p[29]	0.2221	f(29)	636.217 A	网侧 LVRT 无功电流封顶（0x288）
Control_31 网侧不平衡无功最 大	p[30]	0.2221	f(30)	636.217 A	网侧不平衡工况无功封顶（0x2A0）
Control_32 机侧 LVRT 无功最 大	p[31]	0.3456	f(31)	989.989 A	机侧 LVRT 无功电流封顶（0x428）
Control_33 机侧不平衡无功最 大	p[32]	0.3456	f(32)	989.989 A	机侧不平衡工况封顶（0x450）

参数	p 下标	面板值	换算	运行时值	控制什么
Control_34 HVRT 无功计算电压	p[33]	1.0	直接	1.0	HVRT 公式参考电压（网侧 0x640 / 机侧 0x6F8）：ΔV = V⁺/1611.96 − 1.0
Control_35 HVRT 初始系数	p[34]	1.0	直接	1.0	1.0参与HVRT电流合成的励磁叠加，需按门控判断
Control_36 HVRT 负序系数	p[35]	0.0	直接	0.0	网侧 0x5F0 / 机侧 0x638（负序回路未使能）
Control_37 网侧 HVRT 无功最大	p[36]	−0.0187	f(36)	−53.567 A	网侧 HVRT 吸收无功上限（0x298，最负 −424.240 到 −53.567）
Control_38 网侧 HVRT 无功最小	p[37]	−0.1481	f(37)	−424.240 A	网侧 HVRT 吸收无功下限（0x290）
Control_39 机侧 HVRT 无功最大	p[38]	−0.014	f(38)	−40.104 A	机侧 HVRT 吸收无功上限（0x438）
Control_40 机侧 HVRT 无功最小	p[39]	−0.1481	f(39)	−424.240 A	机侧 HVRT 吸收无功下限（0x430）
Control_41 LVRT 有功控制模式	p[40]	1.0	直接	1.0	写入 0x21C8 后被 Control_73 覆盖；当前模式未取用
Control_42 LVRT 有功控制系数	p[41]	1.0	直接	1.0	机侧 0x518（当前控制路径未取用）
Control_43 HVRT 有功功率系数	p[42]	0.0	直接	0.0	机侧 0x520（当前控制路径未取用）
Control_44 转子 d 轴限幅上限	p[43]	0.3801	f(43)	1088.816 A	机侧 LVRT 转子 d 轴电流上限（0x458，覆盖默认值 707）

参数	p 下标	面板值	换算	运行时值	控制什么
Control_45 LVRT 最小 d 轴电流	p[44]	0.074	h(44)	149.913 A	h(44)=149.913A，再 ×1.414≈211.977A，参与If*上界

Control56~75与缺号

表 21 Control56~75与缺号：参数与状态关系

参数	p 下标	面板值	写入位置	含义
Control_56 Rate_Freq	p[55]	50.0	基准频率	额定频率 50 Hz；初始化与主入口都使用（parameters[65]）
Control_57 惯量响应时间	p[56]	10.0	0xBC8	惯量支撑维持 10 s
Control_58 惯量死区	p[57]	0.3	0xC38	df/dt 与频偏死区 0.3 Hz
Control_59 惯量时间常数 Tj	p[58]	0.1	0x720	惯量支撑功率幅值 0.1 pu
Control_60 一次调频响应时间	p[59]	5.0	0x7B0	一次调频最长持续 5 s
Control_61 功频下垂系数	p[60]	0.06	0xC30	调差率 6%
Control_62 一次调频死区	p[61]	0.03	0x738	频偏死区 ±0.03 Hz（外加 0.005 Hz 裕量）
Control_63 调频功率上限	p[62]	0.2	0xC40	×1.05×P* 为上限
Control_64 调频功率下限	p[63]	0.1	0xC48	×1.1×P* 为下限门槛

表 22 Control56~75与缺号：参数与状态关系

参数	p 下标	面板值	换算	运行时值	控制什么
Control_65 恢复斜坡起点	p[64]	0.7	直接	0.7	特殊恢复起点，按模式与门控生效
Control_66 恢复斜坡时间	p[65]	0.5	直接	0.5 s	0.5s恢复参数，并参与特殊恢复超时

参数	p 下标	面板值	换算	运行时值	控制什么
Control_67 网侧 LVRT 无功最小	p[66]	0.015	直接（标幺）	0.015	网侧 LVRT 无功系数下限（0xC58）→ 有名值 9.5445 A
Control_68 机侧 LVRT 无功最小	p[67]	0.008	直接（标幺）	0.008	机侧 LVRT 无功系数下限（0xB68）→ 有名值 7.918 A
Control_72 PWM 200ms 断路器	p[71]	0.0	直接	0.0	0xC98；置 1 时命令入口改为锁存 P*/Q*（本工程为 0，不激活）
Control_73 LVRT d 轴模式	p[72]	2.0	直接	2	当前2：常规励磁公式分支；保持寄存器属于模式1
Control_74 LVRT d 轴限幅	p[73]	0.7	直接（标幺）	0.7	机侧 0x1C30；仅模式 1 时生效（本工程模式 2，未取用）
Control_75 恢复斜坡模式	p[74]	3.0	直接	3	当前3：特殊恢复激活后选择 P_pos均值参考

Protect1~28

表 23　Protect1~28：参数与状态关系

段名	面板参数	面板值	运行时定值	动作逻辑（详见保护专题）
过压快速段	Protect_1 / Protect_2	1.33 / 0.15 s	1516.2 V / 0.15 s	任一相线电压有效值超 1516.2 V，累计 0.15 s → 跳闸（位 10）
过压 1 段	Protect_3 / Protect_4	1.28 / 0.6 s	1459.2 V / 0.6 s	超 1459.2 V 累计 0.6 s → 跳闸（位 10）
过压 2 段	Protect_5 / Protect_6	1.23 / 1.1 s	1402.2 V / 1.1 s	超 1402.2 V 累计 1.1 s → 跳闸（位 10）

段名	面板参数	面板值	运行时定值	动作逻辑（详见保护专题）
过压 3 段	Protect_9 / Protect_10	1.13 / 10.4 s	1288.2 V / 10.4 s	超 1288.2 V 累计 10.4 s → 跳闸（位 10，最慢、最不灵敏的一段）
欠压 1 段	Protect_7 / Protect_8	0.12 / 0.3 s	136.8 V / 0.3 s	低于 136.8 V 累计 0.3 s → 跳闸（位 11，对应深度跌落）
欠压 2 段	Protect_11 / Protect_12	0.4 / 1.4 s	456.0 V / 1.4 s	低于 456.0 V 累计 1.4 s → 跳闸（位 11）
欠压 3 段	Protect_13 / Protect_14	0.6 / 2.2 s	684.0 V / 2.2 s	低于 684.0 V 累计 2.2 s → 跳闸（位 11）
欠压 4 段	Protect_15 / Protect_16	0.83 / 3.0 s	946.2 V / 3.0 s	低于 946.2 V 累计 3.0 s → 跳闸（位 11，最灵敏）
过频 1 段	Protect_17 / Protect_18	54 Hz / 0.2 s	54 Hz / 0.2 s	实测频率超 54 Hz 累计 0.2 s → 跳闸（位 14）
过频 2 段	Protect_19 / Protect_20	53 Hz / 10 s	53 Hz / 10 s	超 53 Hz 累计 10 s → 跳闸（位 14）
过频 3 段	Protect_21 / Protect_22	52 Hz / 2100 s	52 Hz / 2100 s	超 52 Hz 累计 2100 s（35 分钟）→ 跳闸（位 14，实际几乎不动作）
欠频 1 段	Protect_23 / Protect_24	46 Hz / 0.2 s	46 Hz / 0.2 s	低于 46 Hz 累计 0.2 s → 跳闸（位 15）
欠频 2 段	Protect_25 / Protect_26	47 Hz / 10 s	47 Hz / 10 s	低于 47 Hz 累计 10 s → 跳闸（位 15）
欠频 3 段	Protect_27 / Protect_28	47.5 Hz / 2100 s	47.5 Hz / 2100 s	低于 47.5 Hz 累计 2100 s → 跳闸（位 15，实际几乎不动作）

全部运行时数值

表 24　全部运行时数值：参数与状态关系

工况	面板参数及换算	运行时值	存入位置
LVRT 无功系数（两侧共用）	Control_15 = 3.0 → r(14)	2.315876	网侧 0x278、机侧 0x418（同一数 值）
HVRT 无功系数（网侧）	Control_19 = 2.2841 → re(18)	2.223863	网侧 0x280
HVRT 无功系数（机侧）	(r_e(19)×3)/4，p[19]=2.2841（别名）	p[19]=Control19=2.2841； (r_e(19)×3)/4	现行K_R,HV≈1.6678969；需求随滤波抬升变化，参数钳位 [-424.240,-40.104]A

表 25　全部运行时数值：参数与状态关系

对象	运行时范围	来源（面板参数及换算）
网侧 LVRT 无功电流 Iq*	负值置9.5445A；上限745.680A，非负小值按源码保留	下限 Control_67 = 0.015×636.3；上限原 DLL 常数 1.1719×636.3
网侧 LVRT 无功电流上限 Iq*	≤ 636.216 A	由有功给定 Id*（0x58）推无功余量： $\sqrt{(2\times636.3^2 - Id^{*2})}$（0x2C0）， 再按 Control_30 换算值 636.216 A（0x288）截顶，供 q 轴给定合成
网侧 HVRT 无功电流 Iq*	[−424.240, −53.567] A	Control_37 / Control_38 经 f() 换算（负号 = 吸收无功）
机侧 LVRT 励磁电流 Id*	负值置7.918A；上限976.240A，非负小值按源码保留	下限 Control_68 = 0.008×989.8；上限原 DLL 常数 0.9863×989.8
机侧 LVRT 无功电流上限 Iq*	[212.1, 1088.816] A	$\sqrt{(1.21 - Idpu^2)}$×989.8；上限取 0x458 = Control_44 经 f(43) = 1088.816 A（707 A 只是默认初值，每拍被参数入口覆盖），下限 212.1 A 为代码常数
机侧 HVRT 无功电流 Iq*	[−424.240, −40.104] A	Control_39 / Control_40 经 f() 换算（负号 = 吸收无功）

表 26　全部运行时数值：参数与状态关系

调节器	kp	ki	跟踪系数 ki/kp	输出限幅
网侧 PLL（0x550/0x560）	0.08	0.0001	0.01（固定）	跟踪增益 0x1370 由初始化写成 0.01（0.0012/0.12），每拍只刷新 0x1320/0x1368、不刷新 0x1370；角速度 ±15.708 rad/s（正常）；±7.854（LVRT）；±62.832（HVRT），每拍按模式重写
机侧 PLL（0x5A0/0x5B0）	0.08	0.0001	0.01（固定）	参数入口每拍重写 0x5A0/0x5B0 ← Control_1/2（0.12/0.0012 仅为 motor_defaults 初始值）；跟踪增益 0x11E0 = 0.01 初始化算得后不刷新；角速度 ±62.832 rad/s（正常）；±7.854（LVRT）
网侧直流电压外环	5	0.15	0.03	上限 795.375 A（= 1.25×636.3，过/欠压标志时为 0）；下限 = −有功斜坡值
网侧正序电流环	1200	20	0.016667	±0.577 pu（内部定标 2^{23}，8388608 = 1 pu）
网侧负序电流环	1200	20	0.016667	±0.577 pu（同上）
机侧正序电流环	3500	15	0.004286	±0.577 pu（内部定标 2^{23}）；调制输入另有 ±1.1547 限幅（第6.12节）
机侧负序电流环	4500	25	0.005556	±0.577 pu（内部定标 2^{23}）；调制输入另有 ±1.1547 限幅（第6.12节）

内部固定常数

表 27　内部固定常数：参数与状态关系

常数	位置	用途
1.4	0x370 → 0x1FD0	机侧低压穿越限幅初值（标幺），每拍复制到机侧状态 区
2.6889 × 2 = 5.3778	0x368 + 0x1FC8 → 0x1FD8	机侧高压穿越限幅初值（标幺）
0.1 / 1.1 / 0.9	0x378 / 0x380 / 0x388	故障门槛默认值，配套计数器 100/600/100/100 拍 （0x658~0x664）；还原出的控制路径未读取
2500 / 1200	0x390 / 0x398	固定写入的定值；还原出的控制路径未读取，原样保留
1.0	0x778	使能标志
8000 拍 = 2 s	主入口 0x373C	保护潜在自动复归计数；当前整链门控未激活（保护专题）
2π = 6.28318530717958	PLL 内部	额定角速度前馈（314.16 rad/s = 50 Hz）
0.03665 / 0.10472	PLL 内部	序分量一阶低通系数 / dq 前馈滤波系数（第6.11节）
(30x + 994y)/1024	穿越控制内部	LVRT/HVRT 电压偏差低通（滤波系数 30/1024 ≈ 0.03，4.4 节）

关键参数作用表

表 28　关键参数作用表：参数与状态关系

参数	存储与装载	实际控制作用	源码依据（见源码对应函数）
Control_1/2	两侧 PLL 缓存；每有效采样复制 到活动 Kp/Ki	双 PLL 解耦带宽；tracking 不随输入重算	recovered_grid.cpp
Control_3/4	直接写网侧外环 Kp/Ki 并重算 Ki/Kp	P/Q 外环；默认门限 2（每 2 个被放行周期）	recovered_grid.cpp
Control_5…8	直接写网侧四电流环 Kp/Ki 并重 算 Ki/Kp	网侧电流环 d/q × 正/负序 四路	recovered_grid.cpp
Control_9…12	每周期仅写机侧缓存 0x208/0x210/0x218/0x220	就绪时复制到活动环 E10/E90/F10/F90；默认 6000/60	recovered_motor.cpp

参数	存储与装载	实际控制作用	源码依据（见源码对应函数）
Control_56	公开 parameters[65]，无普通 p[55] 字段	初始化与计算函数基频输入	recovered_controller.cpp
Control_57/58	S.d(0xBC8) / S.d(0xC38)	惯量保持时间阈值 / 频率偏差与变化率阈值	recovered_grid.cpp
Control_65/66	恢复参考源与恢复常数	模式 2 参考 C65·548；恢复速率 − (P_ref−a)/C66·M.548·0.00025	特殊恢复起点，按模式与门控生效
Control_72	S.d(0xC98)	=1：P/Q 命令采样保持（0.2s 计时置 0x7D0，下一更新消费）	recovered_grid.cpp
Control_73	S.i(0x21C8)，覆盖 Control_41	=1 跌落>0.2 或锁存：Id=C74·M.548；=2：Id=2B0·410/D0	当前2：常规励磁公式分支；保持寄存器属于模式1
Control_74	低穿电流系数	与 M.548 相乘作低穿 Id 参考	recovered_motor.cpp
Control_75	S.d(0x2618) = M.d(0xB60)	恢复源模式 1/2/3（double 精确比较）；模式 3 改 C65 无效	当前3：特殊恢复激活后选择P_pos均值参考
Cmd_6/7/8	0x21A4 / 0x628 / 0x740	Q 模式（0 功率因数/1 ±0.33/2 电压支撑）；输出模式 0=AVG；频率支持 0/1/2	recovered_commands.cpp
C24/26/36/42/43	仅见装载	未发现独立控制读取（部分确认，勿当可独立调）	各 recovered_*.cpp
C76、Gen_5…8、Gen_11/12	未打包进状态	不直接进入DLL参数槽；Gen11/12已通过工程机械命令及回读影响输入，Gen5～8导入标签与机器字面量参数须区别	contract.py

5.3 参数入口、保护输入与活动值的精确换算

参数输入是选定槽的逐项写入，不是对外层90槽逐一遍历。外层parameters[0..9]由铭牌等入口处理，parameter_input收到其+10指针，读取局部p[0..79]中的指定项。槽复用、面板命名和实际控制读取属于不同层次；这里只以恢复源码证明内部运算。

5.3.1 h、f、r的运算次序与分母

h先把全局容量0x538除以网侧电压0x270，再除1.732、乘1.414，最后乘p[n]。f在h结果上再乘1.414，两个1.414都必须保留。r以容量/1140/1.732为分子基准，乘p[n]；分母是0x110/1.414加0x1B00/1.414乘0x2F58，extra时只把分母第二项再乘0.75。

0x1B00为机侧基址+0x48，0x2F58为机侧基址+0x14A0。r的extra不能解释为整函数返回值乘0.75，机侧HVRT写入还在r(19,true)结果外乘3/4，属于另一步。所有除数没有软件防零处理；NaN、Inf及浮点舍入需保持实际表达式次序，不能为了“化简”改变数值路径。

$$h(n) = p[n]\left(\left(\frac{S}{V_g}\frac{1}{1.732}\right)1.414\right) \tag{5-15}$$

$$f(n) = 1.414\,h(n) \tag{5-16}$$

$$r(n,e) = \frac{p[n](S\ /\ 1140\ /\ 1.732)}{A\ /\ 1.414 + (B\ /\ 1.414)c\gamma_e}, \qquad \gamma_0 = 1,\ \gamma_1 = 0.75 \tag{5-17}$$

recovered_parameters.cpp L124-L145

```
    const auto h = [&s, p](std::size_t n) {
        double q = s.d(0x538) / s.d(0x270);
        q = q / 1.732;
        q = q * 1.414;
        return p[n] * q;
    };
    const auto f = [&h](std::size_t n) {
        return h(n) * 1.414;
    };
    const auto r = [&s, p](std::size_t n, bool extra) {
        double q = s.d(0x538) / 1140.0;
        q = q / 1.732;
        q = p[n] * q;
        double a = s.d(0x110) / 1.414;
        double b = s.d(0x1B00) / 1.414;
        b = b * s.d(0x2F58);
        if (extra) {
            b = b * 0.75;
        }
        a = a + b;
        return q / a;
    };
```

1140是函数硬编码，不能从当前外部铭牌变化自动推定它同步变化。

5.3.2 PI输入、缓存及低穿下一拍积分增益

p[0]/p[1]先写两侧PLL缓存；p[2]/p[3]写网侧电压外环并重算跟踪Ki/Kp；p[4..7]分别写网侧正负序d/q四个电流环及跟踪系数。p[8..11]写入机侧全局0x1CC0～0x1CD8，折成机侧局部0x208～0x220，这四项只是缓存。

如果进入本参数函数时0x1AC8==1，两侧PLL的Ki缓存被后写成p[1]/10。0x1AC8就是机侧局部0x10，motor_calculate会实际写入穿越模式1/2/0；因此不是“没有内部写入”的死分支。因为参数入口先于motor_calculate，模式本拍进入或退出时，Ki变化在下一参数拍反映。机侧电流环缓存则在ready转换处复制活动PI，ready之后单独改缓存不保证立即改变活动环。

$$K_{i,PLL} = \begin{cases} p[1]\ /\ 10, & M.i(0x10) = 1 \\ p[1], & \text{otherwise} \end{cases} \tag{5-18}$$

recovered_parameters.cpp L157–L180

```
    s.d(0x550, p[0]);
    s.d(0x560, p[1]);
    s.d(0x1120, p[2]);
    s.d(0x1168, p[3]);
    s.d(0x1170, s.d(0x1168) / s.d(0x1120));
    s.d(0xF20, p[4]);
    s.d(0xF68, p[5]);
    s.d(0xF70, s.d(0xF68) / s.d(0xF20));
    s.d(0xFA0, p[4]);
    s.d(0xFE8, p[5]);
    s.d(0xFF0, s.d(0xFE8) / s.d(0xFA0));
    s.d(0x1020, p[6]);
    s.d(0x1068, p[7]);
    s.d(0x1070, s.d(0x1068) / s.d(0x1020));
    s.d(0x10A0, p[6]);
    s.d(0x10E8, p[7]);
    s.d(0x10F0, s.d(0x10E8) / s.d(0x10A0));
    s.d(0x2058, p[0]);
    s.d(0x2068, p[1]);
    s.d(0x1CC0, p[8]);
    s.d(0x1CC8, p[9]);
    if (s.i(0x1AC8) == 1) {
        s.d(0x560, p[1] / 10.0);
        s.d(0x2068, p[1] / 10.0);
```

recovered_parameters.cpp L181–L183

```
    }
    s.d(0x1CD0, p[10]);
    s.d(0x1CD8, p[11]);
```

机侧ready复制及活动跟踪系数详见motor_calculate第524～543行。

5.3.3 穿越阈值、别名槽与六项电流限幅

网侧低压门槛及退出门槛分别为0x270乘p[12]和p[12]+p[13]再除1.732；高压退出门槛用p[16]−p[17]。机侧使用0x1EC8电压基准写同类门槛。两侧LVRT系数都用r(14,false)，网侧HVRT用r(18,true)，机侧HVRT用r(19,true)再乘3/4。

这里没有读取p[15]，而p[19]确实独立参与机侧HVRT换算。若外部包装把p[14]/p[15]和p[18]/p[19]打成相同值，那是包装别名约定，不能从本函数证明；缺少独立面板Control16或Control20也不能推定数组槽为零。随后p[20..25]经f分别写入六项限幅，其中0x140连续写两次f(20)，属于重复赋相同值。

$$V_{L,in} = V_g p[12] \,/\, 1.732, \quad V_{L,out} = V_g (p[12] + p[13]) \,/\, 1.732 \tag{5-19}$$

$$V_{H,in} = V_g p[16] \,/\, 1.732, \quad V_{H,out} = V_g (p[16] - p[17]) \,/\, 1.732 \tag{5-20}$$

$$K_{R,H} = \frac{3}{4} r(19, \mathrm{true}) \tag{5-21}$$

recovered_parameters.cpp L190-L212

```
    s.d(0x240, (s.d(0x270) * p[12]) / 1.732);
    s.d(0x248, (s.d(0x270) * (p[12] + p[13])) / 1.732);
    s.d(0x278, r(14, false));
    s.d(0x250, (s.d(0x270) * p[16]) / 1.732);
    s.d(0x258, (s.d(0x270) * (p[16] - p[17])) / 1.732);
    s.d(0x280, r(18, true));
    s.d(0x1E68, (s.d(0x1EC8) * p[12]) / 1.732);
    s.d(0x1E70, (s.d(0x1EC8) * (p[12] + p[13])) / 1.732);
    s.d(0x1ED0, r(14, false));
    s.d(0x1E78, (s.d(0x1EC8) * p[16]) / 1.732);
    s.d(0x1E80, (s.d(0x1EC8) * (p[16] - p[17])) / 1.732);
    s.d(0x1ED8, (r(19, true) * 3.0) / 4.0);
// p[20..25]: 电流限幅（标幺 → 有名值）。
//   网侧有功电流限幅 0.2777 / 无功电流限幅 0.1111 (0x140/0x120)；
//   机侧有功电流限幅 0.4 / 0.0，无功电流限幅 ±0.2895 (0x1B08 起)。
// 注意 0x140 连续写两次—原 DLL 就是这样（后写覆盖前写），原样保留。
    s.d(0x140, f(20));
    s.d(0x140, f(20));
    s.d(0x120, f(21));
    s.d(0x1B08, f(22));
    s.d(0x1B10, f(23));
    s.d(0x1B18, f(24));
    s.d(0x1B20, f(25));
```

p15不读与p19读取是不同证据，不应合并为“重复槽不生效”。

5.3.4 穿越电流、模式与运行时写入

p[26..28]和p[33..35]分别直接写网侧及机侧的LVRT/HVRT参考电压、初始系数、负序系数；p[29..32]、p[36..39]经f写入两侧穿越电流限值。数值为零、存储已写入与当前控制路径是否消费是三个问题，不能仅按参数名称判断。

p[40]先CVTT写机侧LVRT模式0x21C8，p[41]/p[42]写系数0x21D0/0x21D8，p[43]经f写转子d轴上限，p[44]经h写0x2648。后面的p[72]又写同一0x21C8，因此本函数返回时活动模式是后写p[72]，p[40]不能独立决定该拍模式。

$$I_{d,upper} = f(43), \qquad I_{field,base} = h(44) \tag{5-22}$$

recovered_parameters.cpp L220-L243

```
    s.d(0x638, p[26]);
    s.d(0x648, p[27]);
    s.d(0x5E8, p[28]);
    s.d(0x21A8, p[26]);
    s.d(0x21B8, p[27]);
    s.d(0x20E8, p[28]);
    s.d(0x288, f(29));
    s.d(0x2A0, f(30));
    s.d(0x1EE0, f(31));
    s.d(0x1F08, f(32));
    s.d(0x640, p[33]);
    s.d(0x650, p[34]);
    s.d(0x5F0, p[35]);
    s.d(0x21B0, p[33]);
    s.d(0x21C0, p[34]);
    s.d(0x20F0, p[35]);
    s.d(0x298, f(36));
    s.d(0x290, f(37));
    s.d(0x1EF0, f(38));
    s.d(0x1EE8, f(39));
    s.i(0x21C8, cvtt(p[40]));
    s.d(0x21D0, p[41]);
    s.d(0x21D8, p[42]);
    s.d(0x1F10, f(43));
```

recovered_parameters.cpp L244-L244

```
    s.d(0x2648, h(44));
```

对“未使用”结论须继续检查机侧局部地址读者。

5.3.5 固定值覆盖与频率支撑参数装载

每个参数拍无条件覆盖0x370=1.4、0x378=0.1、0x380=1.1、0x388=0.9，并设置四个整数计数门槛100/600/100/100；0x368和0x1FC8均写2.6889，0x390/0x398写2500/1200。它们不是外部保护数组输入，也不是整体初始化的1596、55Hz、45Hz等初值。

p[56..63]直接写惯量持续时间、死区、幅值参数、一次调频持续时间、下垂、死区和上下阈值；p[64]/p[65]写恢复起点和时间，紧随第272～273行的p[66]/p[67]写两侧低穿下限。局部p[55]未在parameter_input读取，额定频率由主入口直接取parameters[65]传给计算函数；不能因为该槽位于局部参数段，就虚构一次普通状态写入。

recovered_parameters.cpp L245-L268

```
    s.d(0x370, 1.4);
    s.i(0x658, 100);
    s.d(0x378, 0.1);
    s.i(0x65C, 600);
    s.d(0x380, 1.1);
    s.i(0x660, 100);
    s.d(0x388, 0.9);
    s.i(0x664, 100);
    s.d(0x368, 2.6889);
    s.d(0x1FC8, 2.6889);
    s.d(0x390, 2500.0);
    s.d(0x398, 1200.0);
// p[56..67]: 频率支撑（惯量响应 + 一次调频）参数。
//   p[56] = 惯量响应时间 10s; p[57] = 惯量死区 0.3; p[58] = 惯量时间常数 0.1;
//   p[59] = 一次调频响应时间 5s; p[60] = 功频下垂系数 0.06;
//   p[61] = 一次调频死区 0.03; p[62]/p[63] = 一次调频功率上/下限;
//   p[64]/p[65] = 恢复斜坡起点/时间; p[66]/p[67] = 网侧/机侧 LVRT 无功最小值。
    s.d(0xBC8, p[56]);
    s.d(0xC38, p[57]);
    s.d(0x720, p[58]);
    s.d(0x7B0, p[59]);
    s.d(0xC30, p[60]);
    s.d(0x738, p[61]);
    s.d(0xC40, p[62]);
```

recovered_parameters.cpp L269-L271

```
    s.d(0xC48, p[63]);
    s.d(0x2600, p[64]);
    s.d(0x2610, p[65]);
```

支撑参数单位及分支有效性由grid_frequency_support证明。

5.3.6 尾部覆盖、命令采样保持与动态量

p[69]/p[68]/p[70]按此顺序写无功电压下垂系数、死区及参考；p[71]写命令采样保持使能0xC98，p[72]再次CVTT写0x21C8并覆盖p[40]，p[73]写低穿模式系数，p[74]写double恢复模式，p[79]写全局0x36F8。

0x36F8减机侧基址0x1AB8为机侧局部0x1C40，因此p[79]是该动态量的入口。源代码不能证明其外部源一定是P_pos，更不能证明面板缺槽已经补零；若宿主契约把它指定为P_pos，应在外部接口证据中另行确认。尾部固定写0x778=1、0x1FD0=0x370、0x1FD8=0x368+0x1FC8，故有效结果为1.4与5.3778。

$$M.i(0x710) = \text{cvtt}\ (p[72]) \tag{5-23}$$

$$M.d(0x1C40) = p[79] \tag{5-24}$$

$$0x1FD8 = 2.6889 + 2.6889 = 5.3778 \tag{5-25}$$

recovered_parameters.cpp L280-L292

```
    s.d(0x3698, p[69]);
    s.d(0x36A0, p[68]);
    s.d(0x36A8, p[70]);
    s.d(0xC98, p[71]);
    s.i(0x21C8, cvtt(p[72]));
    s.d(0x36E8, p[73]);
    s.d(0x2618, p[74]);
    s.d(0x36F8, p[79]);
// 尾部固定常数：0x778 = 1.0（使能标志），
// 0x1FD0/0x1FD8 = 机侧低压/高压限幅初值（0x370 与 0x368+0x1FC8）。
    s.d(0x778, 1.0);
    s.d(0x1FD0, s.d(0x370));
    s.d(0x1FD8, s.d(0x368) + s.d(0x1FC8));
```

恢复模式存double并按精确比较使用；LVRT电流模式存int32。

5.3.7 保护28项的非连续偏移与内部固定定值

protection_input先把第69～88行Setting表中的网侧、机侧内部定值无条件写入。表不包含外部过压/欠压四段的1.33、1.28等“默认28项”；外部28项没有缺值或无效值时回退默认的机制。随后voltage_offsets将前16项写入指定非连续位置，其中j为偶数时乘0x270/1.414，奇数时间项直接存。

后12项input[16..27]由frequency_offsets写入对应频率与延时槽，全部直接赋值。本函数从不读input[28]或[29]，因此源码需要28个可读double；外部包装即使分配30槽，也只能称容量30、内部有效28，不能称“30槽保护定值写入”。第二个double基频参数在定义中无名，函数未使用。

$$V_{protect,base} = d(0x270) \,/\, 1.414 \tag{5-26}$$

$$z_j = \begin{cases} input[j]V_{protect,base}, & j \bmod 2 = 0 \\ input[j], & j \bmod 2 = 1 \end{cases}, \quad 0 \le j < 16 \tag{5-27}$$

recovered_parameters.cpp L90-L106

```
    for (const Setting& setting : settings) {
        s.d(setting.offset, setting.value);
    }
    const std::size_t voltage_offsets[] = {
        0x900, 0x918, 0x920, 0x938, 0x940, 0x958, 0x980, 0x998,
        0x960, 0x978, 0x9A0, 0x9B8, 0x9C0, 0x9D8, 0x9E0, 0x9F8
    };
    for (std::size_t j = 0; j < 16; ++j) {
        s.d(voltage_offsets[j], j % 2 == 0 ? input[j] * voltage_base : input[j]);
    }
    const std::size_t frequency_offsets[] = {
        0xA78, 0xA88, 0xA90, 0xAA0, 0xAA8, 0xAB8,
        0xAC0, 0xAD0, 0xAD8, 0xAE8, 0xAF0, 0xB00
    };
    for (std::size_t j = 0; j < 12; ++j) {
        s.d(frequency_offsets[j], input[j + 16]);
    }
```

输入顺序中第四过压段并非简单排在前四对，必须保留offset表。

5.4 命令入口、上一拍无功模式与复位边沿

command_input每有效控制拍先处理启动和P/Q源，再根据已经存储的无功模式计算Q，最后装载本拍模式、复位及输出/频率选择。P/Q来源冻结与模式延迟是两种不同的状态效应。

5.4.1 启动双写与P/Q采样保持

commands[0]经CVTT分别写网侧启动0x348、机侧启动0x1FAC。若0xC98不严格等于1，p、q直接取本拍commands[1]/[2]；如果0xC98==1，只有0x7D0==1才消费采样请求，清0x7D0并把本拍两个原始给定保存到0x36D8/0x36E0。随后始终读取这两个保存值。

命令保持使能并不保证首次就抓取本拍输入；若使能为1而采样请求未置位，缓存值由既有状态决定。p写0x1D58，相当于机侧局部0x2A0；motor_calculate之后按当前输出模式把这份本拍p乘1.04或1.045，并把本拍Q加0.015，下一正常参数拍由入口重新写原始给定，不能说在上一拍参考上无限累乘。

recovered_commands.cpp L70-L89

```
extern "C" __declspec(dllexport) void command_input(const double* commands, void* state) {
    CommandState s(state);
    s.i(0x348, cvtt_i32(commands[0]));
    s.i(0x1FAC, cvtt_i32(commands[0]));

    double p;
    double q;
    if (s.d(0xC98) == 1.0) {
        if (s.d(0x7D0) == 1.0) {
            s.d(0x7D0, 0.0);
            s.d(0x36D8, commands[1]);
            s.d(0x36E0, commands[2]);
        }
        p = s.d(0x36D8);
        q = s.d(0x36E0);
    } else {
        p = commands[1];
        q = commands[2];
    }
    s.d(0x1D58, p);
```

外部给定与内部功率/电流基准换算须分开。

0x7D0在grid_calculate第650行置1，当前命令拍消费之后的该次写入到下一命令拍才被读取。

5.4.2 功率因数模式的零值与越界输入

switch读取进入函数时已有的0x21A4；本拍commands[8]到末尾才写入，故无功模式延后一拍使用。模式0只将0<q<0.95改成0.95，将−0.95<q<0改成−0.95。q等于0不会被钳为0.95，|q|大于1也不会被钳为1。

先算apparent=p/q，再写apparent平方减p平方；负值才置0，然后sqrt，q<0时取负。q为0时保留IEEE除零/NaN行为，NaN比较不进入负值归零分支；|q|>1时有限实数结果通常被负根号项归零。手册不能笼统写“保证|pf|≥0.95”而遗漏pf=0。

$$q_c = \begin{cases} 0.95, & 0 < q < 0.95 \\ -0.95, & -0.95 < q < 0 \\ q, & \text{otherwise} \end{cases} \tag{5-28}$$

$$Q = \operatorname{sgn}(q_c)\sqrt{\max\left((P / q_c)^2 - P^2, 0\right)} \quad (q_c \neq 0, \text{有限实数}) \tag{5-29}$$

recovered_commands.cpp L92-L109

```
    switch (s.i(0x21A4)) {
    case 0: {
        if (q > 0.0 && q < .95) {
            q = .95;
        } else if (q < 0.0 && q > -.95) {
            q = -.95;
        }
        // Zero is intentionally not guarded: retain IEEE division behavior.
        const double apparent = p / q;
        s.d(0x1D60, apparent * apparent - p * p);
        if (s.d(0x1D60) < 0.0) {
            s.d(0x1D60, 0.0);
        }
        s.d(0x1D60, std::sqrt(s.d(0x1D60)));
        if (q < 0.0) {
            s.d(0x1D60, -s.d(0x1D60));
        }
        break;
```

符号与max数学式仅在明确有限非零输入前提下使用；异常输入按C++顺序解释。

5.4.3 直接无功、电压下垂与非法模式保留

模式2先写原始q，再检查全局0x1AD8==1；该地址是机侧局部运行标志0x20，motor_calculate正常输出路径确实写1，不能称无内部激活路径。偏差为0x36A8−0x1E60/0x1EC8，等于正/负死区边界也进入下垂，因为采用>=和<=，不是严格“超过”。

正负偏差都写同一系数乘偏差到0x3690并加到q；死区内部清0x3690。若运行门控不满足，Q保持本拍原始q，而0x3690不在本分支清除。模式1把原始q钳在±0.33；非法模式default不改0x1D60，因而Q保存旧状态。模式2没有±0.33限幅，与模式1不同。

$$e_V = V_{ref} - V / V_{base} \tag{5-30}$$

$$Q^* = q + K_V e_V \quad \text{if } M.i(0x20) = 1 \ \wedge \ (e_V \geq d \ \vee \ e_V \leq -d) \tag{5-31}$$

$$Q^* = \operatorname{clip}(q, -0.33, 0.33) \quad \text{mode} = 1 \tag{5-32}$$

recovered_commands.cpp L111-L134

```
    case 2:
        s.d(0x1D60, q);
        if (s.i(0x1AD8) == 1) {
            const double deviation = s.d(0x36A8) - s.d(0x1E60) / s.d(0x1EC8);
            if (deviation >= s.d(0x36A0)) {
                s.d(0x3690, s.d(0x3698) * deviation);
                s.d(0x1D60, s.d(0x1D60) + s.d(0x3690));
            } else if (deviation <= -s.d(0x36A0)) {
                s.d(0x3690, s.d(0x3698) * deviation);
                s.d(0x1D60, s.d(0x1D60) + s.d(0x3690));
            } else {
                s.d(0x3690, 0.0);
                s.d(0x1D60, q);
            }
        }
        break;
    case 1:
        s.d(0x1D60, q);
        if (s.d(0x1D60) > .33) {
            s.d(0x1D60, .33);
        } else if (s.d(0x1D60) < -.33) {
            s.d(0x1D60, -.33);
        }
        break;
```

recovered_commands.cpp L135-L137

```
    default:
        break;
    }
```

运行门控来自机侧前拍状态，因为命令入口早于机侧计算。

零死区使等于0也进入第一个分支，但乘积仍按系数与浮点值决定。

5.4.4 复位边沿与同步装载的整数命令

commands[3]/[4]经CVTT双写两侧LVRT/HVRT使能。复位只在commands[5]精确等于1且前拍整数0x3720为0时，把两侧复位请求置1；随后无论是否触发都CVTT保存本拍commands[5]。这不是通用的数值上升沿比较，也没有在命令为0时主动清除既有复位请求。

commands[6]写机侧恢复标志0x2088，commands[7]双写两侧保护字，commands[8]最后更新无功模式，commands[9]双写两侧输出模式，commands[10]写频率支撑模式。0xCA0另外复制机侧全局0x2038。输出0/1/2的模式含义由output实现确定；非法模式由输出函数保留六路已有缓冲，不能把整数转换说成有效枚举校验。

$$R_k = 1 \quad \text{if}\, commands_k[5] = 1 \ \wedge\ \text{cvtt}\,(commands_{k-1}[5]) = 0 \tag{5-33}$$

recovered_commands.cpp L139-L157

```
    s.i(0x518, cvtt_i32(commands[3]));
    s.i(0x1FEC, cvtt_i32(commands[3]));
    s.i(0x51C, cvtt_i32(commands[4]));
    s.i(0x1FF0, cvtt_i32(commands[4]));

    if (commands[5] == 1.0 && s.i(0x3720) == 0) {
        s.i(0x520, cvtt_i32(commands[5]));
        s.i(0x1FF4, cvtt_i32(commands[5]));
    }
    s.i(0x3720, cvtt_i32(commands[5]));

    s.i(0x2088, cvtt_i32(commands[6]));
    s.i(0x7D8, cvtt_i32(commands[7]));
    s.i(0x23D0, cvtt_i32(commands[7]));
    s.i(0x21A4, cvtt_i32(commands[8]));
    s.i(0x628, cvtt_i32(commands[9]));
    s.i(0x21A0, cvtt_i32(commands[9]));
    s.i(0x740, cvtt_i32(commands[10]));
    s.d(0xCA0, s.d(0x2038));
```

连续给定1只触发一次；复位请求的清理属于两侧算法。

第6章　公共算法：数学模型与源码对照

6.1 公共算法的阅读方式

以下25节按函数职责组织。先给数学关系与状态含义，再给当前源码摘录，最后列出时序、边界和工程门控。公式与代码中的有限常数、先后更新和上一拍记忆保持一致；代数等价不保证不同编译器逐位等价。算法行号对应当前recovered_primitives.cpp。

6.2 限幅 clip

$$C(x;L,U)=\begin{cases} U, & x>U,\\ L, & x<L,\\ x, & \text{otherwise.}\end{cases} \tag{6-1}$$

recovered_primitives.cpp L36-L40

```
static double clip(double value, double lower, double upper) {
    if (value > upper) return upper;
    if (value < lower) return lower;
    return value;
}
```

x=value；L=lower；U=upper。先检查上限再检查下限。L≤U 是调用前提而非函数自身检验；x 为 NaN 时两比较均假，返回 NaN。只在有限且有效上下限时，可附理论等价式 min(max(x,L),U)。

6.3 PI 调节器及抗积分饱和

PI依次计算误差、比例、积分、反向差分微分、合成、限幅及抗饱和残差。K_i,step对应ki_per_step（结构基址+0x60），直接乘本拍误差，不再乘周期；K_t对应tracking_gain（+0x68），乘上一拍限幅残差。参数代码原ki写到K_i,step，ki/kp写到K_t。部分源码注释误称后者为积分增益，本文按结构布局及实际读写纠正。

限幅函数定义为：

注意：微分项前的系数 k_d 取差分近似，即用上一步误差与本步误差之差近似误差变化率；当输出撞到限幅时，被削掉的部分通过 k_t 反向灌回积分器，积分值不会无意义地继续增长。

各PLL、电压外环及四轴电流调节器使用regulator内核。参数ki写入ki_per_step，ki/kp通常写入tracking_gain。PLL的模式限幅与四电流环自身限幅应分别说明，不能混用。

$$e_k=r_k-f_k,\qquad P_k=K_p e_k\,. \tag{6-2}$$

$$I_k=C(I_{k-1}+K_{i,\mathrm{step}}e_k+K_t a_{k-1};L,U)\,. \tag{6-3}$$

$$D_k=K_d(e_{k-1}-e_k)\,. \tag{6-4}$$

$$v_k = P_k + I_k + D_k, \qquad u_k = C(v_k; L, U)\,. \tag{6-5}$$

$$a_k = u_k - v_k, \qquad e_{\text{prev}} \leftarrow e_k\,. \tag{6-6}$$

recovered_primitives.cpp L60-L76

```
void regulator(RegulatorState* s) {
    const double error = s->reference - s->feedback;
    const double proportional = s->kp * error;
    const double integral = clip(s->integral + s->ki_per_step * error
        + s->tracking_gain * s->antiwindup_error, s->lower_limit, s->upper_limit);
    const double derivative = s->derivative_gain * (s->previous_error - error);
    const double unsaturated = proportional + integral + derivative;
    const double output = clip(unsaturated, s->lower_limit, s->upper_limit);
    s->error = error;
    s->proportional = proportional;
    s->integral = integral;
    s->derivative = derivative;
    s->unsaturated = unsaturated;
    s->output = output;
    s->antiwindup_error = output - unsaturated;
    s->previous_error = error;
}
```

r/reference、f/feedback、v/unsaturated、u/output；I/integral、a/antiwindup_error；K_i,step/ki_per_step、K_t/tracking_gain、K_d/derivative_gain。本函数不再乘采样周期，微分方向为“过去减现在”，积分与输出共用限幅。参数映射中通常 K_t=原 ki/原 kp，K_i,step=原 ki，不可反置。

k_i是每步积分系数；k_t是抗饱和跟踪系数。参数入口的ki/kp写入tracking_gain，不是替代k_i。微分项按源码用上一拍误差减本拍误差。

6.4 线电压到 α β

输入为两路线电压 v_ab、v_bc（第三路可推算）。注意系数 1.1547 ≈ 2/√3、0.577 ≈ 1/√3 是原 DLL 中被截断的常数，为保留有限常数与执行顺序，明码版本原样保留。

$$v_\alpha = 1.1547 v_{ab} + 0.577 v_{bc}, \qquad v_\beta = v_{bc}\,. \tag{6-7}$$

recovered_primitives.cpp L85-L89

```
void line_voltage_transform(LineVoltageState* s) {
    // Preserve the DLL's truncated coefficients rather than ideal square roots.
    s->alpha = 1.1547 * s->vab + 0.577 * s->vbc;
    s->beta = s->vbc;
}
```

v_ab/vab、v_bc/vbc；输出 alpha/beta。第三路线电压仅放入 reserved，不参与此变换。此坐标尺度不同于“先还原相电压再标准等幅 Clarke”的尺度，不能未经说明混用基准。

6.5 三相电流 Clarke

系数 0.666667 ≈ 2/3、0.866 ≈ √3/2，同样为截断常数。

$$i_\alpha = (i_a - 0.5 i_b - 0.5 i_c)\, 0.666667\,. \tag{6-8}$$

$$i_\beta = (0.866 i_b - 0.866 i_c)\, 0.666667\,. \tag{6-9}$$

recovered_primitives.cpp L96-L99

```
void current_transform(CurrentTransformState* s) {
    s->alpha = (s->ia - 0.5 * s->ib - 0.5 * s->ic) * 0.666667;
    s->beta = (0.866 * s->ib - 0.866 * s->ic) * 0.666667;
}
```

i_a/ia、i_b/ib、i_c/ic；不输出零序。理想等幅系数 2/3 与 √3/2 仅供解释。

6.6 Park

θ 取锁相环角度时，三相正序基波量在 dq 坐标系下变为直流量，PI 调节器才能无静差控制。

$$x_d = x_\alpha \cos\theta + x_\beta \sin\theta\,. \tag{6-10}$$

$$x_q = x_\beta \cos\theta - x_\alpha \sin\theta\,. \tag{6-11}$$

recovered_primitives.cpp L104-L107

```
void park_transform(ParkState* s) {
    s->d = s->alpha * std::cos(s->theta) + s->beta * std::sin(s->theta);
    s->q = s->beta * std::cos(s->theta) - s->alpha * std::sin(s->theta);
}
```

θ/theta 为调用方提供角；d、q 的符号以此定义固定。相量方向、功率正负约定需与这个 q 轴符号统一。

6.7 反 Park

控制环在旋转坐标系里算出的电压参考经反 Park 变回静止坐标系，交给调制模块。

$$x_\alpha = x_d \cos\theta - x_q \sin\theta\,. \tag{6-12}$$

$$x_\beta = x_q \cos\theta + x_d \sin\theta\,. \tag{6-13}$$

recovered_primitives.cpp L111-L114

```
void inverse_park_transform(ParkState* s) {
    s->alpha = s->d * std::cos(s->theta) - s->q * std::sin(s->theta);
    s->beta = s->q * std::cos(s->theta) + s->d * std::sin(s->theta);
}
```

同角 θ 下为第6.6节 Park 变换的逆变换（数学精确运算下）。

6.8 两态滤波

该模块是两个状态相互耦合的离散递推结构：x 根据输入与旧 x、旧 z 的偏差更新，z 再使用新 x 更新。增益 g、尺度 s 与步长 T 共同决定频率响应和稳定性，不能仅凭名称断言任意参数下均稳定。

$$a_k = (g_k / s_k)T_k, \qquad b_k = (g_k s_k)T_k . \tag{6-14}$$

$$e_k = u_k - x_{k-1} . \tag{6-15}$$

$$x_k = x_{k-1} + a_k(e_k - z_{k-1}) . \tag{6-16}$$

$$z_k = z_{k-1} + b_k x_k . \tag{6-17}$$

recovered_primitives.cpp L124-L137

```
void two_state_filter(TwoStateFilter* s) {
    const double a = (s->gain / s->scale) * s->step;
    const double b = (s->gain * s->scale) * s->step;
    s->x_coefficient = a;
    s->z_coefficient = b;
    const double error = s->input - s->x;
    const double next_x = s->x + a * (error - s->z);
    const double next_z = s->z + b * next_x;
    s->error = error;
    s->next_x = next_x;
    s->next_z = next_z;
    s->x = next_x;
    s->z = next_z;
}
```

g/gain、s/scale、T/step、u/input、x/x、z/z；scale 非零。先 x 后 z，不能都用旧 x。名称可说“两态递推结构”，不要仅靠名称保证滤波器对所有参数均稳定。

6.9 一阶递归滤波

本节统一用 b_0 表示当前输入系数、b_1 表示历史输入系数、a_1 表示 previous_output_coefficient；反馈项为 $-a_1 y_{k-1}$。下面公式与源码采用同一套符号。

$$y_k = -a_1 y_{k-1} + b_0 x_k + b_1 x_{k-1} . \tag{6-18}$$

$$H(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}. \tag{6-19}$$

recovered_primitives.cpp L142-L149

```
void recursive_filter(RecursiveFilterState* s) {
    const double output = ((-s->previous_output_coefficient) * s->previous_output
        + s->input_coefficient * s->input)
        + s->previous_input_coefficient * s->previous_input;
    s->output = output;
    s->previous_output = output;
    s->previous_input = s->input;
}
```

b0/input_coefficient、b1/previous_input_coefficient、a1/previous_output_coefficient；递推明确有负反馈符号。H(z) 仅在定系数、零初始条件下适用；本拍末保存输入及输出。

6.10 双通道残差滤波

先低通（0.2042035 + 0.7957965 = 1，保证直流增益为 1），再用"输入 − 低通结果"得高频残差，限幅 ±1131.2。用于提取谐波/纹波分量，反相注入以抑制谐波。d/q 两通道各一套。

$$\ell_{j,k} = 0.20420352248333634\, x_{j,k} + 0.7957964775166637\, \ell_{j,k-1}. \tag{6-20}$$

$$r_{j,k} = C(x_{j,k} - \ell_{j,k}; -1131.2, 1131.2), \qquad j \in \{0, 1\}. \tag{6-21}$$

recovered_primitives.cpp L159-L167

```
void residual_filter(ResidualFilterState* s) {
    for (int channel = 0; channel < 2; ++channel) {
        s->lowpass[channel] = s->input[channel] * 0.20420352248333634
            + s->lowpass[channel] * 0.7957964775166637;
    }
    for (int channel = 0; channel < 2; ++channel) {
        s->residual[channel] = clip(s->input[channel] - s->lowpass[channel], -1131.2, 1131.2);
    }
}
```

x/input、ℓ/lowpass、r/residual；本拍残差减的是已更新低通值。

6.11 正负序解耦 PLL

PLL先求原始αβ矢量模，再用更新前±θ获得正负序原始dq，并用对侧上一拍滤波值解耦。解耦结果低通保存；正序解耦q直接送调节器，不先用本拍低通q。角速度叠加τf_base，积分后单次边界修正，再按新角进行前馈Park及低通。PLL.voltage_magnitude与主控制正序滤波dq模是不同量。

下标 f 表示上一拍的滤波值；2θ 旋转项把对方序分量的 2 倍频纹波投影过来抵消。

四个解耦分量各过一遍一阶低通（系数来自原 DLL，原样保留）：

负序电压幅值：

PI 调节器输入为正序解耦 q 轴分量（reference = q^+_dec，feedback = 0），输出加额定角速度前馈：

新角度>τ则减τ，<0则加τ，否则保持，τ=6.28318530717958；只修正一次，等于τ不归零，大步跳变可能仍超界。不能等同严格取模或保证[0,2π)。随后用新角度再Park并进行前馈低通。

网侧与机侧各有一套 PLL（状态区分别位于 0x1188 与 0xFF8），算法完全相同。本工程 PLL 定值：kp = 0.08、ki = 0.0001（p[0]/p[1]，DLL 内置默认 0.12/0.0012），额定角速度前馈 2πf_base = 2π×50 = 314.16 rad/s。

$$\tau = 6.28318530717958, \qquad V_{\alpha\beta,k} = \sqrt{v_{\alpha,k}^2 + v_{\beta,k}^2}. \tag{6-22}$$

$$\begin{bmatrix} d_{r,k}^+ \\ q_{r,k}^+ \end{bmatrix} = P(\theta_{k-1}) \begin{bmatrix} v_{\alpha,k} \\ v_{\beta,k} \end{bmatrix}, \qquad \begin{bmatrix} d_{r,k}^- \\ q_{r,k}^- \end{bmatrix} = P(-\theta_{k-1}) \begin{bmatrix} v_{\alpha,k} \\ v_{\beta,k} \end{bmatrix}. \tag{6-23}$$

$$c_k = \cos(2\theta_{k-1}), \qquad s_k = \sin(2\theta_{k-1}). \tag{6-24}$$

$$d_k^+ = (d_{r,k}^+ - \bar{d}_{k-1}^- c_k) - \bar{q}_{k-1}^- s_k. \tag{6-25}$$

$$q_k^+ = (q_{r,k}^+ - \bar{q}_{k-1}^- c_k) + \bar{d}_{k-1}^- s_k. \tag{6-26}$$

$$d_k^- = (d_{r,k}^- - \bar{d}_{k-1}^+ c_k) + \bar{q}_{k-1}^+ s_k. \tag{6-27}$$

$$q_k^- = (q_{r,k}^- - \bar{q}_{k-1}^+ c_k) - \bar{d}_{k-1}^+ s_k. \tag{6-28}$$

$$\bar{x}_k = 0.03665191429188089 x_k + 0.9633480857081191 \bar{x}_{k-1}, \quad x \in \{d^+, q^+, d^-, q^-\}. \tag{6-29}$$

$$V_k^- = \sqrt{(\bar{d}_k^-)^2 + (\bar{q}_k^-)^2}. \tag{6-30}$$

$$\omega_k = u_{\mathrm{reg},k} + \tau f_b, \quad u_{\mathrm{reg},k} = \ \mathrm{regulator}\ (r = q_k^+, f = 0). \tag{6-31}$$

$$\tilde{\theta}_k = \theta_{k-1} + \omega_k T. \tag{6-32}$$

$$\theta_k = \begin{cases} \tilde{\theta}_k - \tau, & \tilde{\theta}_k > \tau, \\ \tilde{\theta}_k + \tau, & \tilde{\theta}_k < 0, \\ \tilde{\theta}_k, & \text{otherwise.} \end{cases} \tag{6-33}$$

$$\begin{bmatrix} d_{n,k} \\ q_{n,k} \end{bmatrix} = P(\theta_k) \begin{bmatrix} v_{\alpha,k} \\ v_{\beta,k} \end{bmatrix}. \tag{6-34}$$

$$v_{\mathrm{ff},j,k} = 0.10471975511965965 x_{n,j,k} + 0.8952802448803403 v_{\mathrm{ff},j,k-1}, \quad j \in \{d, q\}. \tag{6-35}$$

recovered_primitives.cpp L186-L209

```
void pll(double base_frequency, PllState* s) {
    const double tau = 6.28318530717958;
    s->stationary.vab = s->line_voltage[0];
    s->stationary.vbc = s->line_voltage[1];
    s->stationary.reserved = s->line_voltage[2];
    line_voltage_transform(&s->stationary);
    s->rotating.alpha = s->stationary.alpha;
    s->rotating.beta = s->stationary.beta;
    s->voltage_magnitude = std::sqrt(s->rotating.alpha * s->rotating.alpha
        + s->rotating.beta * s->rotating.beta);

    s->rotating.theta = s->angle;
    park_transform(&s->rotating);
    s->positive_raw[0] = s->rotating.d;
    s->positive_raw[1] = s->rotating.q;
    s->rotating.theta = -s->angle;
    park_transform(&s->rotating);
    s->negative_raw[0] = s->rotating.d;
    s->negative_raw[1] = s->rotating.q;

    const double cosine = std::cos(2.0 * s->angle);
    const double sine = std::sin(2.0 * s->angle);
    s->positive_decoupled[0] = (s->positive_raw[0] - s->negative_filtered[0] * cosine)
        - s->negative_filtered[1] * sine;
```

recovered_primitives.cpp L210-L233

```
    s->positive_decoupled[1] = (s->positive_raw[1] - s->negative_filtered[1] * cosine)
        + s->negative_filtered[0] * sine;
    s->negative_decoupled[0] = (s->negative_raw[0] - s->positive_filtered[0] * cosine)
        + s->positive_filtered[1] * sine;
    s->negative_decoupled[1] = (s->negative_raw[1] - s->positive_filtered[1] * cosine)
        - s->positive_filtered[0] * sine;

    // All four decouplings consume the previous filter histories.
    for (int axis = 0; axis < 2; ++axis) {
        s->positive_filtered[axis] = s->positive_decoupled[axis] * 0.03665191429188089
            + s->positive_filtered[axis] * 0.9633480857081191;
    }
    for (int axis = 0; axis < 2; ++axis) {
        s->negative_filtered[axis] = s->negative_decoupled[axis] * 0.03665191429188089
            + s->negative_filtered[axis] * 0.9633480857081191;
    }
    s->negative_magnitude = std::sqrt(s->negative_filtered[0] * s->negative_filtered[0]
        + s->negative_filtered[1] * s->negative_filtered[1]);
    s->regulator.reference = s->positive_decoupled[1];
    s->regulator.feedback = 0.0;
    regulator(&s->regulator);
    s->angular_speed = s->regulator.output + tau * base_frequency;
    s->angle = s->angle + s->angular_speed * s->step;
    if (s->angle > tau) s->angle = s->angle - tau;
```

recovered_primitives.cpp L234-L242

```
    else if (s->angle < 0.0) s->angle = s->angle + tau;

    s->rotating.theta = s->angle;
    park_transform(&s->rotating);
    s->feedforward[0] = s->rotating.d * 0.10471975511965965
        + s->feedforward[0] * 0.8952802448803403;
    s->feedforward[1] = s->rotating.q * 0.10471975511965965
        + s->feedforward[1] * 0.8952802448803403;
}
```

P 是第6.6节 Park 变换矩阵；上横线/positive_filtered、negative_filtered；r/raw，无横线/decoupled。四个解耦均用历史滤波值。f_b/base_frequency、T/step；前馈重做 Park 用新 θ。本函数 V_αβ 不是正序模，不能与主控制中正序模混称。

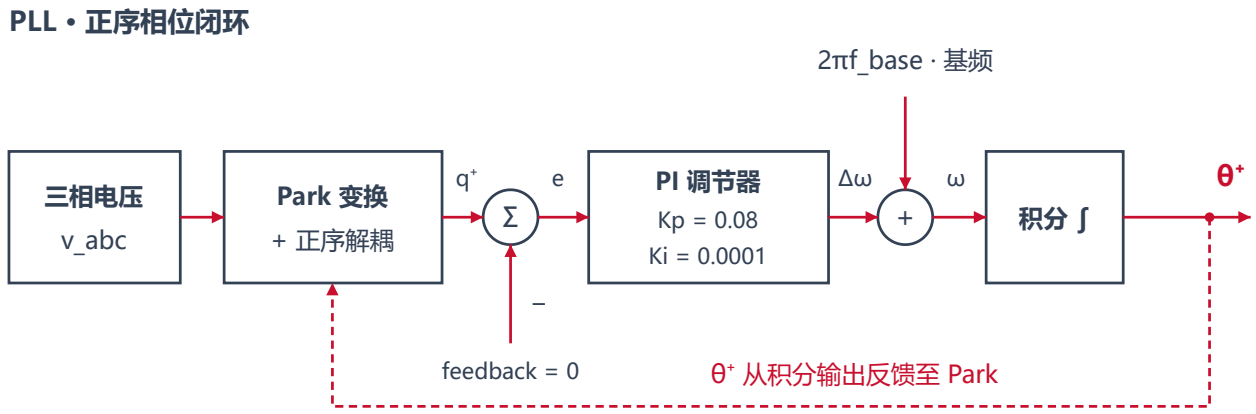


图 5　正负序解耦PLL；图示增益按当前参数0.08/0.0001理解，初始化0.12/0.0012另列

6.12 零序注入与中点平衡调制

把控制环给出的 αβ 电压参考（标幺）转成三相开关管占空比，适用于 NPC 三电平变流器（每相上/下管成对输出）。步骤：αβ 反推三相调制波 → 限幅 ±1.1547（近似 2/√3，保留有限常数）→ 注入由三相中间值的一半构成的公共分量（在线电压差中相消；一般含三倍频及其他零序谐波，不宜称为纯三次谐波）→ 中点电位平衡修正 → 限幅 ±1 → 转占空比。

中点电位平衡：δ = v_dc⁺ − v_dc⁻ 超过死区 deadband 时，按增益 g 比例产生修正量（极性 polarity 可反转），否则为 0：

占空比分配（上管 d_up、下管 d_dn）：

（当 m′_k ≥ 0 时；当 m′_k < 0 时）

$$p_a^0 = 2\alpha, \quad p_b^0 = 2(-0.5\alpha + 0.866\beta), \quad p_c^0 = 2(-0.5\alpha - 0.866\beta). \tag{6-36}$$

$$p_j = C(p_j^0; -1.1547, 1.1547), \quad H = \max_j p_j, \quad h = \min_j p_j. \tag{6-37}$$

$$p_{\mathrm{mid}} = ((p_a + p_b) + p_c) - H - h, \qquad m_0 = p_{\mathrm{mid}} \,/\, 2\,. \tag{6-38}$$

$$\delta = U_+ - U_-, \qquad \sigma = \begin{cases} 1, & \mathrm{polarity} = 1, \\ -1, & \mathrm{otherwise}. \end{cases} \tag{6-39}$$

$$c = \begin{cases} C((\sigma g \delta) \,/\, (U_+ + U_-); -L_c, L_c), & |\delta\,| > d_b, \\ 0, & \mathrm{otherwise}. \end{cases} \tag{6-40}$$

$$m_j = C((p_j + m_0) + c; -1, 1)\,. \tag{6-41}$$

$$(D_{j,0}, D_{j,1}) = \begin{cases} (m_j, 1), & m_j \geq 0, \\ (0, 1 + m_j), & m_j < 0\,. \end{cases} \tag{6-42}$$

recovered_primitives.cpp L260-L283

```
void modulation(ModulationState* s) {
    s->phase[0] = 2.0 * s->alpha;
    s->phase[1] = 2.0 * ((-0.5 * s->alpha) + (0.866 * s->beta));
    s->phase[2] = 2.0 * ((-0.5 * s->alpha) - (0.866 * s->beta));
    for (int phase = 0; phase < 3; ++phase) {
        if (s->phase[phase] > 1.1547) s->phase[phase] = 1.1547;
        else if (s->phase[phase] <= -1.1547) s->phase[phase] = -1.1547;
    }
    s->phase_max = s->phase[0];
    if (s->phase[1] > s->phase_max) s->phase_max = s->phase[1];
    if (s->phase[2] > s->phase_max) s->phase_max = s->phase[2];
    s->phase_min = s->phase[0];
    if (s->phase_min > s->phase[1]) s->phase_min = s->phase[1];
    if (s->phase_min > s->phase[2]) s->phase_min = s->phase[2];
    s->middle = (((s->phase[0] + s->phase[1]) + s->phase[2]) - s->phase_max) - s->phase_min;
    s->delta = s->dc_plus - s->dc_minus;
    if (std::fabs(s->delta) > s->deadband) {
        const double signed_gain = s->polarity == 1 ? s->gain : -s->gain;
        s->correction = (signed_gain * s->delta) / (s->dc_plus + s->dc_minus);
        s->correction = clip(s->correction, -s->limit, s->limit);
    } else {
        s->correction = 0.0;
    }
    for (int phase = 0; phase < 3; ++phase) {
```

recovered_primitives.cpp L284-L298

```
        s->modulation[phase] = (s->phase[phase] + s->middle / 2.0) + s->correction;
    }
    for (int phase = 0; phase < 3; ++phase) {
        s->modulation[phase] = clip(s->modulation[phase], -1.0, 1.0);
    }
    for (int phase = 0; phase < 3; ++phase) {
        if (s->modulation[phase] >= 0.0) {
            s->duty[phase][0] = s->modulation[phase];
            s->duty[phase][1] = 1.0;
        } else {
            s->duty[phase][0] = 0.0;
            s->duty[phase][1] = 1.0 + s->modulation[phase];
        }
    }
}
```

U+/dc_plus、U−/dc_minus、g/gain、L_c/limit、d_b/deadband。只有有效校正分支要求 U++U−≠0；死区边界相等时校正为 0。初始相限幅源码下界使用 ≤，数值结果与有效 C 相同。占空比是 NPC 两路调制控制量，具体开关互补、死区需由外部模型定义。

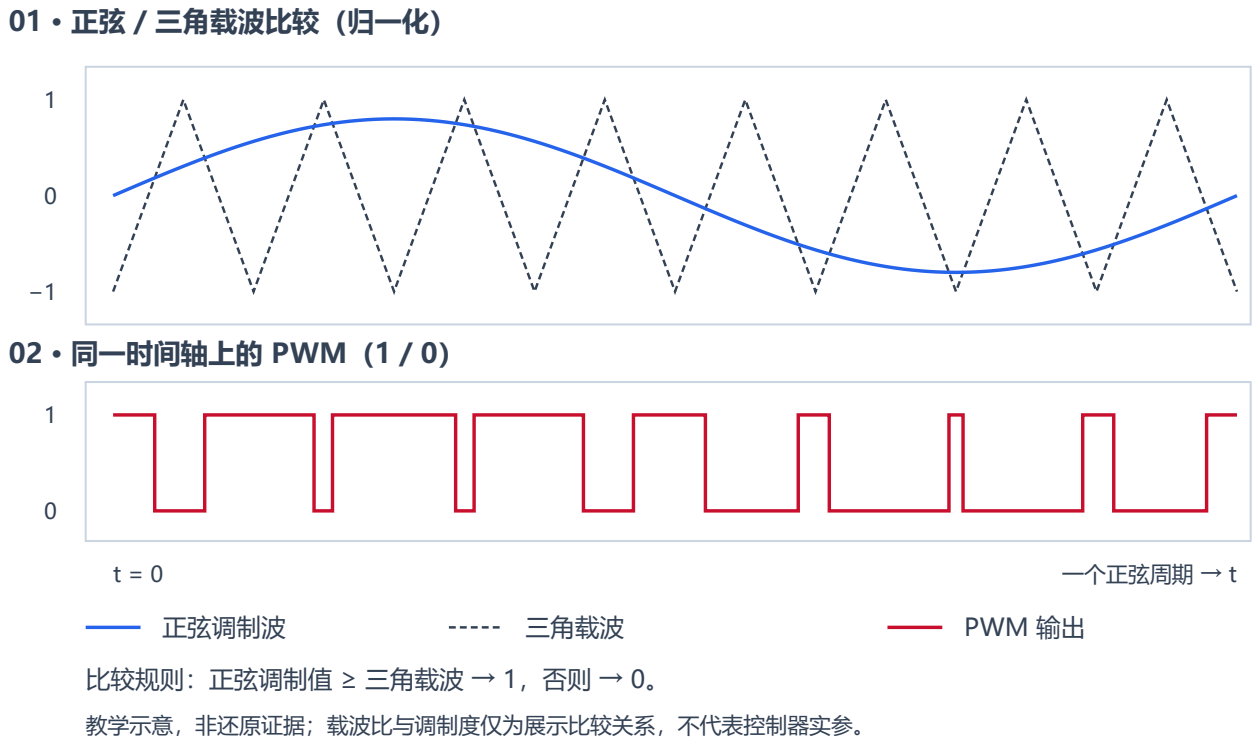


图 6　三相调制、零序注入与NPC占空比映射

6.13 备用调制

输出模式0x6E8==1使用备用调制。它与标准版共用初始换算、相限幅及零序注入，但无中点校正和末级±1限幅。注入后H/h/M为最大/最小/中间值，精确等于H输出(H,1)，否则等于h输出(0,1+h)，其他输出((M+H)/2,1+(M−H)/2)。最大分支优先，H=h也取此分支；不可保证占空比均在[0,1]。

$$p_j = C(p_j^0; -1.1547, 1.1547), \quad m_0 = \frac{((p_a + p_b) + p_c) - \max_j p_j - \min_j p_j}{2}. \tag{6-43}$$

$$w_j = p_j + m_0, \quad H = \max_j w_j, \quad h = \min_j w_j, \quad M = ((w_a + w_b) + w_c) - H - h. \tag{6-44}$$

$$A = (M + H) / 2, \qquad B = (M - H) / 2. \tag{6-45}$$

$$(D_{j,0}, D_{j,1}) = \begin{cases} (H, 1), & w_j = H, \\ (0, 1 + h), & w_j \neq H \text{ and } w_j = h, \\ (A, 1 + B), & \text{otherwise.} \end{cases} \tag{6-46}$$

recovered_primitives.cpp L324-L347

```
void alternate_modulation(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto put = [state](std::size_t offset, double value) { set_state_double(state, offset,
value); };
    put(0x10, 2.0 * d(0));
    put(0x18, 2.0 * ((-0.5 * d(0)) + (0.866 * d(8))));
    put(0x20, 2.0 * ((-0.5 * d(0)) - (0.866 * d(8))));
    for (std::size_t offset = 0x10; offset <= 0x20; offset += 8) {
        if (d(offset) > 1.1547) put(offset, 1.1547);
        else if (-1.1547 >= d(offset)) put(offset, -1.1547);
    }
    put(0x28, d(0x10));
    if (d(0x18) > d(0x28)) put(0x28, d(0x18));
    if (d(0x20) > d(0x28)) put(0x28, d(0x20));
    put(0x30, d(0x10));
    if (d(0x30) > d(0x18)) put(0x30, d(0x18));
    if (d(0x30) > d(0x20)) put(0x30, d(0x20));
    put(0x38, ((((d(0x10) + d(0x18)) + d(0x20)) - d(0x28)) - d(0x30)) / 2.0);
    put(0x88, d(0x10) + d(0x38));
    put(0x90, d(0x18) + d(0x38));
    put(0x98, d(0x20) + d(0x38));
    put(0x138, d(0x88));
    if (d(0x90) > d(0x138)) put(0x138, d(0x90));
    if (d(0x98) > d(0x138)) put(0x138, d(0x98));
    put(0x140, d(0x88));
```

recovered_primitives.cpp L348-L367

```
    if (d(0x140) > d(0x90)) put(0x140, d(0x90));
    if (d(0x140) > d(0x98)) put(0x140, d(0x98));
    put(0x158, (((d(0x88) + d(0x90)) + d(0x98)) - d(0x138)) - d(0x140));
    put(0x148, (d(0x158) + d(0x138)) / 2.0);
    put(0x150, (d(0x158) - d(0x138)) / 2.0);
    for (std::size_t phase = 0; phase < 3; ++phase) {
        const double value = d(0x88 + phase * 8);
        const std::size_t duty = 0xF0 + phase * 16;
        if (value == d(0x138)) {
            put(duty, d(0x138));
            put(duty + 8, 1.0);
        } else if (value == d(0x140)) {
            put(duty, 0.0);
            put(duty + 8, 1.0 + d(0x140));
        } else {
            put(duty, d(0x148));
            put(duty + 8, d(0x150) + 1.0);
        }
    }
}
```

p^0 对应第6.12节初始三相调制波。H/h/M 分别在 0x138/0x140/0x158，A/B 在 0x148/0x150。w 在 0x88/0x90/0x98。精确浮点相等，最大优先；无末级 ±1 限幅与中点校正，不应声称占空比必在 [0,1]。

备用调制不经过普通调制的中点修正和末级±1钳位。最大/最小值并列时分支顺序影响结果，不能把占空比自动解释为总在[0,1]。

6.14 状态字 ×100

$$r = (100s) \bmod 2^{32}, \qquad Y = \begin{cases} r, & r < 2^{31}, \\ r - 2^{32}, & r \geq 2^{31}. \end{cases} \tag{6-47}$$

recovered_primitives.cpp L372-L377

```
static double status_times_100(const void* state, std::size_t offset) {
    const std::uint32_t product = state_value<std::uint32_t>(state, offset) * 100u;
    std::int32_t signed_product;
    std::memcpy(&signed_product, &product, sizeof(product));
    return static_cast<double>(signed_product);
}
```

s 是 offset 处 uint32 状态字；乘法回绕，按 int32 位模式解释后转 double。Y 是最终数值，不是普通浮点 100s。

6.15 网侧输入映射与相电压

$$d_{o_j} \leftarrow u_j, \quad (o_0, \ldots, o_7) = (0\text{xC0}, 0\text{xC8}, 0\text{xD0}, 0\text{x100}, 0\text{x108}, 0\text{xD8}, 0\text{xE0}, 0\text{xE8}). \tag{6-48}$$

$$v_a = (2v_{ab} + v_{bc}) / 3, \quad v_b = (-v_{ab} + v_{bc}) / 3, \quad v_c = (-v_{ab} - 2v_{bc}) / 3 . \tag{6-49}$$

recovered_primitives.cpp L385-L393

```
void grid_input(const double* input, void* state) {
    const std::size_t offsets[] = {0xC0, 0xC8, 0xD0, 0x100, 0x108, 0xD8, 0xE0, 0xE8};
    for (int index = 0; index < 8; ++index) set_state_double(state, offsets[index], input[index]);
    const double vab = state_value<double>(state, 0xC0);
    const double vbc = state_value<double>(state, 0xC8);
    set_state_double(state, 0x1718, (2.0 * vab + vbc) / 3.0);
    set_state_double(state, 0x1720, (-vab + vbc) / 3.0);
    set_state_double(state, 0x1728, (-vab - 2.0 * vbc) / 3.0);
}
```

输入依次为三路线电压、正负母线电压、三相电流；v_ab=d_0xC0，v_bc=d_0xC8。相电压分别写0x1718/0x1720/0x1728，以相电压和为零为前提，线电压不能确定零序。

6.16 机侧输入与转差

$$d_{o_j} \leftarrow u_j, \quad (o_0, \ldots, o_7) = (0\text{x}228, 0\text{x}230, 0\text{x}238, 0\text{x}240, 0\text{x}248, 0\text{x}250, 0\text{x}290, 0\text{x}6\text{E}0) . \tag{6-50}$$

$$f_{\text{slip}} = np / 60 - f_b, \qquad d_{0\text{x}5\text{F}8} \leftarrow f_{\text{slip}} . \tag{6-51}$$

$$v_{a,\text{lag}} = (2d_{0\text{x}258} + d_{0\text{x}260}) / 3 . \tag{6-52}$$

$$v_{b,\text{lag}} = (-d_{0\text{x}258} + d_{0\text{x}260}) / 3 . \tag{6-53}$$

$$v_{c,\text{lag}} = (-d_{0\text{x}258} - 2d_{0\text{x}260}) / 3 . \tag{6-54}$$

recovered_primitives.cpp L402-L414

```
void rotor_input(const double* input, double base_frequency, void* state) {
    const std::size_t offsets[] = {0x228, 0x230, 0x238, 0x240, 0x248, 0x250, 0x290, 0x6E0};
    for (int index = 0; index < 8; ++index) set_state_double(state, offsets[index], input[index]);
    const double speed = state_value<double>(state, 0x6E0);
    const double pole_pairs = state_value<double>(state, 0x1E0);
    set_state_double(state, 0x5F8, (speed * pole_pairs) / 60.0 - base_frequency);
    // The main routine copies this step's grid voltages only after this helper returns.
    const double previous_vab = state_value<double>(state, 0x258);
    const double previous_vbc = state_value<double>(state, 0x260);
    set_state_double(state, 0x19B8, (2.0 * previous_vab + previous_vbc) / 3.0);
    set_state_double(state, 0x19C0, (-previous_vab + previous_vbc) / 3.0);
    set_state_double(state, 0x19C8, (-previous_vab - 2.0 * previous_vbc) / 3.0);
}
```

输入依次为转子三相电流、1140V低压接入测量电流、编码器角、转速；n=d_0x6E0，p=d_0x1E0。相电压写 0x19B8/0x19C0/0x19C8。此处 0x258/0x260 为随后主调度才复制的电网电压，正常运行使用上一拍；若铭牌极对数本拍改变，转差仍使用 nameplate 更新前 p。

6.17 网侧 20 路输出

$$\varepsilon = \ \text{double}\ (i_{0\text{x}50\text{C}}),\quad y_0 = \varepsilon,\quad M = i_{0\text{x}628}\,. \tag{6-55}$$

$$(y_1,\ldots,y_6) = \begin{cases} \varepsilon(d_{0\text{x}4\text{B}8}, d_{0\text{x}4\text{D}0}, d_{0\text{x}4\text{E}8}, d_{0\text{x}4\text{A}8}, d_{0\text{x}4\text{B}0}, 0), & M = 0, \\ \varepsilon(d_{0\text{x}4\text{C}0}, d_{0\text{x}4\text{C}8}, d_{0\text{x}4\text{D}8}, d_{0\text{x}4\text{E}0}, d_{0\text{x}4\text{F}0}, d_{0\text{x}4\text{F}8}), & M = 1, \\ \varepsilon(d_{0\text{x}\text{D}88}, d_{0\text{x}\text{D}90}, d_{0\text{x}\text{D}98}, 0, 0, 0), & M = 2, \\ \text{retain}, & \text{otherwise}. \end{cases} \tag{6-56}$$

$$(y_7, y_8, y_9, y_{10}) = (i_{0\text{x}504}, i_{0\text{x}508}, i_{0\text{x}510}, i_{0\text{x}514})\,. \tag{6-57}$$

$$y_{11} = S_{100}(s_{0\text{x}18}),\quad (y_{12}, y_{13}, y_{14}, y_{15}) = (d_{0\text{x}\text{F}08}, d_{0\text{x}\text{F}10}, d_{0\text{x}\text{F}88}, d_{0\text{x}\text{F}90}) \,/\, 1.414\,. \tag{6-58}$$

$$y_{16} = S_{100}(s_{0\text{x}\text{C}60}),\quad (y_{17}, y_{18}, y_{19}) = (d_{0\text{x}1218}, d_{0\text{x}270}, d_{0\text{x}278})\,. \tag{6-59}$$

recovered_primitives.cpp L422-L445

```
void grid_output(double* output, const void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto i = [state](std::size_t offset) { return state_value<std::int32_t>(state, offset); };
    const double enable = static_cast<double>(i(0x50C));
    output[0] = enable;
    switch (i(0x628)) {
    case 0:
        output[1] = d(0x4B8) * enable;
        output[2] = d(0x4D0) * enable;
        output[3] = d(0x4E8) * enable;
        output[4] = d(0x4A8) * enable;
        output[5] = d(0x4B0) * enable;
        output[6] = 0.0;
        break;
    case 1: {
        const std::size_t offsets[] = {0x4C0, 0x4C8, 0x4D8, 0x4E0, 0x4F0, 0x4F8};
        for (int index = 0; index < 6; ++index) output[index + 1] = d(offsets[index]) * enable;
        break;
    }
    case 2:
        output[1] = d(0xD88) * enable;
        output[2] = d(0xD90) * enable;
        output[3] = d(0xD98) * enable;
        output[4] = 0.0;
```

recovered_primitives.cpp L446-L463

```
        output[5] = 0.0;
        output[6] = 0.0;
        break;
    }
    output[7] = static_cast<double>(i(0x504));
    output[8] = static_cast<double>(i(0x508));
    output[9] = static_cast<double>(i(0x510));
    output[10] = static_cast<double>(i(0x514));
    output[11] = status_times_100(state, 0x18);
    output[12] = d(0xF08) / 1.414;
    output[13] = d(0xF10) / 1.414;
    output[14] = d(0xF88) / 1.414;
    output[15] = d(0xF90) / 1.414;
    output[16] = status_times_100(state, 0xC60);
    output[17] = d(0x1218);
    output[18] = d(0x270);
    output[19] = d(0x278);
}
```

S_{100} 为第6.14节状态字转换；ε 是整数直接转浮点，并非强制布尔。公式向量中的零通道在代码中直接写 0，不乘 ε；这对 ε=NaN/Inf 的非正常输入有差异，所以向量表达仅表示正常有限 ε 下值。模式 0 的 1…3 是相电压，4…5 是平均模型母线电流；模式 1 是六路占空比，模式 2 是三相调制波。非法模式保持调用前输出缓冲。

Cmd_7输出模式按当前输出函数：0为平均模型相电压/母线电流，1为占空比，2为调制波。仅选中模式的1～6路乘使能；其他指定路单独赋值，非法模式会保留已有1～6路缓存。

6.18 机侧 20 路输出

$$\varepsilon = d_{0x350}, \quad y_0 = \varepsilon, \quad M = i_{0x6E8}. \tag{6-60}$$

$$(y_1, \ldots, y_6) = \begin{cases} \varepsilon(d_{0x300}, d_{0x318}, d_{0x330}, d_{0x2F0}, d_{0x2F8}, 0), & M = 0, \\ \varepsilon(d_{0x308}, d_{0x310}, d_{0x320}, d_{0x328}, d_{0x338}, d_{0x340}), & M = 1, \\ \varepsilon(d_{0xC78}, d_{0xC80}, d_{0xC88}, 0, 0, 0), & M = 2, \\ \text{retain}, & \text{otherwise}. \end{cases} \tag{6-61}$$

$$y_7 = 0, \quad (y_8, y_9) = (i_{0x3A0}, i_{0x3A4}), \quad y_{10} = S_{100}(s_{0x10}). \tag{6-62}$$

$$(y_{11}, y_{12}, y_{13}, y_{14}) = (d_{0xDF8}, d_{0xE00}, d_{0xE78}, d_{0xE80}) \,/\, 1.414. \tag{6-63}$$

$$y_{15} = i_{0x3E0}, \quad y_{16} = \text{double}\,(i_{0x5D4}\ \text{OR}\ i_{0x5D8}), \quad y_{17} = B. \tag{6-64}$$

$$y_{18} = \left(\left(\frac{V_s}{1.732}\right) / \left(\frac{V_r}{V_s}\right)\right) / B \times 1.414. \tag{6-65}$$

$$y_{19} = A 1_{S=1} + (C \,/\, D) 1_{S \neq 1}. \tag{6-66}$$

recovered_primitives.cpp L470-L493

```
void rotor_output(double* output, const void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto i = [state](std::size_t offset) { return state_value<std::int32_t>(state, offset); };
    const double enable = d(0x350);
    output[0] = enable;
    switch (i(0x6E8)) {
    case 0:
        output[1] = d(0x300) * enable;
        output[2] = d(0x318) * enable;
        output[3] = d(0x330) * enable;
        output[4] = d(0x2F0) * enable;
        output[5] = d(0x2F8) * enable;
        output[6] = 0.0;
        break;
    case 1: {
        const std::size_t offsets[] = {0x308, 0x310, 0x320, 0x328, 0x338, 0x340};
        for (int index = 0; index < 6; ++index) output[index + 1] = d(offsets[index]) * enable;
        break;
    }
    case 2:
        output[1] = d(0xC78) * enable;
        output[2] = d(0xC80) * enable;
        output[3] = d(0xC88) * enable;
        output[4] = 0.0;
```

recovered_primitives.cpp L494-L513

```
        output[5] = 0.0;
        output[6] = 0.0;
        break;
    }
    output[7] = 0.0;
    output[8] = static_cast<double>(i(0x3A0));
    output[9] = static_cast<double>(i(0x3A4));
    output[10] = status_times_100(state, 0x10);
    output[11] = d(0xDF8) / 1.414;
    output[12] = d(0xE00) / 1.414;
    output[13] = d(0xE78) / 1.414;
    output[14] = d(0xE80) / 1.414;
    output[15] = static_cast<double>(i(0x3E0));
    output[16] = static_cast<double>(i(0x5D4) | i(0x5D8));
    output[17] = d(0x6C8);
    output[18] = (((d(0x6D0) / 1.732) / (d(0x6D8) / d(0x6D0))) / d(0x6C8)) * 1.414;
    // Both terms are evaluated even when their weight is zero.
    output[19] = d(0x4E0) * static_cast<double>(i(0x10) == 1)
        + (d(0x3A8) / d(0x410)) * static_cast<double>(i(0x10) != 1);
}
```

Vs=d_0x6D0，Vr=d_0x6D8，B=d_0x6C8；A=d_0x4E0、C=d_0x3A8、D=d_0x410、S=i_0x10。OR 为按位或，不是逻辑或；两个条件权重均计算对应项。分母 Vs/Vr/B/D 的原求值关系保持，不能约去 B。零通道亦直接写 0，见第6.17节非有限 ε 边界。

Cmd_7输出模式按当前输出函数：0为平均模型相电压/母线电流，1为占空比，2为调制波。仅选中模式的1～6路乘使能；其他指定路单独赋值，非法模式会保留已有1～6路缓存。

6.19 平均模型相电压与母线电流

$$U_h = (U_+ + U_-)\,/\,2, \qquad v_j = U_h m_j\,. \tag{6-67}$$

$$I_{\mathrm{nonpos}} = \eta \sum_{j:m_j \le 0} m_j i_j, \qquad I_{\mathrm{pos}} = \eta \sum_{j:m_j > 0} m_j i_j\,. \tag{6-68}$$

$$\eta = \begin{cases} -1, & \mathrm{GSC}, \\ +1, & \mathrm{RSC}\,. \end{cases} \tag{6-69}$$

recovered_primitives.cpp L521-L544

```
void average_converter_output(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const double half_dc = (d(0x100) + d(0x108)) / 2.0;
    for (int side = 0; side < 2; ++side) {
        const std::size_t modulation_base = side == 0 ? 0xD88 : 0x2730;
        const std::size_t current_base = side == 0 ? 0xD8 : 0x1CE0;
        const std::size_t phase_base = side == 0 ? 0x4B8 : 0x1DB8;
        const std::size_t dc_current_base = side == 0 ? 0x4A8 : 0x1DA8;
        double nonpositive[3];
        double positive[3];
        for (int phase = 0; phase < 3; ++phase) {
            const double m = d(modulation_base + 8 * phase);
            const double current = d(current_base + 8 * phase);
            if (m > 0.0) {
                nonpositive[phase] = 0.0;
                positive[phase] = m * current;
            } else {
                nonpositive[phase] = m * current;
                positive[phase] = 0.0;
            }
        }
        for (int phase = 0; phase < 3; ++phase) {
            set_state_double(state, phase_base + 24 * phase, half_dc * d(modulation_base + 8 *
phase));
        }
```

recovered_primitives.cpp L545-L550

```
        const double nonpositive_sum = (nonpositive[0] + nonpositive[1]) + nonpositive[2];
        const double positive_sum = (positive[0] + positive[1]) + positive[2];
        set_state_double(state, dc_current_base, side == 0 ? -nonpositive_sum : nonpositive_sum);
        set_state_double(state, dc_current_base + 8, side == 0 ? -positive_sum : positive_sum);
    }
}
```

U+/U−为全状态 0x100/0x108，两侧共用；网侧 m 基址 0xD88、i 基址 0xD8、相电压基址 0x4B8（每相跨 24 字节）、母线电流基址 0x4A8；机侧对应 0x2730/0x1CE0/0x1DB8/0x1DA8，以上机侧这里用的是全状态偏移。分配条件取调制波正负，不取电流正负。精确代码条件 m>0，其 else 包括 m=0 及 NaN，故若表达 IEEE 边界，用“非正分支=otherwise”替换数学 m≤0。求和使用 ((a+b)+c)。

6.20 41 通道监控映射

$$y_j=\begin{cases}\mathrm{double}\,(i_{o_j}), & j=28,\\ d_{o_j}, & j\in\{0,\dots,40\}\backslash\{28\}.\end{cases}\tag{6-70}$$

recovered_primitives.cpp L554-L566

```
void view_output(double* output, const void* state) {
    const std::size_t offsets[] = {
        0xB18, 0x468, 0x908, 0x910, 0x918, 0x928, 0x930, 0x938,
        0x948, 0x950, 0x958, 0x988, 0x990, 0x998, 0xA80, 0xA88,
        0xA98, 0xAA0, 0xAB0, 0xAB8, 0xAC8, 0xAD0, 0xAE0, 0xAE8,
        0xAF8, 0xB00, 0xB10, 0xB20, 0x7DC, 0x968, 0x970, 0x978,
        0x9A8, 0x9B0, 0x9B8, 0x9C8, 0x9D0, 0x9D8, 0x9E8, 0x9F0, 0x9F8
    };
    for (int index = 0; index < 41; ++index) {
        if (index == 28) output[index] = static_cast<double>(state_value<std::int32_t>(state,
offsets[index]));
        else output[index] = state_value<double>(state, offsets[index]);
    }
}
```

o_j 顺序：0xB18、0x468、0x908、0x910、0x918、0x928、0x930、0x938、0x948、0x950、0x958、0x988、0x990、0x998、0xA80、0xA88、0xA98、0xAA0、0xAB0、0xAB8、0xAC8、0xAD0、0xAE0、0xAE8、0xAF8、0xB00、0xB10、0xB20、0x7DC、0x968、0x970、0x978、0x9A8、0x9B0、0x9B8、0x9C8、0x9D0、0x9D8、0x9E8、0x9F0、0x9F8。第 29 路按整数读，其余按 double；这里无滤波和使能门控。

6.21 电机铭牌参数换算

$$P=d_{0x598}\leftarrow q_0,\quad V_s=d_{0x6D0}\leftarrow 1000q_1,\quad V_r=d_{0x6D8}\leftarrow 1000q_2.\tag{6-71}$$

$$B=d_{0x6C8}\leftarrow q_3,\qquad p=d_{0x1E0}\leftarrow q_4.\tag{6-72}$$

$$d_{0x5F0}\leftarrow V_r\,/\,V_s.\tag{6-73}$$

$$k_P=(((V_s\,/\,1.732)\,/\,(V_r\,/\,V_s))\,/\,B)\times 1.414.\tag{6-74}$$

$$k_T=(((((-P\times 3000)\,/\,n)\,/\,p)\,/\,V_r)\,/\,1.732)\times 1.414\times(d_{0xB28}\,/\,f_b).\tag{6-75}$$

$$k_\psi=((P\,/\,V_r)\,/\,1.732)\times 1.414.\tag{6-76}$$

recovered_primitives.cpp L571-L578

```
void rotor_nameplate(const double* parameters, void* state) {
    set_state_double(state, 0x598, parameters[0]);
    set_state_double(state, 0x6D0, parameters[1] * 1000.0);
    set_state_double(state, 0x6D8, parameters[2] * 1000.0);
    set_state_double(state, 0x6C8, parameters[3]);
    set_state_double(state, 0x1E0, parameters[4]);
    set_state_double(state, 0x5F0, state_value<double>(state, 0x6D8) / state_value<double>(state,
0x6D0));
}
```

q_j=parameters[j]（铭牌组，下标与控制参数组 p[j] 不同），q1/q2 从 kV 换 V。工程 4 MW、Vs=1.14 kV、Vr=3.541 kV、p=2 是外部配置，不是此函数内硬编码。要求 Vs 非零。B 独立保留。关联 motor_calculate 换算应另列： 以上是执行表达式，并不把 1.732/1.414 换为根式，也不把原程序 3000 推定为本机理论同步转速；n、p、Vr、B、fb 等除数须非零。

6.22 自适应振荡器

g=d(0)为增益、u=d(0x40)为输入，x/y在0x48/0x50；幅值用旧状态模并下限5。频率修正为(−20g/A)(u−x)yT，限幅后叠加τf_base；先更新x再用新x更新y。相位只单次±τ修正。

g为偏移0的增益，u为0x40的唯一输入；T在8，x/y在0x48/0x50。A由调用前状态计算；相位单次修正，等于τ不归零，大步可仍超界。

$$A_k = \max\left(\sqrt{x_{k-1}^2 + y_{k-1}^2}, 5\right). \tag{6-77}$$

$$q_k = \frac{-20g}{A_k}(u_k - x_{k-1})y_{k-1}T. \tag{6-78}$$

$$\omega_{c,k} = C(\omega_{c,k-1} + q_k; L, U), \qquad \omega_k = \omega_{c,k} + \tau f_b. \tag{6-79}$$

$$e_k = g(u_k - x_{k-1}) - y_{k-1}. \tag{6-80}$$

$$x_k = x_{k-1} + (\omega_k T)e_k. \tag{6-81}$$

$$y_k = y_{k-1} + (\omega_k T)x_k. \tag{6-82}$$

$$\tilde{\phi}_k = \phi_{k-1} + \omega_k T. \tag{6-83}$$

$$\phi_k = \begin{cases} \tilde{\phi}_k - \tau, & \tilde{\phi}_k > \tau, \\ \tilde{\phi}_k + \tau, & \tilde{\phi}_k < 0, \\ \tilde{\phi}_k, & \text{otherwise.} \end{cases} \tag{6-84}$$

recovered_primitives.cpp L589-L610

```
void adaptive_oscillator(void* state, double base_frequency) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto put = [state](std::size_t offset, double value) { set_state_double(state, offset,
value); };
    const double tau = 6.28318530717958;
    put(0x68, std::sqrt(d(0x48) * d(0x48) + d(0x50) * d(0x50)));
    if (5.0 > d(0x68)) put(0x68, 5.0);
    double q = (-d(0)) * 20.0;
    q = q / d(0x68);
    q = q * (d(0x40) - d(0x48));
    q = q * d(0x50);
    q = q * d(8);
    put(0x20, d(0x20) + q);
    put(0x20, clip(d(0x20), d(0x30), d(0x28)));
    put(0x18, d(0x20) + tau * base_frequency);
    put(0x10, d(0x40) - d(0x48));
    put(0x10, d(0) * d(0x10) - d(0x50));
    put(0x48, d(0x48) + (d(0x18) * d(8)) * d(0x10));
    put(0x50, d(0x50) + (d(0x18) * d(8)) * d(0x48));
    put(0x38, d(0x38) + d(0x18) * d(8));
    if (d(0x38) > tau) put(0x38, d(0x38) - tau);
    else if (0.0 > d(0x38)) put(0x38, d(0x38) + tau);
}
```

g=d0， T=d8， u=d_0x40， x=d_0x48， y=d_0x50， ωc=d_0x20， ω=d_0x18， φ=d_0x38， A=d_0x68；L=d_0x30、U=d_0x28、τ同 PLL。g 不是输入；A 是更新前状态模。代码按 (-g)×20、除 A、乘(u−x)、乘 y、乘 T 顺序计算 q；普通代数式不替代顺序。A 为 NaN 时“5>A”比较假，不会修成 5。

6.23 网侧电流正负序分解

电流Clarke后分别按调用方θ+、θ−做Park，以Δθ=θ+−θ−和对侧上一拍滤波值解耦；之后两组一阶低通并行地取同一解耦结果，非串级。机侧θ−一般不等于−θ+。

网侧第一组低通（用于解耦反馈历史）：

网侧第二组并行低通：

$$\begin{bmatrix} d^{+}_{r,k} \\ q^{+}_{r,k} \end{bmatrix} = P(\theta_{+}) \begin{bmatrix} i_{\alpha} \\ i_{\beta} \end{bmatrix}, \qquad \begin{bmatrix} d^{-}_{r,k} \\ q^{-}_{r,k} \end{bmatrix} = P(\theta_{-}) \begin{bmatrix} i_{\alpha} \\ i_{\beta} \end{bmatrix}. \tag{6-85}$$

$$\Delta\theta = \theta_{+} - \theta_{-}, \quad c = \cos\Delta\theta, \quad s = \sin\Delta\theta\,. \tag{6-86}$$

$$d^{+}_{k} = (d^{+}_{r,k} - \ell^{-}_{d,k-1} c) - \ell^{-}_{q,k-1} s\,. \tag{6-87}$$

$$q^{+}_{k} = (q^{+}_{r,k} - \ell^{-}_{q,k-1} c) + \ell^{-}_{d,k-1} s\,. \tag{6-88}$$

$$d^{-}_{k} = (d^{-}_{r,k} - \ell^{+}_{d,k-1} c) + \ell^{+}_{q,k-1} s\,. \tag{6-89}$$

$$q_{k}^{-} = (q_{r,k}^{-} - \ell_{q,k-1}^{+} c) - \ell_{d,k-1}^{+} s\,. \tag{6-90}$$

$$\ell_{j,k} = 0.0314159265358979 x_{j,k} + 0.9685840734641021 \ell_{j,k-1}\,. \tag{6-91}$$

$$h_{j,k} = 0.5115273563044781 x_{j,k} + 0.4884726436955219 h_{j,k-1}\,. \tag{6-92}$$

recovered_primitives.cpp L620-L643

```
void current_sequence(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto put = [state](std::size_t offset, double value) { set_state_double(state, offset,
value); };
    put(0xA8, d(0));
    put(0xB0, d(8));
    put(0xB8, d(0x10));
    CurrentTransformState stationary = state_value<CurrentTransformState>(state, 0xA8);
    current_transform(&stationary);
    put(0xC0, stationary.alpha);
    put(0xC8, stationary.beta);
    put(0xD0, d(0xC0));
    put(0xD8, d(0xC8));
    put(0xE0, d(0x18));
    ParkState rotating = state_value<ParkState>(state, 0xD0);
    park_transform(&rotating);
    put(0xE8, rotating.d);
    put(0xF0, rotating.q);
    put(0x68, d(0xE8));
    put(0x70, d(0xF0));
    put(0xE0, d(0x20));
    rotating.theta = d(0xE0);
    park_transform(&rotating);
    put(0xE8, rotating.d);
    put(0xF0, rotating.q);
```

recovered_primitives.cpp L644-L664

```
    put(0x78, d(0xE8));
    put(0x80, d(0xF0));

    double t = d(0x68) - d(0x58) * std::cos(d(0x18) - d(0x20));
    put(0x28, t - d(0x60) * std::sin(d(0x18) - d(0x20)));
    t = d(0x70) - d(0x60) * std::cos(d(0x18) - d(0x20));
    put(0x30, t + d(0x58) * std::sin(d(0x18) - d(0x20)));
    t = d(0x78) - d(0x48) * std::cos(d(0x18) - d(0x20));
    put(0x38, t + d(0x50) * std::sin(d(0x18) - d(0x20)));
    t = d(0x80) - d(0x50) * std::cos(d(0x18) - d(0x20));
    put(0x40, t - d(0x48) * std::sin(d(0x18) - d(0x20)));
    const std::size_t raw[] = {0x28, 0x30, 0x38, 0x40};
    const std::size_t filtered[] = {0x48, 0x50, 0x58, 0x60};
    for (int axis = 0; axis < 4; ++axis) {
        put(filtered[axis], d(raw[axis]) * 0.0314159265358979 + d(filtered[axis]) *
0.9685840734641021);
    }
    for (int axis = 0; axis < 4; ++axis) {
        const std::size_t offset = 0x88 + 8 * axis;
        put(offset, d(raw[axis]) * 0.5115273563044781 + d(offset) * 0.4884726436955219);
    }
}
```

输入 i_a/i_b/i_c 在 0/8/0x10，Clarke 用第6.5节；θ+/θ−在 0x18/0x20，不强制互为相反数。j 顺序 d+、q+、d−、q−；x 是解耦结果在 0x28/0x30/0x38/0x40；ℓ 在 0x48/0x50/0x58/0x60，h 在 0x88/0x90/0x98/0xA0；r 为原始序分量在 0x68…0x80。全部解耦先于低通更新，两组并行。

两组低通并行读取对应解耦分量，各自保持历史；不是前后串级滤波。机侧两个旋转角由PLL角和编码器角合成，不必互为相反数。

6.24 机侧电流正负序分解

机侧第一组低通：

机侧第二组并行低通：

正序 dq 给电流环做反馈，负序 dq 用于负序电流抑制。

$$\ell_{j,k} = 0.07853981633974474\,x_{j,k} + 0.9214601836602553\,\ell_{j,k-1}\,. \tag{6-93}$$

$$h_{j,k} = 0.523709146364097\,x_{j,k} + 0.47629085363590296\,h_{j,k-1}\,. \tag{6-94}$$

recovered_primitives.cpp L671-L694

```
void rotor_current_sequence(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto put = [state](std::size_t offset, double value) { set_state_double(state, offset,
value); };
    put(0xA8, d(0));
    put(0xB0, d(8));
    put(0xB8, d(0x10));
    CurrentTransformState stationary = state_value<CurrentTransformState>(state, 0xA8);
    current_transform(&stationary);
    put(0xC0, stationary.alpha);
    put(0xC8, stationary.beta);
    put(0xD0, d(0xC0));
    put(0xD8, d(0xC8));
    put(0xE0, d(0x18));
    ParkState rotating = state_value<ParkState>(state, 0xD0);
    park_transform(&rotating);
    put(0xE8, rotating.d);
    put(0xF0, rotating.q);
    put(0x68, d(0xE8));
    put(0x70, d(0xF0));
    put(0xE0, d(0x20));
    rotating.theta = d(0xE0);
    park_transform(&rotating);
    put(0xE8, rotating.d);
    put(0xF0, rotating.q);
```

recovered_primitives.cpp L695-L715

```
    put(0x78, d(0xE8));
    put(0x80, d(0xF0));

    double t = d(0x68) - d(0x58) * std::cos(d(0x18) - d(0x20));
    put(0x28, t - d(0x60) * std::sin(d(0x18) - d(0x20)));
    t = d(0x70) - d(0x60) * std::cos(d(0x18) - d(0x20));
    put(0x30, t + d(0x58) * std::sin(d(0x18) - d(0x20)));
    t = d(0x78) - d(0x48) * std::cos(d(0x18) - d(0x20));
    put(0x38, t + d(0x50) * std::sin(d(0x18) - d(0x20)));
    t = d(0x80) - d(0x50) * std::cos(d(0x18) - d(0x20));
    put(0x40, t - d(0x48) * std::sin(d(0x18) - d(0x20)));
    const std::size_t raw[] = {0x28, 0x30, 0x38, 0x40};
    const std::size_t filtered[] = {0x48, 0x50, 0x58, 0x60};
    for (int axis = 0; axis < 4; ++axis) {
        put(filtered[axis], d(raw[axis]) * 0.07853981633974474 + d(filtered[axis]) *
0.9214601836602553);
    }
    for (int axis = 0; axis < 4; ++axis) {
        const std::size_t offset = 0x88 + 8 * axis;
        put(offset, d(raw[axis]) * 0.523709146364097 + d(offset) * 0.47629085363590296);
    }
}
```

输入、Park、四条解耦、状态偏移同第6.23节；只改滤波系数。θ+、θ−由调用方包含电网角/编码器角/补偿角后提供，并不一定是 ±同一 θ。比较实际时间常数须结合调用周期。

两组低通并行读取对应解耦分量，各自保持历史；不是前后串级滤波。机侧两个旋转角由PLL角和编码器角合成，不必互为相反数。

6.25 二阶谐振滤波

谐振滤波每次调用重算系数；偏移0为步长T、8为中心角频率a、0x10为带宽b、0x80为增益K。网侧/机侧中心频率默认约2π×300/2π×350，带宽约2π×10；重算不代表参数每拍变化。K初始化0且全部历史0时谐振输出为0，非零历史仍可能有自由响应。

比例辅助状态P=K_p e写0x58，积分辅助状态I←I+(K_iT)e写0x60；最终0x68仅clip(−y,L,U)，不把P/I相加到输出。上下限0xB0/0xA8；±0.05×8388608为初始化默认。

$$\Delta_k = (4 + 4bT) + (a^2T)T\,. \tag{6-95}$$

$$b_{0,k} = ((4Kb)T)\,/\,\Delta_k\,. \tag{6-96}$$

$$a_{1,k} = ((2a^2T)T - 8)\,/\,\Delta_k\,. \tag{6-97}$$

$$a_{2,k} = ((4 - 4bT) + (a^2T)T)\,/\,\Delta_k\,. \tag{6-98}$$

$$e_k = r_k - f_k\,. \tag{6-99}$$

$$y_k = (b_{0,k}(e_k - e_{k-2}) - a_{1,k}y_{k-1}) - a_{2,k}y_{k-2}\,. \tag{6-100}$$

$$P_k = K_p e_k, \qquad I_k = I_{k-1} + (K_i T)e_k\,. \tag{6-101}$$

$$u_k = C(-y_k; L, U)\,. \tag{6-102}$$

$$H(z) = \frac{b_0(1 - z^{-2})}{1 + a_1 z^{-1} + a_2 z^{-2}}\,. \tag{6-103}$$

recovered_primitives.cpp L724-L741

```
void resonant_filter(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    const auto put = [state](std::size_t offset, double value) { set_state_double(state, offset,
value); };
    put(0x90, (4.0 + (4.0 * d(0x10)) * d(0)) + ((d(8) * d(8)) * d(0)) * d(0));
    put(0x88, (((4.0 * d(0x80)) * d(0x10)) * d(0)) / d(0x90));
    put(0x98, (((((2.0 * d(8)) * d(8)) * d(0)) * d(0)) - 8.0) / d(0x90));
    put(0xA0, ((4.0 - (4.0 * d(0x10)) * d(0)) + ((d(8) * d(8)) * d(0)) * d(0)) / d(0x90));
    put(0x28, d(0x18) - d(0x20));
    put(0x40, (d(0x88) * (d(0x28) - d(0x38)) - d(0x98) * d(0x48)) - d(0xA0) * d(0x50));
    put(0x38, d(0x30));
    put(0x30, d(0x28));
    put(0x50, d(0x48));
    put(0x48, d(0x40));
    put(0x58, d(0x70) * d(0x28));
    put(0x60, d(0x60) + (d(0x78) * d(0)) * d(0x28));
    put(0x68, -d(0x40));
    put(0x68, clip(d(0x68), d(0xB0), d(0xA8)));
}
```

T=d0、a=d8、b=d_0x10、K=d_0x80；r=d_0x18、f=d_0x20；e=d_0x28、历史 e_{k−1}/e_{k−2} 为 0x30/0x38；y=d_0x40、历史 y 为 0x48/0x50；Kp=d_0x70、Ki=d_0x78、P/I=0x58/0x60、u=0x68、L/U=0xB0/0xA8。分母 Δ非零。P 和 I 虽更新，却没有相加到 u；输出只为 −y 限幅。 系数每调用重算，若 T/a/b/K 变化则系统时变，H(z) 只作定系数零初始对照；偏移 0 的 T 不能误称“频率”。

谐振器初始增益G=0；在全部历史为0且无外部改写时输出保持0。非零历史可能产生自由响应，不能单凭中心频率存在就声称该支路在当前运行中主动抑谐。辅助比例/积分通道不直接加入限幅后的谐振输出。

6.26 差分滤波

该模块按四个系数和一拍输入、输出历史提取信号变化量。以下直接以状态偏移定义 T、a、b、g，具体数值由初始化调用方写入，不能把系数名中的 T 理解为全部都具有时间单位。

$$c = \frac{abg}{T(a+b)}, \qquad d = \frac{a-b}{a+b}. \tag{6-104}$$

$$y_k = cx_k - cx_{k-1} - dy_{k-1}. \tag{6-105}$$

$$H(z) = \frac{c(1-z^{-1})}{1+dz^{-1}}. \tag{6-106}$$

recovered_primitives.cpp L745-L754

```
void difference_filter(void* state) {
    const auto d = [state](std::size_t offset) { return state_value<double>(state, offset); };
    double u = (((d(8) * d(0x10)) * d(0x18)) * d(0x20)) / (d(0) * (d(8) + d(0x10)));
    double v = (((d(8) * d(0x10)) * d(0x18)) * d(0x28)) / (d(0) * (d(8) + d(0x10)));
    u = u - v;
    v = ((d(8) - d(0x10)) * d(0x38)) / (d(8) + d(0x10));
    set_state_double(state, 0x30, u - v);
    set_state_double(state, 0x38, d(0x30));
    set_state_double(state, 0x28, d(0x20));
}
```

T=d0、a=d8、b=d_0x10、g=d_0x18；当前 x=d_0x20、历史 x=d_0x28，y=d_0x30、历史 y=d_0x38。要求 T≠0 且 a+b≠0。代码先分别计算 abg×当前输入/分母与 abg×历史输入/分母，再相减，再减反馈；不可为了逐位比对改成一次乘 c(x−xprev)。H(z) 限定定系数、零初始条件。

差分滤波公式保留T0/T1/T2/T3定义与源码除法顺序，避免在重排公式时把乘法项误写为求和。

第7章　网侧变流器 GSC 控制

7.1 网侧控制链与当前定值

本章保留公式中的V_base须按调用点解释：PLL原始幅值及控制归一化使用0x270=1611.96V；三相相基波提取器与门槛比较使用约930.69V。两种尺度不互换。所有电流圆先把负根号被开方数置0，再开方。



图 7　网侧主链：需求与最终输出分层

控制实现与逐拍关系

控制结构总览

信号流：输入与 PLL → 三相基波幅值提取 → 直流母线电压平均 → 软启动 → 直流电压外环（有功电流给定）→ LVRT/HVRT 状态机（无功电流给定）→ 电流给定合成 → 四个电流内环 PI + 300Hz 谐振 + 差分修正 → 电压前馈反 Park → 调制 → 占空比输出。

输入与锁相

PLL 锁三相电网线电压（0xC0/0xC8/0xD0），得到电网角度 θ（0x218）、角速度 ω_PLL、正序电压幅值 V_mag 与负序电压幅值 V_neg（PLL 内部）。正序幅值：

$$V_{mag} = \sqrt{d_f^{+2} + q_f^{+2}} \tag{7-1}$$

三相基波幅值提取

每相电压先过自适应振荡器（中心 2π×50 = 314.16 rad/s），再过两态滤波器（动态角频率参数 = 2·ω_PLL，随频率更新），输出三相基波幅值，除以相电压幅值基准 V_base = 930.69（= 658.1×√2，1140 V 系统的相电压峰值）得标幺值，并取三相最大/最小值供 LVRT/HVRT 判定：

$$u_k = \frac{V_k}{V_{\text{base}}} \tag{7-2}$$

$$u_{\max} = \max\left(u_a, u_b, u_c\right), u_{\min} = \min\left(u_a, u_b, u_c\right) \tag{7-3}$$

直流母线电压平均

每步累加正负母线电压之和，每 80 步取一次平均：

$$V_{\mathrm{dc,avg}} = \frac{1}{N}\sum_{k=1}^{N}\left(v_{\mathrm{dc}}^{+} + v_{\mathrm{dc}}^{-}\right) \tag{7-4}$$

软启动

启动后前 400 步（0.1 s）：直流电压给定直接跟随实测值（经两态滤波平滑），调制输出保持为零，避免冲击；400 步后转入正常控制。启动初期（前 800 步）锁相角附加 0x220 角度补偿，之后按 40 步窗口累计锁相频率误差并限幅修正。

直流电压外环（有功电流给定）

母线电压给定按0x178=1V/拍上升、按0x178/64=0.015625V/拍下降，逼近动态目标，再与每拍滤波后的实际电压送PI（每2拍执行一次）。工程kp=5.0、ki=0.15；外环上限正常为1.25×636.3=795.375A，母线迟滞异常标志成立时上限为0；下限为−0x138的有功限幅斜坡值。PI输出0x1158直接成为d轴给定0x58；8/1024低通副本0x588用于另一统计通路，d电流环仍读0x58：

$$e_k = V_{\mathrm{dc}}^{*} - V_{\mathrm{dc},f} \tag{7-5}$$

$$i_{d,\mathrm{ref}} = u_{PI,k} \tag{7-6}$$

$$i_{d,\mathrm{avg},k} = \frac{8i_{d,k} + 1016i_{d,\mathrm{avg},k-1}}{1024} \tag{7-7}$$

正常模式下 q 轴电流给定为交叉解耦补偿项（0x48 为备用输入槽，本工程保持 0）：

$$i_{q,\mathrm{target}} = \mathrm{clip}\left(u_{\mathrm{aux}} - k_{\mathrm{dec}}V_{+} / V_{\mathrm{LL,pk}}, -I_{q,\,\mathrm{lim}}, I_{q,\,\mathrm{lim}}\right) \tag{7-8}$$

交叉解耦系数按额定频率折算：k_dec = 658.1986 × (0.0028274·f_base) × 1.414，50 Hz 时约为 131.6；限幅 ±0x120。

LVRT / HVRT 状态机

模式0/1/2分别为正常/LVRT/HVRT。初始化包含固定门槛，但当前参数入口每拍向两侧重写门槛：LVRT进入837.624V、退出884.158V；HVRT进入1051.683V、参数滞回退出1005.148V，另有1.02pu兜底分支。初始化值与运行值须分列。

进入 LVRT：三相基波最小值低于 0.9 标幺（门槛 0x240 = 837.6 V）且 LVRT 使能、1611.96 ≥ V_mag，计数从0起，第21个连续满足的控制拍进入；有功限幅斜坡目标0x140写0x5A0（565.6A）。

网侧LVRT退出：最大保持门控解除后，相基波最小值高于884.158V，退出计数严格大于20时切换，通常第21个满足条件的拍动作。

HVRT进入检查最大相基波、使能、幅值和阻断标志。计数阈值4；严格大于计数阈值的切换在第5个满足条件的控制拍发生。有功支路出现1.25×I_G=795.375A的临时赋值，后续参数拍和斜坡逻辑仍可改写。

网侧参数HVRT退出门槛0x258=(0x270×(p16−p17))/1.732；当前1.13−0.05=1.08pu，约1005.15V。源码另有固定1.02×930.69的复合退出条件，需分列，不能把参数门槛写成1.02pu。退出计数先增后比较>4，通常第5个满足退出条件的控制拍动作。

穿越退出时模式0x18已经恢复0；穿越后标志0x1C另保持，计数0x20从0累计800次后，第801次正常分支才清0x1C。该标志选择q斜坡快速步长。每次新进入事件才增加0xB38/0xB3C。

LVRT 控制（无功优先）

网侧LVRT需求z=K_G,LV×电压偏差，当前K_G,LV≈2.315876。负值时置0.015，超过1.1719时置上界，其余原样，再乘I_G=636.3A。注意该源码不是把所有正小值钳到0.015；典型深度跌落需求上限745.680A。最终q目标另受636.216A天花板、电流圆、解耦与斜坡。

$$\Delta V = k_{q0} - \frac{V_{mag}}{V_{\text{base}}} \tag{7-9}$$

$$\Delta V_{f,k} = \frac{30\Delta V_k + 994\Delta V_{f,k-1}}{1024} \tag{7-10}$$

$$z_k = K_{G,LV}\Delta V_{f,k}, \quad i_{q,\text{demand}} = I_G \begin{cases} c_{\min}, & z_k < 0, \\ 1.1719, & z_k > 1.1719, \\ z_k, & \text{otherwise}. \end{cases} \tag{7-11}$$

d/q 轴补偿电流（来自负序电压，限幅 ±0.48828125，经 30/1024 低通；过压预判标志 0x308 与使能 0x620 同时成立时才输出，乘基准后限幅 ±0x5F8）：

$$\begin{aligned} t_d &= \text{clip}\left(\frac{V_{d,-}}{V_{LL,\text{pk}}}, -0.48828125, 0.48828125\right), \\ c_d &= \text{clip}\left(k_n t_d, -0.48828125, 0.48828125\right). \end{aligned} \tag{7-12}$$

网侧无功余量按当前d轴给定由总电流圆计算；需求、余量、参数天花板和历史保持分别在不同槽位更新，必须按执行顺序区分。

$$i_{q,\text{room}} = \sqrt{\max\left\{0, 2I_G^2 - (i_d^*)^2\right\}} \tag{7-13}$$

HVRT 控制（吸收无功）

HVRT滤波抬升量乘网侧当前换算系数2.223863，加负序幅值附加项后限制到[0,0.4883]；取负并乘I_G=636.3A，再接受参数上下界、历史保持及电流余量约束。

$$\Delta V = \frac{V_{d,\text{PLL}}}{V_{\text{base}}} - k_{q0,h} \tag{7-14}$$

$$\Delta V_{f,k} = \frac{30\Delta V_k + 994\Delta V_{f,k-1}}{1024} \tag{7-15}$$

$$z_{\mathrm{HV}} = K_{\mathrm{HV}} \Delta_{\mathrm{sel}} \tag{7-16}$$

$$\Delta V_{add} = \mathrm{clip}\left(\frac{0.5 V_{neg,LP}}{V_{\mathrm{base}}}, 0, 0.1 \right) \tag{7-17}$$

$$i_{q,HV} = -I_G \ \mathrm{clip}\ (K_{G,HV} \Delta V_f + \Delta V_{\mathrm{add}}, 0, 0.4883) \tag{7-18}$$

当前网侧HVRT需求参数区间为[-424.240,-53.567]A；旧初始化或保持支路中的-282.8A不能替代当前参数上界。最终给定仍须按逐拍合成顺序判断。

$$I_{d,\mathrm{room}} = \sqrt{\max\{0, 2I_G^2 - I_{q,\mathrm{selected}}^2\}} \tag{7-19}$$

有功给定 0x140 不超过该上限。

三种模式下的电流给定合成

模式1/2更新q轴斜坡目标0x50。0x2C0在当前分支代表选中的无功需求/余量，备用量0x48乘相应系数后扣除交叉解耦。d轴给定0x58仍来自电压外环。本拍较早执行q斜坡，因此这里生成的新目标供下一拍斜坡读取。

$$i_{q,\mathrm{target}} = I_{q,\mathrm{selected}} + u_{\mathrm{aux}} k_1 - \frac{k_{\mathrm{dec}} V_{\mathrm{mag}}}{V_{LL,\mathrm{pk}}} \tag{7-20}$$

正常模式d轴给定0x58为电压外环输出；q轴目标0x50=clip(备用输入−交叉解耦，±0x120)，输出0x60按斜坡逼近。负序0x610/0x618有独立目标、斜坡和门控。

电流内环与谐波补偿

四个电流环分别控制正负序dq通路；PLL电压前馈与电流环输出组成dq参考，经反Park后叠加差分修正。两组300Hz谐振器每拍执行并更新内部状态，但本函数电压参考和调制未读取其结果。±7.853981633974475/±62.8318530717958/±15.70796326794895 的模式限幅在 PLL 内嵌调节器 0x1348/0x1350，不能称四电流环积分限幅；电流环按自身结构限幅槽执行。

$$B_j = \min\left(\frac{0.577 V_{\mathrm{ff},j}}{\bar{V}_{\mathrm{dc}}}, 0.61 \right), \quad j \in \{d, q\} \tag{7-21}$$

差分滤波修正（限幅 ±0.1）：

$$\Delta v_d = \mathrm{clip}\left(\frac{0.577 y_{diff,d}}{V_{\mathrm{dc,avg}}}, -0.1, 0.1 \right) \tag{7-22}$$

电压前馈与调制

正序电压参考 = 谐波补偿 − 电流环输出/标幺基准（0x80 = 8388608），反 Park（θ）回 αβ；负序电压参考 = −负序电流环输出/标幺基准，反 Park（−θ）。两者与差分修正相加后送调制模块：

$$v_{d,\mathrm{ref}}^{+} = B_d - \frac{u_{PI,d}^{+}}{S_{pu}} \tag{7-23}$$

$$v_{d,\mathrm{ref}}^{-} = -\frac{u_{PI,d}^{-}}{S_{pu}} \tag{7-24}$$

$$v_{\alpha} = v_{\alpha}^{+} + v_{\alpha}^{-} + \Delta v_{\alpha} \tag{7-25}$$

调制计算六路占空比缓存并由输出模式选用。0x31C阶段和正常阶段均置PWM使能0x50C=1；0x24先增后以>0x28判断，0x28=20时第21次该分支结束，随后正常分支置运行门0x2C=1。停机计数0x358先增后以>0x35C判断，0x35C=800时第801次停机尾段执行整块网侧复位。

GSC穿越状态机

表 29　GSC穿越状态机：参数与状态关系

项目	LVRT（模式 1）	HVRT（模式 2）	说明
进入门槛	Vmin < 0.9 pu = 837.62 V	Vmax > 1.13 pu = 1051.68 V	相基波幅值体系（基 930.69 V）
退出门槛	Vmin > 0.95 pu = 884.16 V	① Vmax < 1.08 pu = 1005.15 V 且 |V⁻| < 56.56 V；② Vmax < 1.02 pu = 949.31 V（两条件并存，任一满足即计数）	|V⁻| = PLL 每拍实时负序幅值（0x1218），故障后负序电压显著、56.56 V 条件一般先破，② 为"负序消失后仍可退出"的兜底路径（原样披露）
确认拍数	进入 / 退出均 20 拍（5 ms）	进入 / 退出均 4 拍（1 ms）	计数阈值 20 / 4：连续满足（中断即清零重数），从首次满足起第 21 / 5 拍切换
附加条件	对应使能命令字 = 1；LVRT 还要求 |V⁺| ≤ 1611.96 V（防过压时误触发）、HVRT 还要求 |V⁺| > 1611.96 V 且另一侧标志未封锁；进入后 40 拍（10 ms）内退出计数被门控（0x30C/0x310 = 1，穿越至少保持 40 拍）		使能来自外部命令（见参数换算专题命令表）
进入动作	有功限幅目标写565.6A；进入条件持续满足时每拍赋值	有功限幅目标写795.375A；进入条件持续满足时每拍赋值	下一拍被参数入口重写为 795.49 A
退出动作	有功斜坡目标写 795.375 A；无功保持寄存器清零；进入"穿越后"状态		穿越后状态持续 800 拍（0.2 s）

项目	LVRT（模式 1）	HVRT（模式 2）	说明
穿越后	q 给定斜坡改用快速步长 14.14 A/拍 （固定常数 0x070，≈2π×2.25）； 正常时 2.3567 A/拍（= 7.07/3，固 定常数 0x068）		让无功指令尽快回到正常值

GSC汇总下与基准链

表 30　GSC汇总下与基准链：参数与状态关系

HVRT 穿越	进入 / 退出门槛	**1.13 pu（1051.68 V）/ ① 1.08 pu（1005.15 V）且 $\|V^-\| < 56.56$ V；② 1.02 pu（949.31 V）**	**Control_17 = 1.13 / Control_18 = 0.05（滞回）；退出双条件并 存、任一满足即计数（$\|V^-\|$ = PLL 每拍实时负序幅值 0x1218）**
	确认拍数	进入 / 退出均阈值 4（1 ms），第 5 拍 切换	中断即清零重数；进入后 40 拍内退出计数被门控（最少保持）
	无功系数	2.223863	Control_19 = 2.2841 经 $r_e(18)$ 换算
	需求限幅	[0, 0.4883] pu + 修 正项	0.4883 为固定常数；修正项 = clip(0.5×$\|V^-\|$f/1611.96, 0, 0.1)
	输出限幅	[−424.240, −53.567] A	Control_38 = −0.1481 / Control_37 = −0.0187 × 2864.5517
谐振状态计算	两组 300 Hz 谐振器	每拍更新内部状态；结果未被本函数电压参考和调制读取	+0.61 上界属于 PLL 电压前馈通路，不能标为谐振输出限幅
	差分滤波修正	±0.1	固定限幅
调制	零序注入	三相中间值的一半	作为三相公共分量注入，在线电压差中相消；一般不等于纯三次谐波，见第6.12节。
	中点平衡	死区 + 比例增益 + 限幅 + 极性	把直流上/下母线电压差拉回平衡（第6.12节）
软启动 / 停机	软启动 400 拍；停机 复位 800 拍	0.1 s / 0.2 s	固定常数

表 31　GSC汇总下与基准链：参数与状态关系

参数	设计值	运行实况	连带效果
LVRT 无功系数	2.315876	2.315876	LVRT 期间需求按 G×ΔV̄×636.3 全额计算，限 [9.5445, 745.68] A
LVRT 无功天花板	636.216 A	636.216 A	天花板与恢复期 282.8 A 上限同时生效
HVRT 无功系数	2.223863	2.223863	吸收需求按 G×(V̄−额定)×636.3 全额计算
HVRT 输出限幅	[−424.240, −53.567] A	[−424.240, −53.567] A	Iq* 限幅按设计值生效
有功给定目标值	795.49 A	795.49 A	参数入口每拍先写795.49A示例值；后续穿越分支可同拍改写，须按目标读写先后判断。
无功给定限幅（正常模式）	±318.25 A	±318.25 A	限幅生效（正常模式给定为解耦项，见对应源码对照）

7.2 网侧源码逐段对照

本节按实际执行顺序解释网侧控制。state[offset]表示字节偏移处状态；0x270为PLL/控制电压归一化基准，0x1780为相基波幅值基准，I_G为0x110电流基准。代码片段保留一基行号，省略区间的条件与时序在文字中明确说明。

7.2.1 斜坡算法与运行前的计算范围

grid_calculate定义的斜坡先比较目标与当前值，再加上升步长或减去下降步长/divisor，最后仅在越过目标时截到目标。clip按先上界、后下界顺序判断。正常数值下三数最大/最小函数用于提取相基波极值。

控制函数在判断运行状态前已经执行PLL、相幅值提取、电流序分解和母线平均；只有状态0x350为1或2时才继续外环、穿越、内环和调制。因此停机或跳闸状态仍更新前端估计，不能把运行门解释为整函数停算。

$$x_k = \begin{cases} \min\,(x_{k-1} + h, t), & t > x_{k-1}, \\ \max\,(x_{k-1} - h\,/\,d, t), & x_{k-1} > t, \\ x_{k-1}, & x_{k-1} = t\,. \end{cases} \tag{7-26}$$

recovered_grid.cpp L547-L568

```
    const auto ramp = [&s](std::size_t value, std::size_t target, std::size_t step, double divisor) {
        if (s.d(target) > s.d(value)) {
            s.d(value, s.d(value) + s.d(step));
            if (s.d(value) > s.d(target)) s.d(value, s.d(target));
        } else if (s.d(value) > s.d(target)) {
            s.d(value, s.d(value) - s.d(step) / divisor);
            if (s.d(target) > s.d(value)) s.d(value, s.d(target));
        }
    };
    const auto clip = [](double value, double lower, double upper) {
        if (value > upper) return upper;
        if (lower > value) return lower;
        return value;
    };
    const auto maximum = [](double a, double b, double c) {
        const double t = a > b ? a : b;
        return t > c ? (a > b ? a : b) : c;
    };
    const auto minimum = [](double a, double b, double c) {
        const double t = a > b ? b : a;
        return t > c ? c : (a > b ? b : a);
    };
```

divisor=64表示下降步长缩小64倍。NaN比较不满足普通大小关系，不能用连续实数饱和公式承诺异常数值行为。运行门实际位于645行。

7.2.2 PLL参数、角度与正序幅值

每拍先把0x550/0x560写入PLL内嵌调节器参数，再按进入本函数时的0x18选择积分限幅：LVRT为±7.853981633974475，HVRT为±62.8318530717958，正常为±15.70796326794895。这些槽位属于PLL调节器，四个电流环各自使用自己的结构。

将三路线电压写入PllState，调用pll(f)，把完整结构写回后，取0x11A8为角度，0x11D0/0x11D8的欧氏范数为0x230。穿越模式稍后才更新，故本拍PLL限幅使用此前模式，新模式通常在下一拍参与该选择。

$$V_{+}=\sqrt{v_{d,+}^{2}+v_{q,+}^{2}} \tag{7-27}$$

recovered_grid.cpp L570-L593

```
    s.d(0x1320, s.d(0x550));
    s.d(0x1368, s.d(0x560));
    if (s.i(0x18) == 1) {
        s.d(0x1348, 7.853981633974475);
        s.d(0x1350, -7.853981633974475);
    } else if (s.i(0x18) == 2) {
        s.d(0x1348, 62.8318530717958);
        s.d(0x1350, -62.8318530717958);
    } else {
        s.d(0x1348, 15.70796326794895);
        s.d(0x1350, -15.70796326794895);
    }
    s.d(0x1188, s.d(0xC0));
    s.d(0x1190, s.d(0xC8));
    s.d(0x1198, s.d(0xD0));
// ---- 第 2 步: 锁相环 (把 0x1188 起的一段状态区当 PllState 使用) ----
    {
        PllState p;
        std::memcpy(&p, s.at(0x1188), sizeof(p));
        pll(f, &p);
        std::memcpy(s.at(0x1188), &p, sizeof(p));
    }
    s.d(0x218, s.d(0x11A8));
    s.d(0x230, std::sqrt(s.d(0x11D0) * s.d(0x11D0) + s.d(0x11D8) * s.d(0x11D8)));
```

recovered_grid.cpp L594-L594

```
// ---- 第 3 步: 三相自适应振荡器 + 两态滤波器, 提取每相基波幅值 ----
```

PI参数刷新不等于重置积分状态；本段未重算0x1370的ki/kp比值。

7.2.3 三相基波估计与角度单次折返

每相0x1718起的电压先送自适应振荡器，再把其+0x68输出作为两态滤波器输入。filter基址处写2ωPLL，它是滤波器动态参数，不能笼统称输入增益。0x1730起保存有名幅值，0x1748起保存除以0x1780后的标幺值；极值0x1760/0x1768是有名值，0x1770/0x1778是标幺值。

启动补偿标志0x228成立时使用PLL角度加0x220，否则用PLL角度。角度超过2π仅减一次，小于0仅加一次，恰为2π不折返。随后把±角度分别交给电流正负序分解。

$$u_a = V_a \,/\, V_{\text{phase,pk}},\quad V_{\max} = \max\left(V_a, V_b, V_c\right),\quad V_{\min} = \min\left(V_a, V_b, V_c\right) \tag{7-28}$$

recovered_grid.cpp L597-L620

```
    for (int phase = 0; phase < 3; ++phase) {
        const std::size_t adaptive = 0x1890 + 0xB8 * phase;
        const std::size_t filter = 0x1788 + 0x58 * phase;
        s.d(adaptive + 0x40, s.d(0x1718 + 8 * phase));
        adaptive_oscillator(s.at(adaptive), f);
        s.d(filter + 0x10, s.d(adaptive + 0x68));
        s.d(filter, s.d(0x11A0) * 2.0);
        s.apply(filter, two_state_filter);
        s.d(0x1730 + 8 * phase, s.d(filter + 0x18));
        s.d(0x1748 + 8 * phase, s.d(0x1730 + 8 * phase) / s.d(0x1780));
    }
    s.d(0x1760, maximum(s.d(0x1730), s.d(0x1738), s.d(0x1740)));
    s.d(0x1768, minimum(s.d(0x1730), s.d(0x1738), s.d(0x1740)));
    s.d(0x1770, maximum(s.d(0x1748), s.d(0x1750), s.d(0x1758)));
    s.d(0x1778, minimum(s.d(0x1748), s.d(0x1750), s.d(0x1758)));
    if (s.i(0x228) == 1) s.d(0x218, s.d(0x11A8) + s.d(0x220));
    else s.d(0x218, s.d(0x11A8));
    if (s.d(0x218) > 6.28318530717958) s.d(0x218, s.d(0x218) - 6.28318530717958);
    else if (0.0 > s.d(0x218)) s.d(0x218, s.d(0x218) + 6.28318530717958);
// ---- 第 4 步：电流正负序分解（0x1388 起的状态区，θ 用 0x218 及其负值） ----
// 0x668 选择用哪一组滤波结果：0 = 短时间常数组（0x13B0 起），1 = 另一组（0x1410 起）。
    s.d(0x1388, s.d(0xD8));
    s.d(0x1390, s.d(0xE0));
    s.d(0x1398, s.d(0xE8));
```

recovered_grid.cpp L621-L623

```
    s.d(0x13A0, s.d(0x218));
    s.d(0x13A8, -s.d(0x218));
    current_sequence(s.at(0x1388));
```

PLL归一化0x270=1611.96V与相幅值归一化0x1780约930.69V分开使用。本段不使用无限次取模。

7.2.4 序分量选择、母线平均与启动修正

current_sequence执行后，0x668为0时选0x13B0起的四路dq序分量，否则选0x1410起的另一组，分别写到0x90/0x98/0xA0/0xA8。选组条件是等于0与其余整数，不限于0/1。

母线瞬时电压0xF8为两段直流电压之和。每拍先累加计数和电压，达到0x1D0后以设定窗口长度作分母，写0x1B8并清累加状态。656—685行另外实现运行步数上限4000、前0x8拍角度补偿、后续40拍幅值偏差平均和非负修正、最大值保持/800拍刷新，以及穿越中参考的向上跟踪。

$$V_{\mathrm{dc}} = V_{\mathrm{dc},+} + V_{\mathrm{dc},-}, \quad \bar{V}_{\mathrm{dc}} = \frac{\sum V_{\mathrm{dc}}}{N_{\mathrm{set}}} \tag{7-29}$$

recovered_grid.cpp L624-L645

```
    if (s.i(0x668) == 0) {
        s.d(0x90, s.d(0x13B0));
        s.d(0x98, s.d(0x13B8));
        s.d(0xA0, s.d(0x13C0));
        s.d(0xA8, s.d(0x13C8));
    } else {
        s.d(0x90, s.d(0x1410));
        s.d(0x98, s.d(0x1418));
        s.d(0xA0, s.d(0x1420));
        s.d(0xA8, s.d(0x1428));
    }
    s.d(0xF8, s.d(0x100) + s.d(0x108));
// ---- 第 5 步：直流母线电压平均（0xF8 = 正负母线电压和，每 0x1D0 步平均） ----
    s.d(0x1C8, s.d(0x1C8) + 1.0);
    s.d(0x1C0, s.d(0x1C0) + s.d(0xF8));
    if (s.d(0x1C8) >= s.d(0x1D0)) {
        s.d(0x1B8, s.d(0x1C0) / s.d(0x1D0));
        s.d(0x1C8, 0.0);
        s.d(0x1C0, 0.0);
    }
// ---- 闸门：只有“运行/停机过程”状态才执行下面的控制，否则直接跳到最后 ----
    if (s.i(0x350) != 1 && s.i(0x350) != 2) goto final_flags;
```

频率误差的说法不准确：663行使用0x11B0−0x1A8的正序解耦d电压分量与参考之差乘1.5。计数到设定值时使用设定分母，不是实际累计次数。

7.2.5 软启动、母线区间和迟滞标志

启动计数小于0x4时，先增计数，置0x510=1、0x514=0，令母线参考0x130跟随当前0xF8，并运行反馈滤波器，然后跳到exit_tail。该路径跳过本拍外环PI、电流环和调制；它没有在这里再次清六路占空比缓存。

计数达到0x4后置相反阶段标志，计数再走至1.5倍启动长度才置0x30。根据先前参考0x128/0x130选择两组母线迟滞阈值；正、负两段母线分别高于上门槛置标志、低于下门槛清标志，中间区间及等号保持此前状态。

$$b_k = \begin{cases} 1, & V > V_H, \\ 0, & V < V_L, \\ b_{k-1}, & V_L \le V \le V_H . \end{cases} \tag{7-30}$$

recovered_grid.cpp L690–L713

```
    if (s.i(0) < s.i(4)) {
        s.increment(0);
        s.i(0x510, 1);
        s.i(0x514, 0);
        s.d(0x130, s.d(0xF8));
        s.d(0x14E0, s.d(0xF8));
        s.apply(0x14D0, two_state_filter);
        goto exit_tail;
    }
    s.i(0x510, 0);
    s.i(0x514, 1);
    if (static_cast<double>(s.i(4)) * 1.5 > static_cast<double>(s.i(0))) s.increment(0);
    else s.i(0x30, 1);
// ---- 直流电压水平判定：根据当前直流电压落在哪个区间，
//      选择母线过压/欠压的限幅基准（0x208/0x210）与标志 0x504/0x508 ----
    if (s.d(0x128) > s.d(0x1F8)) {
        s.d(0x208, s.d(0x1E8));
        s.d(0x210, s.d(0x1F0));
    } else if (s.d(0x200) > s.d(0x130)) {
        s.d(0x208, s.d(0x1D8));
        s.d(0x210, s.d(0x1E0));
    }
    if (s.d(0x100) > s.d(0x208)) {
        s.i(0x504, 1);
```

recovered_grid.cpp L714–L717

```
        s.i(0x32C, 1);
    } else if (s.d(0x210) > s.d(0x100)) {
        s.i(0x504, 0);
        s.i(0x32C, 0);
```

初始化启动长度400对应400次本路径；正常首次调制还取决于运行状态的调度时序。0x510/0x514是阶段标志，不能称解耦项。负段母线对应719—725行，逻辑相同。

7.2.6 母线外环、非对称斜坡与当拍给定

0x158=1时，有功限幅斜坡0x138按0x150接近0x140，外环下限0x1150写成−0x138；上限0x1148在两段母线任一迟滞标志成立时为0，否则为1.25×I_G。电压参考目标为基础0x168加修正0x188，再以上界0x170封顶；0x130向其上升每拍0x178、下降每拍0x178/64。

反馈滤波器每个到达本段的控制拍执行；PI每0x160拍执行，未到时保持此前输出。正序d给定0x58直接取PI输出0x1158；0x588另按8/1024更新平滑副本，后续d电流环读取0x58而不是0x588。q斜坡0x60在本拍穿越模式和新目标计算之前，向此前0x50接近；步长按进入本段时0x1C选择。

$$U_H=\begin{cases}0, & b_+\vee b_-,\\ 1.25I_G, & \text{其他.}\end{cases}\qquad U_L=-I_{\text{ramp}} \tag{7-31}$$

$$i_{d,k}^{*}=u_{\text{PI},k},\quad \bar{i}_{d,k}=(8i_{d,k}^{*}+1016\bar{i}_{d,k-1})\,/\,1024 \tag{7-32}$$

recovered_grid.cpp L728-L751

```
    if (s.i(0x158) == 1) {
        ramp(0x138, 0x140, 0x150, 1.0);
        s.d(0x1150, -s.d(0x138));
        if (s.i(0x32C) == 1 || s.i(0x330) == 1) s.d(0x1148, 0.0);
        else s.d(0x1148, 1.25 * s.d(0x110));
    }
    s.d(0x128, s.d(0x168) + s.d(0x188));
    if (s.d(0x128) > s.d(0x170)) s.d(0x128, s.d(0x170));
    ramp(0x130, 0x128, 0x178, 64.0);
// ---- 电压外环 PI (0x1108) : 输入 0x130 (直流电压给定) 与 0x14E8 (滤波后实际值)  ----
    s.increment(0x15C);
    s.d(0x14E0, s.d(0xF8));
    s.apply(0x14D0, two_state_filter);
    if (s.i(0x15C) >= s.i(0x160)) {
        s.i(0x15C, 0);
        s.d(0x1108, s.d(0x130));
        s.d(0x1110, s.d(0x14E8));
        s.apply(0x1108, regulator);
    }
// ---- 电流给定合成: 0x58 = 外环输出 (d 轴给定) ,
//      0x588 对外环输出做平滑 (8/1024 的低通) ----
    s.d(0x58, s.d(0x1158));
    s.d(0x588, (8.0 * s.d(0x58) + 1016.0 * s.d(0x588)) / 1024.0);
    s.d(0x78, s.i(0x1C) == 0 ? s.d(0x68) : s.d(0x70));
```

recovered_grid.cpp L752-L752

```
    ramp(0x60, 0x50, 0x78, 1.0);
```

I_G=636.3A时正常上限795.375A；0x178初始化为1V/拍，而7.069999999999999A/拍的0x150管理电流限幅斜坡。

7.2.7 负序预判与LVRT进入

负序幅值0x1218严格超过0x260时，0x2EC先走到20，下一次仍满足才置0x308；当0x308已置位且幅值严格低于0x268时，退出确认0x304按同样先计数后动作顺序清标志。该预判有独立滞回。

LVRT进入要求相幅最小值低于0x240、HVRT最少保持标志0x310为0、使能0x518为1，且正序幅值不超过0x270。0x2D8从0开始，第21个连续满足的控制拍置模式1。事件计数只在此前模式不是1时增加，但0x140=0x5A0位于模式写入后，无模式变化限定，进入条件持续满足时会每拍写入。

$$V_{\min} < V_{\mathrm{LV,in}} \ \wedge \ m_{\mathrm{HV,hold}} = 0 \ \wedge \ e_{\mathrm{LV}} = 1 \ \wedge \ V_{+} \le V_{\mathrm{LL,pk}} \tag{7-33}$$

recovered_grid.cpp L761-L782

```
    if (s.d(0x1218) > s.d(0x260)) {
        s.i(0x304, 0);
        if (s.i(0x2EC) < 20) s.increment(0x2EC);
        else s.i(0x308, 1);
    } else s.i(0x2EC, 0);
    if (s.i(0x308) == 1) {
        if (s.d(0x268) > s.d(0x1218)) {
            if (s.i(0x304) < 20) s.increment(0x304);
            else s.i(0x308, 0);
        } else s.i(0x304, 0);
    }
    if (s.d(0x240) > s.d(0x1768) && s.i(0x310) == 0 && s.i(0x518) == 1 && s.d(0x270) >= s.d(0x230)) {
        s.i(0x2E0, 0);
        s.i(0x2F0, 0);
        s.i(0x2F4, 0);
        s.increment(0x334);
        if (s.i(0x2D8) < 20) s.increment(0x2D8);
        else {
            if (s.i(0x18) != 1) s.increment(0xB38);
            s.i(0x18, 1);
            s.d(0x140, s.d(0x5A0));
        }
```

连续条件中断后才进入后续分支清确认计数。目标赋值发生在本拍0x138斜坡之后，下一拍参数入口还会先改写0x140。

7.2.8 HVRT进入与模式退出

LVRT进入条件不成立后才检查HVRT：最大相基波严格大于0x250、0x51C=1、正序幅值严格大于0x270，并且0xCA0浮点值恰为0。计数0x2E0从0起，第5个满足拍置模式2并写有功目标1.25I_G；事件计数只在新进入时增加，目标在持续进入条件满足时反复写入。

两种进入条件均不满足时，确认及相关计数清零，才有机会退出已有模式。LVRT还要求0x30C=0和最小相基波严格大于0x248，先增退出计数，计数>20时退出。因此进入和退出通常各需21次满足，且进入判定优先于退出判定。

$$m_{\mathrm{HV,in}}:\quad V_{\max} > V_{\mathrm{HV,in}} \wedge V_{+} > V_{\mathrm{LL,pk}} \wedge e_{\mathrm{HV}} = 1 \wedge c_{\mathrm{block}} = 0 \tag{7-34}$$

recovered_grid.cpp L783–L806

```
    } else if (s.d(0x1760) > s.d(0x250) && s.i(0x51C) == 1 && s.d(0x230) > s.d(0x270) && s.d(0xCA0)
 == 0.0) {
        s.i(0x334, 0);
        s.i(0x2D8, 0);
        s.i(0x2DC, 0);
        s.i(0x2F8, 0);
        s.i(0x2FC, 0);
        if (s.i(0x2E0) < 4) s.increment(0x2E0);
        else {
            if (s.i(0x18) != 2) s.increment(0xB3C);
            s.i(0x18, 2);
            s.d(0x140, 1.25 * s.d(0x110));
        }
    } else {
        s.i(0x2DC, 0);
        s.i(0x2E4, 0);
        s.i(0x2D8, 0);
        s.i(0x2E0, 0);
        s.i(0x334, 0);
        if (s.i(0x18) == 1 && s.i(0x30C) == 0) {
            if (s.d(0x1768) > s.d(0x248)) {
                s.increment(0x2F0);
                if (s.i(0x2F0) > 20) {
                    s.i(0x2F0, 0);
                    s.i(0x18, 0);
```

recovered_grid.cpp L807–L810

```
                    s.d(0x2C8, 0.0);
                    s.d(0x140, 1.25 * s.d(0x110));
                }
            } else s.i(0x2F0, 0);
```

等于进入/退出电压门槛不属于越限。退出同时清0x2C8并写目标1.25I_G。

7.2.9 HVRT复合退出与穿越后标志

HVRT退出要求此前保持门0x310为0。第一条路径为Vmax严格低于参数门槛0x258且实时负序幅值严格小于56.559999999999995；否则第二条路径只要求Vmax严格小于1.02×0x1780。两路径共用0x2F8计数，连续从路径一切到路径二不清零；均不满足才清零。先增计数再判断>4，第5次满足退出。

模式0x18非0时置穿越后标志0x1C=1并清0x20；模式恢复0后，0x1C仍保持，0x20先累计800次，随后第801次正常分支清0x1C。模式已经在退出拍恢复0，延后解除的是快速q斜坡选择标志，不是穿越模式。

$$C_{\mathrm{HV,out}} = (V_{\max} < V_{\mathrm{HV,out}} \wedge V_{-} < 56.56) \vee (V_{\max} < 1.02 V_{\mathrm{phase,pk}}) \tag{7-35}$$

recovered_grid.cpp L811-L834

```
        } else if (s.i(0x18) == 2 && s.i(0x310) == 0) {
            if (s.d(0x258) > s.d(0x1760) && 56.559999999999995 > s.d(0x1218)) {
                s.increment(0x2F8);
                if (s.i(0x2F8) > 4) {
                    s.i(0x2F8, 0);
                    s.i(0x18, 0);
                    s.d(0x2C8, 0.0);
                    s.d(0x140, 1.25 * s.d(0x110));
                }
            } else if (1.02 * s.d(0x1780) > s.d(0x1760)) {
                s.increment(0x2F8);
                if (s.i(0x2F8) > 4) {
                    s.i(0x2F8, 0);
                    s.i(0x18, 0);
                    s.d(0x2C8, 0.0);
                    s.d(0x140, 1.25 * s.d(0x110));
                }
            } else s.i(0x2F8, 0);
        }
    }
    if (s.i(0x18) != 0) {
        s.i(0x1C, 1);
        s.i(0x20, 0);
    } else if (s.i(0x1C) == 1) {
```

recovered_grid.cpp L835-L838

```
        if (s.i(0x20) < 800) s.increment(0x20);
        else {
            s.i(0x20, 0);
            s.i(0x1C, 0);
```

退出计数只有在保持门释放后才执行，最少保持与确认延时不能简单相加成名义4拍。

7.2.10 LVRT需求、滤波和最少保持

LVRT分支清HVRT滤波状态并维护0x314保持计数：小于0x324时增计数且0x30C=1，到阈值后的下一次才清0x30C。需求输入为0x638−V+/0x270，30/1024低通每拍更新。0x5C4=1时使用低通值，否则使用当前偏差；乘0x278后，负值替换为0xC58，大于1.1719截到上界，0与非负小值保持，再乘I_G。

两路负序补偿分别取0x11E0/0x11E8除0x270，先截±0.48828125，分别乘±0x5E8后再截一次；按同一开关选择滤波或直通。输出基址分别为0x5C8和0x5D8，配套滤波缓存是0x5D0和0x5E0，不能把0x5D0称第二路输出。

$$\Delta_k = c_{q0} - V_+ \,/\, V_{\mathrm{LL,pk}},\quad \Delta_{f,k} = (30\Delta_k + 994\Delta_{f,k-1}) \,/\, 1024 \tag{7-36}$$

$$z = K_{\mathrm{LV}}\Delta_{\mathrm{sel}},\quad I_{\mathrm{dem}} = I_G\begin{cases} c_{\min}, & z < 0, \\ 1.1719, & z > 1.1719, \\ z, & \text{其他}. \end{cases} \tag{7-37}$$

recovered_grid.cpp L845-L868

```
    if (s.i(0x18) == 1) {
        s.d(0x590, 0.0);
        s.d(0x578, 0.0);
        s.i(0x318, 0);
        if (s.i(0x314) < s.i(0x324)) {
            s.increment(0x314);
            s.i(0x30C, 1);
        } else s.i(0x30C, 0);
        s.d(0x2D0, s.d(0x638) - s.d(0x230) / s.d(0x270));
        s.d(0x570, (30.0 * s.d(0x2D0) + 994.0 * s.d(0x570)) / 1024.0);
        if (s.i(0x5C4) == 1) s.d(0x2D0, s.d(0x278) * s.d(0x570));
        else s.d(0x2D0, s.d(0x278) * s.d(0x2D0));
        if (0.0 > s.d(0x2D0)) s.d(0x2D0, s.d(0xC58));
        else if (s.d(0x2D0) > 1.1719) s.d(0x2D0, 1.1719);
        s.d(0x2D0, s.d(0x2D0) * s.d(0x110));
        for (int axis = 0; axis < 2; ++axis) {
            const std::size_t output = 0x5C8 + 0x10 * axis;
            const std::size_t filtered = output + 8;
            const double t = clip(s.d(0x11E0 + 8 * axis) / s.d(0x270), -0.48828125, 0.48828125);
            const double multiplier = axis == 0 ? s.d(0x5E8) : -s.d(0x5E8);
            s.d(output, multiplier * t);
            s.d(output, clip(s.d(output), -0.48828125, 0.48828125));
            if (s.i(0x5C4) == 1) s.d(filtered, (30.0 * s.d(output) + 994.0 * s.d(filtered)) /
1024.0);
            else s.d(filtered, s.d(output));
```

recovered_grid.cpp L869-L869

```
            if (s.i(0x308) == 1 && s.i(0x620) == 1) {
```

当前0x5C4=1由初始化直接确认。负值替代语义不等于clip(z,c_min,1.1719)。

7.2.11 LVRT负序门控与无功余量

只有0x308=1且0x620=1时，负序补偿输出取滤波结果×I_G并截到±0x5F8，否则该拍两路输出强制为0；滤波缓存仍然更新。默认0x620=0使该路径不输出，但不删除四路电流环或其状态。

0x5C4=1时需求保持槽0x2C8直接覆盖；否则仅在新需求更大时更新。无功余量由本拍未滤波d给定0x58计算，负被开方数先置0，再开方，然后顺次取需求保持上界、负序预判额外上界0x2A0、恢复幅值条件触发的0x2A8、以及参数天花板0x288。恢复检查读正序解耦d分量0x11B0而非滤波dq范数0x230，不能把两者混为一个幅值。

$$I_{q,\mathrm{room}} = \sqrt{\max\left(0, 2I_G^2 - \left(i_d^*\right)^2\right)} \tag{7-38}$$

$$I_{q,\mathrm{selected}} = \min\left(I_{q,\mathrm{room}}, I_{\mathrm{hold}}, I_{\mathrm{conditional}}, I_{\mathrm{ceiling}}\right) \tag{7-39}$$

recovered_grid.cpp L869-L885

```
            if (s.i(0x308) == 1 && s.i(0x620) == 1) {
                s.d(output, s.d(filtered) * s.d(0x110));
                s.d(output, clip(s.d(output), -s.d(0x5F8), s.d(0x5F8)));
            } else s.d(output, 0.0);
        }
        if (s.i(0x5C4) == 1) s.d(0x2C8, s.d(0x2D0));
        else if (s.d(0x2D0) > s.d(0x2C8)) s.d(0x2C8, s.d(0x2D0));
        s.d(0x2C0, (2.0 * s.d(0x110)) * s.d(0x110) - s.d(0x58) * s.d(0x58));
        if (0.0 > s.d(0x2C0)) s.d(0x2C0, 0.0);
        s.d(0x2C0, std::sqrt(s.d(0x2C0)));
        if (s.d(0x2C0) > s.d(0x2C8)) s.d(0x2C0, s.d(0x2C8));
        if (s.i(0x308) == 1 && s.d(0x2C0) > s.d(0x2A0)) s.d(0x2C0, s.d(0x2A0));
        if (s.d(0x11B0) > s.d(0x270) * 0.95) {
            if (s.d(0x60) > s.d(0x2A8)) s.d(0x60, s.d(0x2A8));
            if (s.d(0x2C0) > s.d(0x2A8)) s.d(0x2C0, s.d(0x2A8));
        }
        if (s.d(0x2C0) > s.d(0x288)) s.d(0x2C0, s.d(0x288));
```

0x11B0>0.95×0x270时，本拍q输出0x60也先受0x2A8硬上限；各条件限幅按源码顺序作用。

7.2.12 HVRT滤波需求及负序附加量

HVRT清LVRT滤波与保持计数，并由0x318/0x320控制0x310最少保持。电压抬升输入使用PLL滤波d分量0x11D0/0x270−0x640；它在锁相稳态可接近正序幅值，但逐拍算法不把它替换成欧氏范数0x230。

抬升量按30/1024滤波，再根据0x5C4选滤波或直通，乘0x280。实时负序幅值0x1218另按20/1024滤波，乘0.5/0x270截[0,0.1]后加入需求。合成量截[0,0.4883]，取负×I_G生成吸收无功需求。

$$a_k = (20V_{-,k} + 1004a_{k-1}) / 1024, \quad c_k = \operatorname{clip}\left(0.5a_k / V_{\mathrm{LL,pk}}, 0, 0.1\right) \tag{7-40}$$

$$I_{q,\mathrm{raw}} = -I_G \operatorname{clip}\left(K_{\mathrm{HV}}\Delta_{\mathrm{sel}} + c_k, 0, 0.4883\right) \tag{7-41}$$

recovered_grid.cpp L889-L907

```
    } else if (s.i(0x18) == 2) {
        s.d(0x570, 0.0);
        s.i(0x314, 0);
        if (s.i(0x318) < s.i(0x320)) {
            s.increment(0x318);
            s.i(0x310, 1);
        } else s.i(0x310, 0);
        s.d(0x2D0, s.d(0x11D0) / s.d(0x270) - s.d(0x640));
        s.d(0x578, (30.0 * s.d(0x2D0) + 994.0 * s.d(0x578)) / 1024.0);
        if (s.i(0x5C4) == 1) s.d(0x2D0, s.d(0x280) * s.d(0x578));
        else s.d(0x2D0, s.d(0x280) * s.d(0x2D0));
        s.d(0x590, (20.0 * s.d(0x1218) + 1004.0 * s.d(0x590)) / 1024.0);
        s.d(0x598, (0.5 * s.d(0x590)) / s.d(0x270));
        if (0.0 > s.d(0x598)) s.d(0x598, 0.0);
        else if (s.d(0x598) > 0.1) s.d(0x598, 0.1);
        s.d(0x2D0, s.d(0x2D0) + s.d(0x598));
        if (0.0 > s.d(0x2D0)) s.d(0x2D0, 0.0);
        else if (s.d(0x2D0) > 0.4883) s.d(0x2D0, 0.4883);
        s.d(0x2D0, (-s.d(0x2D0)) * s.d(0x110));
```

负序附加量在HVRT真实参与，不受0x620门控。0.4883×636.3=310.70529A。

7.2.13 HVRT分层限幅与三模式目标

HVRT先把需求0x2D0和本拍q输出0x60分别以0x298限制上界；0x5C4=1时直接覆盖保持槽，否则只向更负方向保持。随后保持槽和q输出分别以0x290限制下界，再将保持槽送0x2C0。电流余量由已经经过保持与参数上下界的0x2C0计算，限制0x140；它不直接裁切本拍0x58。

模式1/2的q目标为选中需求加备用输入×相应系数，减去本拍V+/0x270计算的解耦项；没有读取0x630缓存，也没有正常模式的±0x120裁切。LVRT负序目标0x600取0x5D8、0x608取0x5C8，顺序交叉；HVRT把它们清0。正常模式清相关计数/滤波并生成clip(0x48−解耦,±0x120)。

$$I_{d,\mathrm{room}} = \sqrt{\max\left(0, 2I_G^2 - I_{q,\mathrm{selected}}^2\right)} \tag{7-42}$$

$$i_{q,\mathrm{target}} = I_{q,\mathrm{selected}} + u_{\mathrm{aux}} K_m - k_{\mathrm{dec}} V_+ \,/\, V_{\mathrm{LL,pk}} \tag{7-43}$$

recovered_grid.cpp L908-L931

```
        if (s.d(0x2D0) > s.d(0x298)) s.d(0x2D0, s.d(0x298));
        if (s.d(0x60) > s.d(0x298)) s.d(0x60, s.d(0x298));
        if (s.i(0x5C4) == 1) s.d(0x2C8, s.d(0x2D0));
        else if (s.d(0x2C8) > s.d(0x2D0)) s.d(0x2C8, s.d(0x2D0));
        if (s.d(0x290) > s.d(0x2C8)) s.d(0x2C8, s.d(0x290));
        if (s.d(0x290) > s.d(0x60)) s.d(0x60, s.d(0x290));
        s.d(0x2C0, s.d(0x2C8));
        s.d(0x148, (2.0 * s.d(0x110)) * s.d(0x110) - s.d(0x2C0) * s.d(0x2C0));
        if (0.0 > s.d(0x148)) s.d(0x148, 0.0);
        s.d(0x148, std::sqrt(s.d(0x148)));
        if (s.d(0x140) > s.d(0x148)) s.d(0x140, s.d(0x148));
    }
// ---- 三种模式下的 d/q 电流给定合成（0x50/0x600/0x608），
//      含交叉解耦项（0x238×电压/基准）与限幅（±0x120）；
//      0x610/0x618 是给负序电流环的给定（正常模式为 0）----
    if (s.i(0x18) == 1) {
        s.d(0x50, (s.d(0x2C0) + s.d(0x48) * s.d(0x648)) - (s.d(0x238) * s.d(0x230)) / s.d(0x270));
        s.d(0x600, s.d(0x5D8));
        s.d(0x608, s.d(0x5C8));
    } else if (s.i(0x18) == 2) {
        s.d(0x50, (s.d(0x2C0) + s.d(0x48) * s.d(0x650)) - (s.d(0x238) * s.d(0x230)) / s.d(0x270));
        s.d(0x600, 0.0);
        s.d(0x608, 0.0);
    } else {
```

recovered_grid.cpp L932-L935

```
        s.d(0x600, 0.0);
        s.d(0x608, 0.0);
        s.i(0x318, 0);
        s.i(0x314, 0);
```

有功限幅赋值在本拍有功斜坡之后；下一拍参数入口会先写0x140。q目标在本拍q斜坡之后生成，但HVRT硬上下界直接作用本拍0x60。

7.2.14 负序斜坡、谐振器与四路电流环

正常模式重算0x630为解耦监测槽并限制q目标；两路负序斜坡0x610/0x618在新目标生成之后执行，因此它们与本拍q斜坡的时序不同。两组谐振器分别用0x13F0/0x13F8反馈并给定0，运行谐振器并更新其内部状态，但本函数后续电压参考未读取这两组输出（分别在0x1610/0x16C8），因此不能据调用存在就画成参与调制的并联补偿。

四路regulator分别在0xF08、0xF88、0x1008、0x1088调用，输入依次为正序d(0x58/0x90)、正序q(0x60/0x98)、负序d(0x610/0xA0)、负序q(0x618/0xA8)。第4路调用位于965—967行。即使负序给定为0，负序反馈误差和电流环状态仍可产生输出，不能据给定休眠宣称整个负序环不计算。

$$e_d^+ = i_d^* - i_d^+, \quad e_q^+ = i_q^* - i_q^+, \quad e_d^- = i_d^{-*} - i_d^-, \quad e_q^- = i_q^{-*} - i_q^- \tag{7-44}$$

recovered_grid.cpp L938-L961

```
        s.d(0x2C0, 0.0);
        s.d(0x590, 0.0);
        s.d(0x630, (s.d(0x238) * s.d(0x230)) / s.d(0x270));
        s.d(0x50, s.d(0x48) - s.d(0x630));
        s.d(0x50, clip(s.d(0x50), -s.d(0x120), s.d(0x120)));
    }
    ramp(0x610, 0x600, 0x78, 1.0);
    ramp(0x618, 0x608, 0x78, 1.0);
// ---- 300Hz 谐振滤波（0x15A8/0x1660 两组），用于抑制特定次谐波 ----
    s.d(0x15C0, 0.0);
    s.d(0x15C8, s.d(0x13F0));
    resonant_filter(s.at(0x15A8));
    s.d(0x1678, 0.0);
    s.d(0x1680, s.d(0x13F8));
    resonant_filter(s.at(0x1660));
// ---- 第 9 步：四个电流内环 PI：
//      0xF08：正序 d 轴（给定 0x58、反馈 0x90）；0xF88：正序 q 轴（0x60/0x98）；
//      0x1008：负序 d 轴（0x610/0xA0）；0x1088：负序 q 轴（0x618/0xA8） ----
    s.d(0xF08, s.d(0x58));
    s.d(0xF10, s.d(0x90));
    s.apply(0xF08, regulator);
    s.d(0xF88, s.d(0x60));
    s.d(0xF90, s.d(0x98));
    s.apply(0xF88, regulator);
```

recovered_grid.cpp L962-L965

```
    s.d(0x1008, s.d(0x610));
    s.d(0x1010, s.d(0xA0));
    s.apply(0x1008, regulator);
    s.d(0x1088, s.d(0x618));
```

谐振滤波器与PI的参数和状态结构见公共算法；本段只解释调用点。

7.2.15 电压前馈单侧上限与差分修正

0x1220/0x1228是PLL电压前馈滤波输出，而非这两组300Hz谐振器结果。0xB0/0xB8由0x1220/0x1228除母线平均0x1B8再乘0.577，代码只在大于0.61时压到0.61，没有−0.61下界。不能写成对称clip。差分滤波器输入0x1260/0x1268，结果经0.577/母线平均缩放后才截±0.1。

正序dq电压参考为前馈项减正序PI输出/0x80，以+角度反Park；负序为负PI输出/0x80，以−角度反Park。前馈分母未在此加零值保护，启动阶段通过流程避开该计算而不是在除法处钳位。

$$B_j = \min\left(0.577V_{\mathrm{ff},j} / \bar{V}_{\mathrm{dc}}, 0.61\right), \quad C_j = \mathrm{clip}\left(0.577y_{\mathrm{diff},j} / \bar{V}_{\mathrm{dc}}, -0.1, 0.1\right) \tag{7-45}$$

$$v_{dq}^{+*} = B_{dq} - u_{\mathrm{PI}}^{+} / S_{\mathrm{pu}}, \quad v_{dq}^{-*} = -u_{\mathrm{PI}}^{-} / S_{\mathrm{pu}} \tag{7-46}$$

recovered_grid.cpp L971-L990

```
    s.d(0xB0, (s.d(0x1220) / s.d(0x1B8)) * 0.577);
    s.d(0xB8, (s.d(0x1228) / s.d(0x1B8)) * 0.577);
    if (s.d(0xB0) > 0.61) s.d(0xB0, 0.61);
    if (s.d(0xB8) > 0.61) s.d(0xB8, 0.61);
    s.d(0x1548, s.d(0x1260));
    s.d(0x1588, s.d(0x1268));
    difference_filter(s.at(0x1528));
    difference_filter(s.at(0x1568));
    s.d(0x540, (0.577 * s.d(0x1558)) / s.d(0x1B8));
    s.d(0x548, (0.577 * s.d(0x1598)) / s.d(0x1B8));
    s.d(0x540, clip(s.d(0x540), -0.1, 0.1));
    s.d(0x548, clip(s.d(0x548), -0.1, 0.1));
    s.d(0x1498, s.d(0xB0) - s.d(0xF58) / s.d(0x80));
    s.d(0x14A0, s.d(0xB8) - s.d(0xFD8) / s.d(0x80));
    s.d(0x1490, s.d(0x218));
    s.apply(0x1480, inverse_park_transform);
    s.d(0x14C0, (-s.d(0x1058)) / s.d(0x80));
    s.d(0x14C8, (-s.d(0x10D8)) / s.d(0x80));
    s.d(0x14B8, -s.d(0x218));
    s.apply(0x14A8, inverse_park_transform);
```

min公式限于普通有限数；实现仍以严格>分支为准。

7.2.16 调制输入、缓存与PWM阶段切换

调制结构装入三相瞬时电流以及正负直流分段电压。正负序反Park得到的αβ参考先相加，再分别加差分修正，调用modulation后拷贝六路输出缓存0xDF0至0xE18到0x4C0、0x4C8、0x4D8、0x4E0、0x4F0、0x4F8。

0x31C=1分支也置PWM使能0x50C=1；当配置0x28>1时先清q给定、五个调节器积分状态及有功限幅斜坡，并禁用0x158，然后先增0x24、以>0x28结束该阶段、恢复0x158。积分清除发生在本拍调制计算之后，会影响后续拍。否则正常分支在1020—1023行同时置0x50C与0x2C。

$$v_{\alpha\beta}^{*} = v_{\alpha\beta}^{+*} + v_{\alpha\beta}^{-*} + C_{\alpha\beta} \tag{7-47}$$

recovered_grid.cpp L993-L1016

```
    s.d(0xEC0, s.d(0xD8));
    s.d(0xEC8, s.d(0xE0));
    s.d(0xED0, s.d(0xE8));
    s.d(0xDC8, s.d(0x100));
    s.d(0xDD0, s.d(0x108));
    s.d(0xD00, (s.d(0x1480) + s.d(0x14A8)) + s.d(0x540));
    s.d(0xD08, (s.d(0x1488) + s.d(0x14B0)) + s.d(0x548));
    s.apply(0xD00, modulation);
    s.d(0x4C0, s.d(0xDF0));
    s.d(0x4C8, s.d(0xDF8));
    s.d(0x4D8, s.d(0xE00));
    s.d(0x4E0, s.d(0xE08));
    s.d(0x4F0, s.d(0xE10));
    s.d(0x4F8, s.d(0xE18));
// ---- 软启动使能 (0x31C) ：启动阶段 PWM 使能 1，超过 0x28 步后转正常 ----
    if (s.i(0x31C) == 1) {
        s.i(0x50C, 1);
        if (s.i(0x28) > 1) {
            for (std::size_t offset : {0x60, 0xF30, 0xFB0, 0x1030, 0x10B0, 0x1130, 0x138})
s.d(offset, 0.0);
            s.i(0x158, 0);
        }
        s.increment(0x24);
        if (s.i(0x24) > s.i(0x28)) {
            s.i(0x24, 0);
```

recovered_grid.cpp L1017-L1019

```
            s.i(0x31C, 0);
            s.i(0x158, 1);
        }
```

计数0x24从0开始且0x28=20时，第21次该分支结束；运行门在随后到达正常分支时才置1。六路缓存不应与跳闸清零的三路调制量混称。

7.2.17 停机复位与最终运行标志

到达停机尾段且0x350=2时，先增0x358，再以严格>0x35C整块清零0x1AB8字节、调用grid_defaults重写默认。从零起、0x35C=800时第801次尾段执行才复位；仅考虑该计数相当于200.25ms，外部停机命令首次被post识别还带有调度时序。

final_flags对所有路径执行，依据当时状态1或2置0x360=1，其余为0。停机过程允许完整控制计算，直至尾段复位；post在此后还可能处理命令，因此输出可见状态必须按完整调度解释。

recovered_grid.cpp L1020-L1036

```
    } else {
        s.i(0x50C, 1);
        s.i(0x2C, 1);
    }
// ---- exit_tail: 停机过程 (0x350==2) 持续 0x35C 步 (0.2 秒) 后整块复位 ----
exit_tail:
    if (s.i(0x350) == 2) {
        s.increment(0x358);
        if (s.i(0x358) > s.i(0x35C)) {
            std::memset(state, 0, 0x1AB8);
            grid_defaults(f, state);
        }
    }
// ---- final_flags: 0x360 = 运行标志 (1/2 → 1) , 供输出与监控使用 ----
final_flags:
    s.i(0x360, s.i(0x350) == 1 || s.i(0x350) == 2 ? 1 : 0);
}
```

复位重新写默认值后，final_flags读的是新状态0。

第8章　机侧变流器 RSC 控制

8.1 机侧控制链与当前定值

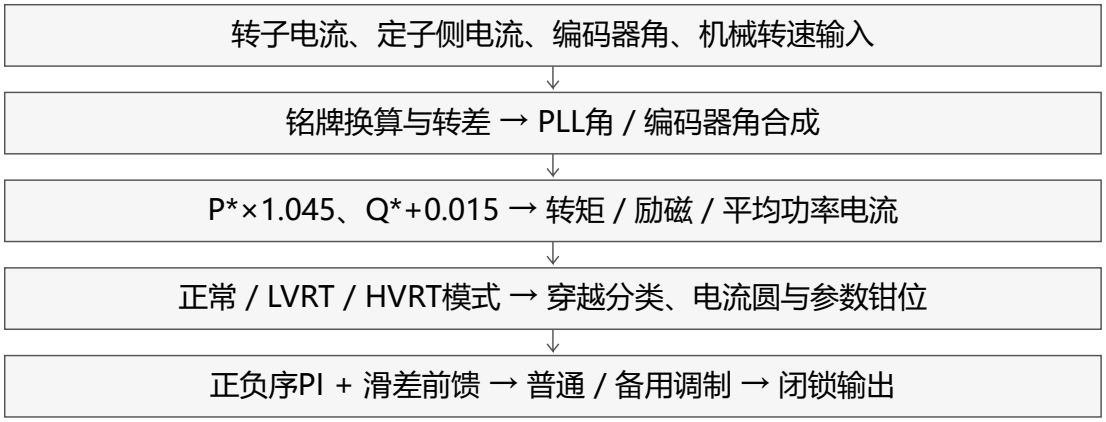


图 8　机侧主链与功率、励磁、电流参考的关系

机侧公式中的控制电压尺度通常为局部0x410=1611.96V，三相基波提取尺度约930.69V；按变量读者与对应偏移区分。

当前K_R,HV由包装别名得到约1.6678969；与记录“p19无槽所以为0”的推论相反。Control73=2采用常规励磁公式分支；保持寄存器属于另一分支。Control75=3的特殊恢复参考可能来自动态P_pos均值，不能只看静态初值0就认定永远休眠。

控制实现与逐拍关系

输入、转差频率与换算系数

转子输入含三相转子电流、1140V低压接入测量电流、编码器角、转速。转差频率：

$$f_{slip} = \frac{np}{60} - f_{\text{base}} \tag{8-1}$$

换算使用Vs=d(0x6D0)=定子额定电压×1000、Vr=d(0x6D8)=转子额定电压×1000、B=d(0x6C8)独立参数、P=d(0x598)额定功率、n=d(0x6E0)转速、p=d(0x1E0)极对数；0xB28为独立内部参数。0x6D8不是转子电流。

$$k_P = \left(\left((V_s \,/\, 1.732) \,/\, (V_r \,/\, V_s) \right) \,/\, B \right) 1.414 \tag{8-2}$$

$$k_T = \left(\left(\left(((-P \times 3000) \,/\, n) \,/\, p \right) \,/\, V_r \right) \,/\, 1.732 \right) 1.414 \left(d_{0\text{xB28}} \,/\, f_b \right) \tag{8-3}$$

$$k_\psi = ((P \,/\, V_r) \,/\, 1.732) 1.414 \tag{8-4}$$

系数每拍按当前铭牌及转速重算；本工程Vs=1140V、Vr=3541V、P=4000000W，B独立保留。初始化362.43648等默认值不代表运行重算值必相等；代入应明确B、n、p、0xB28和f_base，并按源码顺序求值。

机侧电网锁相与基波提取

机侧 PLL 使用主入口从网侧复制的三路线电压，在机侧局部 0x258/0x260/0x268 读取，得到电网相位角（0x1E8）及正序滤波幅值（0x1040/0x1048 的平方和开方）。0x1088 是独立槽位，不能把它等同 PLL 算得的负序幅值。转差角度（用于序分解，θ_enc 为编码器角，0x1F0 = 3.0665 为相位补偿角）：

$$\theta_p = \theta_r - \theta_{enc} + \theta_{comp} \tag{8-5}$$

$$\theta_n = -\theta_r - \theta_{enc} + \theta_{comp} \tag{8-6}$$

三相基波提取同网侧（自适应振荡器 + 两态滤波器，增益 = 2·ω_PLL），标幺基准 0x1A20 = 930.69，三相最大/最小值供机侧 LVRT/HVRT 判定。

转子电流正负序分解

两套序分解：0x11F8处理转子电流并用θp/θn，0x12F0处理0x240/248/250三相电压并用±θPLL，滤波系数见 4.7 节机侧参数。结果按启动标志 0x1C 与选择位 0x728 取正序 d/q（0x2C0/0x2C8）与负序 d/q（0x2D0/0x2D8）。

启动与预同步

前400次调用在PLL、幅值及序分解后跳过公共控制尾段；预同步以本拍重算的0x1C8作目标，累计A8==A0的精确相等观察，计数到200后的下一次相等才置ready并复制PI。不相等不清计数。之后100次调用仍执行公共斜坡、PI和调制，再进入正常参考和穿越判断。

转矩/励磁电流给定

命令入口每拍接收新的P*、Q*。机侧按输出模式把本拍P*乘1.045（标准）或1.04（备用），把本拍Q*加0.015作为参考；这不是在上一拍参考上无限递增。

$$i_T = T^* k_T \tag{8-7}$$

$$i_\psi = Q^* k_\psi \tag{8-8}$$

负序预判标志0x3E0/0x3F8按各自门槛进入/退出；计数小于20时递增，已到20的下一次满足才更新，从零需第21次满足调用。输入0x1088是否实际刷新须另查。

机侧 LVRT 控制

机侧LVRT初始化门槛0.854/0.942与系数1.8会在参数拍被覆盖。当前两侧进入/退出参数门槛837.624/884.158V，机侧LVRT换算系数2.315876；进入计数阈值20，退出计数阈值4，严格比较的实际动作拍须核对源码。恢复保持与模式门控分别执行。

--

$$\Delta P = 1 - \frac{V_{d,rot}}{V_{\text{base}}} \tag{8-9}$$

$$\Delta V = 0x6F0 - \frac{V_{mag,rot}}{V_{\text{base}}} \tag{8-10}$$

电压偏差按选择位使用30/1024滤波值或瞬时值，乘活动系数0x418（默认1.8，参数入口可覆盖）；需求大于0.9863限到0.9863，小于0时写0xB68，再乘989.8A：

$$\Delta V_{f,k} = \frac{30\Delta V_k + 994\Delta V_{f,k-1}}{1024} \tag{8-11}$$

$$z_{R,k} = K_{R,LV}\Delta V_{f,k}, \quad i_{R,LV} = I_R \begin{cases} c_{R,\,\min}, & z_{R,k} < 0, \\ 0.9863, & z_{R,k} > 0.9863, \\ z_{R,k}, & \text{otherwise}. \end{cases} \tag{8-12}$$

有功（d 轴）电流上限按电流圆约束（$1.1^2 = 1.21$），限幅 [212.1, 707.0]：

$$i_{d,\max} = \text{clip}\left(\sqrt{1.21 - i_{q,n}^2}\, I_{\text{base}}, 212.1, 707.0\right) \tag{8-13}$$

负序补偿电流：负序电压标幺限幅 ±0.48828125 后乘系数 ±0x630，经 30/1024 低通；当过压预判 0x3E0 与使能 0x668 同时成立时乘电流基准输出并限幅 ±494.9。

功率环：功率电流给定 = 80 步平均功率 + 有功上限 + 励磁项（0x700由参数入口写入p[27]，不能按零值删去该项），限幅 ≤ I_base；无功上限按电流圆重算：

$$I_A^* = \min\left(I_{avg} + d(0x460) + i_\psi d(0x700), I_b\right) \tag{8-14}$$

$$i_{q,\max} = \sqrt{1.21 I_{\text{base}}^2 - i_p^{*2}} \tag{8-15}$$

励磁电流给定被限制在 [−i_q,max, 0]（上限槽 0xB90 保持 0，即本工程励磁给定非正）。

LVRT档位状态机0x5D4按电压跌落量ΔV=1−0x1040/0x410与0x57C有功命令阈值标志分类。0x57C在P参考修正前按P*≥0.6置位，不代表物理并网状态：

表 32　状态分支与定值

分支条件	并网（57C=1）档位	脱网（57C=0）档位
正常（负序小、电压正常）	ΔP>0.6→121；0.35<ΔP<0.45→481；0.1<ΔP<0.15→861	ΔP>0.8→122；ΔP>0.4→482；0.1<ΔP<0.15→862
异常（过压预判或负序超限）	ΔP>0.41→123；>0.18→483；>0.12→703；>0.1→863	ΔP>0.4→124；>0.22→484；>0.15→704；>0.05→864

每档允许“条件成立或当前编码相同”，但整条if/else if按先后优先；前面的新条件可覆盖后面的旧档位。编码记录穿越分类，当前读写证据没有证明器件动作。

机侧 HVRT 控制

机侧HVRT初始化1.123/1.02、系数1.5与运行值不同。当前参数拍重写门槛1051.683/1005.148V，并从别名p[19]=Control_19=2.2841得到K_R,HV≈1.6678969。滤波抬升乘活动系数后限[0,0.3418]，取负乘I_R=989.8A；负序附加量0x5E8虽被计算，源码未叠加到需求；参数钳位区间[-424.240,-40.104]A。需求不恒等于-40.104A。

$$I_{H,raw} = -I_R \ \mathrm{clip}\ (K_{R,HV}\Delta V_f, 0, 0.3418) \tag{8-16}$$

机侧HVRT参数钳位区间为[-424.240,-40.104]A，历史保持另行更新；有功余量由实际无功需求与I_R的电流圆计算。

$$i_{d,\max} = \sqrt{I_{\mathrm{base}}^2 - i_q^2} \tag{8-17}$$

HVRT档位状态机0x5D8按正序向量幅值标幺（正常分类）或0x1088（异常分类），结合有功命令等级0x57C编码：

表 33　状态分支与定值

分支条件	并网（57C=1）档位	脱网（57C=0）档位
正常（负序幅值 < 42.42）	V_pu>1.2→1361；>1.16→1241；>1.05→1131	V_pu>1.25→1362；>1.2→1242；>1.05→1132
异常（过压预判或负序超限）	V_neg>219.17→1363；>169.68→1243；>98.98→1133	V_neg>226.24→1364；>169.68→1244；>113.12→1134

正常模式

正常模式的励磁相关电流限幅取参数入口更新后的机侧局部0x60，当前为829.288A；494.9A仅初始化。随后按电压比折算，功率电流合成叠加平均功率与励磁折算值，穿越分类编码清零。

$$i_\psi^* = \mathrm{clip}\ (i_\psi, -829.288, 829.288)\ \frac{V_{\mathrm{base}}}{V_{d,\mathrm{rot}}} \tag{8-18}$$

$$i_p^* = P_{\mathrm{avg}} + i_\psi^* \tag{8-19}$$

滑差相关的中点平衡极性与增益

普通调制以0xBF0为基址；0xD98映射polarity，0xD10映射gain。滑差大于0.2Hz选择正极性和增益5.0，滑差小于−2Hz选择负极性和增益5.0，其余区间含等号将增益置0并保留极性。5.0是增益，不是5秒延时。这些字段用于直流母线中点电压修正，不能据此推断撬棒投退。备用调制不读取这两个字段。

谐波抑制

对正序原始 dq 电流做 0.01/0.99 低通，残差过递归滤波器后乘增益 0x740/0x748，限幅 ±5%×|i_q*|；仅在0x750不为0且正常模式下保留注入量：

$$y_k^{lp} = 0.01x_k + 0.99y_{k-1}^{lp} \tag{8-20}$$

$$B_h = \text{clip}\left|G_h y_k^{res}, -0.05\left|i_q^*\right|, 0.05\left|i_q^*\right|\right| \tag{8-21}$$

电流内环与前馈

四个电流环 PI：q 轴（给定 0xB0+谐波补偿，反馈 0x2C0）、d 轴（给定 0xA8+谐波补偿，反馈 0x2C8）、负序 d/q（给定 0x658/0x660，反馈 0x2D0/0x2D8；过压标志 0x3F8 == 1 时清零输出）。电压前馈（共享电网电压正序 dq、变比与转差缩放；按实际写入槽位限幅）：

$$\begin{gathered} a = 0.577\frac{V_d^+ k_v}{V_{dc}}\frac{f_{slip}}{50}, \quad b = 0.577\frac{V_q^+ k_v}{V_{dc}}\frac{f_{slip}}{50}, \\ v_1 = \min(a, 0.577), \\ v_2 = \text{clip}\left(\begin{cases} -0.577, & a < -0.577, \\ b, & a \geq -0.577, \end{cases} -0.577, 0.577\right). \end{gathered} \tag{8-22}$$

负序补偿（限幅 ±0.1）：

$$v_c = \text{clip}\left(\frac{0.577\left(-\text{V}_{rot,neg}\right)k_{\psi r}\left(2 + f_{slip}/50\right)}{V_{\text{dc}}}, -0.1, 0.1\right) \tag{8-23}$$

两组谐振滤波器（中心 2π×350 rad/s）并联。残差滤波器谐波抑制量（增益 0.3）：正序残差源经递归滤波后乘 0.3，标幺化（×√3×0.577/V_dc），限幅 ±0.2。

调制波合成、调制与过流保护

正序电压参考 = 前馈 + q/d 轴电流环输出/标幺基准（0x358 = 8388608）+ 谐波抑制量 + 谐振输出/基准，反 Park（转差角 θ_p）；负序电压参考 = 负序补偿 + 负序电流环输出/基准，反 Park（θ_n）。两者相加送调制模块（0x6E8为0或2时用普通调制，其它值用备用调制）：

$$u_1^+ = v_1 + u_{PI,1}/S + B_{6A0} - B_{6B0} + y_{r1}/S, \quad u_2^+ = v_2 + u_{PI,2}/S - B_{698} + B_{6A8} + y_{r2}/S \tag{8-24}$$

$$v_{\alpha,\text{ref}} = v_{\alpha,\text{ref}}^+ + v_{\alpha,\text{ref}}^- \tag{8-25}$$

过流保护：任一相转子电流绝对值超过 5×I_base，或直流电压超过 3000，则闭锁输出（输出使能 0x350 = 0，电流给定清零）。停机计数严格大于800才整块复位；从零第801个停机调用动作。

$$K_{R,HV} = \frac{3}{4}r_e(19) \simeq 1.6678969 \tag{8-26}$$

$$I_H = \text{clip}(I_{H,hist}, -424.240, -40.104), \quad I_{H,raw} = -989.8\,\text{clip}(K_{R,HV}\Delta V_f, 0, 0.3418) \tag{8-27}$$

此式给出需求层的钳位结构；电流圆、历史保持、励磁合成和调制仍在后续环节作用。不能由此直接断言端子电流达到该值。

RSC启动与参考

表 34　RSC启动与参考：参数与状态关系

条件（每拍检查）	新状态	含义
网侧跳闸联动位 0x4F8 = 1	3 跳闸	网侧已跳闸，机侧跟着跳；跳闸后所有输出清零
0x4FC = 1 且 启动位 0x4F4 = 0	2 停机中	收到停机指令，进入"停机过程中"，持续 800 拍（0.2 s）后整块复位、重写默认值
0x4FC = 0 且 启动位 0x4F4 = 1	1 运行	收到启动指令，进入运行

RSC电流合成与门槛

表 35　RSC电流合成与门槛：参数与状态关系

项目	**LVRT（模式 1）**	**HVRT（模式 2）**	说明
进入门槛	Vmin < 837.62 V 且 $\|V^+\| \le$ 1611.96 V 且 HVRT 保持窗未激活（0x4C4 = 0）	Vmax > 1051.68 V 且 $\|V^+\| >$ 1611.96 V 且惯量响应计时器 0x580 = 0	门槛 = Control_13/14/17/18 × 930.6928 V，与网侧同值（见下方注）
退出门槛	Vmin > 884.16 V	Vmax < 1005.15 V（"$\|V^-\| <$ 56.56 V"因输入 0x1088 恒 0 恒成立；"< 949.31 V"分支不生效）	滞回带防抖动
确认拍数	进入阈值 20（第 21 拍）、退出阈值 4（第 5 拍）	进入 / 退出均阈值 4（第 5 拍）	计数阈值 20 / 4：连续满足（中断即清零重数），从首次满足起第 21 / 5 拍切换
使能条件	LVRT 使能命令 = 1（0x534）	HVRT 使能命令 = 1（0x538）	来自命令第 4/5 路
最少保持	进入后最少保持 40 拍（10 ms）：0x4C0 / 0x4C4 = 1 期间退出条件不评价；HVRT 保持窗同时屏蔽 LVRT 进入		0x4C8 / 0x4CC 计数到 0x4D4 / 0x4D0 = 40

RSC内环、前馈与中点平衡

表 36　RSC内环、前馈与中点平衡：参数与状态关系

回路	比例 **kp**	积分 **ki**	跟踪增益 **ki/kp**	参数来源
正序 d 轴	3500	15	0.004286/拍	Control_9 / Control_10

回路	比例 kp	积分 ki	跟踪增益 ki/kp	参数来源
正序 q 轴	3500	15	0.004286/拍	Control_9 / Control_10
负序 d 轴	4500	25	0.005556/拍	Control_11 / Control_12
负序 q 轴	4500	25	0.005556/拍	Control_11 / Control_12

8.2 机侧源码逐段对照

依照三个导出函数的实际读写与执行顺序说明机侧控制。局部地址以0x1AB8为基址；默认值、参数拍覆盖、本拍读取和后写覆盖分别说明。注释与执行语义冲突时以可执行表达式为准。

8.2.1 默认值、参数覆盖和全区复位边界

motor_defaults只写指定默认字段，不先清空状态。全区0x1C68字节清零由外部初始化或post/停机复位的memset实现；不能根据一次孤立defaults调用推断全部积分器、历史值归零。机侧局部地址相对总状态区0x1AB8。

默认等待阈值400、800、800拍，电流基准989.8，参考限值0x50=940.31、0x60=494.9，瞬时过流4949，直流总电压阈值3000。活动参数可能被每拍参数入口重写，默认值不等于运行值。

四个PI活动比例/积分默认正序22000/150、负序6000/60，初始化另保存6000/60就绪复制来源；输出限幅±0.577×8388608，积分比例ki/kp。默认0x1190/11D8是PLL调节器参数，不是四个电流内环。

三组振荡器默认2π×50，三组幅值滤波器步长250μs。LVRT门槛0.854/0.942、HVRT门槛1.123/1.02乘930.6928406466512；负序预判门槛0.088/0.06、0.08/0.02则乘1611.96。两种电压尺度不得混用。默认穿越增益1.8/1.5。

RMS窗口0xE4/1AC在224–225按额定频率计算并截断；CVTT非法或超范围转换保留INT_MIN硬件结果。50Hz窗口为80拍。

$$T_s = 0.00025\ s,\quad I_b = 989.8,\quad I_{\mathrm{trip}} = 5I_b = 4949 \tag{8-28}$$

$$N_f = \ \mathrm{cvtt}_{32}\left(\frac{1\ /\ f_b}{T_s}\right) \tag{8-29}$$

recovered_motor.cpp L113-L136

```
    s.i(0x8, 400);
    s.i(0xC, 800);
    s.i(0x504, 800);
    s.i(0x4FC, 0);
    s.d(0x358, 8388608.0);
    s.d(0x1F8, 22000.0);
    s.d(0x200, 150.0);
    s.d(0x208, 6000.0);
    s.d(0x210, 60.0);
    s.d(0x218, 6000.0);
    s.d(0x220, 60.0);
    s.d(0x1E0, 2.0);
    s.i(0x1C, 0);
// 更多基准与限幅：0x1F0 = 3.0665（转矩系数类）、0x50 = 0.95×989.8、
// 0x60 = 0.5×989.8、0x40 = 5×989.8（过流门槛）、0x28 = 3000；
// 0x1190/0x11D8 = 电流环 kp 0.12 / ki 0.0012；0x11B8/0x11C0 = ±2π×75 限幅。
    s.d(0x1F0, 3.0665434957540336);
    s.d(0x48, 989.8);
    s.d(0x1C8, 362.43647999999996);
    s.d(0x1C0, s.d(0x1C8));
    s.d(0x50, s.d(0x48) * .95);
    s.d(0x60, s.d(0x48) * .5);
    s.d(0x378, 2.828);
    s.d(0x370, 14.139999999999999);
```

recovered_motor.cpp L137-L140

```
    s.d(0x360, 2.828);
    s.d(0x368, .42419999999999997);
    s.d(0x40, 5.0 * s.d(0x48));
    s.d(0x28, 3000.0);
```

motor_defaults完整104–261；数值表必须分初始化与参数覆盖两列。

8.2.2 350Hz谐振参数与默认零增益

两组谐振器基址0x14A8、0x1560，中心2199.114857512853rad/s即350Hz，带宽62.8318530717958rad/s，步长250μs，输出限幅±419430.4即±0.05标幺。源码注释300Hz与执行常数冲突，按常数说明。

0x1528、0x15E0分别为相对0x80增益，默认均为0。resonant_filter驱动系数与此增益相乘。零初态且无外部改写时，支路虽在最终调制合成中实接入，仍无补偿贡献；孤立defaults未清历史时不能保证瞬时输出为零。

$$\omega_0 = 2\pi \cdot 350 \approx 2199.1148575 \text{ rad / s} \tag{8-30}$$

$$b_r = \frac{4K_r\omega_c\omega_0}{D}, \quad K_r = 0 \text{ at defaults} \tag{8-31}$$

recovered_motor.cpp L249-L260

```
    s.d(0x14B0, 2199.114857512853);
    s.d(0x14B8, 62.8318530717958);
    s.d(0x1528, 0.0);
    s.d(0x14A8, .00025);
    s.d(0x1550, .05 * s.d(0x358));
    s.d(0x1558, -.05 * s.d(0x358));
    s.d(0x1568, 2199.114857512853);
    s.d(0x1570, 62.8318530717958);
    s.d(0x15E0, 0.0);
    s.d(0x1560, .00025);
    s.d(0x1608, .05 * s.d(0x358));
    s.d(0x1610, -.05 * s.d(0x358));
```

公共算法增益与历史更新见recovered_primitives.cpp:724–740。

8.2.3 后处理状态机、跳闸清理与复位优先级

post先读网侧联锁4F8，等于1直接置状态3；否则状态1在启动命令4F4撤销时改2，状态0在启动命令为1时改1。状态2不因启动命令自动改1，状态3也不靠启动命令复归。

此时状态3就清模式10、ready1C、状态3A0/3A4，以及列举的使能、输出与参考字段。它不是整块清零，也不把六路占空比字段全部擦除。之后若复位命令53C==1，memset全区、defaults并return，本拍RMS与末尾故障判断跳过。

$$0 \xrightarrow{\text{Start}=1} 1, \quad 1 \xrightarrow{\text{Start}=0} 2, \quad \text{GridTrip} = 1 \Rightarrow 3 \tag{8-32}$$

recovered_motor.cpp L274-L297

```
extern "C" __declspec(dllexport) void motor_post(double p, void* state) {
    MotorState s(state);
    if (s.i(0x4F8) == 1) {
        s.i(0x4FC, 3);
    } else if (s.i(0x4FC) == 1 && s.i(0x4F4) == 0) {
        s.i(0x4FC, 2);
    } else if (s.i(0x4FC) == 0 && s.i(0x4F4) == 1) {
        s.i(0x4FC, 1);
    }

    if (s.i(0x4FC) == 3) {
        s.i(0x10, 0);
        s.i(0x1C, 0);
        s.i(0x3A0, 0);
        s.i(0x3A4, 0);
        const std::size_t clear_offsets[] = {
            0x350, 0x348, 0x300, 0x318, 0x330,
            0x2F0, 0x2F8, 0xC78, 0xC80, 0xC88
        };
        for (std::size_t offset : clear_offsets) {
            s.d(offset, 0.0);
        }
    }
```

recovered_motor.cpp L298-L301

```
    if (s.i(0x53C) == 1) {
        std::memset(state, 0, 0x1C68);
        motor_defaults(p, state);
        return;
```

状态机发生在calculate之后；本拍改变的运行状态由后续调用读取。

8.2.4 RMS实际输入、采样计数与滑差零点

第一组九路输入258/260/268、240/248/250、19B8/19C0/19C8，在304–321先递增计数并累计平方，达到1AC后用实际计数结算全部RMS，再统一清全部累加器，清计数并置完成位1B0。

第二组输入实际是转子电流228/230/238，输出148/150/158，不能因注释而称转子电压。窗口由滑差5F8计算：仅0<f<0.1钳0.1、−0.1<f<0钳−0.1；恰好0不钳位。零点倒数无穷，经CVTT得到INT_MIN，已递增计数可立即满足比较。

窗口每拍可改变，RMS分母仍取结算时实际采样计数。±0.1Hz窗口40000拍即10s；不能描述为每种低频均自动选择这个窗口。

$$X_{\mathrm{rms}} = \sqrt{\frac{\sum_{j=1}^{N_{\mathrm{actual}}} x_j^2}{N_{\mathrm{actual}}}} \tag{8-33}$$

$$N_{\mathrm{slip}} = \mathrm{cvtt}_{32}\left(\left|\frac{1}{f_{\mathrm{slip}} T_s}\right|\right) \tag{8-34}$$

recovered_motor.cpp L323-L341

```
    const RmsRow rows_b[] = {
        {0x228, 0x160, 0x148},
        {0x230, 0x168, 0x150},
        {0x238, 0x170, 0x158}
    };
    s.increment(0x1B4);
    accumulate(s, rows_b);
    double x = s.d(0x5F8);
    if (x < .1 && x > 0.0) {
        x = .1;
    } else if (x > -.1 && x < 0.0) {
        x = -.1;
    }
    s.i(0x1B8, cvtt_i32(std::fabs((1.0 / x) / .00025)));
    if (s.i(0x1B4) >= s.i(0x1B8)) {
        finish_rms(s, rows_b, s.i(0x1B4));
        s.i(0x1B4, 0);
        s.i(0x1BC, 1);
    }
```

工具77–95保证所有输出先写完才清平方和。

8.2.5 后写故障、4秒延时与下一次后处理清输出

934的bit0优先，置位即置状态3；否则bit1且20==1时938加250μs，达到4s后置3并饱和计时到4；bit1且20不为1则立即置3。两位都未置时才清938，bit0分支不清该计时器。

这段晚于前面的跳闸清理，末尾新置状态3不回跳清理块，需要下一次post才清前面列举的输出。若本拍其它保护已经闭锁须另查，不能把状态3写入与当拍所有输出清零等同。

$$t_{2,k} = \min\left(t_{2,k-1} + T_s, 4\right) \quad (\text{bit1} = 1,\ 0x20 = 1) \tag{8-35}$$

recovered_motor.cpp L343–L359

```
// ---- 故障锁存：本拍先置状态，输出清零要等下一拍（原 DLL 的行为，原样保留） ----
    // Late faults latch now; output clearing waits until the next call.
    if ((s.i(0x934) & 1) != 0) {
        s.i(0x4FC, 3);
    } else if ((s.i(0x934) & 2) != 0) {
        if (s.i(0x20) == 1) {
            s.d(0x938, s.d(0x938) + .00025);
            if (s.d(0x938) >= 4.0) {
                s.i(0x4FC, 3);
                s.d(0x938, 4.0);
            }
        } else {
            s.i(0x4FC, 3);
        }
    } else {
        s.d(0x938, 0.0);
    }
```

局部20为调制成功的运行标志，与4FC、ready各异。

8.2.6 每拍铭牌换算、PLL参数和命令顺序

先重算kP=1C8、kT=548及无功系数558/560，再将5A0/5B0刷新到PLL参数1190/11D8。kT含转速、极对数、转子额定电压与B28/base_frequency，分母为零无替代保护。

PLL输出限幅按本拍进入时的模式选择：模式1为±2π×1.25，其余±2π×10。后面的穿越判定本拍改变模式不追溯改此限幅，参数入口中的KI/10同样需结合调度解释次拍影响。

先以未修正P参考2A0≥0.6写57C，再按6E8==1将P乘1.04，其余值乘1.045，Q均加0.015。57C为有功命令阈值标志，不是断路器物理并网状态。命令入口每拍重写目标时才等价于修正新命令；孤立连续calculate会累乘/累加。

$$k_P = \frac{V_s / 1.732}{V_r / V_s}\frac{1.414}{B} \tag{8-36}$$

$$k_T = \frac{(-P_b)3000}{n_p V_r 1.732}1.414\,\frac{d(0xB28)}{f_b} \tag{8-37}$$

$$k_{\psi,+} = k_{\psi,-} = \frac{P_b}{V_r 1.732}1.414 \tag{8-38}$$

recovered_motor.cpp L413-L435

```
    s.d(0x1C8, (((s.d(0x6D0) / 1.732) / (s.d(0x6D8) / s.d(0x6D0))) / s.d(0x6C8)) * 1.414);
    s.d(0x548, ((((((-s.d(0x598)) * 3000.0) / s.d(0x6E0)) / s.d(0x1E0)) / s.d(0x6D8)) / 1.732) *
1.414 * (s.d(0xB28) / base_frequency));
    s.d(0x558, ((s.d(0x598) / s.d(0x6D8)) / 1.732) * 1.414);
    s.d(0x560, ((s.d(0x598) / s.d(0x6D8)) / 1.732) * 1.414);
    s.d(0x1190, s.d(0x5A0));
    s.d(0x11D8, s.d(0x5B0));
// ---- 电流环 PI 参数与限幅刷新（按模式 0x10 选 ±2π×1.25 或 ±2π×10） ----
    if (s.i(0x10) == 1) {
        s.d(0x11B8, 7.853981633974475);
        s.d(0x11C0, -7.853981633974475);
    } else {
        s.d(0x11B8, 62.8318530717958);
        s.d(0x11C0, -62.8318530717958);
    }
    if (s.d(0x2A0) >= 0.6) s.i(0x57C, 1);
    else if (0.6 > s.d(0x2A0)) s.i(0x57C, 0);
    if (s.i(0x6E8) == 1) {
        s.d(0x2A0, s.d(0x2A0) * 1.04);
        s.d(0x2A8, s.d(0x2A8) + 0.015);
    } else {
        s.d(0x2A0, s.d(0x2A0) * 1.045);
        s.d(0x2A8, s.d(0x2A8) + 0.015);
    }
```

rotor_input已在外层写本拍滑差；固定50Hz前馈另见后节。

8.2.7 PLL与三相基波幅值的实际数据源

PLL输入258/260/268复制到FF8开始的PllState，调用pll后1018作为角度，3A8由1040/1048平方和开方。电压物理来源必须结合controller跨区搬移确认，不能把机侧局部所有电压都称转子电压。

三组自适应振荡器实际输入19B8/19C0/19C8；相对68输出送两态滤波器，滤波器参数每拍设2×PLL角速度1010。三相幅值既保留有名值又除1A20标幺化；1A00/1A08为有名最大/最小，1A10/1A18为标幺最大/最小。穿越判断使用有名最大/最小。

$$V_{+} = \sqrt{d(0x1040)^2 + d(0x1048)^2} \tag{8-39}$$

$$V_{j,\mathrm{pu}} = d(0x19D0 + 8j) \,/\, d(0x1A20) \tag{8-40}$$

recovered_motor.cpp L437-L460

```
    s.d(0xFF8, s.d(0x258));
    s.d(0x1000, s.d(0x260));
    s.d(0x1008, s.d(0x268));
    {
        PllState p;
        std::memcpy(&p, s.at(0xFF8), sizeof(p));
        pll(base_frequency, &p);
        std::memcpy(s.at(0xFF8), &p, sizeof(p));
    }
    s.d(0x1E8, s.d(0x1018));
    s.d(0x3A8, std::sqrt(s.d(0x1040) * s.d(0x1040) + s.d(0x1048) * s.d(0x1048)));
// ---- 第 3 步：三相自适应振荡器 + 两态滤波器（转子电压基波提取） ----
    for (int phase = 0; phase < 3; ++phase) {
        const std::size_t adaptive = 0x1790 + 0xB8 * phase;
        const std::size_t filter = 0x1688 + 0x58 * phase;
        s.d(adaptive + 0x40, s.d(0x19B8 + 8 * phase));
        adaptive_oscillator(s.at(adaptive), base_frequency);
        s.d(filter + 0x10, s.d(adaptive + 0x68));
        s.d(filter, s.d(0x1010) * 2.0);
        s.apply(filter, two_state_filter);
        s.d(0x19D0 + 8 * phase, s.d(filter + 0x18));
        s.d(0x19E8 + 8 * phase, s.d(0x19D0 + 8 * phase) / s.d(0x1A20));
    }
    s.d(0x1A00, maximum(s.d(0x19D0), s.d(0x19D8), s.d(0x19E0)));
```

recovered_motor.cpp L461-L463

```
    s.d(0x1A08, minimum(s.d(0x19D0), s.d(0x19D8), s.d(0x19E0)));
    s.d(0x1A10, maximum(s.d(0x19E8), s.d(0x19F0), s.d(0x19F8)));
    s.d(0x1A18, minimum(s.d(0x19E8), s.d(0x19F0), s.d(0x19F8)));
```

本恢复PLL路径没有自动将负序dq平方和写入1088，不能推定该字段自动刷新。

8.2.8 母线平均、角度合成与控制门控

278为两路母线280+288，C8先加计数并累加，达到CC后用CC作平均分母、清计数和累计值，默认80拍。D8虽累计1040，E4窗口结束只清计数/累计值，不写平均输出；D0每拍直接复制1040。

编码器角290先一次加减2π，合成θp=θPLL−θenc+θcomp和θn=−θPLL−θenc+θcomp，再各一次加减。wrap不是fmod，多圈越界一次调用后未必归一。

只有4FC为1或2执行后续序分解、启动、穿越和公共控制；0/3直接跳final_status。前面的换算、PLL和幅值仍执行，goto本身不清输出。528只把状态1/2映射1.0，不等于350输出使能或ready。

$$\theta_p = \theta_{\mathrm{PLL}} - \theta_{\mathrm{enc}} + \theta_{\mathrm{comp}},\quad \theta_n = -\theta_{\mathrm{PLL}} - \theta_{\mathrm{enc}} + \theta_{\mathrm{comp}} \tag{8-41}$$

recovered_motor.cpp L465-L486

```
    s.d(0x278, s.d(0x280) + s.d(0x288));
    s.increment(0xC8);
    s.d(0xC0, s.d(0xC0) + s.d(0x278));
    if (s.i(0xC8) >= s.i(0xCC)) {
        s.d(0xB8, s.d(0xC0) / static_cast<double>(s.i(0xCC)));
        s.i(0xC8, 0);
        s.d(0xC0, 0.0);
    }
    s.increment(0xE0);
    s.d(0xD8, s.d(0xD8) + s.d(0x1040));
    if (s.i(0xE0) >= s.i(0xE4)) {
        s.i(0xE0, 0);
        s.d(0xD8, 0.0);
    }
    s.d(0xD0, s.d(0x1040));
    wrap(0x290);
    s.d(0x2E0, (s.d(0x1E8) - s.d(0x290)) + s.d(0x1F0));
    s.d(0x2E8, ((-s.d(0x1E8)) - s.d(0x290)) + s.d(0x1F0));
    wrap(0x2E0);
    wrap(0x2E8);
// ---- 闸门：只有“运行/停机过程”状态才执行下面的控制 ----
    if (s.i(0x4FC) != 1 && s.i(0x4FC) != 2) goto final_status;
```

状态2仍会运行控制直到停机计数复位。

8.2.9 两套序分解与ready前后反馈选择

第一套11F8输入转子电流228/230/238，使用θp、θn；第二套12F0输入240/248/250三相电压，使用θPLL、−θPLL。函数同名rotor_current_sequence不代表两套数据都是电流，θn也不是简单−θp。

ready为0，反馈2C0/2C8取原始旋转坐标1260/1268，负序反馈置零。ready后728==0取1220/1228与1230/1238，否则取1280/1288与1290/1298。本拍后段才置ready时，此处已按旧ready完成选择。

recovered_motor.cpp L489-L512

```
    s.d(0x11F8, s.d(0x228));
    s.d(0x1200, s.d(0x230));
    s.d(0x1208, s.d(0x238));
    s.d(0x1210, s.d(0x2E0));
    s.d(0x1218, s.d(0x2E8));
    rotor_current_sequence(s.at(0x11F8));
    s.d(0x12F0, s.d(0x240));
    s.d(0x12F8, s.d(0x248));
    s.d(0x1300, s.d(0x250));
    s.d(0x1308, s.d(0x1E8));
    s.d(0x1310, -s.d(0x1E8));
    rotor_current_sequence(s.at(0x12F0));
    if (s.i(0x1C) == 0) {
        s.d(0x2C0, s.d(0x1260));
        s.d(0x2C8, s.d(0x1268));
        s.d(0x2D0, 0.0);
        s.d(0x2D8, 0.0);
    } else if (s.i(0x728) == 0) {
        s.d(0x2C0, s.d(0x1220));
        s.d(0x2C8, s.d(0x1228));
        s.d(0x2D0, s.d(0x1230));
        s.d(0x2D8, s.d(0x1238));
    } else {
        s.d(0x2C0, s.d(0x1280));
```

recovered_motor.cpp L513-L516

```
        s.d(0x2C8, s.d(0x1288));
        s.d(0x2D0, s.d(0x1290));
        s.d(0x2D8, s.d(0x1298));
    }
```

原始/滤波序分量与轴序见公共算法，地址优先于不一致的d/q注释。

8.2.10 400拍等待、累计就绪和PI一次复制

启动计数0小于8时加一并goto exit_reset，跳过公共斜坡、PI、调制和瞬时闭锁，但前面PLL、角度、序分解已执行；不是全区复位或直接return。400个等待调用之后下一拍开始预同步。

ready为0时A0写本拍重算kP。精确相等A8==A0且计数4<200时递增，已经200的下一次相等才置ready及3A0、清计数，并复制缓存208/210、218/220到四个活动PI的比例、积分与ki/kp。失败不清计数，因此累计相等观察，非连续200拍；从零要第201次相等观察。

预同步本拍即使置ready也goto common运行斜坡、PI、调制。之后100次调用计数4递增并goto common，再下一拍进正常参考和穿越判断。运行中参数入口更新缓存没有另一条ready后重复复制活动PI的路径。

$$0xA0 = k_P, \quad d(0xA8) = d(0xA0) \text{ uses exact double equality} \tag{8-42}$$

recovered_motor.cpp L520-L543

```
    if (s.i(0) < s.i(8)) {
        s.increment(0);
        goto exit_reset;
    }
    if (s.i(0x1C) == 0) {
        s.d(0xA0, s.d(0x1C8));
        if (s.d(0xA8) == s.d(0xA0)) {
            if (s.i(4) < 200) s.increment(4);
            else {
                s.i(0x1C, 1);
                s.i(0x3A0, 1);
                s.i(4, 0);
                s.d(0xE10, s.d(0x208));
                s.d(0xE58, s.d(0x210));
                s.d(0xE60, s.d(0xE58) / s.d(0xE10));
                s.d(0xE90, s.d(0x208));
                s.d(0xED8, s.d(0x210));
                s.d(0xEE0, s.d(0xED8) / s.d(0xE90));
                s.d(0xF10, s.d(0x218));
                s.d(0xF58, s.d(0x220));
                s.d(0xF60, s.d(0xF58) / s.d(0xF10));
                s.d(0xF90, s.d(0x218));
                s.d(0xFD8, s.d(0x220));
                s.d(0xFE0, s.d(0xFD8) / s.d(0xF90));
```

完整520–553；预同步目标并非固定初始化362.44A。

8.2.11 负序预判标志与21次观察的滞回

3E0、3F8都按1088判定：前者超过3D0进入、低于3D8退出，后者超过3E8进入、低于3F0退出。均严格比较，等号不计入相应确认。

进入/退出都先检查计数<20并递增，已达20的下一次满足才改标志，从零需第21次观察。未满足只清相应计数，不直接改标志。进入检查晚于本拍PLL、早于退出检查，进入段先清退出计数。3E0影响需求及档位，3F8决定负序PI执行/清理。

recovered_motor.cpp L565-L586

```
    if (s.d(0x1088) > s.d(0x3D0)) {
        s.i(0x4BC, 0);
        if (s.i(0x4A4) < 20) s.increment(0x4A4);
        else s.i(0x3E0, 1);
    } else s.i(0x4A4, 0);
    if (s.i(0x3E0) == 1) {
        if (s.d(0x3D8) > s.d(0x1088)) {
            if (s.i(0x4BC) < 20) s.increment(0x4BC);
            else s.i(0x3E0, 0);
        } else s.i(0x4BC, 0);
    }
    if (s.d(0x1088) > s.d(0x3E8)) {
        s.i(0x404, 0);
        if (s.i(0x400) < 20) s.increment(0x400);
        else s.i(0x3F8, 1);
    } else s.i(0x400, 0);
    if (s.i(0x3F8) == 1) {
        if (s.d(0x3F0) > s.d(0x1088)) {
            if (s.i(0x404) < 20) s.increment(0x404);
            else s.i(0x3F8, 0);
        } else s.i(0x404, 0);
    }
```

1088在当前恢复代码没有内部自动刷新，零初态与外部宿主写入应区别。

8.2.12 LVRT/HVRT进入合取条件和优先级

LVRT进入要求Vmin<3B0、4C4==0、534==1、V+≤410；计数490<20时递增，已20的下一次置模式1，所以第21次动作。新进入才递增事件数970，同时清高穿进入与低穿退出计数。

高穿为else if，只有低穿合取不成立才评价：Vmax>3C0、538==1、V+>410、恢复计时580精确为0。计数498阈值4对应第5次观察，事件数974在模式变化时增加。恢复计时中的高穿进入门控与频率支撑开关不是同一条件。

$$\text{LVRT:} V_{\min} < V_{L,on} \wedge V_{+} \le V_{b} \wedge 0x534 = 1 \wedge 0x4C4 = 0 \tag{8-43}$$

$$\text{HVRT:} V_{\max} > V_{H,on} \wedge V_{+} > V_{b} \wedge 0x538 = 1 \wedge 0x580 = 0 \tag{8-44}$$

recovered_motor.cpp L587-L608

```
    if (s.d(0x3B0) > s.d(0x1A08) && s.i(0x4C4) == 0 && s.i(0x534) == 1 && s.d(0x410) >= s.d(0x3A8)) {
        s.i(0x498, 0);
        s.i(0x4A8, 0);
        if (s.i(0x490) < 20) s.increment(0x490);
        else {
            if (s.i(0x10) != 1) s.increment(0x970);
            s.i(0x10, 1);
        }
    } else if (s.d(0x1A00) > s.d(0x3C0) && s.i(0x538) == 1 && s.d(0x3A8) > s.d(0x410) && s.d(0x580)
== 0.0) {
        s.i(0x408, 0);
        s.i(0x490, 0);
        s.i(0x4B0, 0);
        if (s.i(0x498) < 4) s.increment(0x498);
        else {
            if (s.i(0x10) != 2) s.increment(0x974);
            s.i(0x10, 2);
        }
    } else {
        s.i(0x490, 0);
        s.i(0x498, 0);
        s.i(0x408, 0);
        if (s.i(0x10) == 1 && s.i(0x4C0) == 0) {
```

进入总else时两个进入计数归零。

8.2.13 穿越退出、兜底阈值与恢复保持

退出只在进入条件均不成立的else评价。低穿还需4C0==0、Vmin>3B8，退出计数先加一再检查>4，故第5次动作；清保持量和参考88，置恢复请求570。

高穿需4C4==0，首分支Vmax<3C8且1088<56.56；else if只需Vmax<1.02×1A20，两支共用4B0且>4才退出。固定1.02pu兜底与参数退出门槛不同，不能混称同一个门槛。

641–652在模式非0时置14=1、清18，模式0且14==1后计数18<800就加一，否则下一拍清14与478。此为恢复阶段保持，模式10并不继续保持穿越。4C0/4C4在退出判断后才由各模式块更新，40拍阈值不等于可以忽略先后顺序的最早退出拍。

recovered_motor.cpp L609-L632

```
            if (s.d(0x1A08) > s.d(0x3B8)) {
                s.increment(0x4A8);
                if (s.i(0x4A8) > 4) {
                    s.i(0x4A8, 0);
                    s.i(0x10, 0);
                    s.d(0x480, 0.0);
                    s.d(0x488, 0.0);
                    s.d(0x468, 0.0);
                    s.i(0x408, 0);
                    s.d(0x88, 0.0);
                    s.i(0x570, 1);
                    s.d(0x1C10, 0.0);
                }
            } else s.i(0x4A8, 0);
        } else if (s.i(0x10) == 2 && s.i(0x4C4) == 0) {
            if (s.d(0x3C8) > s.d(0x1A00) && 56.559999999999995 > s.d(0x1088)) {
                s.increment(0x4B0);
                if (s.i(0x4B0) > 4) {
                    s.i(0x4B0, 0);
                    s.i(0x10, 0);
                    s.d(0x468, 0.0);
                }
            } else if (1.02 * s.d(0x1A20) > s.d(0x1A00)) {
                s.increment(0x4B0);
```

recovered_motor.cpp L633-L636

```
                if (s.i(0x4B0) > 4) {
                    s.i(0x4B0, 0);
                    s.i(0x10, 0);
                    s.d(0x468, 0.0);
```

高穿完整631–638；恢复保持实际800次递增后下次解除。

8.2.14 LVRT需求、负序门控及后写覆盖

4E0=1−1040/410是电压跌落量，非功率测量误差。470先为6F0−V+/Vb，按30/1024滤波并由60C选择滤波或瞬时值，乘活动增益418得到标幺需求4D8。>0.9863钳0.9863，<0写B68而非0，再乘989.8。

60C==1或新需求较大时更新468/460及余量478，初次余量限[212.1,458]。680–705负序两轴先限电压标幺±0.48828125、乘±630并滤波；仅3E0==1且668==1才乘基准并限±640，否则电流输出0，668默认0。

706–717按3E0和1020等条件限制460、A8，最终限制460上限428。但788–794在60C==1时又把468/460重写为470并用合成A0重算余量，早期限值不能当作所有字段最终值。

$$\Delta V = d(0x6F0) - V_+ / V_b, \quad \Delta V_{f,k} = \frac{30\Delta V_k + 994\Delta V_{f,k-1}}{1024} \tag{8-45}$$

$$z = K_{R,LV}\Delta V_f, \quad z' = \begin{cases} 0.9863 & z > 0.9863 \\ d(0xB68) & z < 0 \\ z & \text{otherwise} \end{cases} \tag{8-46}$$

$$I_{\mathrm{rem},1} = \ \mathrm{clip}\ (I_b\sqrt{1.21 - z'^2}, 212.1, d(0x458)) \tag{8-47}$$

recovered_motor.cpp L657–L679

```
    if (s.i(0x10) == 1) {
        s.increment(0x408);
        s.d(0x5C8, 0.0);
        s.i(0x4CC, 0);
        if (s.i(0x4C8) < s.i(0x4D4)) {
            s.increment(0x4C8);
            s.i(0x4C0, 1);
        } else s.i(0x4C0, 0);
        s.d(0x4E0, 1.0 - s.d(0x1040) / s.d(0x410));
        s.d(0x470, s.d(0x6F0) - s.d(0x3A8) / s.d(0x410));
        s.d(0x5C0, (30.0 * s.d(0x470) + 994.0 * s.d(0x5C0)) / 1024.0);
        if (s.i(0x60C) == 1) s.d(0x4D8, s.d(0x418) * s.d(0x5C0));
        else s.d(0x4D8, s.d(0x418) * s.d(0x470));
        if (s.d(0x4D8) > 0.9863) s.d(0x4D8, 0.9863);
        else if (0.0 > s.d(0x4D8)) s.d(0x4D8, s.d(0xB68));
        s.d(0x470, s.d(0x4D8) * s.d(0x48));
        if (s.i(0x60C) == 1 || s.d(0x470) > s.d(0x468)) {
            s.d(0x468, s.d(0x470));
            s.d(0x460, s.d(0x468));
            s.d(0x478, std::sqrt(1.21 - s.d(0x4D8) * s.d(0x4D8)) * s.d(0x48));
            if (s.d(0x478) > s.d(0x458)) s.d(0x478, s.d(0x458));
            if (212.1 > s.d(0x478)) s.d(0x478, 212.1);
        }
```

60C==1默认实时覆盖；其它门控可选择历史峰值。

8.2.15 HVRT附加量未接入、历史保持和电流圆

470取1040/410−6F8，按30/1024滤波并选值，乘活动系数420。另对1088按20/1024滤波生成5E0，并算5E8=clip(0.5×5E0/410,0,0.1)；源码没有把5E8加到470，不能将附加量写入高穿需求公式。

470仅自身限[0,0.3418]后取负乘989.8。60C==1或新需求更负时更新468/460；随后先按上限438、再下限430钳460，默认区间[−424.2,−212.1]可被参数覆盖。余量478在747–749用max(Ib²−460²,0)开方。

7A8保存上一拍模式，高穿块先读取其是否1来置570，751–752才写当前模式。高穿保持计数4CC与标志4C4的更新晚于进入/退出判断。

$$I_{H,raw} = \ -I_b\ \mathrm{clip}\ (K_{R,HV}\Delta V_f, 0, 0.3418) \tag{8-48}$$

$$I_H = \ \mathrm{clip}\ (I_{H,hist}, d(0x430), d(0x438)) \tag{8-49}$$

$$\Delta V_{add} = \ \mathrm{clip}\left(\frac{0.5d(0x5E0)}{V_b}, 0, 0.1\right) \quad \text{not added to demand} \tag{8-50}$$

recovered_motor.cpp L720-L743

```
    } else if (s.i(0x10) == 2) {
        if (s.i(0x7A8) == 1) s.i(0x570, 1);
        s.d(0x5C0, 0.0);
        s.d(0x480, 0.0);
        s.i(0x408, 0);
        s.i(0x4C8, 0);
        if (s.i(0x4CC) < s.i(0x4D0)) {
            s.increment(0x4CC);
            s.i(0x4C4, 1);
        } else s.i(0x4C4, 0);
        s.d(0x470, s.d(0x1040) / s.d(0x410) - s.d(0x6F8));
        s.d(0x5C8, (30.0 * s.d(0x470) + 994.0 * s.d(0x5C8)) / 1024.0);
        if (s.i(0x60C) == 1) s.d(0x470, s.d(0x420) * s.d(0x5C8));
        else s.d(0x470, s.d(0x420) * s.d(0x470));
        s.d(0x5E0, (20.0 * s.d(0x1088) + 1004.0 * s.d(0x5E0)) / 1024.0);
        s.d(0x5E8, (0.5 * s.d(0x5E0)) / s.d(0x410));
        if (0.0 > s.d(0x5E8)) s.d(0x5E8, 0.0);
        else if (s.d(0x5E8) > 0.1) s.d(0x5E8, 0.1);
        if (0.0 > s.d(0x470)) s.d(0x470, 0.0);
        else if (s.d(0x470) > 0.3418) s.d(0x470, 0.3418);
        s.d(0x470, (-s.d(0x470)) * s.d(0x48));
        if (s.i(0x60C) == 1 || s.d(0x468) > s.d(0x470)) {
            s.d(0x468, s.d(0x470));
            s.d(0x460, s.d(0x468));
```

recovered_motor.cpp L744-L747

```
        }
        if (s.d(0x460) > s.d(0x438)) s.d(0x460, s.d(0x438));
        if (s.d(0x430) > s.d(0x460)) s.d(0x460, s.d(0x430));
        s.d(0x478, s.d(0x48) * s.d(0x48) - s.d(0x460) * s.d(0x460));
```

附加量计算存在不等于接入。

8.2.16 平均电流、低穿目标与档位优先级

751–759平均量为Σ(kP×1040/410)/80，单位电流，非W功率。88先取转矩电流2B0×Vb/D0并限幅，但768–785的710及跌落条件可能后写88，所以前段负限不是全部分支最终限值。

低穿A0=平均电流+460+无功电流2B8×700，仅限上限Ib。60C==1重写468/460后按合成A0重算sqrt(max(1.21Ib²−A0²,0))，再将88限制在负余量与−B90×1.414之间。负序目标648/650取620/610。

档位5D4正常分类为(3E0==0且1088<120.19)或V+<0.55Vb；低压优先被前分支吸收。正常大功率档121/481/861门槛为>0.6、0.35<跌落<0.45、0.1<跌落<0.15；小功率122/482/862为>0.8、>0.4、0.1<跌落<0.15。815–835异常大功率123/483/703/863为>0.41/0.18/0.12/0.1，小功率124/484/704/864为>0.4/0.22/0.15/0.05。

每档虽允许条件或已有编码，但整个else if有先后优先级，前面新的条件可覆盖后面的旧编码，并非无条件自锁。这些字段是穿越分类编码；当前读写证据没有证明它们对应器件投入动作。

$$I_{avg} = \frac{1}{80}\sum \frac{k_P V_d^{+}}{V_b} \tag{8-51}$$

$$I_A^* = \min\left(I_{avg} + d(0x460) + i_\psi d(0x700), I_b\right) \tag{8-52}$$

$$I_{\mathrm{rem},2} = \sqrt{\max\left(1.21 I_b^2 - (I_A^*)^2, 0\right)} \tag{8-53}$$

recovered_motor.cpp L786-L809

```
        s.d(0xA0, (s.d(0x1C0) + s.d(0x460)) + s.d(0x2B8) * s.d(0x700));
        if (s.d(0xA0) > 1.0 * s.d(0x48)) s.d(0xA0, 1.0 * s.d(0x48));
        if (s.i(0x60C) == 1) {
            s.d(0x468, s.d(0x470));
            s.d(0x460, s.d(0x468));
            s.d(0x478, (1.21 * s.d(0x48)) * s.d(0x48) - s.d(0xA0) * s.d(0xA0));
            if (0.0 > s.d(0x478)) s.d(0x478, 0.0);
            else s.d(0x478, std::sqrt(s.d(0x478)));
        }
        if (-s.d(0x478) > s.d(0x88)) s.d(0x88, -s.d(0x478));
        if (s.d(0x88) > (-s.d(0xB90)) * 1.414) s.d(0x88, (-s.d(0xB90)) * 1.414);
        s.d(0x648, s.d(0x620));
        s.d(0x650, s.d(0x610));
        if ((s.i(0x3E0) == 0 && 120.19 > s.d(0x1088)) || 0.55 * s.d(0x410) > s.d(0x3A8)) {
            if ((s.d(0x4E0) > 0.6 && s.i(0x57C) == 1) || s.i(0x5D4) == 121) {
                s.i(0x5D4, 121);
                s.i(0x774, 1);
            } else if ((s.d(0x4E0) > 0.35 && s.i(0x57C) == 1 && 0.45 > s.d(0x4E0)) || s.i(0x5D4) ==
481) {
                s.i(0x5D4, 481);
                s.i(0x778, 1);
            } else if ((s.d(0x4E0) > 0.1 && s.i(0x57C) == 1 && 0.15 > s.d(0x4E0)) || s.i(0x5D4) ==
861) {
                s.i(0x5D4, 861);
            } else if ((s.d(0x4E0) > 0.8 && s.i(0x57C) == 0) || s.i(0x5D4) == 122) {
                s.i(0x5D4, 122);
```

recovered_motor.cpp L810-L813

```
            } else if ((s.d(0x4E0) > 0.4 && s.i(0x57C) == 0) || s.i(0x5D4) == 482) {
                s.i(0x5D4, 482);
            } else if ((0.15 > s.d(0x4E0) && s.d(0x4E0) > 0.1 && s.i(0x57C) == 0) || s.i(0x5D4) ==
862) {
                s.i(0x5D4, 862);
```

700被参数入口以p[27]写入，本工程不能写“保持0”。

8.2.17 高穿目标、档位与等号空隙

高穿A0=平均电流+460+2B8×708，88仅受负余量下限，负序目标清零。本段没有低穿的A0上限Ib，不能复制低穿限幅到高穿。

正常分类3E0==0且1088<42.42，异常分类3E0==1或1088>42.42；3E0==0且恰等42.42两类都不更新。正常大功率1361/1241/1131按V+/Vb>1.2/1.16/1.05，小功率1362/1242/1132按>1.25/1.2/1.05。

异常大功率1363/1243/1133按负序量>219.17/169.68/98.98，小功率1364/1244/1134按>226.24/169.68/113.12。各条件严格大于，旧编码保持仍受else if优先顺序。

$$I_{A,H}^{*} = I_{avg} + d(0x460) + i_{\psi} d(0x708) \tag{8-54}$$

recovered_motor.cpp L839–L862

```
    } else if (s.i(0x10) == 2) {
        s.d(0xA0, (s.d(0x1C0) + s.d(0x460)) + s.d(0x2B8) * s.d(0x708));
        if (-s.d(0x478) > s.d(0x88)) s.d(0x88, -s.d(0x478));
        s.d(0x648, 0.0);
        s.d(0x650, 0.0);
        if (s.i(0x3E0) == 0 && 42.419999999999995 > s.d(0x1088)) {
            if ((s.d(0x3A8) / s.d(0x410) > 1.2 && s.i(0x57C) == 1) || s.i(0x5D8) == 1361) {
                s.i(0x5D8, 1361);
            } else if ((s.d(0x3A8) / s.d(0x410) > 1.16 && s.i(0x57C) == 1) || s.i(0x5D8) == 1241) {
                s.i(0x5D8, 1241);
            } else if ((s.d(0x3A8) / s.d(0x410) > 1.05 && s.i(0x57C) == 1) || s.i(0x5D8) == 1131) {
                s.i(0x5D8, 1131);
            } else if ((s.d(0x3A8) / s.d(0x410) > 1.25 && s.i(0x57C) == 0) || s.i(0x5D8) == 1362) {
                s.i(0x5D8, 1362);
            } else if ((s.d(0x3A8) / s.d(0x410) > 1.2 && s.i(0x57C) == 0) || s.i(0x5D8) == 1242) {
                s.i(0x5D8, 1242);
            } else if ((s.d(0x3A8) / s.d(0x410) > 1.05 && s.i(0x57C) == 0) || s.i(0x5D8) == 1132) {
                s.i(0x5D8, 1132);
            }
        } else if (s.i(0x3E0) == 1 || s.d(0x1088) > 42.419999999999995) {
            if ((s.d(0x1088) > 219.17 && s.i(0x57C) == 1) || s.i(0x5D8) == 1363) {
                s.i(0x5D8, 1363);
            } else if ((s.d(0x1088) > 169.67999999999998 && s.i(0x57C) == 1) || s.i(0x5D8) == 1243) {
                s.i(0x5D8, 1243);
```

recovered_motor.cpp L863–L866

```
            } else if ((s.d(0x1088) > 98.97999999999999 && s.i(0x57C) == 1) || s.i(0x5D8) == 1133) {
                s.i(0x5D8, 1133);
            } else if ((s.d(0x1088) > 226.23999999999998 && s.i(0x57C) == 0) || s.i(0x5D8) == 1364) {
                s.i(0x5D8, 1364);
```

异常完整859–870；1088实际更新来源影响分类。

8.2.18 正常参考与公共恢复步长

正常模式清穿越计数、编码、保持量及跌落记录，2B8先限±60再乘Vb/D0，A0=1C0+折算后2B8，负序目标清零并记录7A0=D0。分母无零值替代。

common在897–909按14及ready选择步长：14==0且ready为1时390/398均取360，未ready均取368；14非0时390取378，398按408<180取370否则2×370。参数入口可覆盖默认值。

公共slew在985–988使A8追A0、B0追88、658追648、660追650，目标较大加步长，较小减步长并防越过，相等不写。特殊恢复在slew前后写参考及步长影响本拍环；步长本身不取绝对值。

$$i_{\psi,scaled} = \text{clip}\left(i_{\psi}, -d(0x60), d(0x60)\right)\frac{V_b}{V_d^{+}} \tag{8-55}$$

$$I_{A,N}^{*} = I_{avg} + i_{\psi,scaled} \tag{8-56}$$

recovered_motor.cpp L873-L896

```
    } else {
        s.i(0x568, 0);
        s.d(0x550, 0.0);
        s.i(0x4CC, 0);
        s.i(0x4C8, 0);
        s.i(0x5D4, 0);
        s.i(0x5D8, 0);
        s.d(0x5C0, 0.0);
        s.d(0x5C8, 0.0);
        s.d(0x480, 0.0);
        s.d(0x488, 0.0);
        s.d(0x460, 0.0);
        s.d(0x4E0, 0.0);
        clip(0x2B8, -s.d(0x60), s.d(0x60));
        s.d(0x2B8, (s.d(0x2B8) * s.d(0x410)) / s.d(0xD0));
        s.d(0xA0, s.d(0x1C0) + s.d(0x2B8));
        s.d(0x648, 0.0);
        s.d(0x650, 0.0);
        s.d(0x7A0, s.d(0xD0));
    }

// ---- common: 三种模式共用的后续处理（从这里起每步都执行） ----
common:
// ---- 斜坡步长选择: 0x390/0x398 按运行状态给不同的爬升速度 ----
```

recovered_motor.cpp L897-L900

```
    if (s.i(0x14) == 0) {
        if (s.i(0x1C) == 1) {
            s.d(0x390, s.d(0x360));
            s.d(0x398, s.d(0x360));
```

特殊恢复可能生成负步长，按实际slew分支解释。

8.2.19 四个电流PI与两种谐波补偿

985–1005公共斜坡后，1060/1068经0.01/0.99低通、残差递归滤波及740/748增益，限±5%×|88|；750==0或模式非0清两项补偿。门控实际为非零允许，不只750==1，滤波历史仍更新。

DF8以B0+1678对2C0，E78以A8+1640对2C8；EF8以658对2D0，F78以660对2D8。3F8==0才执行负序两环，否则清F20/FA0误差历史及F48/FC8积分，不调用负序regulator。

1054–1079的690残差支路默认0；0或非正常模式清四项补偿，1以1318/1320经residual_filter与6B8增益、限±0.2，2以2C0/2C8与6C0增益、限±0.2。正常模式其它非零值没有分支，可保留旧补偿值。默认6B8/6C0为0.3，关闭不等于实现没有接入。

$$y_{LP,k} = 0.01x_k + 0.99y_{LP,k-1} \tag{8-57}$$

$$B_h = \text{clip}\left(G_h y_{\text{recursive}}, -0.05 \mid d(0x88) \mid, 0.05 \mid d(0x88) \mid\right) \tag{8-58}$$

recovered_motor.cpp L1009-L1027

```
    s.d(0xDF8, s.d(0xB0) + s.d(0x1678));
    s.d(0xE00, s.d(0x2C0));
    s.apply(0xDF8, regulator);
    s.d(0xE78, s.d(0xA8) + s.d(0x1640));
    s.d(0xE80, s.d(0x2C8));
    s.apply(0xE78, regulator);
    if (s.i(0x3F8) == 0) {
        s.d(0xEF8, s.d(0x658));
        s.d(0xF00, s.d(0x2D0));
        s.apply(0xEF8, regulator);
        s.d(0xF78, s.d(0x660));
        s.d(0xF80, s.d(0x2D8));
        s.apply(0xF78, regulator);
    } else {
        s.d(0xF20, 0.0);
        s.d(0xFA0, 0.0);
        s.d(0xF48, 0.0);
        s.d(0xFC8, 0.0);
    }
```

3F8与瞬时过流位24不是同一保护。

8.2.20 前馈负限写错槽与固定50Hz路径

66C==1计算正序前馈，按0.577×电压×5F0×(滑差/50)/B8归一。670超过0.577钳上限，但小于−0.577时实际写678=−0.577，670未钳负限；随后另对678钳±0.577。不能用一条对称clip公式概括两轴。

负序前馈按−1050/1058和(2+滑差/50)计算，限±0.1。66C不为1但3F8==1时正序清零、负序仍算，否则四轴清零。默认66C=1；此开关与负序电流给定668不同。

前馈比例分母固定50，与base_frequency为独立频率路径。额定频率改变后，输入滑差和kT使用新参数，本段依然除50，不能在教材化公式中默改为fb。

$$a = 0.577\frac{V_d^{+}k_v}{V_{dc}}\frac{f_{slip}}{50},\quad b = 0.577\frac{V_q^{+}k_v}{V_{dc}}\frac{f_{slip}}{50} \tag{8-59}$$

$$v_1 = \min(a, 0.577),\quad v_2 = \mathrm{clip}\left(\begin{cases} -0.577 & a < -0.577 \\ b & \text{otherwise} \end{cases}, -0.577, 0.577\right) \tag{8-60}$$

$$v_{-,j} = \mathrm{clip}\left(-0.577\frac{V_{-,j}k_v}{V_{dc}}\left(2+\frac{f_{slip}}{50}\right), -0.1, 0.1\right) \tag{8-61}$$

recovered_motor.cpp L1030-L1053

```
    if (s.i(0x66C) == 1) {
        s.d(0x670, (((s.d(0x1040) * s.d(0x5F0)) * (s.d(0x5F8) / 50.0)) / s.d(0xB8)) * 0.577);
        s.d(0x678, (((s.d(0x1048) * s.d(0x5F0)) * (s.d(0x5F8) / 50.0)) / s.d(0xB8)) * 0.577);
        if (s.d(0x670) > 0.577) s.d(0x670, 0.577);
        // The original lower-limit store targets 0x678 and leaves 0x670 below its bound.
        else if (-0.577 > s.d(0x670)) s.d(0x678, -0.577);
        clip(0x678, -0.577, 0.577);
        s.d(0x680, ((((-s.d(0x1050)) * s.d(0x5F0)) * (2.0 + s.d(0x5F8) / 50.0)) / s.d(0xB8)) *
0.577);
        s.d(0x688, ((((-s.d(0x1058)) * s.d(0x5F0)) * (2.0 + s.d(0x5F8) / 50.0)) / s.d(0xB8)) *
0.577);
        clip(0x680, -0.1, 0.1);
        clip(0x688, -0.1, 0.1);
    } else if (s.i(0x3F8) == 1) {
        s.d(0x670, 0.0);
        s.d(0x678, 0.0);
        s.d(0x680, ((((-s.d(0x1050)) * s.d(0x5F0)) * (2.0 + s.d(0x5F8) / 50.0)) / s.d(0xB8)) *
0.577);
        s.d(0x688, ((((-s.d(0x1058)) * s.d(0x5F0)) * (2.0 + s.d(0x5F8) / 50.0)) / s.d(0xB8)) *
0.577);
        clip(0x680, -0.1, 0.1);
        clip(0x688, -0.1, 0.1);
    } else {
        s.d(0x670, 0.0);
        s.d(0x678, 0.0);
        s.d(0x680, 0.0);
        s.d(0x688, 0.0);
    }
```

实际错误目标地址是复现语义，说明中不得悄悄修正。

8.2.21 谐振实接入、反Park与中点平衡选择

两组谐振器参考0、反馈第一套未滤波旋转坐标1260/1268，每拍调用。输出1510/15C8除8388608直接接入正序反Park输入；默认零增益使零初态无控制贡献。

正序第一轴670+E48/S+6A0−6B0+1510/S，第二轴678+EC8/S−698+6A8+15C8/S，相位θp；负序680+F48/S和688+FC8/S，相位θn。先旋转坐标合成、反Park，再静止坐标相加，不能直接把dq项标成α轴。

滑差>0.2Hz时D98=1、D10=5.0；滑差<−2Hz时D98=0、D10=5.0；其它区间含等号D10=0且D98保留。普通modulation基址BF0，相对0x120的gain映射D10，相对0x1A8的polarity映射D98：这段选择中点电压平衡的极性与增益。5.0是比例增益，不是5秒延时，也不是撬棒投入状态。正极性时中点修正采用正增益，否则负增益；常规调制把修正量叠加到零序注入后的三相调制波。

普通调制在正、负母线差的绝对值大于deadband时，修正量为有符号增益乘电压差再除母线总电压，限幅±limit；否则修正量为0。默认limit=D18=0.2，deadband=D20=20。滑差中间区将增益写0，因此该分支关闭中点修正。备用调制没有读取这两个字段，不能推定该选择在全部输出模式生效。

$$u_1^{+} = v_1 + u_{PI,1} \,/\, S + B_{6A0} - B_{6B0} + y_{r1} \,/\, S \tag{8-62}$$

$$u_2^{+} = v_2 + u_{PI,2} \,/\, S - B_{698} + B_{6A8} + y_{r2} \,/\, S \tag{8-63}$$

$$\begin{cases} D98 = 1,\ D10 = 5 & f_{slip} > 0.2 \\ D98 = 0,\ D10 = 5 & f_{slip} < -2 \\ D10 = 0 & -2 \le f_{slip} \le 0.2 \end{cases} \tag{8-64}$$

$$c_{np} = \begin{cases} \mathrm{clip}\left(\sigma K_{np} \dfrac{V_{dc+} - V_{dc-}}{V_{dc+} + V_{dc-}}, -L, L\right) & |V_{dc+} - V_{dc-}| > D \\ 0 & \text{otherwise} \end{cases}, \quad \sigma = \begin{cases} 1 & D98 = 1 \\ -1 & D98 \neq 1 \end{cases} \tag{8-65}$$

recovered_motor.cpp L1081-L1104

```
    s.d(0x14C0, 0.0);
    s.d(0x14C8, s.d(0x1260));
    resonant_filter(s.at(0x14A8));
    s.d(0x1578, 0.0);
    s.d(0x1580, s.d(0x1268));
    resonant_filter(s.at(0x1560));
// ---- 调制波合成: 正序/负序电压参考 + 各补偿量 → 反 Park (0x13E8/0x1410)  ----
    s.d(0x1400, (((s.d(0x670) + s.d(0xE48) / s.d(0x358)) + s.d(0x6A0)) - s.d(0x6B0)) + s.d(0x1510) /
s.d(0x358));
    s.d(0x1408, (((s.d(0x678) + s.d(0xEC8) / s.d(0x358)) - s.d(0x698)) + s.d(0x6A8)) + s.d(0x15C8) /
s.d(0x358));
    s.d(0x13F8, s.d(0x2E0));
    s.apply(0x13E8, inverse_park_transform);
    s.d(0x1428, s.d(0x680) + s.d(0xF48) / s.d(0x358));
    s.d(0x1430, s.d(0x688) + s.d(0xFC8) / s.d(0x358));
    s.d(0x1420, s.d(0x2E8));
    s.apply(0x1410, inverse_park_transform);
// ---- crowbar (撬棒) 逻辑: 转速高于 0.2 或低于 -2.0 (标幺) 时
//      切换 0xD98/0xD10 (撬棒投入状态与延时 5 秒)  ----
    if (s.d(0x5F8) > 0.2) {
        s.i(0xD98, 1);
        s.d(0xD10, 5.0);
    } else if (-2.0 > s.d(0x5F8)) {
        s.i(0xD98, 0);
        s.d(0xD10, 5.0);
    } else s.d(0xD10, 0.0);
```

recovered_motor.cpp L1105-L1108

```
    s.d(0xDB0, s.d(0x228));
    s.d(0xDB8, s.d(0x230));
    s.d(0xDC0, s.d(0x238));
    s.d(0xCB8, s.d(0x280));
```

单位Hz非标幺转速；源码实际350Hz，注释300Hz不采信。

映射证据：recovered_primitives.h:165–187；recovered_primitives.cpp:30–31、274–283；普通调制调用recovered_motor.cpp:1113。备用调制324–367无gain/polarity读取。

8.2.22 调制分派、逐拍闭锁与801拍复位

正负序静止坐标相加送BF0/BF8，复制三相电流和两路母线为调制输入；6E8为0或2调用普通modulation，其它所有值调用alternate_modulation。六路占空比先复制到308/310、320/328、338/340。

任一相电流绝对值严格大于40或瞬时总母线278严格大于28置24并清34。闭锁清350使能、B0/A8斜坡参考及E20/EA0/F20/FA0误差历史，然后清24；不擦已复制占空比，不清全部目标，不清E48/EC8/F48/FC8积分。无闭锁时20=1、350=1、3A4=1。

1148–1158在状态2先加508，严格>504才memset并defaults。默认阈值800，从零第801个停机调用复位，初始400拍等待也经过此段。最终528以状态1/2映射1.0。闭锁是逐拍门控，下一次到此段无越限即可恢复；状态0/3和前400拍会跳过瞬时保护。

$$\text{block} = (\,|\,i_a\,| > I_{trip}) \vee (\,|\,i_b\,| > I_{trip}) \vee (\,|\,i_c\,| > I_{trip}) \vee (V_{dc,instant} > 3000) \tag{8-66}$$

$$N_{stop} > 800 \Rightarrow \text{reset on call } 801 \tag{8-67}$$

recovered_motor.cpp L1110–L1133

```
    s.d(0xBF0, s.d(0x13E8) + s.d(0x1410));
    s.d(0xBF8, s.d(0x13F0) + s.d(0x1418));
// ---- 第 8 步：调制 (0x6E8==0/2 用普通版，==1 用备用版) → 占空比输出 0x308 起 ----
    if (s.i(0x6E8) == 0 || s.i(0x6E8) == 2) s.apply(0xBF0, modulation);
    else alternate_modulation(s.at(0xBF0));
    s.d(0x308, s.d(0xCE0));
    s.d(0x310, s.d(0xCE8));
    s.d(0x320, s.d(0xCF0));
    s.d(0x328, s.d(0xCF8));
    s.d(0x338, s.d(0xD00));
    s.d(0x340, s.d(0xD08));
// ---- 过流保护：任一相转子电流超过 ±0x40 门槛 → 置 0x24 闭锁标志 ----
    if (std::fabs(s.d(0x228)) > s.d(0x40) || std::fabs(s.d(0x230)) > s.d(0x40) ||
std::fabs(s.d(0x238)) > s.d(0x40)) {
        s.i(0x24, 1);
        s.i(0x34, 0);
    }
    if (s.d(0x278) > s.d(0x28)) {
        s.i(0x24, 1);
        s.i(0x34, 0);
    }
// ---- 闭锁执行：0x24 == 1 时禁止输出（不擦除刚算出的占空比），
//      否则置运行标志 0x20、输出使能 0x350 = 1 ----
    if (s.i(0x24) == 1) {
        // Protection disables the output without erasing the duties just copied.
```

recovered_motor.cpp L1134–L1137

```
        s.d(0x350, 0.0);
        s.d(0xB0, 0.0);
        s.d(0xA8, 0.0);
        s.d(0xE20, 0.0);
```

完整闭锁1132–1146；等号均不动作。

第9章　频率支撑、穿越恢复与特殊分支

9.1 使能、时序与恢复模式

当前Cmd_8=0，频率支撑配置关闭，但完整实现仍保留在源码并在本章说明。使能后惯量响应与一次调频有不同死区、确认时长、保持时长、功率单位和限幅；不得把Control59的0.1pu直接与以W计的下垂输出相加，需遵循实现中的换算。



图 9　低电压事件检测、穿越保持与恢复过程的概念图

控制实现与逐拍关系

频率测量与变化率

实测频率先经 5 点滑动平均，再用相邻两次平均值差分近似 df/dt，变化率再过 5 点滑动平均：

$$\bar{f}_k = \frac{1}{5}\sum_{i=0}^{4} f_{k-i} \tag{9-1}$$

$$\frac{df}{dt} = \frac{\bar{f}_k - \bar{f}_{k-1}}{\Delta t N_{win}} \tag{9-2}$$

惯量响应（模式 1）

模式1且0xBD0=1时累加0xBE0，达到2s后处理惯量触发。变化率与频偏同方向且|变化率|>p[57]，或|频偏|>p[57]时进入确认；0x748先判<0.005再增，下一次else才触发。幅值A=p[58]=0.1直接写状态，源码未乘4MW；只有频偏超正/负死区时才分别赋−A/+A，仅变化率触发而频偏在死区时保留此前幅值。保持计时严格>p[56]才结束：

$$C_I = (\,|r|\, > \delta \wedge \Delta f\, r > 0) \vee (\,|\Delta f|\, > \delta) \tag{9-3}$$

$$A_k = \begin{cases} -A, & \Delta f > \delta, \\ +A, & \Delta f < -\delta, \\ A_{k-1}, & |\Delta f| \le \delta. \end{cases} \quad P_I \text{ 按保持和边沿时序读取 } A_k \tag{9-4}$$

一次调频（模式 2，功频下垂）

频偏超过死区（p[61] = 0.03 Hz 加 0.005 Hz 滞回）并确认 0.05 s 后，按下垂系数δ=p[60]=0.06计算−Δf×P0/(fnδ)，其中P0=0x7C0为调用前复制的外部有功给定；保持计时严格>p[59]才退出，边沿与裁切另按下列源码解释：

$$P_{\text{droop}} = -\Delta f \frac{P_0}{f_n \delta}, \qquad P_0 = \text{state}\ [0x7C0] \tag{9-5}$$

一次调频限幅使用P0=0x7C0而非额定4MW：正最大幅度1.05p[62]P0、正最小幅度1.1p[63]P0，对应负幅度取负。实现按顺序分别处理正侧越上限、正侧最小幅度、负侧最小幅度和负侧最大幅度，其余置0。幅值≥0.1P0只出现在最大幅度分支，最小幅度分支无此门槛；正侧中间区间可清0，负侧可跳到最大幅度，不能写成对称clip或统一400kW门槛。

合成与叠加

支撑状态0x760=0x750+0x758，0x768以常数1−P0作单侧上限，后由总调度直接叠加到0x1D58。源码未在该链完成惯量标幺转W或调频W转标幺，必须按外部给定实际单位与实现原式评估：

$$P_{fs} = P_{inertia} + P_{droop} \tag{9-6}$$

$$P_{\text{out}} = \min\left(P_I + P_D, 1 - P_0\right) \tag{9-7}$$

特殊恢复与后写覆盖

Control41先写机侧LVRT有功模式，随后Control73再次写同一活动标志；当前后者为2。Control28和Control35分别以1.0参与励磁电流叠加。Control66=0.5s除斜坡参数外参与恢复超时；Control75=3在特殊恢复激活后选择P_pos均值相关参考；P_pos为外部每拍动态量。

9.2 频率支撑源码逐段对照

本节覆盖grid_frequency_support的全部分段。该函数由总调度先计算网侧、再按使能调用，随后支撑输出叠加进有功给定，最后post刷新频率。P0专指0x7C0外部有功给定副本；源码未在此处统一惯量幅值、下垂量与常数1的单位。

9.2.1 频率支撑入口与两级五点窗口

grid_frequency_support由总调度在grid_calculate之后、grid_post之前有条件调用；入口使能取机侧槽0x1E08恰等于1.0。0x7C0在调用前从外部有功给定0x1D58复制，调用后0x768直接叠加回0x1D58。这一调度关系决定本段的基准和时序，不能把0x7C0替换成额定视在功率。

频率缓存更新门为0x478=1，并非频率刷新标志0x47C=1。由于post位于其后，频率累计计数复零后的下一拍支撑读取时为0，再后一拍读到1，按固定频率测量周期采一次。第一环保存0x468，按0x708点求平均；差分用当前0x464×250μs作为时间尺度，第二环保存差分并在填满后开始按0x6FC点平均。

$$\bar{f}_k = \frac{1}{N_f}\sum_{j=0}^{N_f-1} f_{k-j}, \quad r_k = \frac{\bar{f}_k - \bar{f}_{k-1}}{0.00025 N_{\mathrm{RMS},k}} \tag{9-8}$$

recovered_grid.cpp L331-L354

```
extern "C" __declspec(dllexport) void grid_frequency_support(double f, void* state) {
    GridState s(state);
    if (s.i(0x478) == 1) {
        s.d(0x670 + 8 * static_cast<std::ptrdiff_t>(s.i(0x704)), s.d(0x468));
        s.increment(0x704);
        if (s.i(0x704) >= s.i(0x708)) {
            s.i(0x704, 0);
            if (s.i(0x70C) == 0) {
                s.i(0x70C, 1);
            }
        }
        if (s.i(0x70C) == 1) {
            s.d(0x6D8, 0.0);
            for (std::int32_t j = 0; j < s.i(0x708); ++j) {
                s.d(0x6D8, s.d(0x6D8) + s.d(0x670 + 8 * static_cast<std::ptrdiff_t>(j)));
            }
            s.d(0x6E0, s.d(0x6D8) / static_cast<double>(s.i(0x708)));
            s.d(0x6F0, ((s.d(0x6E0) - s.d(0x6E8)) / 0.00025) / static_cast<double>(s.i(0x464)));
            s.d(0x6E8, s.d(0x6E0));
            s.d(0x698 + 8 * static_cast<std::ptrdiff_t>(s.i(0x6F8)), s.d(0x6F0));
            s.increment(0x6F8);
            if (s.i(0x6F8) >= s.i(0x6FC)) {
                s.i(0x6F8, 0);
                if (s.i(0x700) == 0) {
```

recovered_grid.cpp L355-L358

```
                    s.i(0x700, 1);
                }
            }
            if (s.i(0x700) == 1) {
```

初始化Nf=Nr=5、首级就绪0x70C=1且缓存清零，初次首级平均含零填充；第二级0x700=0直到环绕才输出。刷新间隔由0x3A0决定，差分分母由可变0x464决定，两者不能自动视为相同。

9.2.2 变化率选择、惯量延时与保持

第二级五点平均写0x6D0，作为变化率参与触发。测量选择以0x70C恰为0/1分别乘原始或均值；其他整数下两项系数均0。两个惯量阈值变量都读取0xC38，源码没有独立配置变化率动作/返回阈值。

模式0x740=1且0xBD0=1时累计0xBE0，达到2.0s后才处理惯量保持。保持0x788=1时先累计0xBC0，严格超过0xBC8才清保持和幅值；否则输出0x790。保持判断位于本拍触发更新之前，因此新幅值一般在下一次调用成为0x750输出。

$$f_{\mathrm{meas}} = f_{\mathrm{raw}}\,1_{c=0} + \bar{f}\,1_{c=1},\quad \Delta f = f_{\mathrm{meas}} - f_n \tag{9-9}$$

recovered_grid.cpp L358-L381

```
            if (s.i(0x700) == 1) {
                s.d(0x6C0, 0.0);
                for (std::int32_t j = 0; j < s.i(0x6FC); ++j) {
                    s.d(0x6C0, s.d(0x6C0) + s.d(0x698 + 8 * static_cast<std::ptrdiff_t>(j)));
                }
                s.d(0x6C8, s.d(0x6C0) / static_cast<double>(s.i(0x6FC)));
                s.d(0x6D0, s.d(0x6C8));
            }
        }
    }

    const double measured = s.d(0x468) * static_cast<double>(s.i(0x70C) == 0)
                          + s.d(0x6E0) * static_cast<double>(s.i(0x70C) == 1);
    const double threshold_a = s.d(0xC38);
    const double threshold_b = s.d(0xC38);
    const double amplitude = s.d(0x720);
    const double derivative = s.d(0x6D0);
    const double product = (measured - f) * derivative;
    const double amplitude_scaled = amplitude * 1.0;
    if (s.i(0x740) == 1) {
        if (s.i(0xBD0) == 1) {
            s.d(0xBE0, s.d(0xBE0) + 0.00025);
            if (s.d(0xBE0) >= 2.0) {
                if (s.i(0x788) == 1) {
```

recovered_grid.cpp L382-L385

```
                    s.d(0xBC0, s.d(0xBC0) + 0.00025);
                    if (s.d(0xBC0) > s.d(0xBC8)) {
                        s.d(0xBC0, 0.0);
                        s.i(0x788, 0);
```

0xBE0在本函数没有退出使能时的清零分支，重复启用不能一律假设重等2s。0xBD0不满足时也没有统一清输出动作。

9.2.3 惯量触发、频偏选向与边沿

触发判据为(|变化率|>C38且频偏×变化率>0)或|频偏|>C38。确认计时0x748先在<0.005s时加250μs，下一次进入else才置0x780；浮点累计和先判后增都应保留。动作幅值只有频偏严格高于正死区时写−A、严格低于负死区时写+A；只满足变化率条件但频偏在死区内时，本段不写幅值，不能套用−sign(Δf)A。

0x780从0到1且旧边沿0x784=0时置保持0x788；随后更新旧边沿。触发不成立而变化率严格小于阈值时才清确认及触发，等于变化率阈值时可保留状态。最后保持为0就清幅值及惯量输出。模式非1时仅清输出0x750与确认计时0x748，未把所有内部边沿/保持计时复位。

$$C_I = (\,|\,r\,|\, > \delta \wedge \Delta f\, r > 0) \vee (\,|\,\Delta f\,|\, > \delta) \tag{9-10}$$

$$A_k = \begin{cases} -A, & \Delta f > \delta, \\ +A, & \Delta f < -\delta, \\ A_{k-1}, & |\Delta f\,| \le \delta\,. \end{cases} \tag{9-11}$$

recovered_grid.cpp L396–L419

```
                if ((std::fabs(derivative) > threshold_a && product > 0.0)
                    || std::fabs(measured - f) > s.d(0xC38)) {
                    if (0.005 > s.d(0x748)) {
                        s.d(0x748, s.d(0x748) + 0.00025);
                    } else {
                        s.i(0x780, 1);
                        if (measured - f > s.d(0xC38)) {
                            s.d(0x790, -amplitude_scaled);
                        } else if (-s.d(0xC38) > measured - f) {
                            s.d(0x790, amplitude_scaled);
                        }
                    }
                } else if (threshold_b > std::fabs(derivative)) {
                    s.d(0x748, 0.0);
                    s.i(0x780, 0);
                }
                if (s.i(0x780) == 1 && s.i(0x784) == 0) {
                    s.i(0x788, 1);
                }
                s.i(0x784, s.i(0x780));
                if (s.i(0x788) == 0) {
                    s.d(0x790, 0.0);
                    s.d(0x750, 0.0);
                }
```

recovered_grid.cpp L420–L423

```
            }
        }
    } else {
        s.d(0x750, 0.0);
```

A=0x720=p[58]直接使用，未在源码乘4MW；频率与变化率共用C38数值不代表两者物理量纲相同。

9.2.4 一次调频频率边界与确认

上下频率边界分别为(f+C738)+0.005和(f−C738)−0.005，严格越过才累计对应确认0xBF8或0xBF0。模式2且0xBD0=1时共用2s延时0xBE0；确认计时先判<0.05再加步长，下一次才置触发方向0x798=1或2。

落入两边界之间及等号时清触发、保持模式0x7A0、两个确认计时和输出。若直接从低频越限跳到高频越限，当前分支不清另一方向确认计时；边沿检测还要求旧方向0x79C=0，因此方向1直接变2不必然开启新保持。

$$f_L = f_n - \delta_f - 0.005, \quad f_H = f_n + \delta_f + 0.005 \tag{9-12}$$

$$P_H = 1.05 c_H P_0, \quad P_L = 1.1 c_L P_0 \tag{9-13}$$

recovered_grid.cpp L427-L450

```
    const double upper_frequency = (f + s.d(0x738)) + 0.005;
    const double lower_frequency = (f - s.d(0x738)) - 0.005;
    const double positive_max = (s.d(0xC40) * 1.05) * s.d(0x7C0);
    const double negative_max = ((-s.d(0xC40)) * 1.05) * s.d(0x7C0);
    const double positive_min = (s.d(0xC48) * 1.1) * s.d(0x7C0);
    const double negative_min = ((-s.d(0xC48)) * 1.1) * s.d(0x7C0);
    if (s.i(0x740) == 2) {
        if (s.i(0xBD0) == 1) {
            s.d(0xBE0, s.d(0xBE0) + 0.00025);
            if (s.d(0xBE0) >= 2.0) {
                if (lower_frequency > measured) {
                    if (0.05 > s.d(0xBF0)) {
                        s.d(0xBF0, s.d(0xBF0) + 0.00025);
                    } else {
                        s.i(0x798, 1);
                    }
                } else if (measured > upper_frequency) {
                    if (0.05 > s.d(0xBF8)) {
                        s.d(0xBF8, s.d(0xBF8) + 0.00025);
                    } else {
                        s.i(0x798, 2);
                    }
                } else {
                    s.i(0x798, 0);
```

recovered_grid.cpp L451-L454

```
                    s.i(0x7A0, 0);
                    s.d(0xBF0, 0.0);
                    s.d(0xBF8, 0.0);
                    s.d(0x758, 0.0);
```

P0为0x7C0外部有功基准。声明中的negative_max虽为负值，后续负侧判断使用positive_max作为比较条件，不能擅自对称化。

9.2.5 一次调频保持、下垂与实际裁切

保持方向1/2各有独立计时0x7A8/0x7B8；先加250μs后严格超过0x7B0清模式与输出，否则计算−(实测频率−额定频率)×((1/fn)×P0/C30)。边沿置保持发生在计算之后，因此本拍新激活通常先保留0输出，再经后续限幅分支可能抬到最小动作幅度，不能声称只输出原始下垂。

483—492行必须按顺序解释：方向1且输出>正上限、幅值≥0.1P0时压到正上限；否则方向1且0≤输出≤正最小幅度时写正最小；否则方向2且负最小≤输出≤0时写负最小；否则方向2且正上限>输出、幅值≥0.1P0时写负最大；其余清0。正侧介于最小与最大之间且未命中首条件的值会清0，负侧可跳到负最大，该实现不是连续对称饱和器。

$$P_{\mathrm{droop}} = -\Delta f \frac{P_0}{f_n \delta}, \quad M = 0.1P_0 \tag{9-14}$$

$$P = \begin{cases} P_H, & m = 1 \wedge P_{\mathrm{raw}} > P_H \wedge |P_{\mathrm{raw}}| \geq M, \\ P_L, & m = 1 \wedge 0 \leq P_{\mathrm{raw}} \leq P_L, \\ -P_L, & m = 2 \wedge -P_L \leq P_{\mathrm{raw}} \leq 0, \\ -P_H, & m = 2 \wedge P_H > P_{\mathrm{raw}} \wedge |P_{\mathrm{raw}}| \geq M, \\ 0, & \text{其他.} \end{cases} \tag{9-15}$$

recovered_grid.cpp L457-L480

```
            if (s.i(0x7A0) == 1) {
                s.d(0x7A8, s.d(0x7A8) + 0.00025);
                if (s.d(0x7A8) > s.d(0x7B0)) {
                    s.d(0x7A8, 0.0);
                    s.i(0x7A0, 0);
                    s.d(0x758, 0.0);
                } else {
                    s.d(0x758, (-(measured - f)) * (((1.0 / f) * s.d(0x7C0)) / s.d(0xC30)));
                }
            } else if (s.i(0x7A0) == 2) {
                s.d(0x7B8, s.d(0x7B8) + 0.00025);
                if (s.d(0x7B8) > s.d(0x7B0)) {
                    s.d(0x7B8, 0.0);
                    s.i(0x7A0, 0);
                    s.d(0x758, 0.0);
                } else {
                    s.d(0x758, (-(measured - f)) * (((1.0 / f) * s.d(0x7C0)) / s.d(0xC30)));
                }
            }
            if (s.i(0x798) == 1 && s.i(0x79C) == 0) {
                s.i(0x7A0, 1);
            }
            if (s.i(0x798) == 2 && s.i(0x79C) == 0) {
                s.i(0x7A0, 2);
```

recovered_grid.cpp L481-L484

```
            }
            s.i(0x79C, s.i(0x798));
            if (s.d(0x758) > positive_max && std::fabs(s.d(0x758)) >= 0.1 * s.d(0x7C0) && s.i(0x7A0)
 == 1) {
                s.d(0x758, positive_max);
```

最小动作幅度分支没有400kW门槛，不应写所有输出须先超过0.1×额定功率。源码原始比较顺序和负侧条件均需保留。

9.2.6 模式清理、监测模式与单侧总限幅

模式非2时清一次调频输出及0xBD8；方向裁切完成后，按惯量输出非0且模式1、或调频输出非0且模式2选择监测0xC60。然后将两个输出相加写0x760，合成值加P0不超过1则直接送0x768，否则0x768写1−P0。

总限幅没有负侧下限，也没有在这里把惯量幅度乘额定功率或把下垂输出除额定功率。源码把上界常数1与外部有功基准相加比较，工程使用必须追踪外部0x1D58实际单位，不能用理想功频控制图替代该数值行为。

$$P_{\text{sum}} = P_I + P_D, \quad P_{\text{out}} = \min\left(P_{\text{sum}}, 1 - P_0\right) \tag{9-16}$$

recovered_grid.cpp L485–L508

```
            } else if (positive_min >= s.d(0x758) && s.d(0x758) >= 0.0 && s.i(0x7A0) == 1) {
                s.d(0x758, positive_min);
            } else if (s.d(0x758) >= negative_min && 0.0 >= s.d(0x758) && s.i(0x7A0) == 2) {
                s.d(0x758, negative_min);
            } else if (positive_max > s.d(0x758) && std::fabs(s.d(0x758)) >= 0.1 * s.d(0x7C0) &&
s.i(0x7A0) == 2) {
                s.d(0x758, negative_max);
            } else {
                s.d(0x758, 0.0);
            }
        }
    } else {
        s.d(0x758, 0.0);
        s.d(0xBD8, 0.0);
    }

    if (s.d(0x750) != 0.0 && s.i(0x740) == 1) {
        s.i(0xC60, 1);
    } else if (s.d(0x758) != 0.0 && s.i(0x740) == 2) {
        s.i(0xC60, 2);
    } else {
        s.i(0xC60, 0);
    }
    s.d(0x760, s.d(0x750) + s.d(0x758));
    if (1.0 >= s.d(0x760) + s.d(0x7C0)) {
```

recovered_grid.cpp L509–L512

```
        s.d(0x768, s.d(0x760));
    } else if (s.d(0x760) + s.d(0x7C0) > 1.0) {
        s.d(0x768, 1.0 - s.d(0x7C0));
    }
```

min表达式只描述有限正常数；两次条件都遇NaN会不写0x768并保留旧值。后续叠加在controller.cpp:170，发生在motor_calculate之前。

9.3 频率支撑模式的命令入口与总调度门控

频率模式选择和调用许可分开存储。commands[10]只决定支撑函数内部采用哪条模式；总调度是否调用，还受机侧共享状态全局0x1E08的double门控。完整惯量和下垂实现须结合网侧频率支撑章节阅读。

9.3.1 模式0/1/2与共享许可的两层关系

commands[10]经CVTT保存为整数0x740。grid_frequency_support以1为惯量模式、2为一次调频模式，其他值不是自动同时运行两种模式；commands[10]=0也不能单独解释主入口的调用许可。

总调度在controller第167～169行仅当double全局0x1E08==1时调用支撑。0x1E08折成机侧局部0x350，属于机侧输出使能；跨区0xBD0又来自机侧局部0x1C。这些许可与命令模式、启动延时均有各自作用。无论本拍是否执行支撑，主入口第170行仍叠加已存0x768，因此关闭或失去许可后的残留贡献需要按支撑及复位实际写入路径判断，不能凭模式标签承诺为零。

recovered_commands.cpp L150-L158

```
    s.i(0x2088, cvtt_i32(commands[6]));
    s.i(0x7D8, cvtt_i32(commands[7]));
    s.i(0x23D0, cvtt_i32(commands[7]));
    s.i(0x21A4, cvtt_i32(commands[8]));
    s.i(0x628, cvtt_i32(commands[9]));
    s.i(0x21A0, cvtt_i32(commands[9]));
    s.i(0x740, cvtt_i32(commands[10]));
    s.d(0xCA0, s.d(0x2038));
}
```

内部支撑模式与网侧使能、机侧ready及状态恢复共同决定活动行为。

恢复源码不足以证明某外部工程Cmd8当前值或历史支撑输出。

9.4 机侧事件记录与穿越恢复源码对照

机侧低穿事件和恢复链独立于网侧频率变化率、惯量功率及一次调频算法；本节说明参考记录、模式选择、计数、斜坡与高穿进入门控的逐拍关系。

9.4.1 低穿事件计时与10～20ms记录窗

此机侧低穿事件与网侧df/dt/下垂频率支撑不同。57C==1、模式1、跌落4E0≥0.15时B88加250μs，严格>0.15s才置1BD0；其它情形仅57C不为1才清事件和计时。大功率仍成立但离开低穿或跌落不足时没有清计时分支，旧计时可保持。

事件位1后70计时严格>0.02s清90，else if严格>0.01s置90=1，因此记录窗(10ms,20ms]。其它时段不写90；事件位0时清累计值与采样计数但不清90，采集仍同时要求90==1和事件位1。

记录累计外部动态1C40、D0/410和参考88，样本78递增，分别生成P_pos均值1C48、电压均值1C60和参考均值80。恢复模式1/3使用不同均值字段。

$$\Delta V = 1 - V_d^{+} \,/\, V_b, \quad \Delta V \geq 0.15 \wedge 0x57C = 1 \wedge 0x10 = 1 \tag{9-17}$$

$$\mathrm{record} = (0.01 < t \le 0.02) \wedge 0x1BD0 = 1 \tag{9-18}$$

$$P_{pos}^{-} = \frac{\sum d(0x1C40)}{N}, \quad I_{ref}^{-} = \frac{\sum d(0x88)}{N} \tag{9-19}$$

recovered_motor.cpp L911-L934

```
    if (s.i(0x57C) == 1 && s.i(0x10) == 1 && s.d(0x4E0) >= 0.15) {
        s.d(0xB88, s.d(0xB88) + 0.00025);
        if (s.d(0xB88) > 0.15) s.i(0x1BD0, 1);
        else s.i(0x1BD0, 0);
    } else if (s.i(0x57C) != 1) {
        s.i(0x1BD0, 0);
        s.d(0xB88, 0.0);
    }
    if (s.i(0x1BD0) == 1) {
        s.d(0x70, s.d(0x70) + 0.00025);
        if (s.d(0x70) > 0.02) s.d(0x90, 0.0);
        else if (s.d(0x70) > 0.01) s.d(0x90, 1.0);
    } else {
        s.d(0x98, 0.0);
        s.d(0x1C50, 0.0);
        s.d(0x1C58, 0.0);
        s.d(0x78, 0.0);
        s.d(0x70, 0.0);
    }
    if (s.d(0x90) == 1.0 && s.i(0x1BD0) == 1) {
        s.d(0x1C50, s.d(0x1C50) + s.d(0x1C40));
        s.d(0x1C58, s.d(0x1C58) + s.d(0xD0) / s.d(0x410));
        s.d(0x98, s.d(0x98) + s.d(0x88));
        s.d(0x78, s.d(0x78) + 1.0);
```

recovered_motor.cpp L935-L938

```
        s.d(0x80, s.d(0x98) / s.d(0x78));
        s.d(0x1C48, s.d(0x1C50) / s.d(0x78));
        s.d(0x1C60, s.d(0x1C58) / s.d(0x78));
    }
```

浮点累加的严格比较决定实际首动作拍；不只按十进制除法宣称固定调用数。

9.4.2 恢复参考、80次写入与超时后写

恢复需模式非LVRT、5D0==1、570==1、57C==1、1BD0==1同时满足，任一失败973行直接清570。离开低穿并不保证恢复链执行。

56C<80时以double精确比较B60=1/2/3：分别取参考均值80、B48×kT、P_pos均值1C48×kT，同时写88及B0并加56C；其它B60值不加计数。80次合格调用期间已经写参考，下一次才算恢复步长，不是80拍完全无输出的等待。

计数到80后390按目标与所选恢复参考差、B58、kT及Ts生成，再清80/90/1C48/1C60。B0==88精确相等时清570、56C、1BD0及分类标志；判断在公共slew前，可提前结束恢复。

974–983只有570仍为1才累加580，严格>B58清事件、记录使能、计时和56C，但不直接清570；下一拍事件位失效使门控清570，574本拍仍复制当前570。HVRT进入早已读取旧580，本拍末清零影响下一拍进入资格。

$$I_{rec}=\begin{cases}\bar{I_{ref}} & B60=1\\ B48k_T & B60=2\\ \bar{P_{pos}}k_T & B60=3\end{cases} \tag{9-20}$$

$$\Delta I_{rec}=-\frac{P^{*}-P_{rec}}{T_{rec}}k_T T_s \tag{9-21}$$

recovered_motor.cpp L940–L963

```
    if (s.i(0x10) != 1 && s.i(0x5D0) == 1 && s.i(0x570) == 1 && s.i(0x57C) == 1 && s.i(0x1BD0) == 1)
{
        if (s.i(0x56C) < 80) {
            if (s.d(0xB60) == 1.0) {
                s.d(0x88, s.d(0x80));
                s.d(0xB0, s.d(0x88));
                s.increment(0x56C);
            } else if (s.d(0xB60) == 2.0) {
                s.d(0x88, s.d(0xB48) * s.d(0x548));
                s.d(0xB0, s.d(0x88));
                s.increment(0x56C);
            } else if (s.d(0xB60) == 3.0) {
                s.d(0x88, s.d(0x1C48) * s.d(0x548));
                s.d(0xB0, s.d(0x88));
                s.increment(0x56C);
            }
        } else {
            if (s.d(0xB60) == 1.0) {
                s.d(0x390, ((-((s.d(0x2A0) - s.d(0x80) / s.d(0x548)) / s.d(0xB58))) * s.d(0x548)) *
0.00025);
            } else if (s.d(0xB60) == 2.0) {
                s.d(0x390, ((-((s.d(0x2A0) - s.d(0xB48)) / s.d(0xB58))) * s.d(0x548)) * 0.00025);
            } else if (s.d(0xB60) == 3.0) {
                s.d(0x390, ((-((s.d(0x2A0) - s.d(0x1C48)) / s.d(0xB58))) * s.d(0x548)) * 0.00025);
            }
            s.d(0x80, 0.0);
```

recovered_motor.cpp L964–L967

```
            s.d(0x90, 0.0);
            s.d(0x1C48, 0.0);
            s.d(0x1C60, 0.0);
            if (s.d(0xB0) == s.d(0x88)) {
```

清理968–973、超时974–983、slew985–988；Control75模式3参考是动态记录均值，改Control65不改它。

第10章　保护、联锁、锁存与复位

10.1 保护与穿越是两条不同链

穿越按基波幅值及正负序估计切换调节模式；保护按RMS、频率、定时和使能位决定跳闸。相电压幅值基准930.6928V与线电压RMS额定1140V不相同；同一波形在两种统计口径下的数值不能直接比较。



图 10　启动就绪条件与保护、使能之间的关系

控制实现与逐拍关系

运行状态机

两侧各有四态状态机：0停机、1运行、2停机过程中、3跳闸。网侧post优先处理0x34C=1到状态3；否则状态1且启动命令0到状态2、状态0且启动命令1到状态1。网侧跳闸仅清指定五个标志与八个浮点槽，并重算母线极性标志，未在该块清全部六路占空比缓存。网侧复位0x520=1则整块清零0x1AB8字节、重写默认并返回；机侧清理与复位范围按第8章源码逐段说明。

RMS 有效值计算

各路信号平方累加，窗口满后开方并清零累加器：

$$X_{\mathrm{rms}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} x_k^2} \tag{10-1}$$

网侧RMS统计各采样量；机侧包含跨区搬入的电网电压。额定50Hz活动窗口80拍20ms，自适应窗口下限cvtt(4000/(f+10))=66、上限100，不能把下限66当活动窗口。转子电流窗口另按转差频率自适应并保护零转差。

频率测量

每步累加 PLL 角频率，每满一个工频周期算一次平均值，并把 RMS 窗口钳位在 f_n±10 Hz 对应范围内：

$$f = \frac{1}{2\pi N}\sum \omega_{\mathrm{PLL}} \tag{10-2}$$

过压/欠压保护

过压快速段：电压越限即累积时间，超过定值时长跳闸 trip(10)。过压三段式与欠压四段式结构相同：每段一个阈值 + 一个允许时长，A/B 两相任一相越限累积计时（每步 +0.00025 s），恢复即清零，超时跳闸（过压 trip(10)、欠压 trip(11)）。保护均受总门和组使能控制；快速过压不要求运行门或RMS刷新门，三段过压和四段欠压另要求0x2C=1与0x480=1；过频/欠频另要求0x2C=1与0x47C=1。本工程定值（Protect_1～Protect_16，阈值为标幺，存入时乘 1140 有名值化）：

表 37　状态分支与定值

保护段	阈值（标幺）	允许时长（s）	跳闸位
过压 1 段（快速）	1.33	0.15	trip(10)
过压 2 段	1.28	0.6	trip(10)
过压 3 段	1.23	1.1	trip(10)
过压 4 段	1.13	10.4	trip(10)
欠压 1 段	0.12	0.3	trip(11)
欠压 2 段	0.4	1.4	trip(11)
欠压 3 段	0.6	2.2	trip(11)
欠压 4 段	0.83	3.0	trip(11)

$$t_k = \begin{cases} t_{k-1} + \Delta t, & V > V_{th} \\ 0, & V \le V_{th} \end{cases} \tag{10-3}$$

跳闸延时比较分组不同：网侧过压三段为t_k≥T_dur；快速过压、四段欠压、三段过频及三段欠频为t_k>T_dur。门控关闭时计时器冻结，恢复到阈值正常侧时才在有效门控内清零。

过频/欠频保护

过频三段与欠频三段结构同电压保护，跳闸位 trip(14)/trip(15)，受频率更新标志 0x47C 门控。本工程定值（Protect_17～Protect_28）：

表 38　状态分支与定值

保护段	阈值（Hz）	允许时长（s）	跳闸位
过频 1 段	54	0.2	trip(14)
过频 2 段	53	10	trip(14)
过频 3 段	52	2100	trip(14)
欠频 1 段	46	0.2	trip(15)

保护段	阈值（Hz）	允许时长（s）	跳闸位
欠频 2 段	47	10	trip(15)
欠频 3 段	47.5	2100	trip(15)

故障锁存

保护动作写入跳闸位与锁存字（网侧 0x7F0，机侧 0x934）：网侧一类故障bit0在post末段将状态置3；二类bit1在运行门为1时累计到≥4s才置3，运行门不为1时直接置3。两类同时存在时bit0优先。机侧时序以对应post源码为准。锁存置位后本拍不立即清输出，下一拍状态机转 3 才清（原 DLL 行为，原样保留）。

自动复归的可达性与外部状态

自动复归机制的潜在阈值为8000控制拍（2s），受0x3738以及两侧相关门控控制。当前源码内部没有把整链门控激活的写入路径，不能断言当前配置跳闸后自动重启。外部宿主或状态恢复可改变状态；重新验证时需记录实际初始状态。

源码中“无内部写入”不意味着地址在整个宿主系统里不可写。排查时应同时记录初态、快照、命令边沿、门控、RMS/频率更新标志和实际读者。保护阈值达到而没有跳闸，首先检查使能位和更新标志，再看连续/累计计时与锁存优先级。

10路RMS/电压定值

表 39　10路RMS/电压定值：参数与状态关系

序号	输入源	累加器	输出	用途（如实披露）
1	线电压 vab（0xC0）	0x3F0	0x3C0	电压保护全部 8 段的比较对象
2	线电压 vbc（0xC8）	0x400	0x3D0	同上（与 vab 共同判定）
3	线电压 vca（0xD0）	0x410	0x3E0	照常计算，但没有任何保护段读取（披露）
4–6	三相电流 ia/ib/ic（0xD8/0xE0/0xE8）	0x438/0x440/0x448	0x420/0x428/0x430	照常计算，但没有任何保护段读取（披露）
7	网侧相电压 RMS（机侧局部0xF0，由相电压重构量统计）	0x458	0x450	照常计算，但没有任何保护段读取（披露）
8–10	相电压 va/vb/vc（0x1718/0x1720/0x1728，由线电压反推）	0x3F8/0x408/0x418	0x3C8/0x3D8/0x3E8	照常计算，但没有任何保护段读取（披露）

表 40　10路RMS/电压定值：参数与状态关系

段名	标幺门槛	有名值（V）	动作延时（s）	跳闸码	备注
过压快速段	1.33	1516.2	0.15	10	唯一无运行门/拍门的段：第 1 拍起每拍判定
过压 1 段	1.28	1459.2	0.6	10	0.1 s 软启动 + 锁存生效后判定
过压 2 段	1.23	1402.2	1.1	10	同上
过压 3 段	1.13	1288.2	10.4	10	最灵敏、最慢；过压组中唯一可能 在穿越中被"轻轻碰到"的一段
欠压 1 段	0.12	136.8	0.3	11	对应深度跌落（近零电压）
欠压 2 段	0.40	456.0	1.4	11	—
欠压 3 段	0.60	684.0	2.2	11	LVRT 工况中计时逼近 1.998 s（见 §7.7）
欠压 4 段	0.83	946.2	3.0	11	最灵敏；轻微的电压跌落都会让它 开始计时

频率定值与阶梯

表 41　频率定值与阶梯：参数与状态关系

段名	定值（Hz）	动作延时（s）	跳闸码	备注
过频 1 段	54	0.2	14	快速段
过频 2 段	53	10	14	—
过频 3 段	52	2100	14	2100 s = 35 分钟，实际几乎不动作
欠频 1 段	46	0.2	15	快速段
欠频 2 段	47	10	15	—
欠频 3 段	47.5	2100	15	严格大于比较：实测恰为 47.5 Hz 时计时保 持 0（四工况实证）

计时、锁存、跳闸、复位

表 42　计时、锁存、跳闸、复位：参数与状态关系

跳闸码	含义	置位路径
10	过压（快速段 / 1 / 2 / 3 段任一动作）	trip(10)
11	欠压（1 / 2 / 3 / 4 段任一动作）	trip(11)
14	过频（1 / 2 / 3 段任一动作）	trip(14)
15	欠频（1 / 2 / 3 段任一动作）	trip(15)

联锁与四工况计时峰值

表 43　联锁与四工况计时峰值：参数与状态关系

工况	端子扰动（**1 ms** 录波 **20** 样本滚动 **RMS**）	有累积的段与计时峰值	对应定值	结论
独立测试记录（AB-G 5～6s，非现工程相别）	跌落体 ≈ 485 V（0.43 pu）	过压 3：0.277 s；欠压 2：0.06–0.10 s；欠压 3：0.99 s；欠压 4：1.01 s	10.4 / 1.4 / 2.2 / 3.0 s	0x7DC 恒 0，不跳闸
LVRT（8–10 s）	跌落体 ≈ 483 V（0.42 pu）	过压 3：0.2575 s；欠压 3：1.998 s；欠压 4：1.998 s	10.4 / 2.2 / 3.0 s	0x7DC 恒 0，不跳闸（欠压 3 距定值仅 0.2 s）
12 s 两相接地	跌落体 ≈ 716 V（0.63 pu）	过压 3：0.178 s；欠压 3：0.06 s；欠压 4：0.998 s	10.4 / 2.2 / 3.0 s	0x7DC 恒 0，不跳闸
HVRT（16–17 s）	抬升 RMS 峰值 ≈ 1381.3 V（1.21 pu；原始采样峰值 1650–1697 V）	过压 3：0.989 s；欠压 3：0.99 s；欠压 4：1.01 s	10.4 / 2.2 / 3.0 s	0x7DC 恒 0，不跳闸（过压 2 门槛 1402.2 V 未触及）
频率（全部工况）		实测频率 0x468 最低 = 47.5 Hz，恰等于欠频 3 门槛；严格大于号比较 → 计时恒 0	2100 s	0x7DC 恒 0，不跳闸

表 44　联锁与四工况计时峰值：参数与状态关系

通道	口径（源码与工程接线复核）	取值含义
Grid_FRT	PSCAD 组件直通：Grid_FRT = Grid_Output(12) = 0x18 × 100	0 = 正常，100 = LVRT，200 = HVRT
Rotor_FRT	Rotor_FRT = Rotor_Output(11) = 0x10 × 100（机侧）	同上

10.2 网侧保护源码逐段对照

网侧保护由grid_post执行，统计刷新和保护计时在同一函数中。保护总门、组使能、运行门与测量锁存门分别判断；动作阈值、计时等号、跳闸状态和输出撤销必须按分支顺序解释。

10.2.1 运行状态优先级、跳闸清理及复位

post首先处理外部跳闸命令：0x34C=1直接置状态3；否则仅状态1且启动命令0时转2，仅状态0且启动命令1时转1。状态2不会因为启动命令1立即回到状态1；状态3也不会自动重启。

当前状态3时清五个整型标志和八个浮点槽，并按两段母线电压各自严格>0重新生成极性标志。这是明确的槽位清理，不是把全部状态和六路占空比缓存清零。手动复位0x520=1随后整块清0x1AB8字节、重写默认并return，本拍不再进行测量及保护。

recovered_grid.cpp L125-L148

```
extern "C" __declspec(dllexport) void grid_post(double f, void* state) {
    GridState s(state);
    if (s.i(0x34C) == 1) {
        s.i(0x350, 3);
    } else if (s.i(0x350) == 1 && s.i(0x348) == 0) {
        s.i(0x350, 2);
    } else if (s.i(0x350) == 0 && s.i(0x348) == 1) {
        s.i(0x350, 1);
    }

    if (s.i(0x350) == 3) {
        for (std::size_t offset : {0x18, 0x50C, 0x2C, 0x510, 0x514}) {
            s.i(offset, 0);
        }
        for (std::size_t offset : {0x4B8, 0x4D0, 0x4E8, 0x4A8, 0x4B0, 0xD88, 0xD90, 0xD98}) {
            s.d(offset, 0.0);
        }
        s.i(0x504, s.d(0x100) > 0.0 ? 1 : 0);
        s.i(0x32C, s.i(0x504));
        s.i(0x508, s.d(0x108) > 0.0 ? 1 : 0);
        s.i(0x330, s.i(0x508));
    }

    if (s.i(0x520) == 1) {
```

recovered_grid.cpp L149-L152

```
        std::memset(state, 0, 0x1AB8);
        grid_defaults(f, state);
        return;
    }
```

跳闸清理槽为0x18/0x50C/0x2C/0x510/0x514和0x4B8/0x4D0/0x4E8/0x4A8/0x4B0/0xD88/0xD90/0xD98；两种清理范围不同。

10.2.2 频率测量与自适应RMS窗口

每拍累加PLL角频率0x11A0并增0x478，达到固定测量窗口0x3A0后先以实际计数求平均频率0x468，再向零截断4000/频率写RMS窗口0x464，清累加、置刷新锁存0x47C=1。限幅的是整数RMS窗口，不是频率0x468本身。

窗口下限向零截断4000/(fn+10)，上限向零截断4000/|fn−10|；50Hz为66至100，稳态80。正序滤波d量0x11D0另按当前0x464平均后除1.414，写0x3A8；它不是平方平均意义的RMS。

$$f_{\mathrm{meas}} = \frac{\sum_{j=1}^{N} \omega_j}{2\pi N}, \quad N_{\mathrm{RMS}} = \operatorname{clamp}\left(\operatorname{cvtt} \frac{4000}{f_{\mathrm{meas}}},\ \operatorname{cvtt} \frac{4000}{f_n + 10},\ \operatorname{cvtt} \frac{4000}{|f_n - 10|} \right) \tag{10-4}$$

recovered_grid.cpp L155-L175

```
    s.d(0x470, s.d(0x470) + s.d(0x11A0));
    s.increment(0x478);
    if (s.i(0x478) >= s.i(0x3A0)) {
        s.d(0x468, ((s.d(0x470) / static_cast<double>(s.i(0x478))) / 2.0) / 3.14159265358979);
        s.i(0x464, cvtt_i32(4000.0 / s.d(0x468)));
        s.i(0x478, 0);
        s.d(0x470, 0.0);
        s.i(0x47C, 1);
        if (s.i(0x464) < cvtt_i32(4000.0 / (f + 10.0))) {
            s.i(0x464, cvtt_i32(4000.0 / (f + 10.0)));
        } else if (s.i(0x464) > cvtt_i32(4000.0 / std::fabs(f - 10.0))) {
            s.i(0x464, cvtt_i32(4000.0 / std::fabs(f - 10.0)));
        }
    }

    s.d(0x3B0, s.d(0x3B0) + s.d(0x11D0));
    s.increment(0x3B8);
    if (s.i(0x3B8) >= s.i(0x464)) {
        s.d(0x3A8, (s.d(0x3B0) / static_cast<double>(s.i(0x3B8))) / 1.414);
        s.i(0x3B8, 0);
        s.d(0x3B0, 0.0);
```

cvtt_i32使用SSE向零截断，非法或越界转换不能按普通整数转换推断。更新0x464发生在后续RMS判窗之前。

10.2.3 十路RMS、刷新标志与统计边界

十路源/累加/输出映射由RmsRow表明确给出：三路线电压、三相电流、0xF0信号以及三路重建相电压。先增共用计数0x460，逐路累计当前样本平方；计数达到当拍0x464后以实际计数作分母开方，更新全部十路结果，再清各累加器和计数，置0x480=1。

RMS是整窗输出并在窗口间保持，不是每拍滚动窗口。0x480和0x47C在本文件没有普通运行清零分支，首次刷新后保持1直到整块复位，因此后续保护每拍使用最近一次统计结果判断，而非仅在新测量拍计时。

$$X_{\mathrm{RMS}} = \sqrt{\frac{1}{N_{\mathrm{actual}}} \sum_{j=1}^{N_{\mathrm{actual}}} x_j^2} \tag{10-5}$$

recovered_grid.cpp L180-L200

```
    const RmsRow rms_rows[] = {
        {0xC0, 0x3F0, 0x3C0}, {0xC8, 0x400, 0x3D0}, {0xD0, 0x410, 0x3E0},
        {0xD8, 0x438, 0x420}, {0xE0, 0x440, 0x428}, {0xE8, 0x448, 0x430},
        {0xF0, 0x458, 0x450}, {0x1718, 0x3F8, 0x3C8},
        {0x1720, 0x408, 0x3D8}, {0x1728, 0x418, 0x3E8}
    };
    s.increment(0x460);
    for (const RmsRow& row : rms_rows) {
        const double source = s.d(row.source);
        s.d(row.accumulator, s.d(row.accumulator) + source * source);
    }
    if (s.i(0x460) >= s.i(0x464)) {
        for (const RmsRow& row : rms_rows) {
            s.d(row.output, std::sqrt(s.d(row.accumulator) / static_cast<double>(s.i(0x460))));
        }
        for (const RmsRow& row : rms_rows) {
            s.d(row.accumulator, 0.0);
        }
        s.i(0x460, 0);
        s.i(0x480, 1);
    }
```

电压保护实际只读0x3C0与0x3D0；第三路线电压仍统计但不进入本文件的保护判据。

10.2.4 快速过压与过压三段的门控差异

保护总门为(0x7E4&0x8000)或(0x7D8&0x40000000)，组使能enabled(0x400)还检查0x7E0或0x7D8的对应位。快速过压不要求运行0x2C或刷新0x480；它按A/B最近RMS严格大于阈值分别累加250μs，否则清各自计时，计时严格>0x918时trip(10)。

过压三段另要求0x2C=1且0x480=1，先更新三段两相计时，再分别检查任一路计时≥该段允许时长；动作时将该段两个计时都清0并trip(10)。门控不成立时没有清计时动作，会冻结已有计时。

$$t_k = \begin{cases} t_{k-1} + 0.00025, & V > V_{\mathrm{th}}, \\ 0, & V \le V_{\mathrm{th}}. \end{cases} \quad \text{仅在该组门控成立时执行} \tag{10-6}$$

recovered_grid.cpp L205-L228

```
    if ((s.u(0x7E4) & 0x8000) != 0 || (s.u(0x7D8) & 0x40000000) != 0) {
        if (s.enabled(0x400)) {
            s.d(0x908, s.d(0x3C0) > s.d(0x900) ? s.d(0x908) + 0.00025 : 0.0);
            s.d(0x910, s.d(0x3D0) > s.d(0x900) ? s.d(0x910) + 0.00025 : 0.0);
            if (s.d(0x908) > s.d(0x918)) {
                s.trip(10);
            }
            if (s.d(0x910) > s.d(0x918)) {
                s.trip(10);
            }
        }

        if (s.enabled(0x400) && s.i(0x2C) == 1 && s.i(0x480) == 1) {
            const VoltageStage stages[] = {
                {0x920, 0x928, 0x930, 0x938},
                {0x940, 0x948, 0x950, 0x958},
                {0x960, 0x968, 0x970, 0x978}
            };
            for (const VoltageStage& stage : stages) {
                s.d(stage.timer_a, s.d(0x3C0) > s.d(stage.limit) ? s.d(stage.timer_a) + 0.00025 :
0.0);
                s.d(stage.timer_b, s.d(0x3D0) > s.d(stage.limit) ? s.d(stage.timer_b) + 0.00025 :
0.0);
            }
            for (const VoltageStage& stage : stages) {
                if (s.d(stage.timer_a) >= s.d(stage.duration) || s.d(stage.timer_b) >=
s.d(stage.duration)) {
```

recovered_grid.cpp L229-L231

```
                    s.d(stage.timer_a, 0.0);
                    s.d(stage.timer_b, 0.0);
                    s.trip(10);
```

快速段比较>时长；三段过压比较≥时长；阈值等号均视为不越限。

10.2.5 欠压和频率保护的严格比较

欠压四段要求总门、enabled(0x800)、运行0x2C=1及RMS锁存0x480=1。判据为门槛严格大于实测A/B RMS，各自累计，任一路计时严格>延时才清双相计时并trip(11)。欠压延时与过压三段的≥不同。

过频三段要求运行、频率锁存0x47C及enabled(0x4000)，实测频率严格大于门槛时增计时，计时严格>延时trip(14)，否则计时清0。273—287行欠频结构对应enabled(0x8000)、门槛严格大于实测频率、trip(15)。频率恰等于门槛时计时清0。

$$C_{\mathrm{UV}}{:}V < V_{\mathrm{th}},\quad C_{\mathrm{OF}}{:}f > f_{\mathrm{th}},\quad C_{\mathrm{UF}}{:}f < f_{\mathrm{th}} \tag{10-7}$$

$$C_{\mathrm{trip,\,OV3}}{:}t \geq T,\quad C_{\mathrm{trip,\,fast\,/\,UV\,/\,OF\,/\,UF}}{:}t > T \tag{10-8}$$

recovered_grid.cpp L236-L259

```
        if (s.enabled(0x800) && s.i(0x2C) == 1 && s.i(0x480) == 1) {
            const VoltageStage stages[] = {
                {0x980, 0x988, 0x990, 0x998},
                {0x9A0, 0x9A8, 0x9B0, 0x9B8},
                {0x9C0, 0x9C8, 0x9D0, 0x9D8},
                {0x9E0, 0x9E8, 0x9F0, 0x9F8}
            };
            for (const VoltageStage& stage : stages) {
                s.d(stage.timer_a, s.d(stage.limit) > s.d(0x3C0) ? s.d(stage.timer_a) + 0.00025 :
0.0);
                s.d(stage.timer_b, s.d(stage.limit) > s.d(0x3D0) ? s.d(stage.timer_b) + 0.00025 :
0.0);
            }
            for (const VoltageStage& stage : stages) {
                if (s.d(stage.timer_a) > s.d(stage.duration) || s.d(stage.timer_b) >
s.d(stage.duration)) {
                    s.d(stage.timer_a, 0.0);
                    s.d(stage.timer_b, 0.0);
                    s.trip(11);
                }
            }
        }

        if (s.i(0x2C) == 1 && s.i(0x47C) == 1 && s.enabled(0x4000)) {
            const FrequencyStage stages[] = {
                {0xA78, 0xA80, 0xA88}, {0xA90, 0xA98, 0xAA0}, {0xAA8, 0xAB0, 0xAB8}
            };
```

recovered_grid.cpp L260-L263

```
            for (const FrequencyStage& stage : stages) {
                if (s.d(0x468) > s.d(stage.limit)) {
                    s.d(stage.timer, s.d(stage.timer) + 0.00025);
                    if (s.d(stage.timer) > s.d(stage.duration)) {
```

禁用组或总门关闭均冻结而非清计时。trip写0x7E8和0x7DC对应bit，并置0x7F0 bit0；不是把跳闸字覆盖为单一故障码。

10.2.6 故障锁存与次拍清输出

本拍保护在前面trip置0x7F0 bit0后，末段立刻把运行状态0x350改为3；清输出块位于函数前部且已经执行过，故此次新保护动作不会返回前部重清输出。下一次grid_calculate先看状态3跳过控制后，下一次grid_post才执行状态3清理。这一顺序决定故障动作拍与使能撤销拍的差别。

bit0优先于bit1。仅bit1置位且运行门0x2C=1时累计0x7F8，达到4.0s置状态3并封顶计时；运行门不为1时不等待，直接置3。两位均未置时才清延时计时。bit0成立分支不清0x7F8。

recovered_grid.cpp L291-L307

```
// ---- 故障锁存：置位后本拍不立刻清输出，下一拍状态机转 3 才清 ----
    // A fault latched here clears outputs on the next call, not this one.
    if ((s.u(0x7F0) & 1) != 0) {
        s.i(0x350, 3);
    } else if ((s.u(0x7F0) & 2) != 0) {
        if (s.i(0x2C) == 1) {
            s.d(0x7F8, s.d(0x7F8) + 0.00025);
            if (s.d(0x7F8) >= 4.0) {
                s.i(0x350, 3);
                s.d(0x7F8, 4.0);
            }
        } else {
            s.i(0x350, 3);
        }
    } else {
        s.d(0x7F8, 0.0);
    }
```

状态3已被上游命令置位则本次post前部就清输出，不需再等下一次。延时计数只在该分支满足时推进。

第11章 工况验证与结果判据

11.1 验证记录与适用范围

以下数值为项目验证记录，用于说明工况设计和判读方式。本次没有重新执行 PSCAD、调用基准 DLL 或复跑录波比较，因此不能直接证明任意修改后的实现仍与基准一致。静态核对能够确认判据、计数、缩放和读写路径，动态行为须再次测试。

表 45 四工况定义与基线：参数与状态关系

工况	扰动类型与时窗	端子实测表现	穿越模式	保护结论
A 基线	独立测试AB-G两相接地5～6s；现工程三相接地5～6s	跌落体 ≈ 485 V（0.43 pu）	LVRT（100）	不跳闸
B LVRT	三相电压跌落至 0.2 pu，8–10 s	跌落体 ≈ 483 V（0.42 pu）	LVRT（100）	不跳闸
C 12 s 故障	两相接地短路，12–13 s	跌落体 ≈ 716 V（0.63 pu）	LVRT（100）	不跳闸
D HVRT	电压抬升至 1.3 pu，16–17 s（另含 5–6 s AB-G 段）	抬升 RMS 峰值 ≈ 1381.3 V（1.21 pu）	LVRT＋HVRT	不跳闸

表 46 四工况结果：参数与状态关系

工况	触发设置	工况目的	输出对比	结论
25s 基线	独立记录含5～6s AB-G；当前两份pscx均保存三相接地5～6s	常态控制：调度、两侧控制、调制全链路	47/47 文件	逐字节一致
12s AB 两相接地故障	t=12s 电网 A、B 两相接地	不对称跌落：正负序分解、负序抑制、机侧穿越动作	47/47 文件	逐字节一致
8s LVRT	t=8s 电压跌至 0.2pu	深跌落检测、穿越分类、中点平衡、无功支撑与电流闭锁	47/47 文件	逐字节一致
16s HVRT	t=16s 电压抬至 1.3pu	高压穿越：过压检测与恢复	47/47 文件	逐字节一致

扰动设置与端子实测值描述不同位置。设置为 0.2 pu 的跌落，不要求机端滚动 RMS 必然等于 0.2 pu；它还受电气模型、扰动位置、参考基准和窗口影响。LVRT/HVRT 的 100/200 为状态码经 ×100 后的监控值，内部模式仍为 1/2。第14章区分当前工程实际配置和其余验证工况。

11.2 保护计时与穿越判读

记录中的欠压第三段最高计时为 1.998 s，低于 2.2 s 延时；高压扰动的 RMS 峰值约 1381.3 V，低于 1.23×1140=1402.2 V 的过压第二段门槛。频率等于 47.5 Hz 时，严格比较的欠频第三段不累计。实际动作还须结合第10章的独立门控、更新条件与比较符号。

11.3 三层验证方法

表 47　验证层次

层次	检查内容	应保存的结果
函数差分	同初态同输入逐函数比较输出和状态	断言范围、失败项、NaN 掩码、有限差值、源码与 DLL 哈希
真实宿主接口	按 ABI、数组长度与布局连续调用	导出表、调用次数、完整 14152 字节状态快照
闭环同工况	基准与待测实现分别运行同一扰动	文件清单、时间轴、每通道样本数、有限差值、NaN 位置和文件哈希

项目验证记录包含 91 项函数差分、每个 DLL 各 100 次宿主接口调用，以及四工况各 47 文件、470 工程通道的比较。重新验证应重新生成结果，不能仅凭报告名称、编译退出码或一条相同波形判定全链通过。

11.4 数值一致与字节一致

$$\Delta_{\max} = \max_{(j,k)\in F} \left| y_{j,k}^{\text{test}} - y_{j,k}^{\text{base}} \right| \tag{11-1}$$

集合 F 仅包含两侧均为有限值的对应样本。非有限值的位置、类型和时间轴须另外比较。有限差值为零只证明这些样本数值一致；字节一致由逐字节比较或 SHA256 判断。格式、舍入、列序和换行均可能造成数值一致而字节不同。

1 ms 录波间隔对应 1000 Hz 采样率，每个 50 Hz 基波周期有 20 个点，可观察基波和慢动态，不能解析全部开关细节。录波间隔不改变 250 μs 控制周期。

第12章 工程操作与故障排查

12.1 修改与替换流程

先固定源码、编译器、参数、初态和工程版本。只调参数时，核对参数槽、每拍覆盖和 ready 装入条件；改算法时重新编译，依次完成函数差分、真实宿主调用与闭环同工况比较。扰动变化后保存独立工程与结果，先确认扰动已实际施加，再解释控制响应。

停止加载对应 DLL 的仿真实例后替换文件，记录新旧哈希与实际模块路径。更换文件不等于宿主已卸载旧模块。逐层记录参数、活动状态与输出，避免用面板值直接代替电流参考。

12.2 常见问题定位

表 48 故障定位

现象	核对内容	处理方法
编译环境不可用	VCVARS64、cl、py、记录脚本	核对依赖、路径和退出码
中文乱码或 C4819	文件编码与编译代码页	定位诊断行，保留实际编码，不忽略全部告警
新 DLL 未生效	产物哈希、复制位置、加载模块	释放对应模块后替换，重新检查加载路径
改参数未反映到输出	槽位、单位、前拍状态、ready、覆盖	同时记录输入、缓存、活动值和最终读者
越阈值未动作	严格比较、持续计数、刷新与门控	按本控制拍顺序逐项核对
零输入仍有输出	积分、滤波历史、快照、非法模式	统一初态，检查历史状态与缓冲保留
差值零但文件不同	列序、时间轴、格式、NaN、哈希	分别报告数值与字节比较
公式打印异常	纸张、缩放、阅读器字体	A4 纵向、实际尺寸或适合页面，无需 LaTeX

12.3 重新验证的最小记录

保存源码哈希、编译器版本与选项、目标位数、DLL 哈希与导出符号、工程版本、完整参数和动态 P*/Q*/P_pos、初态或快照、控制与录波周期、扰动窗口、输出清单、样本形状、有限误差、非有限值掩码和字节比较。各工况分别记录，缺失通道或配置变化必须说明。

第13章 附录 参数全集与源码索引

本章列出工程配置示例、输入数组契约及实现常数。配置原值、每拍换算、活动对象与最终输出分别解释；电气接线及保护开关的实际保存值见第14章，不能仅凭某一测试调用的默认保护字判断工程保护已启用。

13.1 当前全部109项面板配置

表 49 Cmd配置全集

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Cmd_1	Start_Cmd	1	—
Cmd_2	LVRT_Enable	1	—
Cmd_3	HVRT_Enable	1	—
Cmd_4	Reset_Cmd	0	—
Cmd_5	Flag_LVRT_Restore	1	—
Cmd_6	Q_ctrol_mode	1	—
Cmd_7	Flag_AVG_IGBT	0	—
Cmd_8	Flag_Interia_Freq	0	—

表 50 Control配置全集

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Control_1	PLL_kp	0.08	—；p[0]
Control_2	PLL_ki	0.0001	—；p[1]
Control_3	Grid_side_Voltage_Loop_kp	5	—；p[2]
Control_4	Grid_side_Voltage_Loop_ki	0.15	—；p[3]
Control_5	Grid_side_Positive_Sequence_Current_Loop_kp	1200	—；p[4]
Control_6	Grid_side_Positive_Sequence_Current_Loop_ki	20	—；p[5]
Control_7	Grid_side_Negative_Sequence_Current_Loop_kp	1200	—；p[6]
Control_8	Grid_side_Negative_Sequence_Current_Loop_ki	20	—；p[7]

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Control_9	Machine_side_Positive_Sequence_Current_Loop_kp	3500	—；p[8]
Control_10	Machine_side_Positive_Sequence_Current_Loop_ki	15	—；p[9]
Control_11	Machine_side_Negative_Sequence_Current_Loop_kp	4500	—；p[10]
Control_12	Machine_side_Negative_Sequence_Current_Loop_ki	25	—；p[11]
Control_13	Low_Voltage_Ride_through_Threshold	0.9	—；p[12]
Control_14	Low_Voltage_Hysteresis	0.05	—；p[13]
Control_15	LVRT_Reactive_Power_Coefficient_G	3	—；p[14] / p[15]
Control_17	High_Voltage_Ride_through_Threshold	1.13	—；p[16]
Control_18	High_Voltage_Hysteresis	0.05	—；p[17]
Control_19	HVRT_Reactive_Power_Coefficient_G；网侧 r_e(18)=2.223863；机侧p[19]复用C19，(r_e(19)×3)/4≈1.6678969	2.2841	—；p[18] / p[19]
Control_21	Grid_side_Active_Current_Limit	0.2777	—；p[20]
Control_22	Grid_side_Reactive_Current_Limit	0.1111	—；p[21]
Control_23	Machine_side_Maximum_Active_Current_Limit	0.4	—；p[22]
Control_24	Machine_side_Minimum_Active_Current_Limit	0	—；p[23]
Control_25	Machine_side_Maximum_Reactive_Current_Limit	0.2895	—；p[24]
Control_26	Machine_side_Minimum_Reactive_Current_Limit	-0.2895	—；p[25]
Control_27	LVRT_Reactive_Current_Calculation_Voltage	1	—；p[26]
Control_28	LVRT_Reactive_Current_Initial_Coefficient；1.0参与机侧Ip*中的励磁叠加；不是没有读者	1	—；p[27]

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Control_29	LVRT_Reactive_Current_Negative_Sequence_Coefficient	1	—；p[28]
Control_30	Grid_side_LVRT_Reactive_Current_Maximum	0.2221	—；p[29]
Control_31	Grid_side_Unbalance_Reactive_Current_Maximum	0.2221	—；p[30]
Control_32	Machine_side_LVRT_Reactive_Current_Maximum	0.3456	—；p[31]
Control_33	Machine_side_Unbalance_Reactive_Current_Maximum	0.3456	—；p[32]
Control_34	HVRT_Reactive_Current_Calculation_Voltage	1	—；p[33]
Control_35	HVRT_Reactive_Current_Initial_Coefficient；1.0参与HVRT电流合成的励磁叠加，需按门控判断	1	—；p[34]
Control_36	HVRT_Reactive_Current_Negative_Sequence_Coefficient	0	—；p[35]
Control_37	Grid_side_HVRT_Reactive_Current_Maximum	-0.0187	—；p[36]
Control_38	Grid_side_HVRT_Reactive_Current_Minimum	-0.1481	—；p[37]
Control_39	Machine_side_HVRT_Reactive_Current_Maximum	-0.014	—；p[38]
Control_40	Machine_side_HVRT_Reactive_Current_Minimum	-0.1481	—；p[39]
Control_41	LVRT_Active_Current_Control_Mode	1	—；p[40]
Control_42	LVRT_Active_Current_Control_Coefficient	1	—；p[41]
Control_43	HVRT_Active_Power_Control_Coefficient	0	—；p[42]
Control_44	LVRT_Id_rotor_limit_max	0.3801	—；p[43]
Control_45	LVRT_min_Id_value；h(44)=149.913A，再×1.414≈211.977A，参与If*上界	0.074	—；p[44]
Control_56	Rate_Freq	50	—；p[55]

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Control_57	Interia_respond_time	10	—；p[56]
Control_58	Interia_deadtime	0.3	—；p[57]
Control_59	Interia_Tj	0.1	—；p[58]
Control_60	primary_response_time	5	—；p[59]
Control_61	Freq_Droop	0.06	—；p[60]
Control_62	Primary_deadzone	0.03	—；p[61]
Control_63	Primary_powerMAXlimit	0.2	—；p[62]
Control_64	Primary_powerMINlimit	0.1	—；p[63]
Control_65	recovery_ramp_start_point；特殊恢复起点，按模式与门控生效	0.7	—；p[64]
Control_66	recovery_ramp_time；0.5s恢复参数，并参与特殊恢复超时	0.5	—；p[65]
Control_67	Grid_side_LVRT_Reactive_Current_Minimum	0.015	—；p[66]
Control_68	Motor_side_LVRT_Reactive_Current_Minimum	0.008	—；p[67]
Control_72	PWM_ENABLE_200ms_breaker	0	—；p[71]
Control_73	LVRT_Id_model；当前2：常规励磁公式分支；保持寄存器属于模式1	2	—；p[72]
Control_74	LVRT_Id_limit_max	0.7	—；p[73]
Control_75	recovery_ramp_model；当前3：特殊恢复激活后选择P_pos均值参考	3	—；p[74]
Control_76	未提供面板标签；现行未打包；当前0；面板项存在但未进入当前CONTROL_SLOTS	0	—；未进入控制槽

表 51 Protect配置全集

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Protect_1	AC_RMS_Over_Value1	1.33	—；零基保护槽[0]
Protect_2	over_vol_time1	0.15	—；零基保护槽[1]
Protect_3	AC_RMS_Over_Value2	1.28	—；零基保护槽[2]

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Protect_4	over_vol_time2	0.6	—；零基保护槽[3]
Protect_5	AC_RMS_Over_Value3	1.23	—；零基保护槽[4]
Protect_6	over_vol_time3	1.1	—；零基保护槽[5]
Protect_7	AC_RMS_lower_Value1	0.12	—；零基保护槽[6]
Protect_8	lower_vol_time1	0.3	—；零基保护槽[7]
Protect_9	AC_RMS_Over_Value4	1.13	—；零基保护槽[8]
Protect_10	over_vol_time4	10.4	—；零基保护槽[9]
Protect_11	AC_RMS_lower_Value2	0.4	—；零基保护槽[10]
Protect_12	lower_vol_time2	1.4	—；零基保护槽[11]
Protect_13	AC_RMS_lower_Value3	0.6	—；零基保护槽[12]
Protect_14	lower_vol_time3	2.2	—；零基保护槽[13]
Protect_15	AC_RMS_lower_Value4	0.83	—；零基保护槽[14]
Protect_16	lower_vol_time4	3	—；零基保护槽[15]
Protect_17	frequency_over_pro_value1	54	—；零基保护槽[16]
Protect_18	frequency_overtime1	0.2	—；零基保护槽[17]
Protect_19	frequency_over_pro_value2	53	—；零基保护槽[18]
Protect_20	frequency_overtime2	10	—；零基保护槽[19]
Protect_21	frequency_over_pro_value3	52	—；零基保护槽[20]
Protect_22	frequency_overtime3	2100	—；零基保护槽[21]
Protect_23	frequency_lower_pro_value1	46	—；零基保护槽[22]
Protect_24	frequency_lowertime1	0.2	—；零基保护槽[23]
Protect_25	frequency_lower_pro_value2	47	—；零基保护槽[24]
Protect_26	frequency_lowertime2	10	—；零基保护槽[25]
Protect_27	frequency_lower_pro_value3	47.5	—；零基保护槽[26]
Protect_28	frequency_lowertime3	2100	—；零基保护槽[27]

表 52　Gen配置全集

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Gen_1	Generator_Power_W	4000000	—

唯一ID	面板标签 / 说明	配置原值	面板单位与打包备注
Gen_2	Generator_Power_MW	4	—
Gen_3	Motor_Vrotor	3.541	—
Gen_4	Motor_Vstator	1.14	—
Gen_5	Motor_Rs_pu	0.0097	—
Gen_6	Motor_Xls_pu	0.0774	—
Gen_7	Motor_Rr_pu	0.0087	—
Gen_8	Motor_Xlr_pu	0.0999	—
Gen_9	Motor_Xm_pu	2.6962	—
Gen_10	Motor_Pole_Pairs	2	—
Gen_11	speed	1680	—
Gen_12	syn_speed	3000	—

13.2 外层90槽与局部控制p[0..79]

表 53　完整parameters[0..89]静态契约

外层零基 / Fortran一基	局部p槽	来源	静态值	说明
0 / 1	—	Gen_1（W）	4000000	铭牌 / 辅助 / 保留
1 / 2	—	Gen_4（kV）	1.14	铭牌 / 辅助 / 保留
2 / 3	—	Gen_3（kV）	3.541	铭牌 / 辅助 / 保留
3 / 4	—	Gen_9×Gen_4²/Gen_2	0.87599538	铭牌 / 辅助 / 保留
4 / 5	—	Gen_10（极对数）	2	铭牌 / 辅助 / 保留
5 / 6	—	保留零	0	铭牌 / 辅助 / 保留
6 / 7	—	保留零	0	铭牌 / 辅助 / 保留
7 / 8	—	保留零	0	铭牌 / 辅助 / 保留
8 / 9	—	保留零	0	铭牌 / 辅助 / 保留
9 / 10	—	保留零	0	铭牌 / 辅助 / 保留
10 / 11	p[0]	Control_1	0.08	
11 / 12	p[1]	Control_2	0.0001	
12 / 13	p[2]	Control_3	5	

外层零基 / Fortran一基	局部p槽	来源	静态值	说明
13 / 14	p[3]	Control_4	0.15	
14 / 15	p[4]	Control_5	1200	
15 / 16	p[5]	Control_6	20	
16 / 17	p[6]	Control_7	1200	
17 / 18	p[7]	Control_8	20	
18 / 19	p[8]	Control_9	3500	
19 / 20	p[9]	Control_10	15	
20 / 21	p[10]	Control_11	4500	
21 / 22	p[11]	Control_12	25	
22 / 23	p[12]	Control_13	0.9	
23 / 24	p[13]	Control_14	0.05	
24 / 25	p[14]	Control_15	3	
25 / 26	p[15]	Control_15	3	包装复用别名；不是独立Control16/20
26 / 27	p[16]	Control_17	1.13	
27 / 28	p[17]	Control_18	0.05	
28 / 29	p[18]	Control_19	2.2841	
29 / 30	p[19]	Control_19	2.2841	包装复用别名；不是独立Control16/20
30 / 31	p[20]	Control_21	0.2777	
31 / 32	p[21]	Control_22	0.1111	
32 / 33	p[22]	Control_23	0.4	
33 / 34	p[23]	Control_24	0	
34 / 35	p[24]	Control_25	0.2895	
35 / 36	p[25]	Control_26	-0.2895	
36 / 37	p[26]	Control_27	1	
37 / 38	p[27]	Control_28	1	
38 / 39	p[28]	Control_29	1	
39 / 40	p[29]	Control_30	0.2221	

外层零基 / Fortran一基	局部p槽	来源	静态值	说明
40 / 41	p[30]	Control_31	0.2221	
41 / 42	p[31]	Control_32	0.3456	
42 / 43	p[32]	Control_33	0.3456	
43 / 44	p[33]	Control_34	1	
44 / 45	p[34]	Control_35	1	
45 / 46	p[35]	Control_36	0	
46 / 47	p[36]	Control_37	-0.0187	
47 / 48	p[37]	Control_38	-0.1481	
48 / 49	p[38]	Control_39	-0.014	
49 / 50	p[39]	Control_40	-0.1481	
50 / 51	p[40]	Control_41	1	
51 / 52	p[41]	Control_42	1	
52 / 53	p[42]	Control_43	0	
53 / 54	p[43]	Control_44	0.3801	
54 / 55	p[44]	Control_45	0.074	
55 / 56	p[45]	保留零	0	
56 / 57	p[46]	保留零	0	
57 / 58	p[47]	保留零	0	
58 / 59	p[48]	保留零	0	
59 / 60	p[49]	保留零	0	
60 / 61	p[50]	保留零	0	
61 / 62	p[51]	保留零	0	
62 / 63	p[52]	保留零	0	
63 / 64	p[53]	保留零	0	
64 / 65	p[54]	保留零	0	
65 / 66	p[55]	Control_56	50	
66 / 67	p[56]	Control_57	10	
67 / 68	p[57]	Control_58	0.3	

外层零基 / Fortran一基	局部p槽	来源	静态值	说明
68 / 69	p[58]	Control_59	0.1	
69 / 70	p[59]	Control_60	5	
70 / 71	p[60]	Control_61	0.06	
71 / 72	p[61]	Control_62	0.03	
72 / 73	p[62]	Control_63	0.2	
73 / 74	p[63]	Control_64	0.1	
74 / 75	p[64]	Control_65	0.7	
75 / 76	p[65]	Control_66	0.5	
76 / 77	p[66]	Control_67	0.015	
77 / 78	p[67]	Control_68	0.008	
78 / 79	p[68]	保留零	0	当前无Control69～71配置，补零
79 / 80	p[69]	保留零	0	当前无Control69～71配置，补零
80 / 81	p[70]	保留零	0	当前无Control69～71配置，补零
81 / 82	p[71]	Control_72	0	
82 / 83	p[72]	Control_73	2	
83 / 84	p[73]	Control_74	0.7	
84 / 85	p[74]	Control_75	3	
85 / 86	p[75]	保留零	0	
86 / 87	p[76]	保留零	0	
87 / 88	p[77]	保留零	0	
88 / 89	p[78]	保留零	0	
89 / 90	p[79]	外部动态P_pos	0	本表静态初值0；每拍宿主可替换，不是Control80

13.3 命令、保护及输入输出数组

表 54　8个数组的长度与有效范围

数组	容量	有效内容与边界
grid_inputs / rotor_inputs	各10	前8路为物理输入；[8]为元数据/快照条件，[9]为运行号或快照恢复标记
parameters	90	前10铭牌、80控制；最后动态P_pos
protections	30	28配置定值+2保留零
commands	11	8面板项+P*接口（工程保存1.0）、Q*、保护字
grid_outputs / rotor_outputs	各20	输出按0/1/2模式；范围与使能看第6章源码对照
view_result	130	源码写前41通道；容量130不等于130个参数

表 55　完整11路命令契约

命令零基槽	来源 / 当前值	用途
0	Cmd_1=1	启动
1	P*接口（工程保存1.0）	有功参考
2	Q*接口（工程保存0.33）	无功参考
3	Cmd_2=1	LVRT使能
4	Cmd_3=1	HVRT使能
5	Cmd_4=0	复位边沿
6	Cmd_5=1	恢复标志
7	工程五开关组合0；contract缺省1073794048为独立测试实参	测试接口保护字示例 0x4000CC00；当前工程保护开关保存值对应0，见第14章
8	Cmd_6=1	无功模式，使用前拍模式
9	Cmd_7=0	0平均 / 1占空比 / 2调制波
10	Cmd_8=0	0关闭 / 1惯量 / 2一次调频

表 56　全部28保护定值及2保留槽；阈值达到不自动等于跳闸

Protect编号 / 零基槽	类别	配置门槛	有名值门槛	动作时长 / s
1/2；[0]/[1]	过压快速段	1.33 pu	1516.200 V	0.15
3/4；[2]/[3]	过压1段	1.28 pu	1459.200 V	0.6
5/6；[4]/[5]	过压2段	1.23 pu	1402.200 V	1.1

Protect编号 / 零基槽	类别	配置门槛	有名值门槛	动作时长 / s
7/8；[6]/[7]	欠压1段	0.12 pu	136.800 V	0.3
9/10；[8]/[9]	过压3段	1.13 pu	1288.200 V	10.4
11/12；[10]/[11]	欠压2段	0.4 pu	456.000 V	1.4
13/14；[12]/[13]	欠压3段	0.6 pu	684.000 V	2.2
15/16；[14]/[15]	欠压4段	0.83 pu	946.200 V	3
17/18；[16]/[17]	过频1段	54 Hz	54 Hz	0.2
19/20；[18]/[19]	过频2段	53 Hz	53 Hz	10
21/22；[20]/[21]	过频3段	52 Hz	52 Hz	2100
23/24；[22]/[23]	欠频1段	46 Hz	46 Hz	0.2
25/26；[24]/[25]	欠频2段	47 Hz	47 Hz	10
27/28；[26]/[27]	欠频3段	47.5 Hz	47.5 Hz	2100
29/30；[28]/[29]	保留	0	0	不适用

13.4 有限常数、离散系数与时间

表 57　关键常数与离散权重

常数 / 系数	源码数值	含义
√2 / √3 的近似	1.414 / 1.732	执行中保留有限常数
变换系数	1.1547 / 0.577 / 0.666667 / 0.866	2/√3、1/√3、2/3、√3/2的历史近似
2π	6.28318530717958	额定电角速度与单次角度修正边界
PLL序分量低通	0.03665191429188089 / 0.9633480857081191	输入 / 历史权重
PLL前馈低通	0.10471975511965965 / 0.8952802448803403	输入 / 历史权重
网侧并行序分解	0.0314159265358979；0.5115273563044781	两组滤波器输入权重，各自历史权重1-a
机侧并行序分解	0.07853981633974474；0.523709146364097	两组滤波器输入权重，非串级
残差低通	0.20420352248333634 / 0.7957964775166637	输入 / 历史权重
穿越低通	30/1024、20/1024、8/1024	不同偏差、负序及平均环节各自读取
谐波慢低通	0.01 / 0.99	输入 / 历史权重
谐振中心 / 带宽	网侧2π×300，机侧2π×350；带宽2π×10	增益G初值0，历史为零时不产生主动输出
差分滤波参数	2π×1000 / 2π×2000 / 8000 / 1	T0/T1/T2/T3
机侧相位补偿	3.0665 rad	与PLL和编码器角组合

常数 / 系数	源码数值	含义
机侧过流 / 母线闭锁	5×989.8=4949A / 3000V	一拍检测；后续状态与输出按源码门控

表 58　时间换算与计数比较边界

拍数 / 时间	在**250μs**控制周期下	作用 / 注意事项
4 / 5拍	1 / 1.25ms	计数阈值4若比较严格大于，动作在第5拍
20 / 21拍	5 / 5.25ms	计数阈值20同理需区分阈值与实际动作拍
40拍	10ms	锁相误差统计与部分穿越最小保持门控
80拍	20ms	50Hz工频周期、母线/功率平均及活动RMS窗
100拍	25ms	机侧启动阶段与适应窗口上限
200拍	50ms	机侧启动阶段
400拍	0.1s	软启动阶段；累计就绪与连续确认须分开
800拍	0.2s	穿越恢复保持、停机过程等不同计时器
8000拍	2s	潜在自动复归阈值；当前门控未激活
5s / 10s	20000 / 40000拍	惯量及一次调频保持；按配置与实际调用门控计时
4s	16000拍	部分二类锁存故障延时
0.5s	2000拍	Control66特殊恢复超时/斜坡参数

13.5 源码核对要点

表 59　实现解读要点

检查项	规范解释
PI	ki_per_step 直接乘误差；tracking_gain 乘上一拍饱和残差
限幅	先检查上界，再检查 x<L 的下界分支
PLL 与振荡器角度	仅一次 ±τ 修正，不等于任意输入的严格取模
参数活动层	缓存更新与 ready 装入活动 PI 是不同动作
频率支撑	缩放来源、非对称分段与使能按第9章逐段解释
机侧前馈	保留实际写入槽位，不画成对称限幅
基波与序分量	相基波幅值、αβ 模、正序滤波模分别解释
谐振支路	计算或接入不等于主动补偿；结合增益、历史和最终读者
保护	各段门控、更新与 > / ≥ 比较分别核对

检查项	规范解释
自动复归	8000 拍为潜在门槛，整链门控另行确认
周期	有效控制拍更新输出，1 ms 为录波间隔
PSCAD 工程	电气基准、面板槽位、宿主数组与 DLL 名共同构成契约

13.6 源码定位与版本指纹

下表短码为 SHA256 前16位；完整哈希和节选范围保存在配套核验记录。源文件改变后须重新核对行号。

表 60　源码版本指纹

文件	对照章节	SHA256 短码
recovered_commands.cpp	第5章	0bf6e6d014f06aa9
recovered_controller.cpp	第4章	e87caff52d2c4252
recovered_grid.cpp	第7、9、10章	77c5b9c427062d46
recovered_initialization.cpp	第3章	6470a5964bd19d12
recovered_motor.cpp	第8、9、10章	957a36a4e40caff9
recovered_parameters.cpp	第5章	a315d6a5c822e642
recovered_primitives.cpp	第6章	731570356b2564e4
recovered_primitives.h	第4章	23fab91b16bc2e7d

第14章 PSCAD 工程与宿主接口核对

14.1 PSCAD工程配置与接线核对

以工程文件目录中的pscx为实际工程文件，以原装基准目录同名文件为比较参照。文件身份、保存配置、实际接线、DLL接口和动态运行是不同证据层级；本轮只读核对全部主机电实例与控制接口，没有启动仿真，不能推断哪个文件正在运行。

14.1.1 工作工程与参照的文件身份

两份pscx均为23488508字节、440682行，SHA256均为598c2d568d6944052def2ce6e5c9bd25098ab1460ed6323b2792ed03361ca3f5，已比较字节完全相同。目录名字不代表两份工程有不同参数、模型或故障工况。

两目录主DLL也相同，SHA256为461562da5be06560ef6417b0283d8ce8df76054c08da1937fca9c500af2e4c7e；辅助DLL相同，SHA256为bf6687bb61bedba6ecfeae43c03fbf3bfc5cac0074068cc56f8405088daf00ce。主Fortran包装333/333行，辅助包装372/378行，去空行并对非空行去首尾空白后分别完全一致，差异仅空白、缩进和尾部空行；没有发现有效调用语句、形参顺序或加载名改变。

14.1.2 版本与三个时间尺度

根元素声明version=5.0.0、Target=EMTDC、sealed=false，这不是本机已安装PSCAD版本的证据。Settings保存25s时长、time_step=10μs、sample_step=1000μs；实际控制器实例Sam_Ts=250e-6s、Sim_Ts=DELTA_TIME。调用10μs、控制拍250μs、录波1ms各自独立。

若25s录波包含首尾，理论25001样本；实际样本数须检查本次文件。StartType=0、SnapType=0、SnapTime=4、MrunType=0、Mruns=1不能被简化为本次一定从4s快照恢复。startup_filename是另一环境的绝对路径，宿主snp与DLL的snapshot.txt不同。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L1-L24

```
<project name="DFIG_SANY_IGBT_WTG16840_SK_V1" version="5.0.0" schema="" Target="EMTDC"
sealed="false">
  <paramlist name="Settings">
    <param name="creator" value="Administrator,1590062017" />
    <param name="time_duration" value="25" />
    <param name="time_step" value="10" />
    <param name="sample_step" value="1000" />
    <param name="chatter_threshold" value=".001" />
    <param name="branch_threshold" value=".0005" />
    <param name="StartType" value="0" />
    <param name="PlotType" value="0" />
    <param name="output_filename" value="$(Namespace).out" />
    <param name="SnapType" value="0" />
    <param name="SnapTime" value="4" />
    <param name="snapshot_filename" value="$(Namespace).snp" />
    <param name="MrunType" value="0" />
    <param name="Mruns" value="1" />
    <param name="Scenario" value="" />
    <param name="Advanced" value="6143" />
    <param name="Options" value="48" />
    <param name="Build" value="22" />
    <param name="Warn" value="0" />
    <param name="Check" value="0" />
    <param name="description" value="" />
    <param name="startup_filename"
value="D:\A_Matlabjm\KDF\SYZG_20251105_P_4_12p0_1140_India\new_fold\4.SY_india_project_V4\6.SY_india_
project_v20260206\DFIG_SANY_IGBT_WTG16840_V5_02.if15\$(Namespace).snp" />
```

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L25-L27

```
    <param name="revisor" value="sunyijia, 1786710131" />
    <param name="labels" value="" />
    <param name="Source" value="APP_PROGRAM\DFIG_CONTROLLER_SY_WG4000KDF_NPC.cpp" />
```

表 61　工程保存的全部运行与求解设置

设置键	保存值
creator	Administrator,1590062017
time_duration	25
time_step	10
sample_step	1000
chatter_threshold	.001
branch_threshold	.0005
StartType	0
PlotType	0
output_filename	$(Namespace).out

设置键	保存值
SnapType	0
SnapTime	4
snapshot_filename	$(Namespace).snp
MrunType	0
Mruns	1
Scenario	
Advanced	6143
Options	48
Build	22
Warn	0
Check	0
description	
startup_filename	D:\A_Matlabjm\KDF\SYZG_20251105_P_4_12p0_1140_India\new_fold\4.SY_india_project_V4\6.SY_india_project_v20260206\DFIG_SANY_IGBT_WTG16840_V5_02.if15\$(Namespace).snp
revisor	sunyijia, 1786710131
labels	
Source	APP_PROGRAM\DFIG_CONTROLLER_SY_WG4000KDF_NPC.cpp
Preprocessor	
sparsity_threshold	200
Debug	0

14.1.3 求解位域与资源边界

chatter_threshold=.001 、 branch_threshold=.0005 、 sparsity_threshold=200；Advanced=6143 、 Options=48、Build=22、Warn=0、Check=0、Debug=0是保存原值。没有核查安装版本位定义，不能把位域擅自翻译成某编译器、实数精度或特定开关求解功能已经启用。

Root/output 保 留 TwoLevelConverterDFIG1 、 EMTDC 、 version2010 、 日 期 2020/08/11 、 sample rate200000/end400000等旧元数据，不能覆盖现Settings或证明当前470通道。Resource仅两份相对路径 Fortran 文 件 ， include=true/copy_file=false；Settings 另 存 Source=APP_PROGRAM\DFIG_CONTROLLER_SY_WG4000KDF_NPC.cpp字符串，不能据此证明八份恢复源码自动重编或DLL已替换。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L440637-L440653

```
  <bookmarks />
  <List classid="Resource">
    <Resource classid="SourceResource" id="1366932881" url=".\DFIGSY4MW_DllLink64bit.f">
      <paramlist>
        <param name="filepath" value=".\DFIGSY4MW_DllLink64bit.f" />
        <param name="copy_file" value="false" />
        <param name="include" value="true" />
      </paramlist>
    </Resource>
    <Resource classid="SourceResource" id="269797108" url=".\POWER_FUN_DllLink64bit.f">
      <paramlist>
        <param name="filepath" value=".\POWER_FUN_DllLink64bit.f" />
        <param name="copy_file" value="false" />
        <param name="include" value="true" />
      </paramlist>
    </Resource>
  </List>
```

14.1.4 实例链与未使用定义

全工程有207个Definition，hierarchy只15个call。Main实际引用sungrow324567520、DFIG_CONVERTER_1_1父实例1470556608；控制器908509267、辅助功率1535623755、保护求和650645758是实际脚本组件，虽无独立画布call仍在父画布调用。

父converter、controller均disable=false；sungrow内25个组件禁用。需沿Main实例递归，不能用207定义数量、定义instances属性或旧变体后缀推断实际运行模型。当前实际电网、两SCR换算、机器、两侧平均与开关桥、母线、充电、滤波、PWM和监控全部已核对。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L41760-L41779

```
        <User classid="UserCmp" id="1470556608"
name="DFIG_SANY_IGBT_WTG16840_SK_V1:DFIG_CONVERTER_1_1" x="936" y="342" w="258" h="107" z="150"
orient="0" defn="DFIG_SANY_IGBT_WTG16840_SK_V1:DFIG_CONVERTER_1_1" link="-1" q="4" disable="false">
          <paramlist name="" link="-1" crc="35606437">
            <param name="Name" value="" />
            <param name="CONVERTER_NUM" value="0" />
            <param name="Cmd_1" value="1" />
            <param name="Cmd_2" value="1" />
            <param name="Cmd_3" value="1" />
            <param name="Cmd_4" value="0" />
            <param name="Cmd_5" value="1" />
            <param name="Cmd_7" value="0" />
            <param name="Cmd_8" value="0" />
            <param name="Cmd_6" value="1" />
            <param name="Control_72" value="0" />
            <param name="Control_76" value="0" />
            <param name="Control_1" value="0.08" />
            <param name="Control_2" value="0.0001" />
            <param name="Control_3" value="5" />
            <param name="Control_4" value="0.15" />
            <param name="Control_5" value="1200" />
            <param name="Control_6" value="20" />
```

14.2 主机电拓扑与元件保存参数

主电路为受控三相电网、源阻抗、33/1.14kV变压器与机组低压端，定子和背靠背变流器在低压侧接入。控制器4MW基准、机器4.2MVA和变压器4.4MVA须分别保留，静态连线不证明实际波形或稳定性。

14.2.1 三相主接线的坐标证据

Main电网sungrow经高压测量652046622、变压器1753557961、低压测量508994776到converter三相PHASE端口。Wire1709104859：(288,360)→(378,360)；514837209：(414,360)→(504,360)；1572494239：(576,360)→(666,360)；1205370237：(702,360)→(810,360)。810,360等于converter936,342加PHASE相对端口−126,+18。

高压标签Iabc_35kV_kA1/Vabc_35kV_kV1虽含35kV，实际电源SyskV与变压器V1都是33kV；不能按标签当额定值。测量块Freq60Hz字段亦不能覆盖受控电源外接SysHz50。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L41636-L41655

```
        <User classid="UserCmp" defn="master:xfmr-3p2w" id="1753557961" x="540" y="360" w="120"
h="89" z="1" orient="0" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="86463138">
            <param name="Name" value="TWTG1_1" />
            <param name="Tmva" value="4.4 [MVA]" />
            <param name="f" value="50 [Hz]" />
            <param name="YD1" value="1" />
            <param name="YD2" value="0" />
            <param name="Lead" value="2" />
            <param name="Xl" value="0.082111205 [pu]" />
            <param name="Ideal" value="0" />
            <param name="NLL" value="0.000888889 [pu]" />
            <param name="CuL" value="0.008 [pu]" />
            <param name="Tap" value="0" />
            <param name="View" value="1" />
            <param name="Dtls" value="1" />
            <param name="V1" value="33 [kV]" />
            <param name="V2" value="1.140 [kV]" />
            <param name="Enab" value="0" />
            <param name="Sat" value="0" />
            <param name="Xair" value="0.16422241 [pu]" />
```

14.2.2 变压器4.4MVA与连接枚举

TWTG1_1保存4.4MVA、50Hz、33kV/1.140kV、漏抗.082111205pu、NLL=.000888889pu、CuL=.008pu、Ideal=0、Sat=0。YD1=1、YD2=0、Lead=2和地元件973940857的位置540,396保留为明确接线证据。

精确矢量组时钟号需核查该版本master:xfmr-3p2w枚举定义，不能从Lead2擅自命名。控制器4e6功率换算不会自动继承4.4MVA变压器基准。

表 62　变压器保存参数

字段	值
Tmva	4.4 [MVA]
f	50 [Hz]
V1	33 [kV]
V2	1.140 [kV]
YD1	1
YD2	0
Lead	2
Xl	0.082111205 [pu]
Ideal	0
NLL	0.000888889 [pu]
CuL	0.008 [pu]
Tap	0
Enab	0
Sat	0
Xair	0.16422241 [pu]
Tdc	0.0 [s]
Xknee	1.17 [pu]
Txk	0.0 [s]
Im1	1 [%]
Hys	0

14.2.3 绕线转子机器4.2MVA与参数

机器 master:wound_rotor1662057210 保存 Pbase4.2MVA、Vbase1.14kV、OMEG314.15926rad/s、TRN.3219、J.7267s、D.01pu；ERC1/MM0/SNP1/Cage0/NCW1，Msat/Lsat0。R1/R2/Xa/Xkd1/Xmd为.0097/.0087/.0774/.0999/2.6962pu。

它们是机器元件直接字面量，不是Gen5～Gen9变量引用。当前同名面板数字恰好相同，不意味着改Gen5自动改变机器R1。额外笼条/耦合槽有保存值但是否活动受机器模式约束。TRN与控制器比值倒数也略有差异，不能自动统一。

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L415659-L415678

```
        <User classid="UserCmp" name="master:wound_rotor" defn="master:wound_rotor" id="1662057210"
 x="2142" y="774" w="103" h="116" z="2510" orient="7" link="-1" q="4" disable="false">
           <paramlist link="-1" name="" crc="68671592">
             <param name="Name" value="DFIG" />
             <param name="Pbase" value="4.2 [MVA]" />
             <param name="Vbase" value="1.14 [kV]" />
             <param name="OMEG" value="314.15926 [rad/s]" />
             <param name="TRN" value="0.3219" />
             <param name="J" value="0.7267 [s]" />
             <param name="D" value="0.01 [pu]" />
             <param name="View" value="1" />
             <param name="ERC" value="1" />
             <param name="MM" value="0" />
             <param name="SNP" value="1" />
             <param name="Cage" value="0" />
             <param name="NCW" value="1" />
             <param name="Msat" value="0" />
             <param name="Lsat" value="0" />
             <param name="R1" value="0.0097 [pu]" />
             <param name="R2" value="0.0087 [pu]" />
             <param name="R3" value="0.298 [pu]" />
```

表 63　机器活动实例参数摘要

字段	保存值
Pbase	4.2 [MVA]
Vbase	1.14 [kV]
OMEG	314.15926 [rad/s]
TRN	0.3219
J	0.7267 [s]
D	0.01 [pu]
ERC	1
MM	0
SNP	1
Cage	0
NCW	1
Msat	0
Lsat	0
R1	0.0097 [pu]
R2	0.0087 [pu]

字段	保存值
R3	0.298 [pu]
R4	0.018 [pu]
R5	0.018 [pu]
Xmd	2.6962 [pu]
Xa	0.0774 [pu]
Xkd1	0.0999 [pu]
Xkd2	0.05 [pu]
Xkd3	0.05 [pu]
Xkd4	0.05 [pu]
Xkd12	0.02 [pu]
Xkd13	0.02 [pu]
Xkd14	0.02 [pu]
Xkd23	0.02 [pu]
Xkd24	0.02 [pu]
Xkd34	0.02 [pu]
Spd	Speed_pu
ThetaR	Rotor_Theta
Vwdg	0.3983

14.2.4 机械速度命令与回读

Gen11=1680、Gen12=3000、Gen10=2通过两个除法549540976/1108088538生成1680/(3000/2)=1.12送机器机械输入；机器Speed_pu回读乘3000再除极对数得到Rotor_Speed，正常1.12对应1680r/min。

Gen11/12虽未直接打包进90槽，但真实影响工程机械输入和控制器速度回读。3000/2才是两对极机械同步1500r/min，不能把Gen12说成完全无作用的备注。稳态是否严格跟随给定需运行证据。

$$n_s = Gen_{12} \,/\, Gen_{10} = 1500 \text{ r} \,/\, \text{min} \tag{14-1}$$

$$\omega_{cmd,pu} = Gen_{11} \,/\, n_s = 1.12 \tag{14-2}$$

$$n_{fb} = Speed_{pu}\, 3000 \,/\, Gen_{10} \tag{14-3}$$

14.2.5 分裂母线、充电和斩波支路

上下两只13500μF电容147597610/518476044形成分裂直流母线；电压测量736080629/696377356产生VdcP_fed_pu/VdcN_fed_pu，经各1000倍增益为Vdc_fed_P/N。标签pu不表示已除以900V；1000换算与kV测量基准共同确定V输入。

上下.35Ω电阻1557909446/2066629896和开关1008249152/1234318921构成斩波支路（RON.01Ω/ROFF1e6Ω）；DC_CHARGE827509444含六只peswitch和1Ω电阻。1800V目标来自控制器0x168，不是电容预设电压；不据静态配置宣称实际母线恒±900V。

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L422088-L422107

```
        <User classid="UserCmp" name="master:capacitor" id="147597610" x="1044" y="738" w="34" h="68"
z="1" orient="3" defn="master:capacitor" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="108275881">
            <param name="C" value="13500 [uF]" />
            <param name="Name" value="" />
          </paramlist>
        </User>
        <User classid="UserCmp" name="master:voltmeter" id="736080629" x="990" y="702" w="80" h="44"
z="30" orient="0" defn="master:voltmeter" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="26842314">
            <param name="Name" value="VdcP_fed_pu" />
          </paramlist>
        </User>
        <User classid="UserCmp" name="master:pin" id="1597573252" x="918" y="666" w="12" h="12"
z="-1" orient="0" defn="master:pin" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="1329007" />
        </User>
        <User classid="UserCmp" name="master:pin" id="1637404777" x="1044" y="882" w="12" h="12"
z="-1" orient="0" defn="master:pin" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="1329007" />
        </User>
        <Wire classid="WireOrthogonal" id="94908274" name="" x="882" y="378" w="10" h="514"
orient="0" disable="false">
          <vertex x="0" y="0" />
          <vertex x="0" y="504" />
```

14.2.6 电感、电阻与实体交流滤波

网侧2089018339/.35mH及407344720/.001Ω，转子120122880/.015mH及1154078703/.001Ω。GRID_FILTER656705492内部三相各.08Ω和450μF，是实体阻容滤波，不是软件正负序或谐波滤波。

另有3只1e−6Ω电阻、.1Ω电阻及30Ω/.1μF支路、三个主断路器和四个模型选择断路器。主断路器通常RON.001Ω/ROFF1e6Ω，平均模型PWM断路器RON.0001Ω；组件disable=false不代表所有支路同时导通。

表 64　主要支路参数

部位	实例	值
网侧串联L/R	2089018339/407344720	.35mH/.001Ω
转子串联L/R	120122880/1154078703	.015mH/.001Ω

部位	实例	值
每相交流滤波	656705492内部	450μF/.08Ω
上下直流母线电容	147597610/518476044	13500μF各
上下斩波电阻	1557909446/2066629896	.35Ω各

14.2.7 开关桥、平均模型与模式选支路

网侧与机侧开关子模块各30只peswitch；两侧平均子模块各三只受控电压源、两只直流受控电流源与一个PWM断路器。Cmd7=0经Flag_AVG_IGBT同时控制输出模式和BREAKER_POWER2/5；反相72054582驱动BREAKER_POWER1/4。输出解释和实体支路选择是同一工程命令链。

PWM_GENERATOR保存12信号发生器、12比较器及逻辑，由G_PWM_IN/M_PWM_IN生成两侧IGBT驱动。受控源启用外接控制，残留132.79kV/60Hz默认值不能当实际输出。该活动链没有独立CROWBAR自定义模块，不能用未实例化模板证明实体撬棒投入；电流闭锁及物理支路应分开核验。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L421952-L421971

```
        <User classid="UserCmp"
name="DFIG_SANY_IGBT_WTG16840_SK_V1:GRID_CONVERTER_2_2_1_2_1_1_4_1_1_1_2_1_1" id="1735587818" x="756"
y="774" w="226" h="207" z="3550" orient="0"
defn="DFIG_SANY_IGBT_WTG16840_SK_V1:GRID_CONVERTER_2_2_1_2_1_1_4_1_1_1_2_1_1" link="-1" q="4"
disable="false">
          <paramlist link="-1" name="" crc="83256274">
            <param name="Name" value="" />
          </paramlist>
          <paramlist link="1753825436" name="">
            <param name="xmin" value="0" />
            <param name="xmax" value="10" />
          </paramlist>
          <paramlist link="1240335870" name="">
            <param name="ymin" value="-2" />
            <param name="ymax" value="2" />
          </paramlist>
        </User>
        <User classid="UserCmp"
name="DFIG_SANY_IGBT_WTG16840_SK_V1:MOTOR_CONVERTER_2_2_1_2_1_1_4_1_1_1_2_1_1" id="784293272"
x="1296" y="774" w="246" h="208" z="3540" orient="0"
defn="DFIG_SANY_IGBT_WTG16840_SK_V1:MOTOR_CONVERTER_2_2_1_2_1_1_4_1_1_1_2_1_1" link="-1" q="4"
disable="false">
          <paramlist link="-1" name="" crc="63023770">
            <param name="Name" value="" />
          </paramlist>
          <paramlist link="708824424" name="">
            <param name="xmin" value="0" />
            <param name="xmax" value="10" />
```

14.2.8 绘图用电气节点、桥臂与完整支路参数

按实际PHASE/DC_P/DC_CENTER/DC_N端口绘图：33kV电源经R_Source/L_Source到故障接入节点，继而经4.4MVA的33/1.14kV变压器到低压PCC。低压侧分别接定子STATOR_BREAKER支路、网侧GRID_BREAKER及.35mH/.001Ω到所选GSC功率模型、DC_CHARGE_BREAKER到三相预充电桥。机器转子经.015mH/.001Ω到所选RSC。两变流器通过正、中点、负三导体连接上下13500μF电容，另外有上下.35Ω斩波支路。

父画布用DC_P_1/DC_C_1/DC_N_1标网侧桥端及对应母线侧节点，DC_P_2/DC_C_2/DC_N_2标机侧端；两组三节点在电容正端/中点/负端的相应导线上贯通，是同一分裂母线，不是两套独立直流源。各桥PHASE维数3，三个DC端口各维数1。开关模型DERIVE维数12，平均模型DERIVE维数7。

两个NPC开关桥均30只peswitch，Type3为12个受控开关，Type0为18个二极管型元件，按每相4个受控开关与6个二极管（包含钳位）表达。均RON=.01Ω、ROFF=1e6Ω、SNUB=0、TEXT=0μs；RD=5000Ω/CD=.05μF是保存的吸收槽参数，SNUB=0时不能画成已投入RC吸收支路。18/12数目不应误写成30个IGBT。

交流GRID_FILTER子模块每相.08Ω串450μF；父画布.1Ω元件788777312另位于导线306,774→306,810→306,846→306,864→342,864通向滤波PHASE入口的支路，需在图中注明父支路.1Ω与内部.08Ω是两个元件。图只作审校后的结构示意，开合、测量方向及动态电压不能由图形位置推断。

表 65　实际拓扑绘图节点与设备表

节点/支路	元件或实例	关键参数/连接
高压电源	Main/sungrow324567520	SyskV33kV、SysHz50、SCR1.5、X/R10
源串联阻抗	1299718745	R_Source/L_Source，由SCR模块产生
故障并联支路	692003422+1746392663	Rf/Lf，ABC-G，5～6s，Ratio.2
主变压器	1753557961	4.4MVA，33/1.14kV，Xl.082111205pu
低压PCC	converter PHASE三相端	机器和变流器交流接入
定子并网支路	STATOR_BREAKER1879643222	至wound_rotor1662057210定子
机器	1662057210	4.2MVA、1.14kV、TRN.3219、J.7267s
网侧交流支路	GRID_BREAKER180527573+2089018339+407344720	.35mH/.001Ω
GSC模型选择	1693989980开关/1735587818平均	三相端与DC_P_1/DC_C_1/DC_N_1
转子交流支路	120122880+1154078703	.015mH/.001Ω
RSC模型选择	1745905423开关/784293272平均	三相端与DC_P_2/DC_C_2/DC_N_2
上半母线	147597610	13500μF：正端至中点
下半母线	518476044	13500μF：中点至负端

节点/支路	元件或实例	关键参数/连接
上/下斩波	1008249152/1234318921+1557909446/2066629896	.35Ω各，开关RON.01Ω
预充电	DC_CHARGE_BREAKER1713789474+827509444	三相六二极管型器件，1Ω电阻
交流滤波	656705492+父788777312	每相450μF/.08Ω；父支路另.1Ω

图中SRC/RSC/GSC和电容为实体模块；软件滤波、PI和算法限值放控制信号图，不混在主电路。

不得把两套模型并画为同时并联运行；用Cmd7选择开关/平均的互斥模式注记。

14.2.9 实际工程不存在可消费的撬棒触发支路

从Main实际实例递归检查所有未禁用可达模块，唯一含crow名称的组件为DFIG_CONVERTER_1_1内数据标签CROWBAR_DRIVE（1266562357，坐标2106,1458）。其来源是Rotor_Output一基第8项：DataTap698164269 的 Index8，经类型转换 unity817502807 与 Wire742084786 连接到标签。恢复rotor_output第498行对应零基output[7]无条件写0，这不是按转差门槛生成的撬棒触发输出。

当前可达电气拓扑没有CROWBAR自定义模块、读取CROWBAR_DRIVE的导入端口或可控转子短接开关；该标签没有实际电气消费者。全XML中的14个CROWBAR User虽自身disable=false，却位于未从Main调用的旧变体定义中，不能将未实例化模板当实际工程的撬棒支路。Main hierarchy亦不包含CROWBAR。

两只.35Ω电阻1557909446/2066629896配合开关1008249152/1234318921位于上下直流母线，控制信号为CHOPPER_DRIVE_P/N；它们是DC斩波支路，与跨转子三相端的短接/耗能撬棒不同。活动转子侧30Ω/.1μF及小电阻支路也没有接入CROWBAR_DRIVE的控制开关，不能仅凭其转子位置命名为已动作撬棒。

机侧D98/D10必须按以0xBF0为基址的ModulationState字段解释：D98=BF0+1A8为polarity，D10=BF0+120为gain，随后motor_calculate调用modulation(BF0)。这段符号/增益更新不是撬棒投退或5s维持。静态接口与拓扑均不足以支持“当前工程已经投入撬棒”；若要引入真正撬棒，应另设计实体接线、驱动消费和动态验证。

源码/recovered_primitives.cpp L489-L501

```
    case 2:
        output[1] = d(0xC78) * enable;
        output[2] = d(0xC80) * enable;
        output[3] = d(0xC88) * enable;
        output[4] = 0.0;
        output[5] = 0.0;
        output[6] = 0.0;
        break;
    }
    output[7] = 0.0;
    output[8] = static_cast<double>(i(0x3A0));
    output[9] = static_cast<double>(i(0x3A4));
    output[10] = status_times_100(state, 0x10);
```

本轮只读追踪，没有更改任何源工程或输出函数。

没有实时原装DLL输出记录，文件静态审查不代替某次运行状态；但当前工程该标签缺少实体消费者的结论不依赖假定原装驱动值。

14.3 面板、数组和动态反馈接口

控制器实际Port容量为10、10、90、30、11、20、20、130，内部保护函数仅读前28项。PSCAD DataTap/Fortran是一基，C++零基；必须按实际信号来源确定单位，不能只按源码注释命名。

14.3.1 109面板项与80控制槽

父converter有Cmd8、Control61、Protect28、Gen12共109项，Control编号上限76不等于数量。八个10槽组合80控制槽；实际导入2092757296/1489367275重复Control15到p14/p15，2071904302/1777775023重复Control19到p18/p19。

p45～54、p68～70及部分尾槽由常数0填充，p71～74为Control72～75，p79是P_pos。Control76存在但未进入80槽。CONTROL_SLOTS已与当前实际组、重复导入及尾部连接核对。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L41760-L41779

```
        <User classid="UserCmp" id="1470556608"
name="DFIG_SANY_IGBT_WTG16840_SK_V1:DFIG_CONVERTER_1_1" x="936" y="342" w="258" h="107" z="150"
orient="0" defn="DFIG_SANY_IGBT_WTG16840_SK_V1:DFIG_CONVERTER_1_1" link="-1" q="4" disable="false">
          <paramlist name="" link="-1" crc="35606437">
            <param name="Name" value="" />
            <param name="CONVERTER_NUM" value="0" />
            <param name="Cmd_1" value="1" />
            <param name="Cmd_2" value="1" />
            <param name="Cmd_3" value="1" />
            <param name="Cmd_4" value="0" />
            <param name="Cmd_5" value="1" />
            <param name="Cmd_7" value="0" />
            <param name="Cmd_8" value="0" />
            <param name="Cmd_6" value="1" />
            <param name="Control_72" value="0" />
            <param name="Control_76" value="0" />
            <param name="Control_1" value="0.08" />
            <param name="Control_2" value="0.0001" />
            <param name="Control_3" value="5" />
            <param name="Control_4" value="0.15" />
            <param name="Control_5" value="1200" />
            <param name="Control_6" value="20" />
```

表 66　109项面板保存值全集

组	参数	保存值
Cmd	Cmd_1	1.0
Cmd	Cmd_2	1.0
Cmd	Cmd_3	1.0

组	参数	保存值
Cmd	Cmd_4	0.0
Cmd	Cmd_5	1.0
Cmd	Cmd_6	1.0
Cmd	Cmd_7	0.0
Cmd	Cmd_8	0.0
Control	Control_1	0.08
Control	Control_2	0.0001
Control	Control_3	5.0
Control	Control_4	0.15
Control	Control_5	1200.0
Control	Control_6	20.0
Control	Control_7	1200.0
Control	Control_8	20.0
Control	Control_9	3500.0
Control	Control_10	15.0
Control	Control_11	4500.0
Control	Control_12	25.0
Control	Control_13	0.9
Control	Control_14	0.05
Control	Control_15	3.0
Control	Control_17	1.13
Control	Control_18	0.05
Control	Control_19	2.2841
Control	Control_21	0.2777
Control	Control_22	0.1111
Control	Control_23	0.4
Control	Control_24	0.0
Control	Control_25	0.2895
Control	Control_26	-0.2895

组	参数	保存值
Control	Control_27	1.0
Control	Control_28	1.0
Control	Control_29	1.0
Control	Control_30	0.2221
Control	Control_31	0.2221
Control	Control_32	0.3456
Control	Control_33	0.3456
Control	Control_34	1.0
Control	Control_35	1.0
Control	Control_36	0.0
Control	Control_37	-0.0187
Control	Control_38	-0.1481
Control	Control_39	-0.014
Control	Control_40	-0.1481
Control	Control_41	1.0
Control	Control_42	1.0
Control	Control_43	0.0
Control	Control_44	0.3801
Control	Control_45	0.074
Control	Control_56	50.0
Control	Control_57	10.0
Control	Control_58	0.3
Control	Control_59	0.1
Control	Control_60	5.0
Control	Control_61	0.06
Control	Control_62	0.03
Control	Control_63	0.2
Control	Control_64	0.1
Control	Control_65	0.7

组	参数	保存值
Control	Control_66	0.5
Control	Control_67	0.015
Control	Control_68	0.008
Control	Control_72	0.0
Control	Control_73	2.0
Control	Control_74	0.7
Control	Control_75	3.0
Control	Control_76	0.0
Gen	Gen_1	4000000.0
Gen	Gen_2	4.0
Gen	Gen_3	3.541
Gen	Gen_4	1.14
Gen	Gen_5	0.0097
Gen	Gen_6	0.0774
Gen	Gen_7	0.0087
Gen	Gen_8	0.0999
Gen	Gen_9	2.6962
Gen	Gen_10	2.0
Gen	Gen_11	1680.0
Gen	Gen_12	3000.0
Protect	Protect_1	1.33
Protect	Protect_2	0.15
Protect	Protect_3	1.28
Protect	Protect_4	0.6
Protect	Protect_5	1.23
Protect	Protect_6	1.1
Protect	Protect_7	0.12
Protect	Protect_8	0.3
Protect	Protect_9	1.13

组	参数	保存值
Protect	Protect_10	10.4
Protect	Protect_11	0.4
Protect	Protect_12	1.4
Protect	Protect_13	0.6
Protect	Protect_14	2.2
Protect	Protect_15	0.83
Protect	Protect_16	3.0
Protect	Protect_17	54.0
Protect	Protect_18	0.2
Protect	Protect_19	53.0
Protect	Protect_20	10.0
Protect	Protect_21	52.0
Protect	Protect_22	2100.0
Protect	Protect_23	46.0
Protect	Protect_24	0.2
Protect	Protect_25	47.0
Protect	Protect_26	10.0
Protect	Protect_27	47.5
Protect	Protect_28	2100.0

表 67　80槽控制段与外层/Fortran编号

p下标	parameters零基	Fortran一基	来源	值
0	10	11	Control_1	0.08
1	11	12	Control_2	0.0001
2	12	13	Control_3	5.0
3	13	14	Control_4	0.15
4	14	15	Control_5	1200.0
5	15	16	Control_6	20.0
6	16	17	Control_7	1200.0

p下标	parameters零基	Fortran一基	来源	值
7	17	18	Control_8	20.0
8	18	19	Control_9	3500.0
9	19	20	Control_10	15.0
10	20	21	Control_11	4500.0
11	21	22	Control_12	25.0
12	22	23	Control_13	0.9
13	23	24	Control_14	0.05
14	24	25	Control_15	3.0
15	25	26	Control_15	3.0
16	26	27	Control_17	1.13
17	27	28	Control_18	0.05
18	28	29	Control_19	2.2841
19	29	30	Control_19	2.2841
20	30	31	Control_21	0.2777
21	31	32	Control_22	0.1111
22	32	33	Control_23	0.4
23	33	34	Control_24	0.0
24	34	35	Control_25	0.2895
25	35	36	Control_26	-0.2895
26	36	37	Control_27	1.0
27	37	38	Control_28	1.0
28	38	39	Control_29	1.0
29	39	40	Control_30	0.2221
30	40	41	Control_31	0.2221
31	41	42	Control_32	0.3456
32	42	43	Control_33	0.3456
33	43	44	Control_34	1.0
34	44	45	Control_35	1.0
35	45	46	Control_36	0.0

p下标	parameters零基	Fortran一基	来源	值
36	46	47	Control_37	-0.0187
37	47	48	Control_38	-0.1481
38	48	49	Control_39	-0.014
39	49	50	Control_40	-0.1481
40	50	51	Control_41	1.0
41	51	52	Control_42	1.0
42	52	53	Control_43	0.0
43	53	54	Control_44	0.3801
44	54	55	Control_45	0.074
45	55	56	常数0	0.0
46	56	57	常数0	0.0
47	57	58	常数0	0.0
48	58	59	常数0	0.0
49	59	60	常数0	0.0
50	60	61	常数0	0.0
51	61	62	常数0	0.0
52	62	63	常数0	0.0
53	63	64	常数0	0.0
54	64	65	常数0	0.0
55	65	66	Control_56	50.0
56	66	67	Control_57	10.0
57	67	68	Control_58	0.3
58	68	69	Control_59	0.1
59	69	70	Control_60	5.0
60	70	71	Control_61	0.06
61	71	72	Control_62	0.03
62	72	73	Control_63	0.2
63	73	74	Control_64	0.1
64	74	75	Control_65	0.7

p下标	parameters零基	Fortran一基	来源	值
65	75	76	Control_66	0.5
66	76	77	Control_67	0.015
67	77	78	Control_68	0.008
68	78	79	常数0	0.0
69	79	80	常数0	0.0
70	80	81	常数0	0.0
71	81	82	Control_72	0.0
72	82	83	Control_73	2.0
73	83	84	Control_74	0.7
74	84	85	Control_75	3.0
75	85	86	常数0	0.0
76	86	87	常数0	0.0
77	87	88	常数0	0.0
78	88	89	常数0	0.0
79	89	90	P_pos动态	动态

14.3.2 外层前10槽与机器模型分层

DFIG_Paras合并411914729的10项为Gen1、Gen4、Gen3、Gen9×Gen4²/Gen2、Gen10、五个0，当前数值4000000、1.14、3.541、.87599538、2、五个0。Control56在后控制段外层65/Fortran66作为额定频率。

.87599538是工程换算铭牌Xm，不能与机器Xmd=2.6962pu互换。Gen5～8虽被导入Motor_Rs/Xls/Rr/Xlr标签却不进入该铭牌合并，实体机器对应参数是直接数字；DLL槽作用、画布引用和实体参数变化三者须区别。

$$X_m = 2.6962 \times 1.14^2 \,/\, 4 = 0.87599538 \tag{14-4}$$

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L416151-L416170

```
        <User classid="UserCmp" defn="master:datamerge" id="411914729" x="1512" y="1224" w="27"
h="206" z="2260" orient="4" link="-1" layer="" q="4" disable="false">
          <paramlist link="-1" name="" crc="43697768">
            <param name="N" value="10" />
            <param name="Type" value="2" />
            <param name="Disp" value="1" />
            <param name="View" value="0" />
            <param name="Dim1" value="1" />
            <param name="Dim2" value="1" />
            <param name="Dim3" value="1" />
            <param name="Dim4" value="1" />
            <param name="Dim5" value="1" />
            <param name="Dim6" value="1" />
            <param name="Dim7" value="1" />
            <param name="Dim8" value="1" />
            <param name="Dim9" value="1" />
            <param name="Dim10" value="1" />
            <param name="Dim11" value="1" />
            <param name="Dim12" value="1" />
            <param name="Name" value="" />
          </paramlist>
```

14.3.3 Grid_Input电压电流与宿主元数据

Grid_Input合并708839930一基1～10为Vab、Vbc、Vca、Vdc_fed_P、Vdc_fed_N、Ia_grid、Ib_grid、Ic_grid、real_time、multiplerun_flag。三相低压采样先乘1000，再差分Va−Vb/Vb−Vc/Vc−Va；网侧电流来自Iabc_inductor_pu后各乘1000。

real_time是宿主时间，multiplerun_flag是master:run_num运行号。恢复主入口比较两侧C++[8]作保存条件、读grid[9]运行号；不能把两个[8]解释成同一个运行号。

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L418710-L418729

```
        <User classid="UserCmp" defn="master:datamerge" id="708839930" x="774" y="1944" w="27"
h="206" z="2290" orient="4" link="-1" layer="" q="4" disable="false">
          <paramlist link="-1" name="" crc="43697768">
            <param name="N" value="10" />
            <param name="Type" value="2" />
            <param name="Disp" value="1" />
            <param name="View" value="0" />
            <param name="Dim1" value="1" />
            <param name="Dim2" value="1" />
            <param name="Dim3" value="1" />
            <param name="Dim4" value="1" />
            <param name="Dim5" value="1" />
            <param name="Dim6" value="1" />
            <param name="Dim7" value="1" />
            <param name="Dim8" value="1" />
            <param name="Dim9" value="1" />
            <param name="Dim10" value="1" />
            <param name="Dim11" value="1" />
            <param name="Dim12" value="1" />
            <param name="Name" value="" />
          </paramlist>
```

表 68　实际网侧10槽

Fortran	C++	标签	说明
1	0	Vab	V，Va−Vb
2	1	Vbc	V，Vb−Vc
3	2	Vca	V，Vc−Va
4	3	Vdc_fed_P	V，上半母线
5	4	Vdc_fed_N	V，下半母线
6	5	Ia_grid	A，网侧支路
7	6	Ib_grid	A
8	7	Ic_grid	A
9	8	real_time	宿主时间s
10	9	multiplerun_flag	运行号

14.3.4 Rotor_Input第4～6项确实是低压电流

合并1433791835一基1～10为Ia/Ib/Ic_rotor、Ia/Ib/Ic_1140V、Rotor_Theta、Rotor_Speed、snapshot_time、snapshot_flag。第4～6项不是三相电压；1140V只标识测量位置。多仪表2020652835有MeasI1、CurI=Iabc_1140V_pu、BaseA1[kA]。

datatap1539844327/78828168/945462682按Index1/2/3取电流，gain1595043209/243786219/671152404各1000倍输出A值；标签1603968875/1545663409/1937877762经Wire2045936342/1860175237/1520496076接merge一基4/5/6。rotor_input写机侧0x240/248/250，motor_calculate495～497送电流序分解。工程与可执行使用一致，错误在手册/部分注释；不未经支路检查再称为定子单支路电流。

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L418612-L418631

```
        <User classid="UserCmp" defn="master:datamerge" id="1433791835" x="972" y="1944" w="27"
h="206" z="2540" orient="4" link="-1" layer="" q="4" disable="false">
          <paramlist link="-1" name="" crc="43697768">
            <param name="N" value="10" />
            <param name="Type" value="2" />
            <param name="Disp" value="1" />
            <param name="View" value="0" />
            <param name="Dim1" value="1" />
            <param name="Dim2" value="1" />
            <param name="Dim3" value="1" />
            <param name="Dim4" value="1" />
            <param name="Dim5" value="1" />
            <param name="Dim6" value="1" />
            <param name="Dim7" value="1" />
            <param name="Dim8" value="1" />
            <param name="Dim9" value="1" />
            <param name="Dim10" value="1" />
            <param name="Dim11" value="1" />
            <param name="Dim12" value="1" />
            <param name="Name" value="" />
          </paramlist>
```

表 69　实际机侧10槽

Fortran	C++	来源	局部偏移/单位
1	0	Ia_rotor	0x228，A
2	1	Ib_rotor	0x230，A
3	2	Ic_rotor	0x238，A
4	3	Ia_1140V	0x240，A
5	4	Ib_1140V	0x248，A
6	5	Ic_1140V	0x250，A
7	6	Rotor_Theta	0x290，机器角
8	7	Rotor_Speed	0x6E0，r/min
9	8	snapshot_time	快照时间元数据
10	9	snapshot_flag	1000请求状态恢复

14.3.5 P/Q固定给定与P_pos反馈

Main常数675879430保存P_pu1.0，629070574保存Q_pu.33，经两维Cmd端口到converter并进入11槽命令第2/3项；接口可每拍读取不等于本工程已安排时变P/Q曲线，当前Q模式为1。

P_pos由辅助POWER_CALCULATE1535623755输入Vabc_1140V/Iabc_1140V产生8槽VIEW_PQ，再经TEST4 的 datatap1324340621 Index4 、 export1909419465 与 父 标 签 1129668064/451225483 回 到 parameters89/Fortran90/p79。它不是控制器130槽View_Output回灌，也不是独立Control80。辅助

Power_Base4、Voltage_Base1.14、FBASE50、Sam_Freq10e3；具体刷新延迟和功率单位须查辅助DLL/动态执行，z顺序元数据不够证明反馈同拍。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L419416-L419435

```
        <User classid="UserCmp"
name="DFIG_SANY_IGBT_WTG16840_SK_V1:POWER_CALCULATE_9_2_1_1_1_1_4_1_1_1_2_1_1" id="1535623755"
x="1296" y="1800" w="214" h="119" z="830" orient="0"
defn="DFIG_SANY_IGBT_WTG16840_SK_V1:POWER_CALCULATE_9_2_1_1_1_1_4_1_1_1_2_1_1" link="-1" q="4"
disable="false">
          <paramlist link="-1" name="" crc="116158968">
            <param name="Name" value="POWER_CALCULATE" />
            <param name="Sam_Time" value="250e-6" />
            <param name="Sim_Time" value="DELTA_TIME" />
            <param name="untitled" value="0" />
            <param name="Power_Base" value="4" />
            <param name="Voltage_Base" value="1.14" />
            <param name="Power_Num" value="Converter_num_1" />
            <param name="FBASE" value="50" />
            <param name="Sam_Freq" value="10e3" />
          </paramlist>
        </User>
        <User classid="UserCmp" id="646696274" name="master:pgb" x="2106" y="1782" w="78" h="40"
z="3220" orient="0" defn="master:pgb" link="-1" q="4" disable="false">
          <paramlist name="" link="-1" crc="65740200">
            <param name="Name" value="M_output1" />
            <param name="Group" value="" />
            <param name="UseSignalName" value="1" />
            <param name="enab" value="1" />
            <param name="Display" value="1" />
```

14.3.6 输出消费与保护数组容量

Grid_Output/Rotor_Output各20槽拆出使能、六路模式输出及状态，供7维G_PWM_AVG/M_PWM_AVG或PWM生成器与开关桥使用；模式0是电压/直流电流，1是占空比，2是调制波，单位随模式变化。

View_Output130槽接TEST3但恢复函数只写41项，容量不等于有效通道数。辅助VIEW_PQ8槽经TEST4监控正负序量，和控制器View_Output不同。Prot_paras实际Port30，画布Protect1～28加两个0，恢复protection_input只消费28项；应同时保留外部容量与内部有效项。

14.3.7 机侧0xF0是网侧重构相电压RMS

motor_post的RMS rows_a明确将0x19B8、0x19C0、0x19C8平方累计到0x120、0x130、0x140，结果写机侧0xF0、0x100、0x110。因此这些RMS并非采样Rotor_Input第4～6项的电压；第4～6项实际为低压电流，另分别累计到0x190/198/1A0并写0x178/180/188。

0x19B8等由rotor_input以此前状态0x258/260的网侧线电压重构：Va=(2Vab+Vbc)/3，Vb=(−Vab+Vbc)/3，Vc=(−Vab−2Vbc)/3；主调度在该输入函数之后才更新跨区网侧电压。所以机侧0xF0等是正常时序下前拍网侧线电压重构的三相相电压RMS，不能标成“转子电压RMS”。同组在主控制自适应基波提取中使用，也应标相同信号来源。

$$V_a = (2V_{ab} + V_{bc}) / 3, \quad V_b = (-V_{ab} + V_{bc}) / 3, \quad V_c = (-V_{ab} - 2V_{bc}) / 3 \tag{14-5}$$

源码/recovered_motor.cpp L300-L320

```
        motor_defaults(p, state);
        return;
    }

    const RmsRow rows_a[] = {
        {0x258, 0x118, 0xE8},
        {0x260, 0x128, 0xF8},
        {0x268, 0x138, 0x108},
        {0x240, 0x190, 0x178},
        {0x248, 0x198, 0x180},
        {0x250, 0x1A0, 0x188},
        {0x19B8, 0x120, 0xF0},
        {0x19C0, 0x130, 0x100},
        {0x19C8, 0x140, 0x110}
    };
    s.increment(0x1A8);
    accumulate(s, rows_a);
    if (s.i(0x1A8) >= s.i(0x1AC)) {
        finish_rms(s, rows_a, s.i(0x1A8));
        s.i(0x1A8, 0);
        s.i(0x1B0, 1);
```

RMS窗口与每拍累加在motor_post，真正测量方向/幅值须按该链而非源码注释判断。

14.4 故障、保护字与独立测试的边界

本节依据工程保存组件及连接，不把独立测试默认值或旧工况记录写成当前工程运行事实。工作pscx与参照相同，下面的故障和开关结论也同时适用于两者的静态内容。

14.4.1 当前故障为5～6s三相接地

活动tfaultn1298808288保存TF5s/DF1s，tpflt1746392663保存A=B=C=G=1、RON1e−7Ω/ROFF1e6Ω，两者disable=false，故为5～6s三相接地，不能称AB-G两相接地。串联故障支路692003422使用Rf/Lf。

其余八组故障控制/开关disable=true，保存5、7、9、11、13、15、17、19s等时间不能证明本工程已启用四工况。Ratio.2为阻抗计算参数，不保证低压机端实测.2pu；机端波形需本次录波。

工程文件/DFIG_SANY_IGBT_WTG1684O_SK_V1.pscx L323668-L323687

```
        <User classid="UserCmp" defn="master:tfaultn" id="1298808288" x="198" y="1026" w="79" h="59"
z="190" orient="0" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="79534367">
            <param name="TF" value="5 [s]" />
            <param name="DF" value="1 [s]" />
            <param name="Name" value="" />
          </paramlist>
        </User>
        <Wire classid="WireOrthogonal" id="60372655" name="" x="144" y="954" w="10" h="82" orient="0"
disable="false">
          <vertex x="0" y="0" />
          <vertex x="0" y="-72" />
        </Wire>
        <User classid="UserCmp" defn="master:tpflt" id="1746392663" x="126" y="990" w="73" h="80"
z="290" orient="0" link="-1" q="4" disable="false">
          <paramlist link="-1" name="" crc="5223703">
            <param name="Ctype" value="0" />
            <param name="OpCur" value="0" />
            <param name="Grnd" value="1" />
            <param name="View" value="1" />
            <param name="CLVL" value="0.0 [kA]" />
            <param name="RON" value="0.0000001 [ohm]" />
            <param name="ROFF" value="1.0E6 [ohm]" />
```

14.4.2 SCR与故障阻抗推导

电网父实例SCR1.5、XDR10、SyskV33、S_BASE4、SysHz50、Ratio.2。源SCR模块由Vbase²/Sbase、SCR、X/R产生R_Source/L_Source；故障模块明确Ratio/(1−Ratio)乘源阻抗产生Rf/Lf，分别被源1299718745和故障692003422引用。

静态代数给Zbase272.25Ω、源阻抗模181.5Ω、故障比例.25。此为按运算块推导而非动态测量；变压器和机组反馈使端子电压比不能直接按单分压等同。

$$Z_b = V_{LL}^2 / S_b, \quad |Z_s| = Z_b / SCR \tag{14-6}$$

$$R_s = |Z_s| / \sqrt{1 + (X/R)^2}, \quad L_s = R_s(X/R) / (2\pi f) \tag{14-7}$$

$$R_f = \frac{Ratio}{1 - Ratio} R_s, \quad L_f = \frac{Ratio}{1 - Ratio} L_s \tag{14-8}$$

14.4.3 有效N2表与残留扰动点

三相幅值xy_transfer_function1422128719/1497358153/2087336309均N2，仅有效(0,1)、(100,1)；频偏1966824837 N2有效(0,0)、(30,0)，和SysHz50相加得名义50Hz。

N以上X3～X12中残留8.301s/16s/1.3等并不生效。另有812401194的N10表保存5.001～6.501s1.2、10.001～11.501s1.3，要按其实际输出连线确认用途，不能拼成8～10s低穿、12sAB、16～17s高穿。四工况曲线若保留应声明独立测试记录，附对应修改与哈希；静态工程不证明这些场景已运行。

14.4.4 工程保护字0与测试默认0x4000CC00

五var_switch保存Value0：Enable_Protect1645600848、过压976173064、欠压1495621579、过频1608119367、欠频84614029。它们分别乘2^30/2^10/2^11/2^14/2^15；pro_all2为常数0乘2^15。Sum_Protect650645758把8输入相加形成Protect_Word1，Flag_protect0时第2输出为0。

因此按保存值commands零基[7]=0，命令总门与四组未打开。contract.command_array的protection_word1073794048=0x4000CC00是独立函数调用默认，load_project没有提取五开关，不能称为当前工程保护字。运行中手动切换、状态恢复或内部保护门控须另查，不以保存值预测所有保护输出恒零。

$$W = 2^{30}E_{all} + 2^{10}E_{OV} + 2^{11}E_{UV} + 2^{14}E_{OF} + 2^{15}E_{UF} \tag{14-9}$$

$$E_{all} = E_{OV} = E_{UV} = E_{OF} = E_{UF} = 0 \Rightarrow W = 0 \tag{14-10}$$

表 70　保护字来源必须分层

来源	保护字	证据
工程五开关保存值	0	XML Value0与加权求和接线
contract缺省调用	测试接口保护字示例 0x4000CC00；当前工程保护开关保存值对应0，见第14章	函数默认实参
五开关全设1的数学组合	1073794048	假设配置，非当前保存值

14.5 Fortran包装、DLL加载与ABI核对

组件脚本、两包装和二进制PE头共同确定静态入口契约。本轮不编译或启动仿真，文件路径与位数证据不能取代实时加载模块或真实ABI调用验证。

14.5.1 控制器宿主调用次序

活动脚本FIRSTSTEP调用BEGIN与INIT_DYNAMICS，每步DYNAMICS、LASTSTEP END_DYNAMICS。11形参次序为Grid_Input、Rotor_Input、Paras_Input、Prot_paras、Cmd_Input、Grid_Output、Rotor_Output、View_Output、Sim_Ts、Sam_Ts、Converter_Num，与C++主入口一致。

obj_file/lib_file路线注释，dll_file路线活动。BEGIN解析指针，INIT传数组首地址与实例0，END只传实例；注释TIMEZERO不代替FIRSTSTEP。

工程文件/DFIG_SANY_IGBT_WTG16840_SK_V1.pscx L413144-L413163

```
        <segment name="Fortran" id="1116027818" classid="CoreSegment"><![CDATA[
       ! obj_file
       ! IF (FIRSTSTEP) CALL DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT($Paras_Input,$Converter_Num)
       ! CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC($Grid_Input,$Rotor_Input,$Paras_Input,$Cmd_Input,$Grid_Output,$Rotor
_Output,$Sim_Ts,$Sam_Ts,$Converter_Num)

       ! dll_file
       IF (FIRSTSTEP) CALL DFIG_CONTROLLER_SY_WG4000KDF_NPC_BEGIN() ! IF (TIMEZERO) CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_BEGIN()
       IF (FIRSTSTEP) CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT_DYNAMICS($Paras_Input,$Converter_Num)
       CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_DYNAMICS($Grid_Input,$Rotor_Input,$Paras_Input,$Prot_paras,$Cmd_Inpu
t,$Grid_Output,$Rotor_Output,$View_Output,$Sim_Ts,$Sam_Ts,$Converter_Num)
        IF(LASTSTEP) CALL DFIG_CONTROLLER_SY_WG4000KDF_NPC_END_DYNAMICS($Converter_num) !
$#Component/1000000

       ! lib_file
       ! IF (FIRSTSTEP) CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT_FORTRAN($Paras_Input,$Converter_Num)
       ! CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_FORTRAN($Grid_Input,$Rotor_Input,$Paras_Input,$Cmd_Input,$Grid_Outpu
t,$Rotor_Output,$Sim_Ts,$Sam_Ts,$Converter_Num)]]></segment>
        <segment name="Branch" id="2075610534" classid="CoreSegment"><![CDATA[]]></segment>
        <segment name="Computations" id="2120461321" classid="CoreSegment"><![CDATA[]]></segment>
      </script>
      <graphics size="4">
        <paramlist>
          <param name="show_border" value="0" />
```

14.5.2 主DLL加载名、路径和位数

包装按Intel _WIN64或GFortran指针8选择../SY_SG_WG4000KDF_NPC_64bit_SK_V1.dll，否则32bit同名。GetProcAddress查dfig_controller_sy_wg4000kdf_npc及_end/_init；失败报错STOP。

现主DLL302080字节、PE Machine0x8664/AMD64，哈希与原装参照及contract.ORIGINAL_SHA256相同，不能称已由recovered源码生成后替换。相对路径按仿真EXE目录解析；某时刻实际加载绝对路径要进程模块或运行日志证明。

工程文件/DFIGSY4MW_DllLink64bit.f L149-L168

```
#ifdef __INTEL_COMPILER
#ifdef _WIN64
            DLL_HANDLE_DFIG =
LoadLibrary(C_CHAR_'../SY_SG_WG4000KDF_NPC_64bit_SK_V1.dll'//C_NULL_CHAR)      ! add dll file name
#else
            DLL_HANDLE_DFIG =
LoadLibrary(C_CHAR_'../SY_SG_WG4000KDF_NPC_32bit_SK_V1.dll'//C_NULL_CHAR)      ! add dll file name
#endif
#endif
#ifdef __GFORTRAN__
#if __SIZEOF_POINTER__ == 8
            DLL_HANDLE_DFIG =
LoadLibrary(C_CHAR_'../SY_SG_WG4000KDF_NPC_64bit_SK_V1.dll'//C_NULL_CHAR)      ! add dll file name
#else
            DLL_HANDLE_DFIG =
LoadLibrary(C_CHAR_'../SY_SG_WG4000KDF_NPC_32bit_SK_V1.dll'//C_NULL_CHAR)      ! add dll file name
#endif
#endif
!!=============================================================================================================
=========

            IF (DLL_HANDLE_DFIG .EQ. 0) THEN
                CALL print_error_DFIG ('loading DLL')
                STOP
            ENDIF
```

14.5.3 默认REAL与数组首元素传递

包装REAL/INTEGER没有kind，接口数组参数以标量首元素转交，无长度描述符；ISO_C_BINDING提供句柄与函数指针并不使所有REAL自动成为C_DOUBLE。PSCAD Port有datatype3与10/90等维数，C++实际double与int32。

实数宽度还依赖生成Fortran和编译选项，现文本/位域不能单独证明8字节REAL、4字节INTEGER匹配。本轮未做真实ABI调用；两目录包装非空有效行相同，未发现ABI核心文本变化。

工程文件/DFIGSY4MW_DllLink64bit.f L294-L316

```
!! dfig controller function
!!=============================================================================================================
=========
SUBROUTINE
DFIG_CONTROLLER_SY_WG4000KDF_NPC_DYNAMICS(Grid_Input,Motor_Input,Paras_Input,Prot_Paras,Cmd_Input,Gri
d_Output,Motor_Output,View_Output,Sim_Time,Sam_Time,Converter_Num)
        USE DynamicLoad_DFIG
    !!=============================================!!
        REAL
Grid_Input,Motor_Input,Paras_Input,Prot_Paras,Cmd_Input,Grid_Output,Motor_Output,View_Output,Sim_Time
,Sam_Time       ! Defining each of the parameters of this function.
        INTEGER Converter_Num
! Defining each of the parameters of this function.
    CALL
DFIG_CONTROLLER_SY_WG4000KDF_NPC_DLL(Grid_Input,Motor_Input,Paras_Input,Prot_Paras,Cmd_Input,Grid_Out
put,Motor_Output,View_Output,Sim_Time,Sam_Time,Converter_Num)
        !!=============================================!!
    RETURN
END SUBROUTINE DFIG_CONTROLLER_SY_WG4000KDF_NPC_DYNAMICS

!! dfig init function
!!=============================================================================================================
=========
SUBROUTINE DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT_DYNAMICS(Paras_Input,Converter_Num)
        USE DynamicLoad_DFIG
    !!=============================================!!
        REAL Paras_Input        ! Defining each of the parameters of this function.
        INTEGER Converter_Num   ! Defining each of the parameters of this function.
    CALL DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT_DLL(Paras_Input,Converter_Num)
        !!=============================================!!
    RETURN
END SUBROUTINE DFIG_CONTROLLER_SY_WG4000KDF_NPC_INIT_DYNAMICS
```

14.5.4 辅助DLL是独立闭环依赖

POWER_FUN 包装加载 ../POWER_FUN_64bit_SK_V1.dll 并解析 power_calculate/power_init/power_grid_out/autorun。当前活动 POWER_CALCULATE 脚本 FIRSTSTEP BEGIN/INIT，每步CALCULATE；其它声明存在不证明画布调用。

现辅助DLL175104字节、AMD64 Machine0x8664，与参照哈希相同；八份主控制器源码不生成它。完整工程还依赖两包装与PSCAD master库，替换主DLL不等于重建P_pos辅助算法。

工程文件/POWER_FUN_DllLink64bit.f L161-L179

```
#ifdef __INTEL_COMPILER
#ifdef _WIN64
            DLL_HANDLE_POWER = LoadLibrary(C_CHAR_'../POWER_FUN_64bit_SK_V1.dll'//C_NULL_CHAR)      !
add dll file name
#else
            DLL_HANDLE_POWER = LoadLibrary(C_CHAR_'../POWER_FUN_32bit_SK_V1.dll'//C_NULL_CHAR)      !
add dll file name
#endif
#endif
#ifdef __GFORTRAN__
#if __SIZEOF_POINTER__ == 8
            DLL_HANDLE_POWER = LoadLibrary(C_CHAR_'../POWER_FUN_64bit_SK_V1.dll'//C_NULL_CHAR)      !
add dll file name
#else
            DLL_HANDLE_POWER = LoadLibrary(C_CHAR_'../POWER_FUN_32bit_SK_V1.dll'//C_NULL_CHAR)      !
add dll file name
#endif
#endif
!!=============================================================================================
=========

            IF (DLL_HANDLE_POWER .EQ. 0) THEN
                CALL print_error_POWER ('loading DLL')
                STOP
```

14.5.5 Python契约覆盖及遗漏

load_project 固定父1470556608/controller908509267，检查未禁用、Sim_Ts=DELTA_TIME、CONVERTER_NUM0，提取面板109项与time_step/Sam_Ts。parameter_array生成90，protection_array28+2零；已调用并核对实际画布组。

它不检查机器4.2MVA、变压器4.4MVA、电网SCR、故障相别/时窗、五保护开关、动态给定连线和实时DLL加载。p_pos默认0不证明反馈恒0；command_array输入P/Q与默认保护字是调用实参，不是自动读到当前工程信号。契约检查通过仅说明已检查字段。

contract.py L77-L95

```
def parameter_array(config, p_pos=0.0):
    p = config['parameters']
    result = [p['Gen_1'], p['Gen_4'], p['Gen_3'],
              p['Gen_9'] * p['Gen_4'] ** 2 / p['Gen_2'], p['Gen_10']] + [0.0] * 5
    result.extend(p['Control_' + str(k)] if k is not None else 0.0 for k in CONTROL_SLOTS)
    result[-1] = p_pos
    if len(result) != 90:
        raise AssertionError('Incorrect verified parameter layout')
    return finite(result)


def protection_array(config):
    return finite([config['parameters']['Protect_' + str(k)] for k in range(1, 29)] + [0, 0])


def command_array(config, p_ref, q_ref, protection_word=1073794048):
    p = config['parameters']
    return finite([p['Cmd_1'], p_ref, q_ref, p['Cmd_2'], p['Cmd_3'],
                   p['Cmd_4'], p['Cmd_5'], protection_word, p['Cmd_6'], p['Cmd_7'], p['Cmd_8']])
```